On Monte Carlo Tree Search with Multiple Objectives

Michael Painter

Pembroke College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

Hilary 2026

Acknowledgements

First and foremost, thank you to my supervisors, Nick and Bruno, for their support, guidance, understanding and patience throughout my DPhil. I have grown both as a researcher and as a human over the course of this journey, and this would not have been possible without them. I will continue to, unprompted, let people know that I could not have asked for better supervisors.

I would also like to thank my family and friends for their love, support and encouragement throughout my time at Oxford, and for their understanding and patience during the more difficult stretches of this work, when I was not always easy to reach. I am especially grateful to my partner, Ioana, for her love, support and encouragement, and also for taking care of me when I was not taking care of myself.

Finally, I would like to thank my internal and external examiners, Shimon Whiteson and Simon Lucas, for their time reading this thesis, and for their constructive feedback, insightful questions and thoughtful discussion during the viva.

Abstract

Monte Carlo Tree Search (MCTS) is a sample-based planning framework for sequential decision-making problems, which has produced state-of-the-art results across many domains. While most algorithms underpinning these results are designed for a single objective, many real-world problems involve trade-offs between conflicting objectives, whose relative weighting is unknown at planning time. This thesis first improves exploration in single-objective MCTS, and then extends those improvements to the multi-objective setting.

In the single-objective setting, the maximum entropy objective function, which is commonly used to encourage exploration, is shown to not always align with the standard objective of maximising expected return, both theoretically and empirically. Boltzmann Tree Search (BTS) is adapted from Maximum ENtropy Tree Search (MENTS) by removing entropy from the objective function. Decaying ENtropy Tree Search (DENTS) then re-introduces entropy as an exploration bonus during planning, while still recommending actions that maximise expected return. BTS and DENTS are shown to converge under mild conditions, and outperform Upper Confidence Bound applied to Trees (UCT) and MENTS on gridworld environments and on the game of Go.

In the multi-objective setting, two Multi-Objective MCTS (MOMCTS) frameworks are introduced for the unknown weights scenario with linear utilities. The first is Convex Hull MCTS (CHMCTS), which generalises prior MOMCTS work to stochastic environments via convex hull backups, but at high computational cost. The second is the simplex map, an adaptive data structure that uses a hierarchical triangulation of the unit simplex to maintain value and entropy estimates as a function of how the objectives are weighted. Simplex maps integrate cleanly with the entropy bonus used by DENTS, whereas convex hull backups do not. The methods developed here outperform existing MOMCTS methods on multi-objective gridworld and MO-Gymnasium environments, with simplex maps scaling substantially better than CHMCTS in stochastic environments.

Contents

List of Figures **xi**

List of Tables **xiii**

List of Abbreviations **xv**

List of Notation **xvii**

1 Introduction **1**
- 1.1 Overview and Motivation . . . 2
- 1.2 Scope of the Thesis . . . 4
- 1.3 Contributions . . . 6
- 1.4 Structure of Thesis . . . 8
- 1.5 Publications . . . 9

2 Background **11**
- 2.1 Multi-Armed Bandits . . . 12
  - 2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm 13
  - 2.1.2 Exploring Bandits . . . 14
  - 2.1.3 Contextual Bandits . . . 16
- 2.2 Markov Decision Processes . . . 17
- 2.3 Planning and Reinforcement Learning . . . 20
  - 2.3.1 Maximum Entropy Planning and Reinforcement Learning . . 24
  - 2.3.2 Planning Versus Reinforcement Learning . . . 26
- 2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search . . 27
  - 2.4.1 Notation . . . 29
  - 2.4.2 Trial Based Heuristic Tree Search . . . 31
  - 2.4.3 Upper Confidence Bound Applied to Trees (UCT) . . . 36
  - 2.4.4 Predictor-UCT (PUCT) . . . 39
  - 2.4.5 Maximum Entropy Tree Search . . . 40
- 2.5 Convergence of Random Variables . . . 41
- 2.6 Multi-Objective Planning and Reinforcement Learning . . . 43
  - 2.6.1 Convex Hull Value Iteration . . . 51
  - 2.6.2 Implications of Assuming a Linear Utility . . . 52

3 Related Work **55**
3.1 Multi-Armed Bandits . . . 55
3.2 Maximum Entropy Reinforcement Learning . . . 57
3.3 Monte Carlo Tree Search . . . 57
3.4 Multi-Objective Optimisation and Reinforcement Learning . . . 59
3.5 Multi-Objective Monte Carlo Tree Search . . . 61
3.5.1 Multi-Objective Monte Carlo Tree Search with Point Estimates 63

4 Monte Carlo Tree Search with Boltzmann Exploration **65**
4.1 Introduction and Motivation . . . 67
4.1.1 UCT . . . 68
4.1.2 MENTS . . . 70
4.1.3 Simple Regret and Consistency . . . 72
4.2 Boltzmann Search . . . 73
4.2.1 Boltzmann Tree Search . . . 75
4.2.2 Decaying ENtropy Tree Search . . . 79
4.2.3 Stochastic Search Policies . . . 81
4.3 Theoretical MDPs . . . 87
4.3.1 The D-Chain Problem . . . 87
4.3.2 The Entropy Trap . . . 89
4.4 Empirical Results . . . 92
4.4.1 Environments . . . 93
4.4.2 Evaluation Procedure . . . 95
4.4.3 Results and Discussion . . . 97
4.5 Theoretical Results . . . 107
4.5.1 Notation . . . 108
4.5.2 Preliminaries . . . 110
4.5.3 Consistency Results . . . 113
4.5.4 The Entropy Trap . . . 121
4.6 Summary . . . 125

5 Convex Hull Monte Carlo Tree Search **127**
5.1 Introduction and Motivation . . . 128
5.1.1 Consistent Action Selection . . . 129
5.1.2 Contextual Tree Search . . . 131
5.2 Contextual Zooming for Trees . . . 132
5.2.1 Algorithm Definition . . . 132
5.2.2 Differences from Contextual Zooming . . . 135
5.3 Convex Hull Monte Carlo Tree Search . . . 136
5.3.1 Common Framework . . . 137

5.3.2 Convex Hull UCT . . . 138
5.3.3 Convex Hull BTS . . . 138
5.3.4 Convex Hull CZT . . . 138
5.3.5 Prior Search Policies . . . 139
5.3.6 Optimisations . . . 140
5.3.7 Entropy and Convex Hulls . . . 143
5.4 Empirical Results . . . 144
5.4.1 Environments . . . 144
5.4.2 Evaluation Procedure . . . 147
5.4.3 Multi-Objective Gridworld Results . . . 148
5.4.4 Scalability Results . . . 152
5.5 Theoretical Discussion . . . 155
5.5.1 Notation . . . 156
5.5.2 Consistent Action Selection and the Linear Utility Assumption 157
5.5.3 When Local Optimality Implies Global Optimality . . . 161
5.5.4 Convergence of CHMCTS . . . 163
5.6 Summary . . . 164

6 Simplex Maps for Multi-Objective Monte Carlo Tree Search 167
6.1 Introduction and Motivation . . . 168
6.1.1 Limitations of CHMCTS and CZT . . . 168
6.1.2 Motivation for Working in Weight Space . . . 169
6.1.3 Adaptive Subdivisions of the Weight Space . . . 170
6.2 Simplex Maps . . . 171
6.2.1 Interface . . . 171
6.2.2 Implementation . . . 173
6.2.3 Complexity Discussion . . . 178
6.3 Simplex Maps for Tree Search . . . 179
6.4 Empirical Results . . . 181
6.4.1 Evaluation Procedure . . . 182
6.4.2 Multi-Objective Gridworld Results . . . 182
6.4.3 Scalability Results . . . 185
6.5 Summary . . . 187

7 Conclusion 189
7.1 Summary of Contributions . . . 189
7.2 Future Work . . . 191

Appendices

A Additional Algorithm Details for Chapter 4 **195**
A.1 AR-BTS 195
A.2 AR-DENTS 197

B Additional Experimental Details for Chapter 4 **201**
B.1 Additional DeterministicGridWorld, FrozenLake details 202
B.2 Parameter Details for Gridworld Experiments 202
B.2.1 Parameter Summary 202
B.2.2 Integration of Common Parameters into Prior Algorithms . . 205
B.2.3 Parameter Optimisation 207
B.3 Additional Details for Go Experiments 212
B.3.1 AR-MENTS, AR-RENTS and AR-TENTS 212
B.3.2 Go Round Robin Algorithm Parameter Selection 212
B.3.3 Average Returns vs Bellman Updates for Go 218

C Additional Experimental Details for Chapters 5 and 6 **219**
C.1 Parameter Summary 219
C.2 Parameter Optimisation 222

D Additional Theoretical Results for Chapter 4 **227**
D.1 The DENTS Stochastic Process 227
D.1.1 The BTS process. 229
D.2 Exponential Convergence Results 229
D.2.1 Lemmas for Exponential Convergence Theorems 230
D.2.2 Inductive Steps for Exponential Convergence Theorems . . . 232

Bibliography **241**

List of Figures

2.1 The procedure of a multi-armed bandit problem, following a strategy σ. 13
2.2 The procedure of an exploring multi-armed bandit problem, following an exploration strategy σ, and recommendation strategy ψ. 15
2.3 The procedure of a contextual multi-armed bandit problem, following a strategy σ. 17
2.4 An example MDP $\mathcal{M}$. 18
2.5 An overview of reinforcement learning. 21
2.6 Overview of one trial of MCTS-1. 28
2.7 Tree diagrams notation. 30
2.8 Overview of one trial of `THTS++`. 32
2.9 A complication arising from the use of a transposition table. 33
2.10 The unknown weights scenario for multi-objective reinforcement learning [43]. 43
2.11 An example MOMDP $\boldsymbol{\mathcal{M}}$. 44
2.12 The geometry of convex coverage sets. 48
2.13 An example Pareto front. 49
2.14 An example of arithmetic over vector sets. 51

3.1 An example MOMDP to highlight why using point estimates at each node in MOMCTS is insufficient. 63

4.1 Exploration setting in reinforcement learning. 66
4.2 An example gridworld shortest path problem. 68
4.3 Results on the example gridworld problem for a variety of bias and temperature parameters. 69
4.4 The procedure of an exploration planning problem for MDPs. . . . 72
4.5 Example of building and sampling from an alias table. 83
4.6 An illustration of the *(modified) D-chain problem.* 88
4.7 Results on the (modified) D-chain problem. 90
4.8 An illustration of the *Entropy Trap problem.* 90
4.9 Results on the Entropy Trap problem. 92
4.10 An illustration of the FrozenLake environment. 94

4.11 Illustration of the Sailing Problem. 95
4.12 Results on DeterministicGridWorld. 98
4.13 Results on FrozenLake. 99
4.14 Results on the Sailing Problem. 102
4.15 DeterministicGridWorld results with a range of parameters. 103
4.16 FrozenLake results with a range of parameters. 104
4.17 DeterministicGridWorld results with decaying temperatures. 105
4.18 Intuition behind bounding the empirical transition probabilities to the true transition probabilities. 112
4.19 (Repeated). An illustration of the *Entropy Trap problem*. 121

5.1 Example of locally optimal actions that are not globally optimal. . . 129
5.2 A motivating example MOMDP for contextual tree search. 130
5.3 Deep Sea Treasure environment. 146
5.4 Results on MO-Gymnasium environments. 149
5.5 Results on Deep Sea Treasure environments. 151
5.6 Number of backups performed on Deep Sea Treasure environments. 152
5.7 Scalability results on Deep Sea Treasure environments. 153
5.8 A stochastic MOMDP where making locally optimal decisions does not yield globally optimal policies for *Chebyshev* utilities. 158
5.9 Pareto fronts for the MOMDP from Figure 5.8 with contours of the Chebyshev utility. 159
5.10 The consistent action selection problem in a deterministic MOMDP with Chebyshev utilities. 160

6.1 A convex hull value set and its corresponding value curve. 169
6.2 Illustration of a simplex map. 172
6.3 Examples of conforming and non-conforming triangulations. 173
6.4 Closest vertex lookup examples. 176
6.5 Illustration of the `push` operation of a simplex map. 177
6.6 Example of a successful `maybe_split` operation of a simplex map. . 177
6.7 Results on MO-Gymnasium environments for Simplex Maps. 183
6.8 Results on Deep Sea Treasure environments for Simplex Maps. . . . 184
6.9 Number of backups performed on Deep Sea Treasure environments including Simplex Maps algorithms. 185
6.10 Scalability results on Deep Sea Treasure environments for Simplex Maps. 186

B.1 Maps used for the DeterministicGridWorld and FrozenLake environment. 203

List of Tables

4.1 Summary of complexities for different MCTS algorithms. 87
4.2 Results for the Go round-robin tournament. 106

B.1 The parameter search space for gridworld experiments (Section 4.4.3.1). 209
B.2 Algorithm parameters for DeterministicGridWorld experiments. . . 210
B.3 Algorithm parameters for FrozenLake experiments. 210
B.4 Algorithm parameters for Sailing(North) experiments. 211
B.5 Algorithm parameters for Sailing(South-East) experiments. 211
B.6 Round robin parameter selection for α_{init} for BTS. 213
B.7 Round robin parameter selection for α_{init} for BTS. 214
B.8 Round robin parameter selection for $\epsilon_{\tilde{\lambda}}$ for BTS. 214
B.9 Round robin parameter selection for $\epsilon_{\tilde{\lambda}}$ for AR-BTS. 214
B.10 Round robin parameter selection for ϵ for BTS. 215
B.11 Round robin parameter selection for ϵ for AR-BTS. 215
B.12 Round robin parameter selection for α for MENTS. 216
B.13 Round robin parameter selection for α for AR-MENTS. 216
B.14 Round robin parameter selection for α for RENTS. 216
B.15 Round robin parameter selection for α for AR-RENTS. 216
B.16 Round robin parameter selection for α for TENTS. 217
B.17 Round robin parameter selection for α for AR-TENTS. 217
B.18 Round robin parameter selection for β_{init} for DENTS. 217
B.19 Round robin parameter selection for β_{init} for AR-DENTS. 217
B.20 Results for matches between the average return and Bellman versions of BTS, DENTS, MENTS, RENTS and TENTS. 218

C.1 The parameter search space used in the MO-Gymnasium and Deep Sea Treasure experiments (Sections 5.4.3 and 6.4.2). 224
C.2 Algorithm parameters for MO-Gymnasium environments. 225
C.3 Algorithm parameters for DST environments. 226

List of Abbreviations

AR-BTS . . . Boltzmann Tree Search with Average Returns.

AR-DENTS . Decaying ENtropy Tree Search with Average Returns.

AR-MENTS . Maximum ENtropy Tree Search with Average Returns.

AR-RENTS . Relative ENtropy Tree Search with Average Returns.

AR-TENTS . Tsallis ENtropy Tree Search with Average Returns.

BTG . . . . . Bipartite Triangulation Graph.

BTS . . . . . Boltzmann Tree Search.

C-CHMCTS . Chebyshev Convex Hull Monte Carlo Tree Search.

CCS . . . . . Convex Coverage Set.

CDF . . . . . Cumulative Distribution Function.

CH-BTS . . . Convex Hull Boltzmann Tree Search.

CH-CZT . . . Convex Hull Contextual Zooming for Trees.

CHMCTS . . Convex Hull Monte Carlo Tree Search.

CH-UCT . . . Convex Hull UCT.

CHVI . . . . . Convex Hull Value Iteration.

CHVI(PS) . . Convex Hull Value Iteration with Prioritised Sweeping.

CHVS . . . . . Convex Hull Value Set.

CMAB . . . . Contextual Multi-Armed Bandit (problem).

CZ . . . . . Contextual Zooming.

CZT . . . . . Contextual Zooming for Trees.

DENTS . . . . Decaying ENtropy Tree Search.

DGW . . . . . DeterministicGridWorld (environment).

DKW . . . . . Dvoretzky-Kiefer-Wolfowitz (inequality).

DST . . . . . Deep Sea Treasure (environment).

EMAB . . . . Exploring Multi-Armed Bandit (problem).

EUM . . . . . Expected Utility Metric.

GT . . . . . Good-Turing (estimate).

H-CHMCTS . Hypervolume Convex Hull Monte Carlo Tree Search.

H-MCTS . . . Hybrid MCTS.

HOO . . . . . Hierarchical Optimistic Optimisation.

HV . . . . . Hypervolume (indicator).

LEB . . . . . Longest Edge Bisection.

MAB . . . . . Multi-Armed Bandit (problem).

MaxUCT . . . Maximum UCT.

MCCS . . . . . Minimal Convex Coverage Set.

MCTS . . . . . Monte Carlo Tree Search.

MCTS-1 . . . A specific and common presentation of Monte Carlo Tree Search, presented in Section 2.4.

MDP . . . . . Markov Decision Process.

MENTS . . . . Maximum ENtropy Tree Search.

MOMCTS . . Multi-Objective Monte Carlo Tree Search.

MOMDP . . . Multi-Objective Markov Decision Process.

MORL . . . . Multi-Objective Reinforcement Learning.

NHV . . . . . Normalised Hypervolume.

P-CHMCTS . Pareto Convex Hull Monte Carlo Tree Search.

PF . . . . . Pareto Front.

PUCT . . . . . Predictor-UCT.

RENTS . . . . Relative ENtropy Tree Search.

RL . . . . . Reinforcement Learning.

SM-BTS . . . Boltzmann Tree Search with Simplex Maps.

SM-DENTS . Decaying ENtropy Tree Search with Simplex Maps.

TENTS . . . . Tsallis ENtropy Tree Search.

THTS . . . . . Trial-Based Heuristic Tree Search.

`THTS++` . . . . . An extension of THTS used in this thesis.

UCB . . . . . Upper Confidence Bound (algorithm).

UCT . . . . . Upper Confidence Bound Applied to Trees.

List of Notation

General Notation

$\mathbb{1}$	The indicator function, where $\mathbb{1}[A] = 1$ when A is true, and $\mathbb{1}[A] = 0$ when A is false.
$\hat{A}$	The hat notation is used to denote an estimate, i.e. $\hat{A}$ is an estimate for some value A or some optimal value A^*
$\bar{A}$	The bar notation is used to denote sample averages, $\bar{A}$ is a sample average of a number of samples $A_1, ..., A_k$. i.e. $\bar{A} = \frac{1}{k}\sum_{i=1}^{k} A_i$.
$\tilde{A}$	The tilde notation is used to denote a (function) approximation, such as a neural network.
$\mathcal{A}$	Calligraphic font is used to denote variables that are sets or tuples (ordered sets).
$\mathbf{A}$	Bold font is used to denote vectors, and A_i is used to denote the ith component of the vector $\mathbf{A}$.
$\mathtt{A}$	Typewriter text is used to refer to variables relating to an algorithm (or the analysis of an algorithm).
$\boldsymbol{\mathcal{A}}$	The notations above will also be combined at times, so $\boldsymbol{\mathcal{A}}$ is used to denote a set that contains vectors.
$\mathbb{E}$	The expectation operator. $\mathbb{E}[f]$ denotes the expectation of a distribution f, and $\mathbb{E}_{x\sim f}[g(x)]$ denotes the expected value of $g(x)$ under the distribution f.
$O(f(x))$	Big-O notation: $g(x) = O(f(x))$ if there exist constants $c > 0$ and x_0 such that $\lvert g(x)\rvert \leq c \cdot f(x)$ for all $x \geq x_0$. Used to describe an asymptotic upper bound on the growth rate of a function.
$x \sim f$	If $f : X \to \mathbb{R}$ specifies a probability distribution in some way (e.g. a probability density function or cumulative density function), then $x \sim f$ denotes that the variable x is sampled from the distribution f. If f is a set, then $x \sim f$ denotes that the variable x is sampled uniformly randomly from the set f.

$X_i \xrightarrow{p} X$ . . . . The sequence of random variables X_i converges in probability to the random variable X. $\Pr(|X_i - X| > \varepsilon) \to 0$ as $i \to \infty$ for all $\varepsilon > 0$.

$X_i \xrightarrow{as} X$ . . . . The sequence of random variables X_i converges almost surely to the random variable X. $\Pr(\lim_{i\to\infty} X_i = X) = 1$.

(Multi-Objective) Markov Decision Processes (Sections 2.2 and 2.6)

$\mathcal{A}$ . . . . . A (finite) set of actions.

a_t . . . . . The action at the tth timestep of a trajectory.

D . . . . . The dimension of rewards in an MOMDP.

H . . . . . The finite-horizon time bound of an MDP.

$\mathcal{M}$ . . . . . A Markov Decision Process, which is defined as the tuple $\mathcal{M} = (\mathcal{S}, s_0, \mathcal{A}, R, p, H)$.

$\boldsymbol{\mathcal{M}}$ . . . . . A Multi-Objective Markov Decision Process, which is defined as the tuple $\boldsymbol{\mathcal{M}} = (\mathcal{S}, s_0, \mathcal{A}, \mathbf{R}, p, H)$.

p . . . . . The next-state transition distribution of an MDP. $p(s'|s, a)$ is the probability of transitioning to state s' from state s by taking action a.

π . . . . . A policy, mapping a state $s \in \mathcal{S}$ to a probability distribution over actions $\mathcal{A}$. That is, $\pi(a|s)$ is the probability of taking action a at state s. $\pi(s) = a'$ is used a shorthand for $\pi(a|s) = \mathbb{1}[a = a']$.

R . . . . . The reward function of an MDP: $R(s, a)$ is the reward received from taking action a at state s.

$\mathbf{R}$ . . . . . The D dimensional reward function of an MOMDP: $\mathbf{R}(s, a) \in \mathbb{R}^D$ is the vector of rewards received from taking action a at state s.

r_t . . . . . The reward at the tth timestep of a trajectory.

$\mathcal{S}$ . . . . . A (finite) set of states.

$\mathcal{S}_0$ . . . . . The set of reachable states from the starting state s_0 in an MDP.

$\text{Succ}(s, a)$ . . . . The set of successor states of a state-action pair (s, a), with respect to an MDP: $\text{Succ}(s, a) = \{s' \in \mathcal{S} | p(s'|s, a) > 0\}$.

s_0 . . . . . $s_0 \in \mathcal{S}$ is the initial starting state of an MDP.

s_t . . . . . The state at the tth timestep of a trajectory.

`terminal` . . . . `terminal` $\subseteq \mathcal{S}$ is the set of terminal states. A state is terminal if the transition distribution and reward function of the MDP are such that once reached, the state will never be left and no more reward will be received.

τ . . . . . A trajectory, or sequence, of states, actions and rewards that are sampled according to a policy π and an MDP $\mathcal{M}$: $\tau = (s_0, a_0, r_0, s_1, ..., s_{H-1}, a_{H-1}, r_{H-1}, s_H)$.

$\boldsymbol{\tau}$ . . . . . A multi-objective trajectory, of states, actions and rewards that are sampled according to a policy π and an MDP $\mathcal{M}$: $\boldsymbol{\tau} = (s_0, a_0, \mathbf{r}_0, s_1, ..., s_{H-1}, a_{H-1}, \mathbf{r}_{H-1}, s_H)$.

$\tau_{i:j}$ . . . . . A truncated trajectory, starting at timestep i, and ending at timestep j: $\tau_{i:j} = (s_i, a_i, r_i, s_{i+1}, ..., s_{j-1}, a_{j-1}, r_{j-1}, s_j)$.

$\boldsymbol{\tau}_{i:j}$ . . . . . A multi-objective truncated trajectory, starting at timestep i and ending at timestep j: $\boldsymbol{\tau}_{i:j} = (s_i, a_i, \mathbf{r}_i, s_{i+1}, ..., s_{j-1}, a_{j-1}, \mathbf{r}_{j-1}, s_j)$.

Reinforcement Learning (Section 2.3)

$J(\pi)$ . . . . . The objective function for (standard) reinforcement learning: $J(\pi) = V^\pi(s_0; 0)$.

π^* . . . . . The optimal standard policy, that maximises the objective function $J(\pi)$.

Q^* . . . . . The optimal Q-value function. $Q^*(s, a; t)$ denotes the maximal expected value that can be achieved by any policy, starting from state $s_t = s$, with action $a_t = a$.

Q^π . . . . . The Q-value function of a policy π. $Q^\pi(s, a; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$, starting by taking action $a_t = a$.

V^* . . . . . The optimal value function. $V^*(s; t)$ denotes the maximal expected value that can be achieved by any policy, starting from state $s_t = s$.

V^π . . . . . The value of a policy π. $V^\pi(s; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$.

Maximum Entropy Reinforcement Learning (Section 2.3.1)

α . . . . . The temperature parameter, or, the coefficient of the entropy term in the maximum entropy (soft) objective.

$\mathcal{H}$ . . . . . The Shannon entropy, of a probability distribution or policy.

$J_{\text{sft}}(\pi)$ . . . . . The objective function for maximum entropy (soft) reinforcement learning: $J_{\text{sft}}(\pi) = V^{\pi}_{\text{sft}}(s_0; 0)$.

π^{*}_{sft} . . . . . The optimal soft policy, that maximises the soft objective function $J_{\text{sft}}(\pi)$.

Q^{*}_{sft} . . . . . The optimal soft Q-value function. $Q^{*}_{\text{sft}}(s, a; t)$ denotes the maximal expected soft value that can be achieved by any policy, from state $s_t = s$ and taking action $a_t = a$.

Q^{π}_{sft} . . . . . The soft Q-value function of a policy π. $Q^{\pi}_{\text{sft}}(s, a; t)$ is the expected value of the policy from $s_t = s$, starting with action $a_t = a$, with an addition of the entropy of the policy weighted by the temperature α.

V^{*}_{sft} . . . . . The optimal soft value function. $V^{*}_{\text{sft}}(s; t)$ denotes the maximal expected soft value that can be achieved by any policy, from state $s_t = s$.

V^{π}_{sft} . . . . . The soft value of a policy π. $V^{\pi}_{\text{sft}}(s; t)$ is the expected value of the policy from $s_t = s$, with an addition of the entropy of the policy weighted by the temperature α.

Trial Based Heuristic Tree Search (Section 2.4)

`backup_q` . . . . Updates the values at a chance node in the backup phase of a trial `THTS++`, using the chance node's children, the trajectory sampled for the trial and the heuristic value function.

`backup_v` . . . . Updates the values at a decision node in the backup phase of a trial `THTS++`, using the decision node's children, the trajectory sampled for the trial and the heuristic value function.

$H_{\texttt{THTS++}}$ . . . . . The planning horizon used in `THTS++`, with $H_{\texttt{THTS++}} \leq H$.

$N(s)$ . . . . . The number of visits at the decision node corresponding to state s.

$N(s, a)$ . . . . . The number of visits at the chance node corresponding to state-action pair (s, a).

$N(s'|s,a)$ . . . . The conditional visit count at chance node (s,a), which is the number of times successor state s' was sampled from (s,a) across all `THTS++` trials.

`node`(s) . . . . . The decision node corresponding to the state s.

`node`(s)`.chldrn` The set of chance nodes that are children of `node`(s).

`node`(s)`.data` . The set of variables stored at decision node `node`(s), typically used for estimating values.

`node`(s,a) . . . The chance node corresponding to the state-action pair (s,a).

`node`(s,a)`.chldrn` The set of decision nodes that are children of `node`(s,a).

`node`(s,a)`.data` The set of variables stored at chance node `node`(s,a), typically used for estimating Q-values.

$\pi_{\text{heuristic}}$ . . . . . A heuristic policy, which can be used by the search policy to utilise prior knowledge of the environment.

π_{search} . . . . . The search policy used in `THTS++` to sample a trajectory in the selection phase.

$\hat{Q}_{\text{heuristic}}$ . . . . The heuristic action function used in `THTS++`, used to provide a Q-value estimate for any state-action pairs that aren't in the search tree.

`sample_context` A function used in `THTS++` that creates a context, or key-value store, and samples any initial values to be stored in the context.

`sample_outcome` A function used in `THTS++` to sample outcomes (successor states) from the environment (MDP).

`select_action` A function used in `THTS++` to select an action at a decision node.

`single_node_expansion` Specifies if `THTS++` samples a trajectory such that only one decision node is added to the search tree per trial. If not running with `single_node_expansion` set, `THTS++` samples a trajectory until a terminal state or the planning horizon $H_{\texttt{THTS++}}$ is reached.

$\mathcal{T}$ . . . . . The `THTS++` search tree. $\mathcal{T} \subseteq \mathcal{S} \cup \mathcal{S} \times \mathcal{A}$. Initially it consists of the root node, and new nodes are added from the trajectories/trials sampled in the selection phase.

`transposition_table` Specifies if `THTS++` uses a transposition table to store and re-use the decision nodes of the search tree when they are reached again via another path.

$\hat{V}_{\text{heuristic}}$ . . . . . The heuristic value function used in `THTS++`, used to initialise the value of a new decision node.

Upper Confidence Bound applied to Trees (Section 2.4.3)

b_{UCT} . . . . . The bias parameter used in UCT to control exploration.

`normalised_Q` . Specifies if the Q-values are linearly normalised (between 0 and 1) by the UCT search policy to select actions.

π_{rollout} . . . . . A policy used to sample a trajectory that is used to estimate the value of a state being added to a search tree.

π_{UCT} . . . . . The search policy used in UCT.

$\bar{Q}_{\text{UCT}}(s, a)$ . . . The sample-average value estimate used to estimate the Q-value at a chance node (s, a) in UCT.

$\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a)$ . . . The optionally linearly normalised (between 0 and 1) Q-value. Used by the UCT search policy to select actions. If `normalised_Q` is False, then $\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a) = \bar{Q}_{\text{UCT}}(s, a)$.

$\hat{V}^{\pi_{\text{rollout}}}(s)$ . . . An estimate of $V^{\pi_{\text{rollout}}}(s)$ computed using the return from a single trajectory sampled using policy π_{rollout}. It is used to initialise the value of a state being added to the search tree.

$\bar{V}_{\text{UCT}}(s)$ . . . . . The sample-average value estimate used to estimate the value at a decision node s in UCT.

Maximum ENtropy Tree Search (Section 2.4.5)

α_{MENTS} . . . . . The temperature parameter used in MENTS, which is the coefficient of the entropy terms in the maximum entropy objective.

ϵ_{MENTS} . . . . . An exploration parameter used in MENTS.

$\lambda(s, \epsilon)$ . . . . . A decaying function used to control and anneal exploration in MENTS. $\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e+N(s))}\right)$.

$\tilde{\pi}$ . . . . . A policy neural network, used to initialise soft Q-value estimates in MENTS, if available.

π_{MENTS} . . . . . The search policy used in MENTS.

$\hat{Q}_{\text{MENTS}}(s, a)$ . . A soft Q-value estimate at the chance node (s, a) in MENTS.

$\tilde{V}$ . . . . . A function approximation to V^*.

$\hat{V}_{\text{MENTS}}(s)$ . . . A soft value estimate at the decision node s in MENTS.

Multi-Objective Reinforcement Learning (Section 2.6)

CCS(Π) . . . . A convex coverage set of policies, which contains at least one policy that maximises the linear utility $u_{\text{lin}}(\cdot; \mathbf{w})$ for each weight vector $\mathbf{w}$.

CH(Π) . . . . . The undominated set of policies in Π (the set of all possible policies), with respect to the linear utility function u_{lin}. CH(Π) = $U(\Pi; u_{\text{lin}})$.

CS($\Pi; u$) . . . . A coverage set of policies, which contains at least one policy that maximises the utility $u(\cdot; \mathbf{w})$ for each weight vector $\mathbf{w}$.

`cvx_prune` . . . An operation that takes an arbitrary set of vectors, and returns the subset that lie at the vertices of the geometric (partial) convex hull.

Δ^D . . . . . The $(D-1)$-simplex (with D dimensions), or the set of possible weightings over the D objectives of an MOMDP. $\Delta^D = \{\mathbf{w} \in \mathbb{R}^D | w_i \geq 0, \sum_i w_i = 1\}$.

EUM(Ψ) . . . . The expected utility metric of a set of policies Ψ: EUM(Ψ) = $\mathbb{E}_{\mathbf{w} \sim U(\Delta^D)}[\max_{\psi \in \Psi} \mathbf{w}^\top \mathbf{V}^\psi(s_0)]$.

HV($\boldsymbol{\mathcal{V}}; \mathbf{r}_{\text{ref}}$) . . . The hypervolume indicator of a vector set $\boldsymbol{\mathcal{V}}$ with respect to a reference point $\mathbf{r}_{\text{ref}}$: the Lebesgue measure of the region dominated by $\boldsymbol{\mathcal{V}}$ and bounded below by $\mathbf{r}_{\text{ref}}$.

MCCS(Π) . . . The minimal convex coverage set of policies, which has no redundant policies and for each policy in MCCS(Π) there is some weight that it uniquely (with respect to MCCS(Π)) obtains the optimal linear utility.

PF(Π) . . . . . The Pareto front of the set of policies Π.

Π . . . . . The set of all possible policies in an MOMDP.

$\mathbf{Q}^\pi$ . . . . . The multi-objective Q-value function of a policy π. $\mathbf{Q}^\pi(s, a; t)$ denotes the expected cumulative vector reward that policy π will obtain, starting from state $s_t = s$, starting by taking action $a_t = a$.

$U(\Pi; u)$ . . . . . The undominated set of policies in Π (the set of all possible policies), with respect to the utility function u. Each policy in $U(\Pi; u)$ achieves a maximal utility for some weighting over the objectives.

u . . . . . A utility function, mapping multi-objective values to scalar values, that depends on a weighting over objectives. The multi-objective value $\mathbf{V}^\pi(s;t)$ is mapped to $u(\mathbf{V}^\pi(s;t);\mathbf{w})$, where $\mathbf{w}$ is a weighting over the objectives.

u_{cheby} . . . . . The (non-linear) Chebyshev utility function: $u_{\text{cheby}}(\mathbf{v};\mathbf{w},\mathbf{r}) = -\max_{i=1,\ldots,D} w_i|v_i - r_i|$, where $\mathbf{v},\mathbf{r} \in \mathbb{R}^D$ and $\mathbf{w} \in \Delta^D$.

u_{lin} . . . . . The linear utility function: $u_{\text{lin}}(\mathbf{v};\mathbf{w}) = \mathbf{w}^\top\mathbf{v}$.

$\mathbf{V}^\pi$ . . . . . The multi-objective value of a policy π. $\mathbf{V}^\pi(s;t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$.

$\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$ . . . . The set of multi-objective values obtained by a set of policies Π'.

$\mathbf{w}$ . . . . . A weighting over objectives that quantifies preferences over the multiple objectives, to be used in a utility function.

$\succ_P$ . . . . . The Pareto domination relation over vectors: $\mathbf{v} \succ_P \mathbf{v}'$ if and only if $v_i \geq v'_i$ for all i and $v_j > v'_j$ for some j.

Monte Carlo Tree Search with Boltzmann Exploration (Chapter 4)

$\texttt{sim_regr}(s,\psi)$. The simple regret of a recommendation policy ψ at state s: $\texttt{sim_regr}(s,\psi) = V^*(s) - V^\psi(s)$.

Boltzmann Tree Search (Section 4.2.1)

α_{BTS} . . . . . The temperature function used in BTS, $\alpha_{\text{BTS}}(x) > 0$. The temperature at a decision node s is $\alpha_{\text{BTS}}(N(s))$.

ϵ_{BTS} . . . . . An exploration parameter used in BTS, $\epsilon_{\text{BTS}} \in [0,\infty)$.

$\texttt{mv}_{\text{BTS}}$ . . . . . The most-visited recommendation policy of BTS: $\texttt{mv}_{\text{BTS}}(s) = \arg\max_{a\in\mathcal{A}} N(s,a)$.

$N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}, N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ Pseudo-visit weights used in BTS to bias the Q-value and value estimates towards the heuristic functions $\hat{Q}_{\text{heuristic}}$ and $\hat{V}_{\text{heuristic}}$.

π_{BTS} . . . . . The search policy used in BTS.

ψ_{BTS} . . . . . The Q-value based recommendation policy of BTS: $\psi_{\text{BTS}}(s) = \arg\max_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}(s,a)$.

$\hat{Q}_{\text{BTS}}(s,a)$ . . . The Bellman Q-value estimate at chance node (s,a) in BTS.

$\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a)$ . . . The optionally normalised and heuristic-biased Q-value estimate used by the BTS search policy to select actions.

$\hat{V}_{\text{BTS}}(s)$ . . . . . The Bellman value estimate at decision node s in BTS.

Decaying ENtropy Tree Search (Section 4.2.2)

α_{DNTS} . . . . . The temperature function used in DENTS, $\alpha_{\text{DNTS}}(x) > 0$.

β_{DNTS} . . . . . The entropy weighting function used in DENTS, $\beta_{\text{DNTS}}(x) \geq 0$. Typically chosen such that $\beta_{\text{DNTS}}(x) \to 0$.

ϵ_{DNTS} . . . . . An exploration parameter used in DENTS, $\epsilon_{\text{DNTS}} \in [0, \infty)$.

$\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a)$. The Monte Carlo entropy estimate at chance node (s,a) in DENTS.

$\bar{\mathcal{H}}_{V,\text{DNTS}}(s)$ . . . The Monte Carlo entropy estimate at decision node s in DENTS.

$\mathbf{mv}_{\text{DNTS}}$ . . . . . The most-visited recommendation policy of DENTS.

$N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}, N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$ Pseudo-visit weights used in DENTS, analogous to $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$ and $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$.

π_{DNTS} . . . . . The search policy used in DENTS.

ψ_{DNTS} . . . . . The Q-value based recommendation policy of DENTS.

$\hat{Q}_{\text{DNTS}}(s,a)$ . . The Bellman Q-value estimate at chance node (s,a) in DENTS.

$\hat{Q}_{\text{DNTS}}^{\text{sft}}(s,a)$ . . The entropy-augmented Q-value used in the DENTS search policy: $\hat{Q}_{\text{DNTS}}^{\text{sft}}(s,a) = \hat{Q}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a)$.

$\hat{Q}_{\text{DNTS}}^{\text{sft,norm}}(s,a)$ . . The optionally normalised and heuristic-biased soft Q-value used by the DENTS search policy to select actions.

$\hat{V}_{\text{DNTS}}(s)$ . . . . The Bellman value estimate at decision node s in DENTS.

Convex Hull Monte Carlo Tree Search (Chapter 5)

$\texttt{ctx_sim_regr}(s, \psi)$ The contextual simple regret of a recommendation policy ψ at state s: the expected scalarised loss relative to the optimal policy, with the weight vector drawn uniformly from Δ^D.

$\texttt{DST}(W, p_{\texttt{swept}})$. The randomly generated *Deep Sea Treasure* environment with width W and probability of being swept $p_{\texttt{swept}}$ used in Chapter 5.

$p_{\texttt{swept}}$ . . . . . The probability of the agent being swept by a current in the *Deep Sea Treasure* environment used in Chapter 5.

$\pi(a|s; \mathbf{w})$ . . . . A contextual policy, extending the policy notation $\pi(a|s)$ to include a context weight $\mathbf{w} \in \Delta^D$.

Contextual Zooming for Trees (Section 5.2)

$A(s)$ . . . . . The set of all active balls at decision node s in CZT: $A(s) = \cup_{a \in \mathcal{A}} A(s,a)$.

$A(s,a)$ . . . . . The set of active balls maintained at chance node (s,a) in CZT.

$M(B)$ . . . . . The number of times that the value estimate for ball B has been updated.

M_{min} . . . . . The minimum ball visit count required to trigger the CZT activation rule.

$\pi_{\texttt{CZT}}$ . . . . . The search policy used in CZT.

$\psi_{\texttt{CZT}}$ . . . . . The recommendation policy used in CZT.

$\bar{\mathbf{Q}}_{\texttt{CZT}}(B)$ . . . . The multi-objective average return value estimate associated with an active ball B in CZT.

$\texttt{relevant}(s;\mathbf{w})$ A subset of the active balls $A(s)$ at decision node s, containing the relevant balls for CZT to use for vector value estimates: $\texttt{relevant}(s;\mathbf{w}) = \cup_{a \in \mathcal{A}} \texttt{relevant}(s,a;\mathbf{w})$.

$\texttt{relevant}(s,a;\mathbf{w})$ The subset of active balls $A(s,a)$, consisting of the smallest radius balls that contain $(\mathbf{w},a)$.

CHMCTS Common Framework (Sections 5.3.1 and 5.3.6)

δ_{unsolved} . . . . . The confidence parameter used when computing `unsolved` values in CHMCTS, where smaller values give more pessimistic estimates.

K_{max} . . . . . The maximum size of a convex hull value set in CHMCTS, used in the size pruning optimisation.

$\pi_{\text{ALG}}(a|s;\mathbf{w})$ . . The search policy of a CHMCTS algorithm ALG.

$\psi_{\text{ALG}}(s;\mathbf{w})$ . . . The recommendation policy of a CHMCTS algorithm ALG: $\psi_{\text{ALG}}(s;\mathbf{w}) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}}_{\text{ALG}}(s,a)$.

$\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a)$ . . . The convex hull Q-value set estimate maintained at chance node (s,a) by a CHMCTS algorithm ALG.

$\hat{Q}^{\mathbf{w}}_{\text{ALG}}(s,a)$ . . . The contextual Q-value estimate at chance node (s,a) under weight $\mathbf{w}$: $\hat{Q}^{\mathbf{w}}_{\text{ALG}}(s,a) = \max_{\mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a)} \mathbf{w}^\top \mathbf{q}$.

$\texttt{unsolved}_{\text{tol}}$ . . The tolerance for marking a chance node as solved in CHMCTS, which happens when $\texttt{unsolved}(s,a) \leq \texttt{unsolved}_{\text{tol}}$.

$\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)$ . . . . The convex hull value set estimate maintained at decision node s by a CHMCTS algorithm ALG.

$\hat{V}^{\mathbf{w}}_{\text{ALG}}(s)$ . . . . . The contextual value estimate at decision node s under weight $\mathbf{w}$: $\hat{V}^{\mathbf{w}}_{\text{ALG}}(s) = \max_{\mathbf{v} \in \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)} \mathbf{w}^\top \mathbf{v}$.

CHMCTS Algorithms (Sections 5.3.2, 5.3.3, 5.3.4 and 5.3.5)

$\alpha_{\text{CH-BTS}}$ . . . . . The temperature function used in CH-BTS.

$b_{\texttt{CZT}}$ . . . . . The bias parameter used in CZT to control exploration.

$b_{\text{P-CHMCTS}}$ . . . The bias parameter used in P-CHMCTS to control exploration.

$c_{\text{CH-UCT}}$ . . . . . The bias parameter used in CH-UCT to control exploration.

$\epsilon_{\text{CH-BTS}}$ . . . . . An exploration parameter used in CH-BTS.

$\mathbf{r}_{\text{ref}}$ . . . . . The reference point used for the hypervolume indicator in H-CHMCTS.

$\mathbf{r}_{\text{uto}}$ . . . . . The utopian point used in the Chebyshev utility in C-CHMCTS.

Simplex Maps for Multi-Objective Monte Carlo Tree Search (Chapter 6)

$\boldsymbol{\mathcal{E}}_{\text{SM}}$ . . . . . The set of undirected edges of a simplex map, $\boldsymbol{\mathcal{E}}_{\text{SM}} \subseteq \boldsymbol{\mathcal{V}}_{\text{SM}} \times \boldsymbol{\mathcal{V}}_{\text{SM}}$.

$\mathbf{f}_{\text{SM}}$ . . . . . The function mapping the vertices of a simplex map to value estimates.

$\texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w})$ Returns the (approximately) closest vertex in $\boldsymbol{\mathcal{V}}_{\text{SM}}$ to the weight vector $\mathbf{w} \in \Delta^D$.

$\texttt{find_containing_simplex}(\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w})$ Returns the leaf simplex in $\boldsymbol{\mathcal{T}}_{\text{SM}}$ that contains the weight vector $\mathbf{w} \in \Delta^D$.

$\boldsymbol{\mathcal{G}}_{\text{SM}}$ . . . . . A simplex map, $\boldsymbol{\mathcal{G}}_{\text{SM}} = (\boldsymbol{\mathcal{V}}_{\text{SM}}, \boldsymbol{\mathcal{E}}_{\text{SM}}, \boldsymbol{\mathcal{T}}_{\text{SM}}, \mathbf{f}_{\text{SM}})$.

$\texttt{maybe_split}(\boldsymbol{\mathcal{G}}_{\text{SM}}, \boldsymbol{\mathcal{S}}_{\text{SM}})$ Performs a longest edge bisection of the leaf simplex $\boldsymbol{\mathcal{S}}_{\text{SM}} \in \boldsymbol{\mathcal{T}}_{\text{SM}}$ when split conditions are met.

$\texttt{push}(\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w}_{\text{SM}})$ Propagates the value estimate at $\mathbf{w}_{\text{SM}}$ to neighbouring vertices via breadth-first search.

$\texttt{push_radius}_{\text{SM}}$ The maximum push radius used by the `push` operation.

$\texttt{split_min_delta}_{\text{SM}}$ The minimum value-estimate difference within a simplex required to trigger a split.

$\texttt{split_min_radius}_{\text{SM}}$ The minimum radius below which a simplex is not split.

$\texttt{split_min_visit}_{\text{SM}}$ The minimum number of split calls on a simplex before it is split.

$\boldsymbol{\mathcal{T}}_{\text{SM}}$ . . . . . The binary tree of simplices of a simplex map, whose leaf nodes form a triangulation of the unit simplex Δ^D.

$\boldsymbol{\mathcal{V}}_{\text{SM}}$ . . . . . The set of vertices of a simplex map, $\boldsymbol{\mathcal{V}}_{\text{SM}} \subseteq \Delta^D$.

$\mathbf{w}_{\text{SM}}$ . . . . . A vertex of a simplex map, $\mathbf{w}_{\text{SM}} \in \boldsymbol{\mathcal{V}}_{\text{SM}}$.

DENTS with Simplex Maps (Section 6.3)

$\alpha_{\text{SM-D}}$ . . . . . The temperature function used in SM-DENTS.

$\beta_{\text{SM-D}}$ . . . . . The entropy weighting function used in SM-DENTS.

$\epsilon_{\text{SM-D}}$ . . . . . An exploration parameter used in SM-DENTS.

$\bar{\mathcal{H}}_{Q,\text{SM-D}}(s, a; \cdot)$. The entropy value estimate function maintained in the simplex map at chance node (s, a).

$\bar{\mathcal{H}}_{V,\text{SM-D}}(s; \cdot)$ . . The entropy value estimate function maintained in the simplex map at decision node s.

$\pi_{\text{SM-D}}$ . . . . . The search policy used in SM-DENTS.

$\psi_{\text{SM-D}}$ . . . . . The recommendation policy used in SM-DENTS: $\psi_{\text{SM-D}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}}_{\text{SM-D}}(s, a)$.

$\hat{\mathbf{Q}}_{\text{SM-D}}(s, a; \cdot)$ . . The multi-objective Q-value estimate function maintained in the simplex map at chance node (s, a).

$\hat{Q}^{\mathbf{w}}_{\text{SM-D}}(s, a)$ . . . The contextual Q-value estimate of SM-DENTS at chance node (s, a).

$\hat{\mathbf{V}}_{\text{SM-D}}(s; \cdot)$ . . . The multi-objective value estimate function maintained in the simplex map at decision node s.

$\hat{V}^{\mathbf{w}}_{\text{SM-D}}(s)$ . . . . The contextual value estimate of SM-DENTS at decision node s.

1 Introduction

Contents

1.1 Overview and Motivation **2**
1.2 Scope of the Thesis **4**
1.3 Contributions **6**
1.4 Structure of Thesis **8**
1.5 Publications **9**

Sequential decision-making problems arise whenever an agent must choose a sequence of actions to achieve some goal, where each action affects the environment, and hence the decisions that follow. In games such as chess and Go, the agent's actions are moves on a board, and the goal is to win the game. In robotics, the agent's actions are motor commands, and the goal may be to reach a target location without colliding with obstacles. In drug treatment, the agent's actions are doses to administer, and the goal is to maximise the effectiveness of the treatment whilst minimising side effects.

These problems are commonly modelled as Markov Decision Processes (MDPs), where an agent receives a reward signal at each timestep, and the goal is to maximise the cumulative reward over time. Reinforcement learning and planning algorithms have produced state-of-the-art results in such problems, and Monte Carlo Tree Search

(MCTS) is one of the most successful planning frameworks, famously underpinning the AlphaGo and AlphaZero programs that defeated the world's best Go players.

Many real-world problems involve more than one objective. The drug treatment problem mentioned above is multi-objective, with a trade-off between effectiveness and side effects. In robotics, completing a task quickly may be desirable, but moving faster can increase the risk of damage to the robot or its surroundings. Multi-objective Markov Decision Processes (MOMDPs) extend the MDP framework with a vector-valued reward function, and Multi-Objective Reinforcement Learning (MORL) extends reinforcement learning algorithms to operate in MOMDPs. Preferences over the objectives are commonly captured by a vector of weights, one for each objective, together with a utility function. The utility function maps the weights and the vector-valued return to a scalar utility value. When the preferences are not known at the time of planning, the agent must instead return a set of policies that cover the relevant trade-offs, from which one is selected once the preferences become known.

This thesis is concerned with MCTS algorithms for sequential decision-making problems, in both the single- and multi-objective settings. The single-objective work in this thesis is motivated in part by the multi-objective work, and develops MCTS algorithms that are subsequently extended to the multi-objective setting. The remainder of this chapter motivates the work, sets out its scope, states the research questions and contributions, describes the structure of the thesis, and lists the publications arising from it.

1.1 Overview and Motivation

MCTS algorithms are sample-based planning algorithms that often use a simulated environment. The simulator is generally inexpensive to query, and a large number of trials can be run in the simulator before the agent must commit to actions in the real environment. The setting where the agent is evaluated only on the recommendations it makes after planning is referred to in this thesis as the *exploration setting* of reinforcement learning.

Because poor actions in a simulator incur little or no real-world cost, MCTS algorithms in the exploration setting can afford to explore more freely. Despite this, the most widely used MCTS algorithm is the Upper Confidence Bound applied to Trees (UCT) algorithm, which uses a search policy designed for a different problem: minimising the cumulative regret of a multi-armed bandit. In effect, UCT is designed for a setting where actions taken during planning do incur a real-world cost, and tends to select exploiting actions even when further exploration would be more valuable.

Maximum ENtropy Tree Search (MENTS) is a more recent MCTS algorithm that addresses this limitation by using an entropy regularised objective to encourage exploration. However, the maximum entropy objective is not guaranteed to align with the standard objective of maximising expected return, and the temperature parameter that governs the trade-off between entropy and expected return is difficult to tune: it must be large enough to benefit exploration, but small enough to avoid misalignment. A poor choice can result in MENTS recommending suboptimal actions. The first half of this thesis investigates how MCTS algorithms can prioritise exploration whilst still recommending optimal actions.

The second half of this thesis is concerned with MCTS in the multi-objective setting. Prior Multi-Objective Monte Carlo Tree Search (MOMCTS) algorithms have two limitations: they are restricted to deterministic environments, and they do not ensure that actions within a single trial are consistent with each other — without such a mechanism, the search policy ends up optimising for no particular objective. To address these limitations, this thesis introduces two MOMCTS frameworks for stochastic environments. It is also argued that trials in MOMCTS should be *contextual*: a single weight vector is sampled at the start of each trial and used to guide every action selection within it, so that the actions within a trial remain consistent.

1.2 Scope of the Thesis

This section sets out the scope and setting of the work in this thesis. Generally, the single-objective work (Chapter 4) considers the exploration setting described in Section 1.1, and the multi-objective work (Chapters 5 and 6) considers the unknown weights scenario described below.

Simulation. Throughout this thesis, the simulator is assumed to be an accurate model of the environment; model error, and how to compensate for it, are beyond the scope of this work.

Planning and Execution. In the *planning phase*, the agent has access to a simulator of the environment and uses it to decide how to act. In the *execution phase*, the agent acts in the environment itself. Running MCTS in the planning phase is called *offline planning*: it is given a fixed compute budget before the agent must act. Running MCTS in the execution phase is called *online planning*: it uses whatever time is available before each real-world decision to plan further from the current state using the simulator. Single-objective MCTS can be used in either phase, or both.

Planning from a Known Initial State. The methods in this thesis assume that the agent always begins in a known, deterministic initial state. A distribution over initial states could be handled by adding a pseudo-transition at the root of the search that samples the initial state, but this is not considered here: it amounts to several independent searches, unless transpositions (discussed in Section 2.4.2) are used to share nodes between them.

The Unknown Weights Scenario. The multi-objective reinforcement learning literature establishes several scenarios, distinguished by what is known about preferences and when. This thesis considers the *unknown weights scenario* [82, 43]. Preferences over the objectives are specified by a weight vector $\mathbf{w}$, and a utility

function maps a weight vector and a vector-valued return to a scalar utility. The weight vector is unknown during the planning phase, so the agent must plan for all possible preferences. It is revealed during the *weight selection phase*[1], after which the agent aims to maximise the utility for that weight vector during the execution phase. This setting arises whenever preferences are unavailable during planning, or vary between uses of a system.

The Linear Utility Assumption. The utility function is assumed to be linear in the weight vector throughout this thesis. Section 5.5.2 discusses why this assumption is made and how the work could be adapted to non-linear utilities. Section 5.5.3 shows that MOMCTS algorithms that choose actions using the expected reward-to-go at each node yield a utility-optimal policy if and only if the utility is linear.

Optimisation Criterion. Two criteria are distinguished in the literature: the *scalarised expected returns* (SER) criterion applies the utility to the expected return, whereas the *expected scalarised returns* (ESR) criterion takes the expected utility of the return. SER is appropriate when a policy will be executed many times and average performance matters most, so some poor individual executions can be tolerated. ESR is more appropriate when a policy is executed only once; for example, a patient may have only one opportunity to follow a treatment plan [43, 82]. This thesis uses the more widely studied SER criterion, but under the linear utility assumed here the two coincide.

Evaluation. The algorithms in this thesis are evaluated as offline planners, although the single-objective algorithms of Chapter 4 can equally be used for online planning. In the multi-objective unknown weights scenario, each action yields a vector of rewards, denoted $\mathbf{R}(s, a)$, with one component per objective. Once the weight vector $\mathbf{w}$ is revealed, the problem reduces to a single-objective one with scalarised reward $R(s, a; \mathbf{w}) = \mathbf{w}^\top \mathbf{R}(s, a)$, so any further online planning can be carried out by a single-objective MCTS algorithm. Hence, in this scenario, it

makes little sense to use an MOMCTS algorithm for online planning. It follows that the quality of the solution computed during planning matters most at the initial state, where execution begins.

The Role of Planning. An offline planning budget could instead be spent learning a policy or value function that generalises across states, using function approximators such as neural networks. Why plan at all? MCTS algorithms can be shown to be consistent: given enough trials, they recommend an optimal action. On small environments a feasible budget may therefore be enough to recover an optimal decision, whereas a learned approximation carries no such guarantee. On larger environments the two approaches can be complementary: AlphaZero [94] uses a learned prior policy and value function to guide its search, and the results of that search to improve them in turn. The search methods developed here provide the foundation for such a combination, which is proposed as future work in Section 7.2. Planning also has an incidental benefit not pursued here: a search tree can be inspected once planning is complete, which may aid interpretability.

1.3 Contributions

The following questions, related to Monte Carlo Tree Search and multi-objective reinforcement learning, are considered throughout this thesis:

Q1 - Exploration: When planning in a simulator with limited time, how can Monte Carlo Tree Search algorithms prioritise exploration to make effective decisions?

Q1.1 - Entropy: Entropy is commonly added to the objective in reinforcement learning to encourage exploration — can this be done without compromising the optimality of the recommended actions?

[1] Hayes et al. [43] call this the selection phase. The word *weight* is added to avoid confusion with the selection phase of an MCTS trial (Section 2.4.2).

Q1.2 - Multi-Objective Exploration: Can the exploration insights from (maximum-entropy) single-objective Monte Carlo Tree Search algorithms be applied to multi-objective domains?

Q2 - Stochastic Multi-Objective Domains: Multi-objective Monte Carlo Tree Search has been studied in deterministic domains — can these methods be extended to stochastic settings?

Q3 - Scalability: How can the scalability of Monte Carlo Tree Search methods be improved, in both single- and multi-objective settings?

Q3.1 - Complexity: Monte Carlo Tree Search algorithms typically run linearly in the number of trials, actions and decision horizon — can this computational complexity be reduced?

Q3.2 - Multi-Objective Scalability: With respect to the size of environments, how scalable are multi-objective Monte Carlo Tree Search methods?

These questions are addressed by the following contributions:

- Chapter 4 investigates the use of entropy-based exploration in MCTS, addressing **Q1.1 - Entropy**. The maximum entropy objective is shown to be misaligned with the standard objective of maximising expected return: for any fixed temperature there is an MDP in which MENTS either fails to recommend the optimal policy, or requires an exponential number of trials to do so (Theorem 4.3.1). Two MCTS algorithms, *Boltzmann Tree Search* (BTS) and *Decaying ENtropy Tree Search* (DENTS), are introduced to address this misalignment, and are shown to be consistent under mild conditions. The stochastic search policies of BTS and DENTS also enable the use of the Alias Method to address **Q3.1 - Complexity**.

- Chapter 5 addresses **Q2 - Stochastic Multi-Objective Domains** by introducing *Convex Hull MCTS* (CHMCTS), which generalises prior MOMCTS work to stochastic environments by maintaining convex hull value sets at each

node, and updating them via convex hull backups. *Contextual Zooming for Trees* (CZT) is also introduced as a contextual analogue of UCT, in which the contextual zooming algorithm is applied at each decision node.

- Chapter 6 addresses **Q1.2 - Multi-Objective Exploration** and **Q3.2 - Multi-Objective Scalability** by introducing the *simplex map*, an adaptive data structure that maintains value and entropy estimates as a function of the weight vector over the unit simplex. Simplex maps address two limitations of the convex hull approach: the computational cost of convex hull backups in stochastic environments, and the inability to naturally support entropy-based exploration. The simplex map is used to define SM-BTS and SM-DENTS, which extend BTS and DENTS to the multi-objective setting.

1.4 Structure of Thesis

The remainder of this thesis is organised as follows. Chapter 2 introduces the background material used throughout the thesis. This includes the multi-armed bandit problem, Markov Decision Processes, reinforcement learning, the Monte Carlo Tree Search framework, and the multi-objective reinforcement learning setting. The `THTS++` schema, used to define MCTS algorithms modularly throughout this thesis, is introduced in Section 2.4.2.

Chapter 3 discusses related work in bandits, maximum entropy reinforcement learning, MCTS, and multi-objective sequential decision-making. The discussion is brief and focused on the references most relevant to the contributions of this thesis.

Chapter 4 considers MCTS algorithms for single-objective environments, with the optional use of entropy as a second objective for exploration. The maximum entropy objective is shown to not always align with the standard objective of maximising expected return, and BTS and DENTS are introduced as alternative MCTS algorithms. The stochastic search policies of BTS and DENTS also enable the use of the Alias Method, which significantly reduces the per-trial computational cost.

Chapter 5 extends the MCTS framework to the multi-objective unknown weights scenario with linear utilities, where the weight vector is unknown at the time of planning. CZT and CHMCTS are introduced as two approaches to contextual planning in MOMDPs. CHMCTS recovers prior MOMCTS algorithms as special cases of a common framework, and generalises them to stochastic environments.

Chapter 6 introduces simplex maps, an adaptive data structure for maintaining value and entropy estimates as a function of the weight vector. Simplex maps are used to define SM-BTS and SM-DENTS, which extend BTS and DENTS to the multi-objective setting, and are shown to scale better than CHMCTS in stochastic environments.

Chapter 7 summarises the contributions of this thesis and discusses possible directions for future work.

1.5 Publications

The work covered in this thesis also appears in the following publications:

- Painter, M; Lacerda, B; and Hawes, N. "Convex Hull Monte-Carlo Tree-Search." In *Proceedings of the international conference on automated planning and scheduling. Vol. 30. 2020*, ICAPS, 2020.

- Painter, M; Baioumy, M; Hawes, N; and Lacerda, B. "Monte Carlo Tree Search with Boltzmann Exploration." In *Advances in Neural Information Processing Systems, 36, 2023*, NeurIPS, 2023.

2
Background

Contents

2.1 Multi-Armed Bandits 12
2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm . . 13
2.1.2 Exploring Bandits 14
2.1.3 Contextual Bandits 16
2.2 Markov Decision Processes 17
2.3 Planning and Reinforcement Learning 20
2.3.1 Maximum Entropy Planning and Reinforcement Learning 24
2.3.2 Planning Versus Reinforcement Learning 26
2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search 27
2.4.1 Notation 29
2.4.2 Trial Based Heuristic Tree Search 31
2.4.3 Upper Confidence Bound Applied to Trees (UCT) 36
2.4.4 Predictor-UCT (PUCT) 39
2.4.5 Maximum Entropy Tree Search 40
2.5 Convergence of Random Variables 41
2.6 Multi-Objective Planning and Reinforcement Learning 43
2.6.1 Convex Hull Value Iteration 51
2.6.2 Implications of Assuming a Linear Utility 52

This chapter introduces the fundamental concepts which are used throughout this thesis, in particular Section 2.4.2 introduces the `THTS++` schema, which will be used throughout this thesis to define Monte Carlo Tree Search algorithms modularly.

Section 2.1 briefly covers Multi-Armed Bandit problems, which are commonly

used to study decision-making algorithms and as a building block in Monte Carlo Tree Search methods. In particular, *Contextual* Multi-Armed Bandit problems (Section 2.1.3) are covered, which will be relevant in the design and analysis of the multi-objective algorithms discussed in Chapters 5 and 6.

Sections 2.2 and 2.3 outline the sequential decision-making framework used in this thesis. Additionally, the Maximum Entropy Reinforcement Learning framework is covered (Section 2.3.1) which is a major topic discussed in Chapter 4.

Section 2.4 covers the Monte Carlo Tree Search framework which will be used for solving sequential decision-making problems.

Finally, Section 2.6 covers the Multi-Objective Reinforcement Learning framework, which is used to study sequential decision-making algorithms with multiple objectives.

2.1 Multi-Armed Bandits

The Multi-Armed Bandit (MAB) problem [54] is a foundational problem considered in decision theory. In the K-armed bandit problem, an agent must choose which of the K arms to pull over multiple rounds. On each round, after pulling an arm, the agent receives a reward which is sampled from a probability distribution associated with the arm.

The MAB problem was first considered by Robbins [81] in 1952, in the context of clinical trials. In a clinical trial, each patient is treated with one of two possible treatments, where the effectiveness of each treatment is unknown. In this setting, it is essential that the best treatment is given to as many patients as possible, while each treatment needs to be considered to determine its effectiveness. The balance between *exploring* to find the best treatment (arm) and *exploiting* the best treatment (arm) is known as the *exploration-exploitation trade-off*.

The remainder of this section will formalise the MAB problem, give details of the widely used Upper Confidence Bound (UCB) algorithm [4], and consider two variations on the MAB problem (Sections 2.1.2 and 2.1.3) that are relevant in this thesis.

Parameters: K probability distributions for the rewards of each arm $f(1), \ldots, f(K)$.

- For each round $m = 1, 2, \ldots$:
  - the agent selects an arm $x^m \sim \sigma^m$ to pull;
  - the agent receives a reward $y^m \sim f(x^m)$ from the environment;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.1: The procedure of a multi-armed bandit problem, following a strategy σ.

2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm

Figure 2.1 shows how the operation of a K-armed bandit problem proceeds. Let $f(1), \ldots, f(K)$ be the probability distributions for the rewards of each of the K arms, with expected values of $\mu(1), \ldots, \mu(K)$ respectively. On the mth round, if the arm $x^m \in \{1, \ldots, K\}$ is pulled, then a reward $y^m \sim f(x^m)$ is received. To specify a *strategy* σ, on each round m, a probability distribution σ^m over $\{1, \ldots, K\}$ is given, which is sampled to decide which arm to pull, i.e. $x^m \sim \sigma^m$.

Regret is a common metric used to analyse the performance of algorithms for MAB problems, which is the difference between the cumulative reward obtained and the maximal expected reward that could be obtained with full knowledge of $\{f(i)\}$.

Definition 2.1.1. *The* (cumulative) regret *of strategy σ after n rounds in the MAB process is:*

$$\texttt{cum_regr}_{\text{MAB}}(n, \sigma) = n\mu^* - \sum_{m=1}^{n} y^m, \tag{2.1}$$

where $\mu^* = \max_i \mu(i)$.

The expected regret, $\mathbb{E}[\texttt{cum_regr}_{\text{MAB}}(n, \sigma)]$, is often used to analyse the performance of algorithms for MAB problems. Lai and Robbins [53] show a general lower bound of $\Omega(\log n)$ on the expected regret that an agent can achieve. Auer [4] introduces the Upper Confidence Bound (UCB) algorithm, which achieves a matching upper bound of $O(\log n)$ on the expected regret.

To define the UCB strategy, let $N^m(x)$ be the number of times that arm x has been pulled after m rounds, and let $\bar{y}^m(x)$ be the average reward obtained by pulling arm x after m rounds:

$$N^m(x) = \sum_{i=1}^{m} \mathbb{1}[x^i = x], \tag{2.2}$$

$$\bar{y}^m(x) = \frac{1}{N^m(x)} \sum_{i=1}^{m} y^i \mathbb{1}[x^i = x]. \tag{2.3}$$

The strategy followed by the UCB algorithm on the mth round is given by:

$$x^m_{\text{UCB}} = \arg\max_{i \in \{1,..,K\}} \bar{y}^{m-1}(i) + b_{\text{UCB}} \sqrt{\frac{\log(m)}{N^{m-1}(i)}}, \tag{2.4}$$

$$\sigma^m_{\text{UCB}}(x) = \mathbb{1}[x = x^m_{\text{UCB}}], \tag{2.5}$$

where b_{UCB} is a bias parameter used to control how much the strategy explores. In the first K rounds, UCB pulls each arm once to obtain an initial sample for each arm. Alternatively, the division by zero in Equation 2.4 can be considered to give a value of ∞, to give the same effect.

2.1.2 Exploring Bandits

In the pure exploration problem for K-armed bandits [17], the format of each round is changed slightly. The agent still pulls an arm each round and receives a reward from the environment, but after, the agent is given an opportunity to recommend which arms are the best. This better reflects scenarios where time is spent planning before any action is taken in the real world. Because the agent is only assessed on the quality of the recommendations, the agent is free to purely explore using its arm pulls and only needs to exploit when providing a recommendation.

Figure 2.2 gives an overview of the Exploring Multi-Armed Bandit (EMAB) problem. The agent follows an *exploration strategy* σ to pull an arm each round, and receives a reward from the environment. Then the agent provides a recommendation in the form of a probability distribution, or *recommendation strategy* ψ, over the K arms.

Algorithms for EMAB problems can be analysed by considering the expected regret of following the recommendation policy for one round, known as *simple regret*.

Parameters: K probability distributions for the rewards of each arm $f(1), \ldots, f(K)$.

- For each round $m = 1, 2, \ldots$:
  - the agent selects an arm $x^m \sim \sigma^m$ to pull;
  - the agent receives a reward $y^m \sim f(x^m)$ from the environment;
  - the agent outputs a recommendation distribution ψ^m, over the arms $\{1, \ldots, K\}$;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.2: The procedure of an exploring multi-armed bandit problem, following an exploration strategy σ, and recommendation strategy ψ.

Definition 2.1.2. *The* simple regret *of the recommendation strategy ψ is:*

$$\texttt{sim_regr}_{\text{EMAB}}(\psi) = \mathbb{E}_{i\sim\psi}[\mu^* - \mu(i)]. \tag{2.6}$$

Bubeck et al. [17] analyse a range of exploration and recommendation strategies in their work. Two of the recommendation strategies considered are the *empirical best arm* (EBA) and *most played arm* (MPA):

$$x^m_{\text{EBA}} = \arg\max_{i\in\{1,..,K\}} \bar{y}^m(i), \tag{2.7}$$

$$\psi^m_{\text{EBA}}(x) = \mathbb{1}[x = x^m_{\text{EBA}}], \tag{2.8}$$

$$x^m_{\text{MPA}} = \arg\max_{i\in\{1,..,K\}} N^m(i), \tag{2.9}$$

$$\psi^m_{\text{MPA}}(x) = \mathbb{1}[x = x^m_{\text{MPA}}]. \tag{2.10}$$

In addition to the UCB exploration strategy (Equation 2.5), a *uniform exploration strategy* is considered:

$$x^m_{\text{uniform}} = (m \mod K) + 1, \tag{2.11}$$

$$\sigma^m_{\text{uniform}}(x) = \mathbb{1}[x = x^m_{\text{uniform}}]. \tag{2.12}$$

It is shown that exploring with UCB leads to a polynomial upper bound on the simple regret for both recommendation strategies. That is, when ψ^m_{EBA}

and ψ^m_{MPA} are computed using visit counts and sample averages from following σ_{UCB}, then $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{EBA}}) = O(m^{-k_{\text{UCB,EBA}}})$ and $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{MPA}}) = O(m^{-k_{\text{UCB,MPA}}})$ for some appropriate constants $k_{\text{UCB,EBA}}, k_{\text{UCB,MPA}} > 0$.

When the uniform exploration strategy is used instead, and ψ^m_{EBA} is computed similarly using σ_{uniform}, then $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{EBA}}) = O(e^{-k_{\text{uniform}}m})$ for an appropriate constant $k_{\text{uniform}} > 0$.

Simple regret quantifies how suboptimal the recommendation strategy is in expectation, so a simple regret of zero corresponds to recommending only optimal arms. Bounds that decay to zero more quickly in m are better: the exponential bound for uniform exploration is stronger than the polynomial bound obtained under UCB exploration.

2.1.3 Contextual Bandits

The Contextual Multi-Armed Bandit (CMAB) problem [50, 59, 95] is another extension of the MAB problem, where the agent is provided with a context $w \in \mathcal{W}$ at the start of each round. After the arm $x \in \mathcal{X}$ is selected, the reward is sampled from the distribution $f(w, x)$, with mean $\mu(w, x)$, which also depends on the context. For each round m, a strategy maps contexts to arms via $\sigma^m(w)$. Figure 2.3 gives an overview of the CMAB problem.

It is assumed that the *similarity space* $\mathcal{W} \times \mathcal{X}$ is *compact*, and the mean reward function μ is Lipschitz continuous. This requires a *metric* d over the similarity space.

Definition 2.1.3. *d is a* metric *over the similarity space $\mathcal{W} \times \mathcal{X}$ if it satisfies the following conditions for any $z, z', z'' \in \mathcal{W} \times \mathcal{X}$ with $z \neq z' \neq z''$:*

$$d(z, z) = 0, \tag{2.13}$$

$$d(z, z') > 0, \tag{2.14}$$

$$d(z, z') = d(z', z), \tag{2.15}$$

$$d(z, z'') \leq d(z, z') + d(z', z''). \tag{2.16}$$

Parameters: the set of arms $\mathcal{X}$, the set of contexts $\mathcal{W}$ and a mapping from $\mathcal{W} \times \mathcal{X}$ to probability distributions: $f(w, x)$.

- For each round $m = 1, 2, \ldots$:
  - the agent receives a context $w^m \in \mathcal{W}$ from the environment;
  - the agent selects an arm $x^m \sim \sigma^m(w^m)$ to pull;
  - the agent receives a reward $y^m \sim f(w^m, x^m)$ from the environment;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.3: The procedure of a contextual multi-armed bandit problem, following a strategy σ.

μ is then Lipschitz continuous if the following condition holds:

$$|\mu(w, x) - \mu(w, x')| \leq d((w, x), (w, x')). \tag{2.17}$$

The performance of CMAB algorithms is analysed with *contextual regret*, a notion of regret that accounts for the contexts drawn.

Definition 2.1.4. *The* (cumulative) contextual regret *of the strategy σ over n rounds in the process given in Figure 2.3 is defined as:*

$$\texttt{ctx_regr}_{\text{CMAB}}(n, \sigma) = \sum_{m=1}^{n} \mu^*(w^m) - y^m, \tag{2.18}$$

where $\mu^*(w) = \sup_i \mu(w, i)$.

The *contextual zooming* algorithm [50] is an algorithm for CMAB problems which is presented directly in the context of tree search in the *Contextual Zooming for Trees* algorithm (Section 5.2).

2.2 Markov Decision Processes

Markov decision processes (MDPs) provide a mathematical description of sequential decision-making problems [66]. This section provides definitions and notation related to MDPs used throughout this thesis.

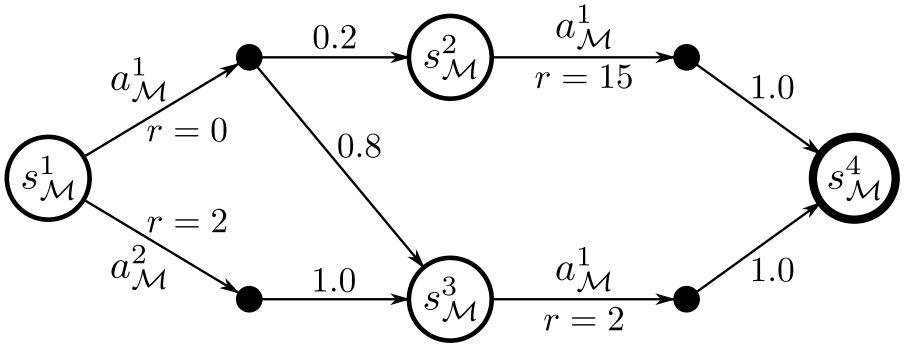


Figure 2.4: An example MDP $\mathcal{M}$, where $s_0 = s^1_{\mathcal{M}}$, the finite horizon is $H = 2$, and terminal states are marked with a thicker border. Edges are appropriately marked with actions and rewards, or marked with transition probabilities.

MDPs are used throughout this thesis to describe the environments in which agents act. An MDP contains, among other things, a set of states and actions. States are sampled according to a transition distribution that depends only on the current state and current action being taken (the Markov assumption). Whenever an action is taken, the agent receives a reward according to a reward function that depends on the current state and action.

In this thesis, unless otherwise stated, MDPs are assumed to have finite state and action spaces, and *trajectories* (sequences of states, actions and rewards) are of finite length.

Definition 2.2.1. *A* (Finite-Horizon) Markov decision process *(MDP) is a tuple* $\mathcal{M} = (\mathcal{S}, s_0, \mathcal{A}, R, p, H)$*, where* $\mathcal{S}$ *is a set of states,* $s_0 \in \mathcal{S}$ *is an initial state,* $\mathcal{A}$ *is a set of actions,* $R(s, a)$ *is the reward function* $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$*,* $p(s'|s, a)$ *is a next-state transition distribution* $\mathcal{S} \times \mathcal{A} \to (\mathcal{S} \to [0, 1])$ *and* $H \in \mathbb{N}$ *is the time horizon.*

An example MDP is shown in Figure 2.4. The set of *successor states* is the set of states that can be reached after taking an action from the current state of the MDP:

Definition 2.2.2. *The set of* successor states $\text{Succ}(s, a)$ *of a state-action pair* (s, a)*, with respect to an MDP, is defined as:*

$$\text{Succ}(s, a) := \{s' \in \mathcal{S} | p(s'|s, a) > 0\}. \tag{2.19}$$

The set of reachable states from the starting state s_0 can then be defined as the smallest set closed under the successor relation:

Definition 2.2.3. *The set of* reachable states $\mathcal{S}_0$ *from the starting state* s_0 *is the smallest set satisfying:*

$$s_0 \in \mathcal{S}_0, \qquad s \in \mathcal{S}_0 \Rightarrow \bigcup_{a \in \mathcal{A}} \text{Succ}(s, a) \subseteq \mathcal{S}_0. \tag{2.20}$$

In this thesis, MDPs may contain *trap states*, which are states that, once reached, can never be left and from which no further reward is obtained.

Definition 2.2.4. *A state* $s \in \mathcal{S}$ *is a* terminal *state (equivalently, a* trap *state), if the transition distribution always returns the same state (i.e.* $p(s|s, a) = 1$ *for all actions* a*), and if no more reward is obtained from that state (*$R(s, a) = 0$ *for all* a*). The set of terminal states is denoted* $\texttt{terminal} \subseteq \mathcal{S}$.

A *policy* maps each state and timestep to a probability distribution over the action space, and defines a strategy that an agent will follow in an MDP. Because the MDPs considered in this thesis have a finite horizon, the optimal behaviour at a state generally depends on how many timesteps remain, and so policies are defined to condition on the timestep. Often it is desirable to define deterministic policies, which always produce the same action when given the same state and timestep, and can be represented as *one-hot* distributions.

Definition 2.2.5. *A* (stochastic) policy $\pi : \mathcal{S} \times \{0, \dots, H-1\} \to (\mathcal{A} \to [0, 1])$ *is a mapping from states and timesteps to a probability distribution over actions.* $\pi(a|s; t)$ *is the probability of sampling action* a *at state* s *at timestep* t*. The policy* π *must satisfy:* $\sum_{a \in \mathcal{A}} \pi(a|s; t) = 1$ *and* $\pi(a|s; t) \geq 0$ *for all* $s \in \mathcal{S}$, $a \in \mathcal{A}$ *and* $t \in \{0, \dots, H-1\}$.

Additionally, a deterministic policy *is defined as a one-hot policy, that is, the policy* π *is deterministic if, for every state* s *and timestep* t*, it can be written as* $\pi(a|s; t) = \mathbb{1}[a = a']$ *for some* $a' \in \mathcal{A}$.

Moreover, the following notations are used for policies:

- $a \sim \pi(\cdot|s; t)$ *denotes sampling an action* a *from the distribution* $\pi(\cdot|s; t)$*;*
- $\pi(s; t) = a'$ *is shorthand to define the deterministic policy* $\pi(a|s; t) = \mathbb{1}[a = a']$*;*

- *$\pi(s;t)$ denotes the action a' of the deterministic policy $\pi(a|s;t) = \mathbb{1}[a = a']$.*

Given an MDP $\mathcal{M}$ and a policy π, a *trajectory* is a sampled sequence of states, actions and rewards, where the policy is used to sample actions and the MDP is used to sample the next state. A trajectory simulates one possible sequence that could occur if an agent follows policy π in $\mathcal{M}$.

Definition 2.2.6. *A* trajectory *τ is a sequence of states, actions and rewards that is induced by a policy π and MDP $\mathcal{M}$. Let the trajectory be $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{H-1}, a_{H-1}, r_{H-1}, s_H)$, where $a_t \sim \pi(\cdot|s_t;t)$, $r_t = R(s_t, a_t)$ and $s_{t+1} \sim p(\cdot|s_t, a_t)$.*

The following notation is also used for trajectories:

- *$\tau \sim \pi$ denotes a trajectory that is sampled using the policy π, where the MDP $\mathcal{M}$ is implicit;*
- *$\tau_{i:j}$ denotes a* truncated trajectory *$\tau_{i:j} := (s_i, a_i, r_i, s_{i+1}, \ldots, s_{j-1}, a_{j-1}, r_{j-1}, s_j)$, between the timesteps $0 \leq i < j \leq H$ inclusive;*
- *$\tau_{:j} := \tau_{0:j}$ denotes a trajectory that is truncated on only one end;*
- *$s \in \tau$ is used as a shorthand for $s \in \{s_i | i = 0, \ldots, H\}$;*
- *$(s, a) \in \tau$ is used as a shorthand for $(s, a) \in \{(s_i, a_i) | i = 0, \ldots, H-1\}$;*
- *and finally, when states, actions or rewards are timestep-indexed (e.g. s_t, a_t, r_t), they always correspond to a trajectory.*

2.3 Planning and Reinforcement Learning

This section provides a brief introduction to the fundamental concepts and definitions of planning and reinforcement learning.

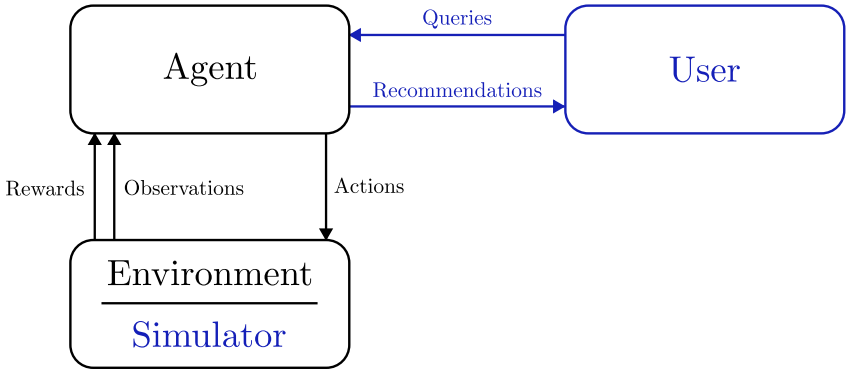


Figure 2.5: An overview of reinforcement learning. Depicted in black is the standard agent-environment interface for reinforcement learning [98], where an agent can perform actions in an environment and is given feedback in the form of observations and rewards. In blue the environment is replaced by a simulated environment which the agent can use for planning, and is queried by a user for recommendations on how to act in the real environment.

It begins by motivating the alternative lens on reinforcement learning taken in this thesis, in which the agent plans in a simulator before recommending how to act. Following on from Section 2.2, it then defines *value functions* and the objectives of planning and reinforcement learning, and covers *Value Iteration*, a tabular dynamic programming approach for planning. Section 2.3.1 covers *maximum entropy* planning and reinforcement learning, and Section 2.3.2 discusses what distinguishes planning from reinforcement learning, and where the methods of this thesis sit.

Typically reinforcement learning is presented as a sequence of agent-environment interactions, where the agent takes actions in an environment and receives *observations* and *rewards* for those actions [98]. The objective of the agent is to maximise the rewards that it receives.

However, in the modern era of reinforcement learning, the agent is often equipped with a simulator in which it can plan, learn and explore. The "target" environment that the user cares about may be the same simulated environment, or a real-world environment where actions carry genuine costs, unlike interactions with the simulator. After the learning phase, one typically cares only about the policy that the agent recommends at the end, rather than the rewards accrued during planning.

Figure 2.5 depicts the classical reinforcement learning setting (in black) where

the agent interacts with the environment directly, and this alternative setting (in blue) where the agent is equipped with a simulator to plan and explore, until a user queries the agent for a recommendation. This perspective shift changes the nature of the exploration-exploitation trade-off, almost separating the two issues. During the learning phase there is a greater emphasis on exploring, while recommendations are concerned with pure exploitation. This perspective on reinforcement learning motivates the research questions around exploration: **Q1 - Exploration**.

The value of a policy π is the expected cumulative reward that is obtained by following the policy:

Definition 2.3.1. *The* value *of a policy π from state s at time t is:*

$$V^{\pi}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} r_i \middle| s_t = s\right]. \tag{2.21}$$

The Q-value *of a policy π, from state s, with action a, at time t is:*

$$Q^{\pi}(s,a;t) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[V^{\pi}(s';t+1)]. \tag{2.22}$$

From the definition of the value functions, the optimal value functions can be defined by taking the maximum value over all policies:

Definition 2.3.2. *The* Optimal (Q-)Value *of a state(-action pair) is defined as:*

$$V^{*}(s;t) = \max_{\pi} V^{\pi}(s;t) \tag{2.23}$$

$$Q^{*}(s,a;t) = \max_{\pi} Q^{\pi}(s,a;t). \tag{2.24}$$

Value functions can also be used to define an objective function:

Definition 2.3.3. *The* (standard) reinforcement learning objective function $J(\pi)$ *is defined as:*

$$J(\pi) = V^{\pi}(s_0;0). \tag{2.25}$$

The objective of (standard) reinforcement learning can then be stated as finding $\max_{\pi} J(\pi)$*; that is, the standard objective is to maximise the expected return obtained from the initial state s_0.*

An *optimal policy* is any policy that maximises the objective function J:

Definition 2.3.4. *An* optimal (standard) policy π^* *is any policy maximising the standard objective function:*

$$\pi^* = \arg\max_{\pi} J(\pi). \tag{2.26}$$

Additionally, there always exists a deterministic optimal (standard) policy, which can be expressed in terms of the optimal Q-value function [66]:

$$\pi^*(s;t) = \arg\max_{a} Q^*(s,a;t). \tag{2.27}$$

To simplify the notation used for theory in this thesis, it is assumed that there is a *unique optimal policy* for each MDP considered:

Assumption 2.3.1. *The* unique optimal policy assumption *is that each MDP $\mathcal{M}$ has a unique optimal policy. This assumption is not necessary; it is made purely to avoid unnecessary complexity in the proofs in this thesis. For example, when reasoning about a sequence of policies $\pi_1, \pi_2, \ldots$ it allows the statement $\pi_n \to \pi^*$ to make sense. Statements about convergence to the unique optimal policy generalise to convergence to one of the optimal policies when this assumption is not made.*

It can be shown that the optimal (Q-)value functions satisfy the *Bellman equations* [9, 66]:

$$V^*(s;t) = \max_{a \in \mathcal{A}} Q^*(s,a;t), \tag{2.28}$$

$$Q^*(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^*(s';t+1)]. \tag{2.29}$$

The Bellman equations admit a *dynamic programming* approach which can be used to compute the optimal value functions, known as *Value Iteration* [9, 66]. In Value Iteration, a table of value estimates $\hat{V}(s;t)$ is kept for each s, t. Given any initial estimate of the value function $\hat{V}^0$, the *Bellman backup* operations are:

$$\hat{V}^{k+1}(s;t) = \max_{a \in \mathcal{A}} \hat{Q}^{k+1}(s,a;t), \tag{2.30}$$

$$\hat{Q}^{k+1}(s,a;t) = \mathbb{E}_{s' \sim p(\cdot|s,a)}[R(s,a) + \hat{V}^k(s';t+1)]. \tag{2.31}$$

In each iteration of Value Iteration, these values are computed for all $s \in \mathcal{S}$, $a \in \mathcal{A}$ and $t \in \{0, 1, \ldots, H\}$. The value estimates computed from the Bellman backups will converge to the optimal values in a finite number of iterations: $V^k \to V^*$.[1]

2.3.1 Maximum Entropy Planning and Reinforcement Learning

In *maximum-entropy reinforcement learning*, the objective function is altered by the addition of an entropy term. This is motivated by the desire to learn stochastic behaviours that better explore large state spaces and are more robust under uncertainty by learning multiple solutions rather than a single deterministic solution [39].

Let $\mathcal{H}$ denote the (Shannon) entropy function [90]:

$$\mathcal{H}(\pi(\cdot|s;t)) = -\mathbb{E}_{a\sim\pi(\cdot|s;t)}[\log \pi(a|s;t)] = -\sum_{a\in\mathcal{A}} \pi(a|s;t) \log \pi(a|s;t). \tag{2.32}$$

In the maximum entropy objective, the relative weighting of entropy terms is included using a coefficient α, called the *temperature*. In the maximum entropy objective, analogues of the value functions can be defined, which are typically referred to as *soft (Q-)values*, and similarly the maximum entropy objective is often referred to as the *soft objective* (Definition 2.3.7 below). Where disambiguation is necessary, the *standard (Q-)value* (Definition 2.3.1) and *standard objective* (Definition 2.3.3) will be used to refer to the standard reinforcement learning counterparts.

Definition 2.3.5. *The* soft value *of a policy π from state s at time t is:*

$$V^\pi_{\text{sft}}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} r_i + \alpha\mathcal{H}(\pi(\cdot|s_i;i))\middle| s_t = s\right]. \tag{2.33}$$

The soft Q-value *of a policy π, from state s, with action a, at time t is:*

$$Q^\pi_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[V^\pi_{\text{sft}}(s';t+1)]. \tag{2.34}$$

[1] Strictly speaking, if backups are performed in reverse order from $t = H$ to $t = 0$, convergence is achieved in a single iteration. The iterative presentation here is adopted to more closely resemble the backup procedures used later in the thesis, where the transition distribution p is unknown.

The optimal soft (Q-)values can be defined by taking the maximum over policies, similar to the standard setting:

Definition 2.3.6. *The* optimal soft (Q-)Value *of a state(-action pair) is defined as:*

$$V^*_{\text{sft}}(s;t) = \max_\pi V^\pi_{\text{sft}}(s;t), \tag{2.35}$$

$$Q^*_{\text{sft}}(s,a;t) = \max_\pi Q^\pi_{\text{sft}}(s,a;t). \tag{2.36}$$

In maximum entropy reinforcement learning, the objective is to find a policy with maximal soft value.

Definition 2.3.7. *The* maximum entropy (or soft) reinforcement learning objective function $J_{\text{sft}}(\pi)$ *is defined as:*

$$J_{\text{sft}}(\pi) = V^\pi_{\text{sft}}(s_0;0). \tag{2.37}$$

The objective of maximum entropy (or soft) reinforcement learning can then be stated as finding $\max_\pi J_{\text{sft}}(\pi)$.

The optimal soft policy is defined as the policy that maximises the soft objective function J_{sft}, and it can be computed uniquely from the optimal soft (Q-)values.

Definition 2.3.8. *The* optimal soft policy π^*_{sft} *is the policy maximising the soft objective function:*

$$\pi^*_{\text{sft}} = \arg\max_\pi J_{\text{sft}}(\pi). \tag{2.38}$$

Given V^*_{sft} *and* Q^*_{sft} *the optimal soft policy is known to take the form [39]:*

$$\pi^*_{\text{sft}}(a|s;t) = \exp\left(\left(Q^*_{\text{sft}}(s,a;t) - V^*_{\text{sft}}(s;t)\right)/\alpha\right). \tag{2.39}$$

The *soft Bellman equations* are the maximum entropy analogues of the Bellman equations. These equations differ from equations (2.28) and (2.29) by the replacement of the max operation with a *softmax* or *log-sum-exp* operation, which

is why the maximum entropy analogues are referred to as the *soft* versions of their standard reinforcement learning counterparts. The *soft Bellman equations* are [39]:

$$V^*_{\text{sft}}(s;t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(Q^*_{\text{sft}}(s,a;t)/\alpha\right), \tag{2.40}$$

$$Q^*_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^*_{\text{sft}}(s';t+1)]. \tag{2.41}$$

Analogous to standard reinforcement learning, the soft Bellman equations can be used in *soft Bellman backups* for a *Soft Value Iteration* algorithm [39]:

$$\hat{V}^{k+1}_{\text{sft}}(s;t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(\frac{1}{\alpha}\hat{Q}^{k+1}_{\text{sft}}(s,a;t)\right), \tag{2.42}$$

$$\hat{Q}^{k+1}_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[\hat{V}^{k}_{\text{sft}}(s';t+1)]. \tag{2.43}$$

2.3.2 Planning Versus Reinforcement Learning

The objective of both planning and reinforcement learning is typically to compute an optimal policy for an MDP $\mathcal{M}$. The difference lies in what each has access to: planning methods compute a policy from the transition distribution p, whereas reinforcement learning methods learn from experience gathered in the environment [98]. Value iteration [9, 66] is therefore a planning method, because it uses p to compute the expectation in Equation 2.31, whereas Q-learning [107] is an example of a pure reinforcement learning method.

MCTS algorithms may fall on either side of this distinction, depending on how they handle the transition distribution: an algorithm that queries p directly is planning, whereas one that samples from a simulator is learning. Technically, the methods in this thesis are model-based reinforcement learning methods [60], as rather than using p directly they form maximum likelihood estimates of the transition probabilities from the *visit counts* at each node. Corollary 4.5.1 shows that these estimates are tightly bounded around the true transition probabilities. They are nonetheless often thought of as planning methods, because the tree search does plan, in the everyday sense of the word, over the estimated transition probabilities and values.

Because MCTS sits at the intersection of the two, the terms are used somewhat interchangeably in this thesis, with *planning* tending to refer to tree search and *reinforcement learning* to the wider literature. To be clear, throughout this thesis neither term is used in the sense of training a function approximation, such as a neural network, which is not considered here.

2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search

When considering MDPs with large state-action spaces, using tabular dynamic programming methods becomes infeasibly slow. One approach for handling large state-action spaces is to use *heuristic methods* that consider a subset of the state-action space and utilise heuristic estimates for the value of states. Often these heuristic methods build a *search tree* from the initial state of the MDP.

Monte Carlo Tree Search (MCTS) refers to heuristic algorithms that build a search tree using *Monte Carlo Trials*, where nodes are added to the tree from randomly sampled trajectories. Two advantages of using MCTS methods in the modern day are: that they allow for statistical analysis, which may provide *probabilistic guarantees* on the performance of the algorithm; and that they offer interpretability, as the search tree can be inspected post-hoc to identify why the algorithm made certain decisions.

MCTS trials are commonly presented using four stages, selection, expansion, simulation/heuristic and backup phases [15], which are depicted in Figure 2.6. To avoid confusion with the broader definition of MCTS above, this thesis refers to this specific form of MCTS as MCTS-1. Each of the four phases of MCTS-1 are described in more detail below:

1. *Selection phase* - a *search policy* is used to traverse the search tree from the root node until it reaches a state not contained in the search tree;

2. *Expansion phase* - a new child node is added to the search tree;

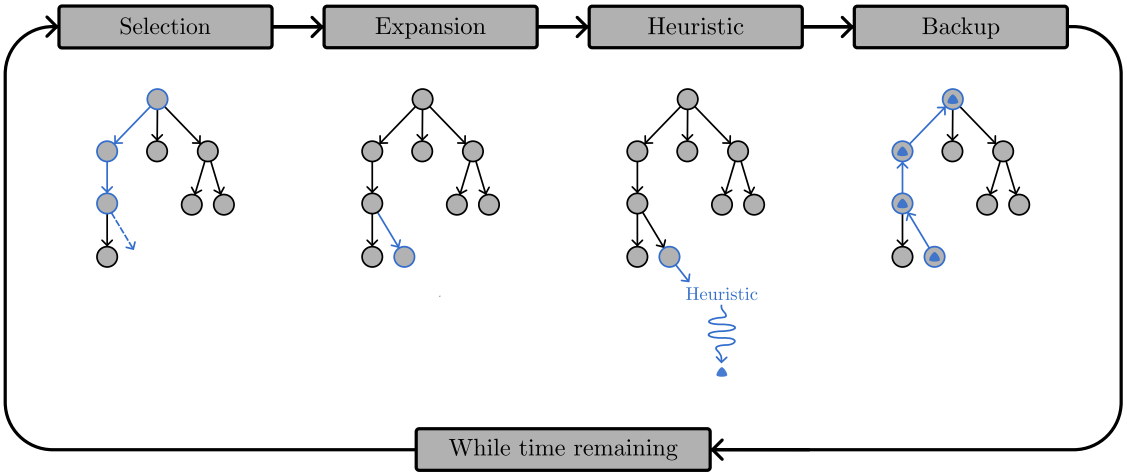


Figure 2.6: Overview of one trial of MCTS-1, depicting the four commonly presented stages [15]. First, in the selection phase a search policy is used to sample a path down the tree. Next a new node is added to the search tree. Then the new node is initialised using a heuristic, often using a *simulation* using a *rollout policy* as depicted. Finally, values (triangles) are updated through *backups* along the path sampled in the selection phase.

3. *Simulation/Heuristic phase* - the new child node's value is initialised using a *heuristic*, often taking the form of a Monte Carlo return, obtained using a *simulation* with a *rollout policy*;

4. *Backup phase* - the value from the heuristic or simulation is used to back up values through the path traversed in the selection phase.

Trial-Based Heuristic Tree Search (THTS) [49, 47] generalises heuristic tree search methods used for planning, such as Trial-Based Real Time Dynamic Programming [8] and LAO* [41]. However, THTS differs slightly from how MCTS-1 is presented above. In a THTS trial, after the expansion/heuristic phases are run, the trial "switches back to the selection phase and alternates between those phases", until the planning horizon is reached.

`THTS++` [68] is introduced in Section 2.4.2, which is an open-source, parallelised extension of the THTS schema — which also generalises MCTS-1 algorithms — and was built to facilitate the work in this thesis. Section 2.4.3 covers the Upper Confidence Bound applied to Trees (UCT) algorithm [51, 52], which is a commonly used MCTS algorithm, and Section 2.4.5 presents the Maximum ENtropy Tree

Search (MENTS) algorithm [113], which is related to the algorithms introduced in Chapter 4 of this thesis.

2.4.1 Notation

To simplify notation in the presentation and analysis of THTS++ algorithms, this thesis assumes a *unique state assumption*, that states and state-action pairs have a one-to-one correspondence with nodes in the search tree.

Assumption 2.4.1. *The* unique state assumption *is that each state s in an MDP $\mathcal{M}$ has a unique path to reach it from the initial state. This implies that states and state-action pairs have a one-to-one correspondence with nodes in the search tree.*

This assumption is not necessary; it is made purely to simplify notation, and any results discussed in this thesis generalise to settings where it does not hold. For example, the environments used in most of the empirical experiments in this thesis do not satisfy this assumption.

Specifically, this assumption allows the notation for value functions and policies to avoid explicitly writing the timestep parameter, because under this assumption the timestep t at which a state s is visited is uniquely determined by s itself. For example, $V^\pi(s)$ can be written instead of $V^\pi(s;t)$, and $\pi(a|s)$ can be written instead of $\pi(a|s;t)$. With the exception of Section 2.6, the explicit timestep parameter is dropped for the remainder of this thesis.

THTS++ builds a *search tree* $\mathcal{T}$ incrementally by running *trials* from the root node and adding them to the tree until a stopping criterion is met. Trees consist of *decision nodes* that correspond to states $s \in \mathcal{T}$, and *chance nodes* that correspond to state-action pairs $(s, a) \in \mathcal{T}$. For brevity, s refers to both states and decision nodes, and (s, a) refers to both state-action pairs and chance nodes. Where necessary for clarity, the notation $\texttt{node}(s)$ and $\texttt{node}(s, a)$ is used to explicitly refer to nodes rather than states or state-action pairs. At the start of search, the search tree is initialised to contain only the root node, $\mathcal{T} \leftarrow \{s_0\}$. Figure 2.7 gives an example of

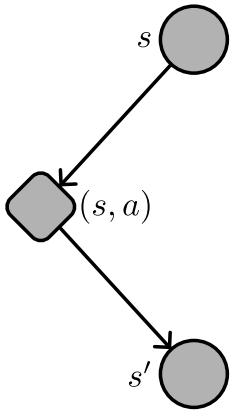


Figure 2.7: Tree diagrams notation, circles denote *decision nodes* that are associated with states, diamonds denote *chance nodes* that are associated with state-action pairs and arrows denote parent/child relationships in the tree.

how decision and chance nodes are depicted as circles and diamonds in diagrams throughout this thesis.

Definition 2.4.1. *A* search tree $\mathcal{T}$ *is a subset of the state and state-action spaces,* $\mathcal{T} \subseteq \mathcal{S} \cup \mathcal{S} \times \mathcal{A}$*, where for each* $s \in \mathcal{T}$*, there exists some truncated trajectory* $\tau_{:h}$ *such that* $s_h = s$*, each* $s' \in \tau_{:h}$ *is also in the tree,* $s' \in \mathcal{T}$*, and each* $s', a' \in \tau_{:h}$ *is also in the tree,* $(s', a') \in \mathcal{T}$*.*

Each decision and chance node stores some algorithm-dependent data, typically including but not limited to a *value estimate* and a *visit count*. Let $N(s)$ and $N(s, a)$ denote the number of times $\texttt{node}(s)$ and $\texttt{node}(s, a)$ have been visited, or equivalently, the number of times s and (s, a) appear in trajectories sampled in `THTS++` trials. $\texttt{node}(s)\texttt{.data}$ denotes the set of variables stored at the decision node $\texttt{node}(s)$, and $\texttt{node}(s, a)\texttt{.data}$ denotes the set of variables stored at the chance node $\texttt{node}(s, a)$. For example $N(s) \in \texttt{node}(s)\texttt{.data}$ and $N(s, a) \in \texttt{node}(s, a)\texttt{.data}$.

Additionally, let $\texttt{node}(s)\texttt{.chldrn}$ be a mapping from actions to the chance nodes that are children of $\texttt{node}(s)$. Likewise, $\texttt{node}(s, a)\texttt{.chldrn}$ is a mapping from outcomes (successor states) to the decision nodes that are children of $\texttt{node}(s, a)$. For example, $\texttt{node}(s)\texttt{.chldrn}[a] = \texttt{node}(s, a)$.

2.4.2 Trial Based Heuristic Tree Search

`THTS++` [68] provides a modular framework for defining and implementing MCTS algorithms. It extends the Trial-Based Heuristic Tree Search (THTS) schema [49] in a way that incorporates MCTS-1 algorithms and more modern MCTS algorithms that use a neural network or heuristic in place of a rollout simulation [92, 93]. It is largely presented in a single-threaded manner, which is sufficient for the purposes of this thesis.

Each trial has an associated *context*, which is a *per-trial* key-value store that can be used to store any algorithm-dependent data. The algorithms defined in Sections 2.4.3 and 2.4.5 will not make use of this context, but it will be used in Chapters 5 and 6 for multi-objective MCTS algorithms where *contextual tree search* is discussed. Contexts allow algorithms to store additional information within a trial without altering the interface. Beyond the scope of this thesis, contexts enable algorithms that require additional per-trial state to be implemented in `THTS++`, such as Partially Observable Monte Carlo Planning [91], where a state is sampled from a *belief state* at the beginning of each trial.

Definition 2.4.2. *A* context *is an arbitrary key-value store used to store any per-trial data.*

The main four steps performed during a `THTS++` trial are (depicted in Figure 2.8):

1. *Context Sampling* - a *context* is sampled for the trial;
2. *Selection Phase* - starting from a decision node s, an action is selected to reach a chance node (s, a), and an outcome is sampled to reach the next decision node s'. (s, a) and s' are added to the search tree $\mathcal{T}$ if they were not already.
3. *Heuristic Step* - any new nodes added from the selection step are initialised using a *heuristic function*;
4. *Backup Phase* - value estimates in the search tree are updated, along the path sampled in the selection step(s).

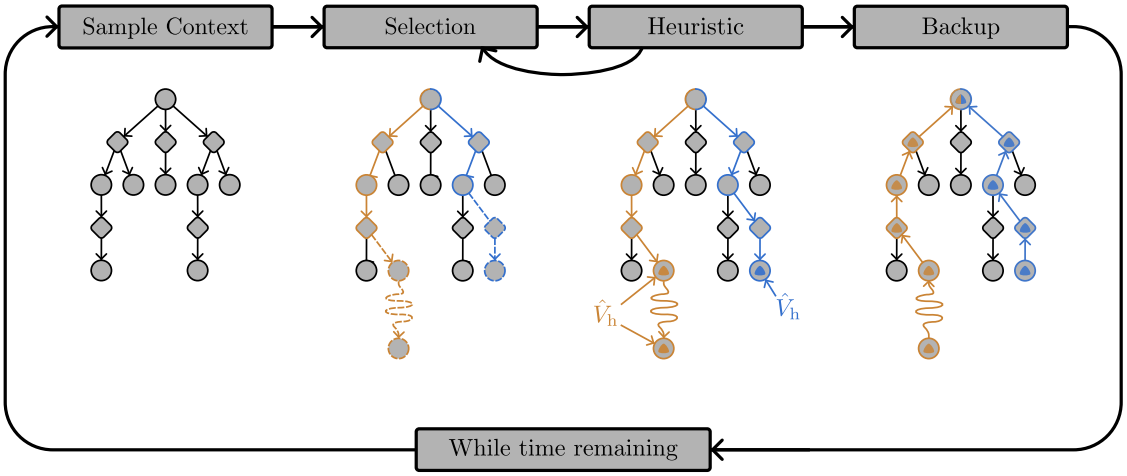


Figure 2.8: Overview of one trial of `THTS++`. Blue shows an example when `single_node_expansion` is True, and orange shows an example when `single_node_expansion` is False. From left to right: first a context is sampled, which stores any necessary per-trial state (not depicted), and the search tree at the beginning of the trial is shown; second shows the selection phase, where a trajectory is sampled and dashed lines indicate any new nodes to be added; third shows new nodes being initialised using the heuristic function $\hat{V}_{\text{heuristic}}$ (written as $\hat{V}_h$ for short); and finally on the right, the backup phase, where the arrows are reversed to show information propagating back up the tree to update the values in the nodes (triangles). The loop illustrates that if `single_node_expansion` is False (orange) the trial returns to the selection phase after the heuristic step, unless a terminal state or the planning horizon has been reached, in which case it proceeds to the backup phase. When `single_node_expansion` is True (blue) the trial instead leaves the loop as soon as a new decision node is added to the tree, and proceeds to the backup phase.

Collectively, the selection and heuristic steps are referred to as the *selection phase* of the trial, and the backup steps are collectively referred to as the *backup phase*.

In `THTS++` there are two options for when to progress from the selection phase to the backup phase, which is indicated by the boolean flag `single_node_expansion`. The selection phase will always end if a terminal state is reached, which includes reaching the planning horizon. When `single_node_expansion` is True, the selection phase will end earlier, once a new decision node is added to the search tree. Figure 2.8 shows an example of a trial when `single_node_expansion` is True (blue), and when it is False (orange).

Definition 2.4.3. *When* `single_node_expansion` *is True, the selection phase will end once a new decision node is added to the search tree.*

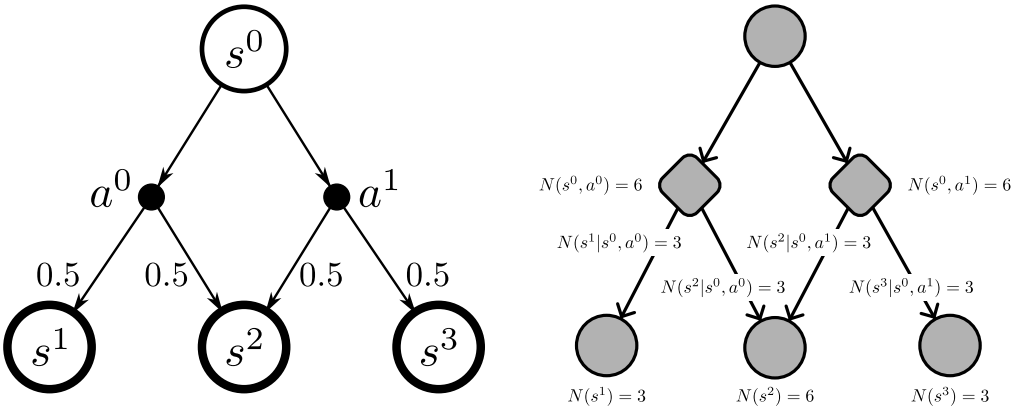


Figure 2.9: A complication arising from the use of a transposition table. Many MCTS algorithms use the ratio $\frac{N(s')}{N(s,a)}$ as an empirical estimate of transition probability $p(s'|s,a)$. Left: an example MDP with horizon $H = 1$ where using this ratio to estimate transition probabilities gives the incorrect ratios. Right: the corresponding *acyclic graph* created by running `THTS++` on the MDP with `transposition_table` set to True and sampling actions uniformly. After 12 trials, the resulting visit counts at the leaf nodes are $N(s^1) = 3, N(s^2) = 6, N(s^3) = 3$. The ratio of visit counts $\frac{N(s^1)}{N(s^0,a^1)} = \frac{1}{2}$ and $\frac{N(s^2)}{N(s^0,a^1)} = 1$, which do not form an (empirical) probability distribution, and even if normalised do not give the correct uniform distribution over successor states. To fix this issue, the *conditional visit counts* $N(s'|s,a)$ for $s' \in \text{Succ}((,s),a)$ are needed instead, that keep track of the number of times s' was sampled from the chance node (s,a). In this example, the conditional visit counts at node (s^0,a^0) would be $N(s^1|s^0,a^0) = 3$ and $N(s^2|s^0,a^0) = 3$, and give the correct ratios of $\frac{N(s^1|s^0,a^0)}{N(s^0,a^0)} = \frac{N(s^2|s^0,a^0)}{N(s^0,a^0)} = \frac{1}{2}$. That is, the ratios $\frac{N(s'|s,a)}{N(s,a)}$ correctly maintain an empirical estimate for the probability $p(s'|s,a)$, even when using a transposition table.

If the search is equipped with an informative heuristic function, then running with `single_node_expansion` set to True can allow the algorithm to avoid wasting resources (time and memory) on parts of the search tree that are not promising. Moreover, if the search is using a large planning horizon or the search will run for a long time, then it may be necessary to run with `single_node_expansion` set to True to avoid running out of memory.

In contrast, a random *rollout* is often used in MCTS-1 algorithms when no informative heuristic is available (for example Section 2.4.3). In such cases, where memory allows, running with `single_node_expansion` set to False can be beneficial, because it avoids throwing away potentially useful information from states visited during the rollout [96, 100].

If the unique state assumption (Assumption 2.4.1) is temporarily set aside,

environments may contain *transpositions*, where the same state can be reached from multiple different paths. When the `transposition_table` flag is set, a *transposition table* is used to re-use the $\texttt{node}(s)$ when the state s is reached again via another path [24, 87, 112].

Many MCTS algorithms, whether explicitly or implicitly, use the ratio of visit counts, $\frac{N(s')}{N(s,a)}$, as an empirical estimate of the transition distribution $p(s'|s,a)$. Figure 2.9 illustrates the issue that arises with this when using a transposition table and how it can be resolved, following an argument similar to that of Wu [112].

To run an algorithm in the `THTS++` schema, the following *modules* and *parameters* need to be specified. Algorithms specify the operation of each module:

Value Estimates: What data will be stored at each decision and chance node. That is, $\texttt{node}(s)\texttt{.data}$ and $\texttt{node}(s,a)\texttt{.data}$ need to be defined. It is assumed that $\texttt{node}(s)\texttt{.data}$ and $\texttt{node}(s,a)\texttt{.data}$ always contain the visit counts $N(s)$, and $N(s,a)$ respectively;

Initialisation: Two functions `init_v` and `init_q` which specify how to initialise $\texttt{node}(s)\texttt{.data}$ and $\texttt{node}(s,a)\texttt{.data}$ respectively;

Context Sampler: A function `sample_context` which creates a context key-value store, and samples any initial values to be stored in the context;

Action Selection (Search Policy): A function `select_action` which selects an action at a decision node. In this thesis, this will be specified as a function π_{search}, and `select_action` will return an action sampled from the search policy: $a \sim \pi_{\text{search}}(s)$;

(Optional) Outcome Sampler: A function `sample_outcome` that samples outcomes from chance nodes. By default, this will sample a state from the transition distribution of the MDP: $s' \sim p(\cdot|s,a)$ and this default will always be used in this thesis[2];

[2]One may want to use a different `sample_outcome` function to implement *double progressive widening* [26] for example.

Backup functions: Two functions `backup_v` and `backup_q` which specify how to update `node`(s)`.data` and `node`(s, a)`.data` respectively.

Parameters are set on a per-environment basis:

(Optional) Heuristic functions: The functions $\hat{V}_{\text{heuristic}}$, $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ are the heuristic functions. $\hat{V}_{\text{heuristic}}$ is used to initialise value estimates in `node`(s)`.data`. $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ can be used by the search policy to incorporate prior knowledge about unexplored actions. If not specified then by default $\hat{V}_{\text{heuristic}}$ and $\hat{Q}_{\text{heuristic}}$ are set to zero, and $\pi_{\text{heuristic}}$ is set to a uniform distribution;

Single Node Expansion: A boolean flag, `single_node_expansion`, specifying whether `THTS++` should only expand one decision node per trial;

Transposition Table: A boolean flag, `transposition_table`, specifying whether `THTS++` should use a transposition table to re-use nodes when the same state is reached via another path;

Planning horizon: A (problem dependent) planning horizon for the tree search $H_{\texttt{THTS++}}$, which is no longer than the horizon of the MDP, $H_{\texttt{THTS++}} \leq H$[3].

Given all of the modules and parameters are specified, the full operation of a trial in `THTS++` is described below. Additionally, pseudocode is given in Listing 2.1.

Sample Context: At the start of the trial, a context is sampled using the `sample_context` function, and the context is passed to all subsequent steps of the trial.

Selection Phase: A trajectory is sampled using the `select_action` and `sample_outcome` functions, and new nodes are initialised using the heuristic functions and added to the search tree. More specifically, starting with $i = 0$:

[3]In this thesis, MDPs have a finite horizon H, so $H_{\texttt{THTS++}}$ is somewhat redundant. However, it is useful in practice for environments with a long or infinite horizon, where a finite planning horizon is still required for the tree search.

Selection Step: Action a_i is selected using `select_action`(s_i), and an outcome s_{i+1} is sampled using `sample_outcome`(s_i, a_i).

Heuristic Step: If $(s_i, a_i) \notin \mathcal{T}$ initialise `node`(s_i, a_i)`.data` using `init_q`, and if $s_{i+1} \notin \mathcal{T}$ initialise `node`(s_{i+1})`.data` using `init_v`. New nodes are added to the tree, $\mathcal{T} \leftarrow \mathcal{T} \cup \{(s_i, a_i), s_{i+1}\}$.

If $s_{i+1} \in$ `terminal` or $i + 1 = H_{\texttt{THTS++}}$, then proceed to the backup phase.

If `single_node_expansion` and s_{i+1} was just added to the tree, then proceed to the backup phase.

Otherwise, increment i and return to the selection step.

Backup Phase: For $i = h - 1, \ldots, 0$, where h is the final value of i in the selection phase, the visit counts $N(s_i)$ and $N(s_i, a_i)$ are incremented, and the `backup_q` and `backup_v` functions are used to update the values of `node`(s_i, a_i)`.data` and `node`(s_i)`.data` respectively.

2.4.3 Upper Confidence Bound Applied to Trees (UCT)

Upper Confidence Bound Applied to Trees (UCT) [51, 52] is a commonly used tree search algorithm, which is based on the Upper Confidence Bound (UCB) algorithm covered in Section 2.1. UCT is a good example of a common paradigm in tree search algorithms for sequential decision-making, where each node in the tree is tasked with making a single decision in the sequence, and so adapts methods from the MAB literature. More specifically, UCT can be viewed as running UCB on a non-stationary MAB at every decision node, where it is non-stationary because the rewards obtained depend on the decisions that children make. The remainder of this subsection will outline how UCT can be defined using the `THTS++` schema given in Section 2.4.2.

The variables used in UCT consist of sample average value estimates $\bar{V}_{\text{UCT}}(s)$ and $\bar{Q}_{\text{UCT}}(s, a)$, visit counts $N(s)$ and $N(s, a)$. The parameters consist of: b_{UCT}, a bias parameter which controls the amount of exploration UCT will perform and `normalised_Q`, a flag indicating whether the Q-value estimates should be normalised when used in the search policy, which is useful if the Q-values can vary in magnitude

```
def run_trial(search_tree: 𝒯,
              search_policy: π_search,
              heuristic_fn: V̂_heuristic,
              heuristic_action_fn: Q̂_heuristic,
              heuristic_policy: π_heuristic,
              single_node_expansion,
              planning_horizon: H_THTS++):
    ctx = sample_context()
    τ_:h = selection_phase(𝒯, π_search, V̂_heuristic, Q̂_heuristic, π_heuristic, ↩
        single_node_expansion, H_THTS++, ctx)
    backup_phase(τ_:h, ctx)

def selection_phase(search_tree: 𝒯,
                    search_policy: π_search,
                    heuristic_fn: V̂_heuristic,
                    heuristic_action_fn: Q̂_heuristic,
                    heuristic_policy: π_heuristic,
                    single_node_expansion,
                    planning_horizon: H_THTS++,
                    ctx):
    i = 0
    while (not single_node_expansion or s_i ∈ 𝒯)
            and (i < H_THTS++)
            and (s_i ∉ terminal):
        # selection step
        a_i ← select_action(s_i, Q̂_heuristic, π_heuristic, ctx)
        r_i ← R(s_i, a_i)
        s_i+1 ← sample_outcome(s_i, a_i, ctx)
        # heuristic step
        if (s_i, a_i) ∉ 𝒯:
            init_q((s_i, a_i), Q̂_heuristic, π_heuristic, ctx)
        if s_i+1 ∉ 𝒯:
            init_v(s_i+1, V̂_heuristic, π_heuristic, ctx)
        𝒯 ← 𝒯 ∪ {(s_i, a_i), s_i+1}
        i += 1
    return (s_0, a_0, r_0, s_1, ..., s_i-1, a_i-1, r_i-1, s_i)

def select_action(s, Q̂_heuristic, π_heuristic, ctx):
    a ∼ π_search(·|s, Q̂_heuristic, π_heuristic, ctx)
    return a

def sample_outcome(s, a, ctx):
    s' ∼ p(·|s, a)
    return s'

def backup_phase(τ_:h, ctx)
    for i in {h − 1, h − 2, ..., 1, 0}:
        N(s_i, a_i) ← N(s_i, a_i) + 1
        N(s_i) ← N(s_i) + 1
        node(s_i, a_i).backup_q(node(s_i, a_i).chldrn, τ_i:h, ctx)
        node(s_i).backup_v(node(s_i).chldrn, τ_i:h, V̂_heuristic(s_h), ctx)
```

Listing 2.1: Pseudocode for running a trial in THTS++.

throughout the search tree. Typically `single_node_expansion` is set to True for UCT, and this section assumes that this is the case.

The search policy that UCT follows is given by:

$$\bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) = \begin{cases} \frac{\bar{Q}_{\text{UCT}}(s,a) - \min_{a \in \mathcal{A}} \bar{Q}_{\text{UCT}}(s,a)}{\max_{a \in \mathcal{A}} \bar{Q}_{\text{UCT}}(s,a) - \min_{a \in \mathcal{A}} \bar{Q}_{\text{UCT}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}_{\text{UCT}}(s,a) & \text{otherwise,} \end{cases} \tag{2.44}$$

$$\pi_{\text{UCT}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) + b_{\text{UCT}} \sqrt{\frac{\log(N(s))}{N(s,a)}}, \tag{2.45}$$

In Equation (2.45), when $N(s,a) = 0$ there is a division by zero, which is taken as ∞, and ties are broken uniformly randomly. Note that like UCB (Section 2.1), this results in every action being taken once before any action is selected for a second time. The heuristic functions $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ are not used by UCT.

Let $\tau_{:h} \sim \pi_{\text{UCT}}$ be the trajectory sampled in the selection phase of UCT. For any chance nodes (s,a) added to the tree, the `init_q` function initialises the visit count $N(s,a) = 0$ and the Q-value estimate $\bar{Q}_{\text{UCT}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$, which will be overwritten by `backup_q` on the first visit (Equation 2.48). The `init_v` function initialises the leaf node s_h using the heuristic function and with a single visit:

$$N(s_h) = 1, \tag{2.46}$$

$$\bar{V}_{\text{UCT}}(s_h) = \hat{V}_{\text{heuristic}}(s_h). \tag{2.47}$$

In UCT the backups `backup_v` and `backup_q` update the (sample average) value estimates incrementally. For $t = h-1, \ldots, 0$:

$$\bar{Q}_{\text{UCT}}(s_t,a_t) \leftarrow \frac{N(s_t,a_t)-1}{N(s_t,a_t)} \bar{Q}_{\text{UCT}}(s_t,a_t) + \frac{\hat{V}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t,a_t)}, \tag{2.48}$$

$$\bar{V}_{\text{UCT}}(s_t) \leftarrow \frac{N(s_t)-1}{N(s_t)} \bar{V}_{\text{UCT}}(s_t) + \frac{\hat{V}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t)}. \tag{2.49}$$

There are two common approaches for implementing $\hat{V}_{\text{heuristic}}$ in UCT. The first approach uses a function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$ [93, 94, 64], where $\tilde{V}$ is often a neural network that aims to approximate the optimal value function V^*. The second approach uses a *rollout*, which completes the trajectory from the selection phase using a *rollout policy* and uses the rewards from the rollout to give a heuristic

value estimate [15, 36, 48, 22]. If no informative policy is available for π_{rollout}, then a uniformly random policy is often used [15, 2].

To use a rollout policy π_{rollout} in `THTS++`, a Monte Carlo estimate $\hat{V}^{\pi_{\text{rollout}}}$ of the value function $V^{\pi_{\text{rollout}}}$ is used for $\hat{V}_{\text{heuristic}}$. Let $\tau_{h:h'} \sim \pi_{\text{rollout}}$ be the rollout trajectory, where $h' = H_{\texttt{THTS++}}$ or $s_{h'} \in \texttt{terminal}$. The Monte Carlo estimate is then the sum of the rewards from the rollout:

$$\hat{V}^{\pi_{\text{rollout}}}(s_h) = \sum_{i=h}^{h'-1} r_i. \tag{2.50}$$

2.4.4 Predictor-UCT (PUCT)

AlphaGo and AlphaGo Zero [92, 93] popularised a variant of UCT which is given in this section and referred to as *Predictor-UCT* (PUCT). In [92], this variant is ambiguously defined as "PUCT", citing *predictor UCB* [86] as the source, while more closely resembling *polynomial UCB* [5].

PUCT uses similar variables to UCT (Section 2.4.3), the sample average value estimates $\bar{V}_{\text{PUCT}}(s)$ and $\bar{Q}_{\text{PUCT}}(s,a)$, visit counts $N(s)$ and $N(s,a)$, the bias parameter b_{PUCT}, and the flag `normalised_Q`.

The PUCT search policy is similar to the UCT search policy (Equation 2.45), with the following changes:

- The logarithmic confidence interval term in Equation (2.45) is replaced by a polynomial term [5];
- The confidence interval term is weighted by the *predictor values* $\pi_{\text{heuristic}}(a|s)$, inspired by predictor UCB [86];
- The $N(s,a)$ denominator is modified to $N(s,a)+1$, which allows actions to be selected more than once before all actions have been tried, making the policy better suited to large action spaces such as in Go.

The operation of PUCT is otherwise identical to UCT, except for the changes to the search policy. The PUCT selection policy is given by:

$$\pi_{\text{PUCT}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}^{\text{norm}}_{\text{PUCT}}(s,a) + b_{\text{PUCT}} \pi_{\text{heuristic}}(a|s) \frac{\sqrt{N(s)}}{N(s,a)+1}. \tag{2.51}$$

In `THTS++`, the values of $\bar{Q}_{\text{PUCT}}(s,a)$ are initialised using the heuristic action value $\hat{Q}_{\text{heuristic}}(s,a)$. When $\bar{Q}_{\text{PUCT}}(s,a)$ is updated via the backup (Equation (2.48)), the heuristic initialisation is replaced by the sample average estimate.

2.4.5 Maximum Entropy Tree Search

Maximum ENtropy Tree Search (MENTS) [113], in contrast to UCT, focuses on the maximum-entropy objective (Definition 2.3.7), and uses soft Bellman backups to update its value estimates. In its original presentation, `single_node_expansion` is set to True, and it uses the soft value estimates $\hat{V}_{\text{MENTS}}$ and $\hat{Q}_{\text{MENTS}}$. The MENTS search policy is given by:

$$\pi_{\text{MENTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{MENTS}}))\rho_{\text{MENTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{MENTS}})}{|\mathcal{A}|} \tag{2.52}$$

$$\rho_{\text{MENTS}}(a|s) = \exp\left(\frac{1}{\alpha_{\text{MENTS}}}\left(\hat{Q}_{\text{MENTS}}(s,a) - \hat{V}_{\text{MENTS}}(s)\right)\right), \tag{2.53}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{2.54}$$

where $\epsilon_{\text{MENTS}} \in (0,\infty)$ is an exploration parameter, α_{MENTS} is the temperature parameter used in MENTS for the maximum entropy objective (the coefficient of the entropy term in Equation (2.33)), and λ is a decay function that is also used in later chapters with different arguments.

Let $\tau_{:h} \sim \pi_{\text{MENTS}}$ be the trajectory sampled in the selection phase of MENTS. For any chance nodes (s,a) added to the tree, the `init_q` function initialises the visit count $N(s,a) = 0$ and the Q-value estimate $\hat{Q}_{\text{MENTS}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$, which will be overwritten by `backup_q` on the first visit (Equation 2.57). The `init_v` function initialises the leaf node s_h using the heuristic function and with a single visit:

$$N(s_h) = 1, \tag{2.55}$$

$$\hat{V}_{\text{MENTS}}(s_h) = \hat{V}_{\text{heuristic}}(s_h). \tag{2.56}$$

The backups `backup_v` and `backup_q` then update the soft value estimates using soft Bellman backups (Equations (2.42) and (2.43)) as follows for $t = h-1, \ldots, 0$:

$$\hat{Q}_{\text{MENTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{MENTS}}(s') \right), \tag{2.57}$$

$$\hat{V}_{\text{MENTS}}(s_t) \leftarrow \alpha_{\text{MENTS}} \log \sum_{a \in \mathcal{A}} \exp \left(\frac{1}{\alpha_{\text{MENTS}}} \hat{Q}_{\text{MENTS}}(s_t, a) \right). \tag{2.58}$$

In [113], the heuristic value function is left as an arbitrary evaluation function, but is set using a function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$ in experiments. The heuristic action function is set to zero, $\hat{Q}_{\text{heuristic}}(s, a) = 0$, however the authors also suggest that if *policy network* $\tilde{\pi}$ is available, then the heuristic action function can alternatively be set to $\hat{Q}_{\text{heuristic}}(s, a) = \log \tilde{\pi}(a|s)$.

2.5 Convergence of Random Variables

This section introduces the two modes of convergence used in the proofs in this thesis and states existing results that will be used.

Definition 2.5.1. *A sequence of random variables* X_i converges in probability *to a random variable* X*, written as* $X_i \xrightarrow{p} X$*, if for all* $\varepsilon > 0$ *the following holds:*

$$\Pr\left(|X_i - X| > \varepsilon\right) \to 0 \text{ as } i \to \infty. \tag{2.59}$$

Definition 2.5.2. *A sequence of random variables* X_i converges almost surely *to a random variable* X*, written as* $X_i \xrightarrow{as} X$*, if the following holds:*

$$\Pr\left(\lim_{i \to \infty} X_i = X\right) = 1. \tag{2.60}$$

Almost sure convergence is a stronger condition than convergence in probability: almost sure convergence implies convergence in probability, but not vice versa [12].

The Dvoretzky-Kiefer-Wolfowitz (DKW) inequality [31] bounds the maximum difference between the empirical distribution of samples and the true distribution:

Theorem 2.5.1. *Let* $\{X_i\}_{i=1}^m$ *be random variables drawn from a probability distribution with a cumulative distribution function of* F*. Let the empirical cumulative*

distribution function be $F_m(x) = \frac{1}{m}\sum_{i=1}^{m} \mathbb{1}[X_i < x]$. *Then the Dvoretzky-Kiefer-Wolfowitz inequality is:*

$$\Pr\left(\sup_x |F_m(x) - F(x)| > \varepsilon\right) \leq 2\exp\left(-2m\varepsilon^2\right). \tag{2.61}$$

The DKW inequality is used in Chapters 4 and 5 to bound the error when using the empirical visit count ratio $\frac{N(s')}{N(s,a)}$ as an approximation to the transition probability $p(s'|s,a)$.

The *Good-Turing estimate* [38] addresses a related but distinct problem: rather than bounding the error in the empirical distribution, it estimates the total probability mass of outcomes that have not yet been observed.

Let f be a probability mass function over the categories $\mathcal{C} = \{C_1, \ldots, C_k\}$, where the number of categories k is unknown or potentially infinite. Let $X_1, \ldots, X_m$ be m independent samples drawn from f, and let $M(C)$ be the number of times category C was observed:

$$M(C) = \sum_{i=1}^{m} \mathbb{1}[X_i = C]. \tag{2.62}$$

Let $\mathcal{C}_{\text{obs}} = \{C \in \mathcal{C} \mid M(C) > 0\}$ be the set of observed categories. The *missing mass* is the total probability of unobserved categories:

$$\texttt{missing_mass} = \sum_{C \in \mathcal{C} - \mathcal{C}_{\text{obs}}} f(C). \tag{2.63}$$

The Good-Turing estimate of the missing mass is:

$$\texttt{missing_mass}_{\text{GT}} = \frac{M_1}{m}, \tag{2.64}$$

where $M_1 = |\{C \in \mathcal{C}_{\text{obs}} \mid M(C) = 1\}|$ is the number of categories observed exactly once (the number of *singletons*).

McAllester and Schapire [62] show that the Good-Turing estimate satisfies the following bound.

Theorem 2.5.2. *For* $0 < \delta \leq 1$, *the Good-Turing estimate (Equation 2.64) satisfies the following bound:*

$$\Pr\left(\texttt{missing_mass} \leq \texttt{missing_mass}_{\text{GT}} + \sqrt{\frac{\log(1/\delta)}{2m}}\right) \geq 1 - \delta. \tag{2.65}$$

The Good-Turing estimate is used in Chapter 5 to bound the probability mass of unobserved successor states when computing the unsolved value of a chance node (s, a) (Section 5.3.6).

2.6 Multi-Objective Planning and Reinforcement Learning

This section defines the multi-objective counterparts to various definitions given in Sections 2.2 and 2.3. Outside of this section, the prefix "multi-objective" may be dropped where it should be clear from context; however, bold typeface will consistently be used to denote any vector variables or functions. In Section 2.6.1, a multi-objective extension of Value Iteration is given.

This thesis follows a utility-based approach to multi-objective reinforcement learning similar to that of Hayes et al. [43] and specifically considers the *unknown weights scenario* [82], which is depicted in Figure 2.10. In the unknown weights scenario, the weight vector $\mathbf{w}$ specifying the preferences over the objectives is unknown during the *planning* (or *learning*) *phase*, so the algorithm computes a *solution set* consisting of multiple *possibly optimal* solutions. The weight vector is revealed during the *weight selection phase*, and the policy that maximises the utility for that weight vector is typically selected from the solution set. The selected policy

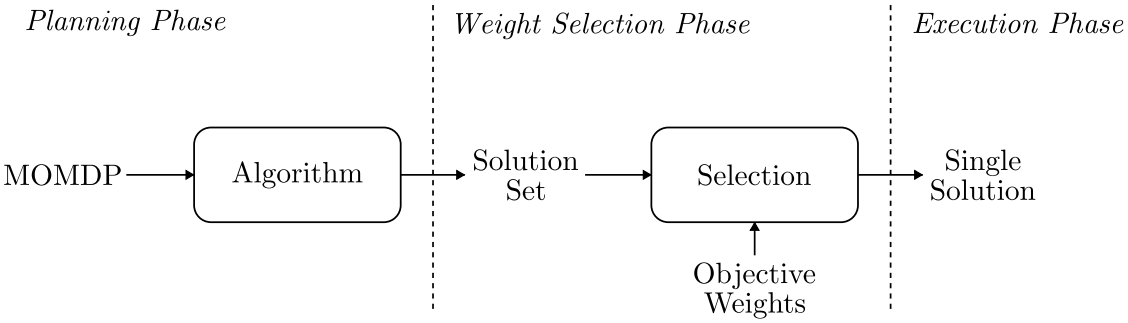


Figure 2.10: The unknown weights scenario for multi-objective reinforcement learning [43]. A multi-objective algorithm produces a solution set during the planning phase; the objective weights are then revealed in the weight selection phase and used to choose a single solution from the set for the execution phase.

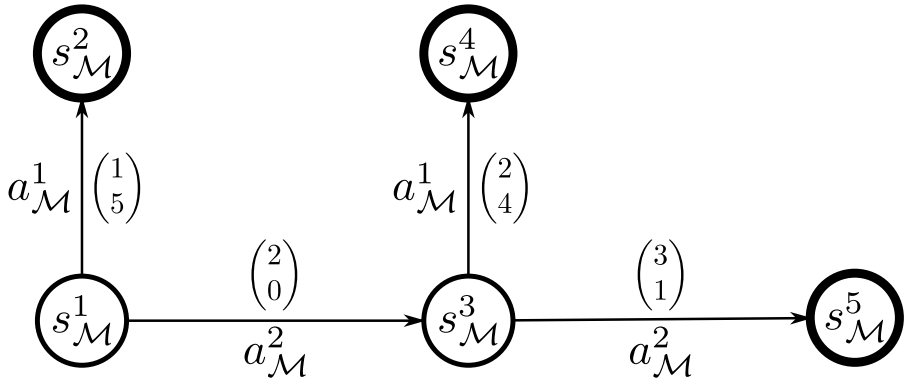


Figure 2.11: An example MOMDP $\boldsymbol{\mathcal{M}}$, where $s_0 = s^1_{\mathcal{M}}$, the finite horizon is $H = 2$ and terminal states are marked with thicker borders. This particular MOMDP is deterministic, so that action nodes are omitted and all transition probabilities are one.

is then followed during the *execution phase*. This scenario arises when preferences are unavailable at the time of planning, or vary between uses of the system.

To specify problems with multiple objectives, the reward function of an MDP is changed to give a vector of rewards, rather than a scalar reward:

Definition 2.6.1. *A* Multi-Objective Markov Decision Process *(MOMDP) is a tuple* $\boldsymbol{\mathcal{M}} = (\mathcal{S}, s_0, \mathcal{A}, \mathbf{R}, p, H)$*, where* $\mathcal{S}$ *is a set of states,* $s_0 \in \mathcal{S}$ *is an initial state,* $\mathcal{A}$ *is a set of actions,* $\mathbf{R}(s, a)$ *is a vector reward function* $\mathcal{S} \times \mathcal{A} \to \mathbb{R}^D$*, where* D *is the number of objectives,* $p(s'|s, a)$ *is a next state transition distribution* $\mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ *and* $H \in \mathbb{N}$ *is a finite-horizon time bound.*

Multi-objective trajectories are defined analogously to single-objective trajectories (Definition 2.2.6):

Definition 2.6.2. *A* multi-objective trajectory $\boldsymbol{\tau}$ *is a sequence of states, actions and vector rewards that is induced by a policy* π *and an MOMDP* $\boldsymbol{\mathcal{M}}$*. Let the trajectory be* $\boldsymbol{\tau} = (s_0, a_0, \mathbf{r}_0, s_1, a_1, \mathbf{r}_1, \ldots, s_{H-1}, a_{H-1}, \mathbf{r}_{H-1}, s_H)$*, where* $a_t \sim \pi(\cdot|s_t; t)$*,* $\mathbf{r}_t = \mathbf{R}(s_t, a_t)$ *and* $s_{t+1} \sim p(\cdot|s_t, a_t)$*.*

The notations used for single-objective trajectories (Definition 2.2.6) will also be used for multi-objective trajectories, such as $\boldsymbol{\tau} \sim \pi$ *for sampling trajectories using policies, and* $\boldsymbol{\tau}_{i:j}$ *for truncated trajectories.*

Similarly, multi-objective variants of the (Q-)value of a policy are defined:

Definition 2.6.3. *The* multi-objective value *of a policy π from state s at time t is:*

$$\mathbf{V}^{\pi}(s;t) = \mathbb{E}_{\boldsymbol{\tau}\sim\pi}\left[\sum_{i=t}^{H-1}\mathbf{r}_i \middle| s_t = s\right]. \tag{2.66}$$

The multi-objective Q-value *of a policy π, from state s, with action a, at time t is:*

$$\mathbf{Q}^{\pi}(s,a;t) = \mathbf{R}(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[\mathbf{V}^{\pi}(s';t+1)]. \tag{2.67}$$

At the corresponding point in Section 2.3, the optimal (Q-)value functions and the objective of single-objective reinforcement learning were defined. However, in a multi-objective setting there is no longer a *total ordering* over values, and so there may be multiple vectors that could be "optimal". For example, consider two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$ which are such that $u_1 > v_1$ and $u_2 < v_2$. To resolve this issue, a *utility function* or *scalarisation function* is used to map multi-objective values to scalar values.

Definition 2.6.4. *The* D-dimensional Unit Simplex *(or the* $(D-1)$-simplex*) consists of the set of D-dimensional vectors, whose entries are non-negative and sum to one*[4]*. More formally, the D-dimensional simplex is*

$$\Delta^D = \{\mathbf{w} \in \mathbb{R}^D | w_i \geq 0, \sum_i w_i = 1\}. \tag{2.68}$$

The elements of the D-dimensional simplex will be referred to as weight vectors *(or just* weights*) in this thesis, as they will be used to specify preferences over the D dimensions of the reward function.*

Definition 2.6.5. *A* utility function *(or* scalarisation function*)* $u : \mathbb{R}^D \times \Delta^D \to \mathbb{R}$ *is used to map from a multi-objective value $\mathbf{v} \in \mathbb{R}^D$ and a weighting over the objectives $\mathbf{w} \in \Delta^D$ to a scalar value. That is, according to the utility function $u(\cdot;\mathbf{w})$ the multi-objective value $\mathbf{v}$ is mapped to the scalar value $u(\mathbf{v};\mathbf{w})$.*

A common choice of utility is the *linear utility function*, where the scalar value takes the form of a dot product:

[4]Note that the D-dimensional simplex is actually the $(D-1)$-simplex; the notation Δ^D is adopted in this thesis to emphasise that weight vectors have D components.

Definition 2.6.6. *The* linear utility function u_{lin} *is the utility function defined by:*

$$u_{\text{lin}}(\mathbf{v}; \mathbf{w}) = \mathbf{w}^\top \mathbf{v}. \tag{2.69}$$

The algorithms in Chapters 5 and 6 assume a linear utility function. Section 2.6.2 discusses the implications of this assumption, and Section 5.5.2 discusses why it is a difficult assumption to avoid in an MOMCTS algorithm.

Equipped with a utility function and a weight vector, any set of multi-objective values can be mapped to scalars and ordered. A policy is *possibly optimal*, or *undominated*, with respect to utility function u if it achieves a maximal utility for some weight $\mathbf{w} \in \Delta^D$. In contrast, a policy is *dominated* if for every weight there is another policy that achieves a better utility.

Let Π be the set of all possible policies for a given MOMDP. When constructing a solution set, a subset of Π, it makes sense to exclude dominated policies. This leads to the first notion of a solution set, called an *undominated set*, consisting of all undominated policies:

Definition 2.6.7. *The* undominated set *of policies* $U(\Pi; u) \subseteq \Pi$*, with respect to a utility function* u*, is the set of policies for which there is a weight vector* $\mathbf{w} \in \Delta^D$ *where the utility (scalarised value) is maximised:*

$$U(\Pi; u) = \left\{ \pi \in \Pi \;\middle|\; \exists \mathbf{w} \in \Delta^D . \forall \pi' \in \Pi : u(\mathbf{V}^\pi(s_0; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}) \right\}. \tag{2.70}$$

In particular, the convex hull *of policies* $\text{CH}(\Pi)$ *is the undominated set with respect to the linear utility function* u_{lin}*. That is* $\text{CH}(\Pi) = U(\Pi; u_{\text{lin}})$*.*

Undominated sets often have an infinite cardinality, and as such are infeasible to compute. However, in undominated sets there are often many redundant policies that obtain the same scalarised values. Instead of computing an undominated set, it is more feasible to compute a *coverage set* that contains at least one policy that maximises the scalarised value given any weight vector $\mathbf{w}$:

Definition 2.6.8. *A set* $\mathrm{CS}(\Pi; u) \subseteq U(\Pi; u)$ *is a* coverage set *with respect to a utility function* u *if for every weight vector* $\mathbf{w} \in \Delta^D$*, there is a policy* $\pi \in \mathrm{CS}(\Pi; u)$ *that maximises the value of* $u(\cdot; \mathbf{w})$*. That is, for* $\mathrm{CS}(\Pi; u)$ *to be a coverage set, the following statement must be true:*

$$\forall \mathbf{w} \in \Delta^D.\ \exists \pi \in \mathrm{CS}(\Pi; u).\ \forall \pi' \in \Pi : \\ u(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}). \tag{2.71}$$

In particular, any set $\mathrm{CCS}(\Pi)$ *is a* convex coverage set *if it is a coverage set with respect to the linear utility function* u_{lin}*.*

A convex coverage set may contain redundant policies and is not necessarily minimal. A minimal convex coverage set is a convex coverage set that contains no redundant policies. That is, for every policy in the minimal convex coverage set, there is some weight for which it uniquely maximises the utility within the set.

Definition 2.6.9. *A set* $\mathrm{MCCS}(\Pi) \subseteq \Pi$ *is a* minimal convex coverage set *if it is a convex coverage set and if the following holds:*

$$\forall \pi \in \mathrm{MCCS}(\Pi).\ \exists \mathbf{w} \in \Delta^D.\ \forall \pi' \in \mathrm{MCCS}(\Pi) - \{\pi\} : \\ u_{\mathrm{lin}}(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) > u_{\mathrm{lin}}(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}). \tag{2.72}$$

When computing solution sets, it is often useful to first compute the multi-objective values that could be obtained, and then use the algorithm's data structures to read out the selected policy or solution set (for example, Convex Hull Value Iteration covered in Section 2.6.1). This thesis will refer to the values obtained by a set of policies as a *value set*:

Definition 2.6.10. *The* (multi-objective) value set *with respect to a set of policies* $\Pi' \subseteq \Pi$ *is defined as* $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi') = \{\mathbf{V}^{\pi}(s_0; 0) | \pi \in \Pi'\}$*.*

Given a value set, the `cvx_prune` operation removes any vectors that are dominated or redundant. That is, for every vector that remains after the pruning

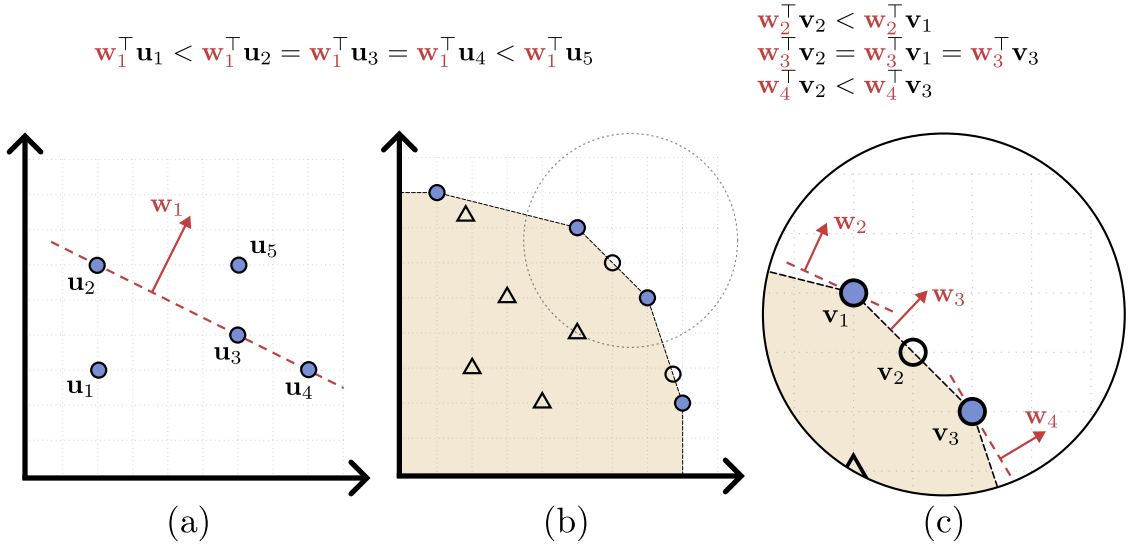


Figure 2.12: The geometry of convex coverage sets, shown with $D = 2$. In each image the points depicted correspond to a value set $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$, for some set of policies Π'. (a) demonstrates that values ($\mathbf{u}_2$,$\mathbf{u}_3$,$\mathbf{u}_4$) obtaining the same utility lie on a hyperplane. Moving in the direction of the weight vector increases the utility ($\mathbf{u}_5$) and likewise moving in the opposite direction decreases utility ($\mathbf{u}_1$); (b) circles correspond to values that are in the convex hull value set, so form the set $\boldsymbol{\mathcal{V}}\mathbf{als}(\text{CH}(\Pi'))$, whereas triangles correspond to values not in the convex hull value set. The blue circles correspond to a minimal convex coverage set MCCS(Π'), so form the set of values $\boldsymbol{\mathcal{V}}\mathbf{als}(\text{MCCS}(\Pi'))$. The unfilled circles and triangles are values that would be pruned by a `cvx_prune` operation on $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$ (Equation (2.73)); (c) shows a magnification of (b) with additional labels, and shows that the value $\mathbf{v}_2$ is redundant because one of $\mathbf{v}_1$ or $\mathbf{v}_3$ will achieve an equal or greater utility. Additionally, it shows $\mathbf{w}_2$ and $\mathbf{w}_4$, which are weights for which the values $\mathbf{v}_1$ and $\mathbf{v}_3$ uniquely maximise the utility within $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$.

operation, there is some weight for which it *uniquely* gives the maximal linear utility. Given a set of vectors $\boldsymbol{\mathcal{V}}$, the `cvx_prune` operation is defined as:

$$\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}) = \{\mathbf{v} \in \boldsymbol{\mathcal{V}} | \exists \mathbf{w} \in \Delta^D . \forall \mathbf{v}' \in \boldsymbol{\mathcal{V}} - \{\mathbf{v}\} . \mathbf{w}^\top \mathbf{v} > \mathbf{w}^\top \mathbf{v}'\}. \tag{2.73}$$

As illustrated in Figure 2.12, the `cvx_prune` operation computes the *extreme points* of a *(partial) geometric convex hull*. As such, for any set of policies Π' the `cvx_prune` operation satisfies $\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')) = \boldsymbol{\mathcal{V}}\mathbf{als}(\text{MCCS}(\Pi'))$, and `cvx_prune` can be implemented using *linear programming* [84].

To distinguish between the convex hull of policies, and the geometric convex hulls of values, this thesis will refer to any value set that forms a geometric convex hull as a *convex hull value set* (CHVS).

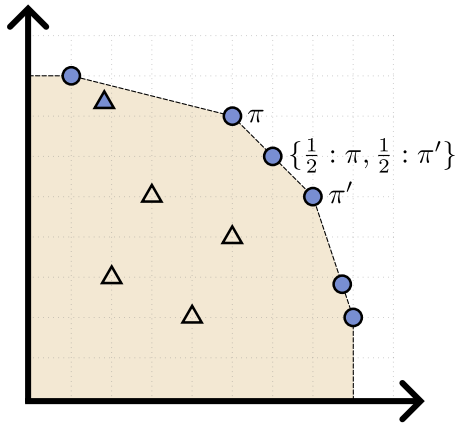


Figure 2.13: An example Pareto front, where the circles correspond to a convex coverage set, and the filled blue points denote the Pareto front. Two points are marked with their corresponding policies π and π', and a mixture policy is shown, denoted $\{\frac{1}{2}:\pi, \frac{1}{2}:\pi'\}$ that follows each of π and π' with probability $\frac{1}{2}$.

Definition 2.6.11. *The set of vectors* $\boldsymbol{\mathcal{V}} \subseteq \mathbb{R}^D$ *is a* convex hull value set *(CHVS) if its points form a partial geometric convex hull. More specifically,* $\boldsymbol{\mathcal{V}}$ *is a convex hull value set if it satisfies the equation:*

$$\boldsymbol{\mathcal{V}} = \texttt{cvx_prune}(\boldsymbol{\mathcal{V}}). \tag{2.74}$$

Whilst this thesis focuses on the linear utility setting, another commonly used solution set is the *Pareto front*, which is defined using a *strict partial ordering* over vectors, known as *Pareto domination*. An example is shown in Figure 2.13.

Definition 2.6.12. *The* Pareto front $\text{PF}(\Pi) \subseteq \Pi$ *is defined as:*

$$\text{PF}(\Pi) = \{\pi \in \Pi | \nexists \pi' \in \Pi . \mathbf{V}^{\pi'}(s_0; 0) \succ_P \mathbf{V}^{\pi}(s_0; 0)\}, \tag{2.75}$$

where the Pareto domination *relation is defined as:*

$$\mathbf{v} \succ_P \mathbf{v}' \iff (\forall i . v_i \geq v'_i) \wedge (\exists j . v_j > v'_j). \tag{2.76}$$

The Pareto front can be related to the previously defined solution sets in two ways. The first is via the *Chebyshev utility* function, which recovers the Pareto front if the utopian point $\mathbf{r}_{\text{uto}}$ is appropriately set [97]:

Definition 2.6.13. *Let* $\mathbf{v}, \mathbf{r}_{\text{uto}} \in \mathbb{R}^D$ *and* $\mathbf{w} \in \Delta^D$*. The* Chebyshev utility *of* $\mathbf{v}$ *with respect to the weight vector* $\mathbf{w}$ *and utopian point* $\mathbf{r}_{\text{uto}}$ *is:*

$$u_{\text{cheby}}(\mathbf{v}; \mathbf{w}, \mathbf{r}_{\text{uto}}) = -\max_{i=1,\ldots,D} w_i |v_i - (r_{\text{uto}})_i|. \tag{2.77}$$

The second is if *mixture policies* are allowed, where a mixture policy between π and π' will follow each policy with some probability[5]. Specifically, if Π is closed with respect to mixture policies, then the Pareto front is a convex coverage set.

Several metrics have been proposed for evaluating the quality of a solution set in multi-objective settings [43]. Two commonly used metrics are the *hypervolume indicator*, which measures the volume of objective space dominated by a value set, and the *expected utility metric*, which measures the expected value of the best policy in the set under a randomly drawn weight vector.

Definition 2.6.14. *Let* $\boldsymbol{\mathcal{V}} \subset \mathbb{R}^D$ *be a finite set of vectors and let* $\mathbf{r} \in \mathbb{R}^D$ *be a* reference point *such that* $r_d \leq \min_{\mathbf{v} \in \boldsymbol{\mathcal{V}}} v_d$ *for all* $d = 1, \ldots, D$*. The* hypervolume indicator *of* $\boldsymbol{\mathcal{V}}$ *with respect to* $\mathbf{r}$ *is the Lebesgue measure of the space dominated by* $\boldsymbol{\mathcal{V}}$ *and bounded below by* $\mathbf{r}$*:*

$$\text{HV}(\boldsymbol{\mathcal{V}}; \mathbf{r}) = \lambda_{\text{Lebesgue}} \left(\bigcup_{\mathbf{v} \in \boldsymbol{\mathcal{V}}} \{\mathbf{x} \in \mathbb{R}^D \mid \mathbf{r} \preceq_P \mathbf{x} \preceq_P \mathbf{v}\} \right), \tag{2.78}$$

where $\lambda_{\text{Lebesgue}}$ *denotes the Lebesgue measure and* $\preceq_P$ *is Pareto domination (Definition 2.6.12).*

Definition 2.6.15. *Given a solution set of policies* Ψ*, the (Uniform) Expected Utility Metric (*EUM*) is defined as:*

$$\text{EUM}(\Psi) = \mathbb{E}_{\mathbf{w} \sim U(\Delta^D)} \left[\max_{\psi \in \Psi} \mathbf{w}^\top \mathbf{V}^\psi(s_0; 0) \right], \tag{2.79}$$

where $U(\Delta^D)$ *is the uniform distribution over vectors in* Δ^D*.*

[5]Mixture policies are only appropriate when a policy is run many times and the aggregated return is of main concern, as under the scalarised expected returns criterion used in this thesis (Section 1.2).

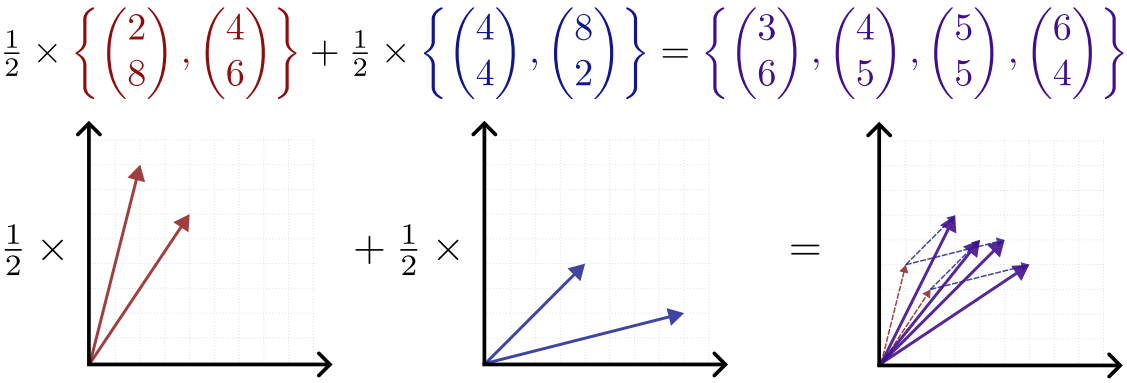


Figure 2.14: An example of arithmetic over vector sets. Two vector sets are shown in red and blue, which are multiplied by $\frac{1}{2}$ using Equation (2.80), and then added using Equation (2.81). On the right the resulting vector set is shown in purple, which are the sum of appropriately scaled red and blue vectors.

2.6.1 Convex Hull Value Iteration

Convex Hull Value Iteration (CHVI) [7] is a tabular dynamic programming algorithm similar to Value Iteration (Section 2.3) that computes optimal convex hull value sets and provides a method for extracting policies for any given weight vector. In CHVI the value estimates of Value Iteration are replaced by CHVSs, which are estimates of the optimal CHVS. These estimates are denoted $\hat{\boldsymbol{\mathcal{V}}}_{\text{CHVI}}(s;t)$ and $\hat{\boldsymbol{\mathcal{Q}}}_{\text{CHVI}}(s,a;t)$.

Firstly, to define a multi-objective version of Value Iteration, arithmetic over sets of vectors needs to be defined. An example of the following arithmetic is given in Figure 2.14. Given the vector sets $\boldsymbol{\mathcal{U}}$ and $\boldsymbol{\mathcal{V}}$, define multiplication by a scalar s, addition with a vector $\mathbf{x}$ and addition between sets as follows:

$$\mathbf{x} + s\boldsymbol{\mathcal{V}} = \{\mathbf{x} + s\mathbf{v} | \mathbf{v} \in \boldsymbol{\mathcal{V}}\} \tag{2.80}$$

$$\boldsymbol{\mathcal{U}} + \boldsymbol{\mathcal{V}} = \{\mathbf{u} + \mathbf{v} | \mathbf{u} \in \boldsymbol{\mathcal{U}}, \mathbf{v} \in \boldsymbol{\mathcal{V}}\}. \tag{2.81}$$

Now to define the multi-objective Bellman backups used in CHVI, let $\hat{\boldsymbol{\mathcal{V}}}^0_{\text{CHVI}}(s;t) = \{\mathbf{0}\}$, where $\mathbf{0} = (0,\ldots,0) \in \mathbb{R}^D$. The CHVI backups are then:

$$\hat{\boldsymbol{\mathcal{V}}}^{k+1}_{\text{CHVI}}(s;t) = \texttt{cvx_prune}\left(\bigcup_{a\in\mathcal{A}} \hat{\boldsymbol{\mathcal{Q}}}^{k+1}_{\text{CHVI}}(s,a;t)\right), \tag{2.82}$$

$$\hat{\boldsymbol{\mathcal{Q}}}^{k+1}_{\text{CHVI}}(s,a;t) = \mathbb{E}_{s'\sim p(\cdot|s,a)}[\mathbf{R}(s,a) + \hat{\boldsymbol{\mathcal{V}}}^{k}_{\text{CHVI}}(s';t+1)]. \tag{2.83}$$

This again parallels the Bellman backups used in single-objective Value Iteration, where the max operation is replaced by the `cvx_prune` operation over the set of achievable values from the current state s.

Once the algorithm has terminated and a weight $\mathbf{w}$ is provided, a policy can be extracted by computing scalar Q-values from the CHVSs [7]:

$$Q^{\mathbf{w}}_{\text{CHVI}}(s,a;t) = \max\{\mathbf{w}^\top \mathbf{q} | \mathbf{q} \in \hat{\mathcal{Q}}_{\text{CHVI}}(s,a;t)\}, \tag{2.84}$$

$$\pi_{\text{CHVI}}(s;t,\mathbf{w}) = \arg\max_{a \in \mathcal{A}} Q^{\mathbf{w}}_{\text{CHVI}}(s,a;t). \tag{2.85}$$

Note that if a policy is extracted for every weight vector $\mathbf{w} \in \Delta^D$, the resulting set of policies forms a convex coverage set.

2.6.2 Implications of Assuming a Linear Utility

The algorithms in Chapters 5 and 6 assume a linear utility, which limits the preferences that can be expressed, because a linear utility trades objectives off at a fixed rate, regardless of how much of each objective has already been obtained. In particular:

- Diminishing returns cannot be expressed, although they are common in practice, with the value of additional money being a common example.
- Lexicographic preferences cannot be expressed, where any improvement in the first objective is preferred to any improvement in the second, the second over the third, and so on.
- An objective with a small weight can be ignored entirely, whereas the Chebyshev utility (Definition 2.6.13) forces a compromise, so that improving a low-weighted objective is still necessary to increase the utility.

In return, a linear utility gives the following properties, the first of which the methods in this thesis rely on:

- Optimal policies can be obtained by selecting actions greedily using the reward-to-go at each node, as in Equation (2.85); Section 5.5.3 shows that this holds if and only if the utility is linear.
- Scalarising the rewards by $\mathbf{w}$ reduces the MOMDP to an MDP, so a stationary deterministic optimal policy exists for each weight vector. When a non-linear utility is used, the optimal policy may be non-stationary, depending on the current timestep or the reward accrued so far [108]. MCTS is able to represent such policies, so this is not in itself a restriction here.
- The scalarised expected returns and expected scalarised returns criteria coincide (Section 1.2), so the algorithms in this thesis can be applied under either criterion.

3
Related Work

Contents

3.1 Multi-Armed Bandits 55
3.2 Maximum Entropy Reinforcement Learning 57
3.3 Monte Carlo Tree Search 57
3.4 Multi-Objective Optimisation and Reinforcement Learning . . 59
3.5 Multi-Objective Monte Carlo Tree Search 61
3.5.1 Multi-Objective Monte Carlo Tree Search with Point Estimates . . 63

This chapter discusses related work on bandits, maximum entropy reinforcement learning, Monte Carlo Tree Search, and multi-objective sequential decision-making. The discussion is brief, and points to the most relevant references rather than giving a full treatment, as background material has already been covered in Chapter 2. Where applicable, references are flagged for use in later chapters.

3.1 Multi-Armed Bandits

Multi-armed bandits (MABs) and their variants are used for action selection throughout this thesis. This section only covers bandit results that are directly relevant to, or used in, later chapters.

Best-arm identification and sequential halving. While UCB targets cumulative regret, several MCTS variants in Section 3.3 target simple regret instead. In K-armed bandits, simple regret measures the suboptimality of a single recommended arm, rather than the total reward earned during exploration rounds. The most influential simple regret algorithm is the Sequential Halving procedure of Karnin et al. [46]. Given a fixed budget of arm pulls, Sequential Halving runs in $\lceil \log_2 K \rceil$ phases, samples the surviving arms uniformly within each phase, and discards the worse half at the end of each phase. The probability of returning the wrong arm decays exponentially in the budget, and the algorithm requires no parameters to be tuned.

Contextual bandits. Contextual bandits extend the standard MAB by giving the agent a context on each round, which can be used when selecting an arm. The contextual zooming algorithm of Kleinberg et al. [50], later refined in [95], assumes a similarity metric on the joint context-arm space, and exploits this structure to obtain regret bounds that adapt to the local geometry of the problem. Li et al. [56] take a different approach, and introduce LinUCB which assumes that the expected reward of each arm is a linear function of the context. The algorithm learns an estimate of the assumed linear function and uses a UCB-style confidence bound for exploration.

Continuous-armed bandits. For bandits with a continuous set of arms, the Hierarchical Optimistic Optimisation (HOO) algorithm [16, 18] maintains a tree-structured covering of the arm space, and assigns optimistic upper-confidence values to nodes. The partitioning is refined around regions of high reward to close in on a region containing the optimal arm.

Multi-objective bandits. Drugan and Nowe [29] generalise UCB to vector-valued rewards by applying an upper confidence bound across all dimensions of the reward vector, and selecting uniformly at random from the perturbed Pareto front. This algorithm, called Pareto-UCB, is shown to attain logarithmic *Pareto regret*. Drugan et al. [30] provide an empirical follow-up that compares several variants on

multi-objective Bernoulli benchmarks. Pareto-UCB is used by Chen and Liu [23] in an MOMCTS algorithm, and is used in Chapter 5 as a baseline search policy.

3.2 Maximum Entropy Reinforcement Learning

Maximum entropy reinforcement learning adds an entropy bonus on the policy to the standard expected return objective (Equation 2.37). Earlier uses of entropy regularisation in sequential decision-making include maximum-entropy inverse reinforcement learning [117, 116], and policy gradient methods that include an entropy regularisation for exploration and to avoid policies degenerating to deterministic ones [110, 63, 89].

In deep reinforcement learning, Haarnoja et al. [39] apply the maximum entropy objective to value-based deep learning methods. Haarnoja et al. [40] subsequently introduce Soft Actor-Critic (SAC), an off-policy actor-critic algorithm that has become a standard baseline for continuous control. SAC also introduces an automatic temperature-tuning rule, to help address the sensitivity of maximum entropy methods to the choice of α.

Lee et al. [55] replace the use of Shannon entropy with Tsallis entropy in the maximum entropy objective, and derive that optimal policies in this objective correspond to a *sparsemax* distribution. The *sparsemax* distribution is also a stochastic approximation to the max function, like the softmax, but it is less uniform in the output probabilities.

3.3 Monte Carlo Tree Search

A broad survey of Monte Carlo Tree Search prior to the deep learning era is given by Browne et al. [15], which catalogues UCT variants, enhancements such as rapid action value estimate and progressive widening, and applications across games and planning. Keller and Helmert [49] introduce the Trial-Based Heuristic Tree Search (THTS) framework, which unifies a number of online MDP planning algorithms (including UCT) into a single template (covered in Section 2.4.2). THTS

is also the framework in which several variants of UCT are first formalised. In particular, MaxUCT replaces the empirical-mean backups of UCT with full Bellman backups that take a maximum over child values. Closely related, but predating MCTS, are Trial-Based Real-Time Dynamic Programming [8], which interleaves Bellman backups with trial-based exploration of the state space, and its labelled variant LRTDP [14], which labels states as *solved* and truncates a trial early when it reaches a solved state.

Simple-regret MCTS. Several MCTS variants replace UCB with action-selection rules that target simple regret, with corresponding theoretical guarantees on the quality of the recommended action at the root node. Sequential Halving applied to Trees (SHOT) [21] applies the Sequential Halving procedure [46] recursively at every level of the tree. Hybrid MCTS (H-MCTS) [71] instead applies Sequential Halving only at the root, and uses UCT at internal nodes. The motivation is that simple regret guarantees are most needed at the root node. BRUE [32] (Best Recommendation with Uniform Exploration) achieves an exponential simple regret bound by choosing a *switching point* depth each trial: actions before that depth are selected uniformly at random to explore, actions after it are selected greedily to estimate values, and only the node at that depth is updated.

Maximum-entropy MCTS. Xiao et al. [113] introduce Maximum ENtropy Tree Search (MENTS), which uses the soft Bellman backup and Boltzmann action selection of maximum entropy RL in MCTS (Section 2.4.5). MENTS is the basis for the Boltzmann tree search algorithms developed in Chapter 4. Dam et al. [27] subsequently generalise MENTS by replacing the Shannon entropy regulariser with other entropy regularisers. RENTS is derived by using relative entropy (KL divergence) regulariser. TENTS is derived by using a Tsallis entropy regulariser similar to that of Lee et al. [55], and results in a sparsemax search policy.

3.4 Multi-Objective Optimisation and Reinforcement Learning

Multi-objective optimisation. Multi-objective optimisation is concerned with computing the Pareto front of a vector-valued objective. Steuer and Choo [97] reduces the problem to minimising the maximum weighted deviation from a utopian reference point, often referred to as the *Chebyshev scalarisation* or utility. Varying the weight vector traces out the entire Pareto front, including its non-convex regions. In the evolutionary setting, NSGA-II [28] is a fast genetic algorithm that combines non-dominated sorting with a crowding distance heuristic to maintain diversity along the front. The crowding distance estimator is used in Chapter 5 to prune points from convex hull value sets. In the multi-agent setting, Roijers et al. [84] introduce algorithms for computing the convex coverage set (CCS) of a multi-objective coordination graph. These include a variable-elimination procedure and an outer-loop linear-support method.

Optimisation criteria. In Section 2.6 the utility function is applied after taking the expectation of the return, which is referred to as the *scalarised expected returns* (SER) criterion. Roijers et al. [82] explain that this is appropriate when the agent is evaluated over many independent executions of the policy, and when the agent is evaluated on a single execution it is more appropriate to use the *expected scalarised returns* (ESR): $\max_\pi \mathbb{E}_\pi[u(\mathbf{G})]$, where $\mathbf{G}$ is the return of the policy. The two criteria coincide when u is linear, which is the setting considered in this thesis, but in general the two criteria yield different optimal policies. The majority of MORL algorithms in the literature target the SER criterion.

Planning algorithms. Planning approaches to MOMDPs predate the reinforcement learning literature. White [108] adapts dynamic programming to find Pareto-optimal policies for infinite-horizon discounted MOMDPs, but the resulting policies are non-stationary. Wiering and De Jong [109] subsequently address this with the CON-MODP algorithm, which uses a consistency operator to enforce

stationarity. Convex Hull Value Iteration [7] extends Bellman's value iteration by storing a convex hull value set at each state-action pair. CHVI is covered in more detail in Section 2.6.1. Lizotte et al. [57, 58] extend this approach by representing the value function in weight space rather than reward space, which gives piecewise-linear representations that scale more favourably with the number of states. The three-objective treatment of Lizotte et al. [58] relies on triangulation primitives from computational geometry, and the approach has been demonstrated mainly in two and three objectives.

Inner-loop methods. Inner-loop methods, which notably include Multi-Objective MCTS methods (Section 3.5), modify the underlying solver to operate directly on vector-valued (or set-valued) value functions, and compute the entire coverage set in a single pass. Van Moffaert and Nowé [103] propose Pareto Q-learning, which is a model-free inner-loop method that maintains, for each state-action pair, a set of non-dominated value vectors, with dominated entries pruned after each update. Castelletti et al. [20] extend Fitted Q-Iteration to the multi-objective case by augmenting the state with the linear scalarisation weights, so that a single training run learns a Q-function parameterised by the weight vector. Perny and Weng [77] apply the Chebyshev scalarisation of Steuer and Choo [97] to multi-objective Q-learning, which allows compromise solutions to be recovered under non-linear preferences.

Outer-loop methods. Outer-loop methods reduce the multi-objective problem to a sequence of single-objective problems, each obtained by fixing a scalarisation. The coverage set is then assembled from the single-objective solutions. The simplest version repeats a single-policy solver on a grid of weight vectors, but adaptive search procedures can identify the relevant weight vectors much more efficiently. Optimistic Linear Support [84] is the most widely used outer-loop framework: it iteratively selects the weight vector that maximises an optimistic upper bound on the value of the current CCS approximation, and adds the corresponding scalarised solution to the set. Outer-loop methods are also common in the (multi-objective) formal

verification literature, where [35] approximate the Pareto curve of a multi-objective probabilistic model-checking problem by repeatedly calling a single-objective value-iteration solver, as implemented in the PRISM model checker.

Single-policy methods. When the utility function and weight vector are fully known a priori, only a single policy needs to be returned. The simplest approach is to apply a scalarisation to the reward vector and solve the resulting single-objective MDP with a standard algorithm such as Q-learning. This is sound when the utility is linear, but for non-linear utilities the additivity of returns is broken, and the Bellman equation no longer holds [82]. For the ESR criterion, Roijers et al. [83] introduce the Expected Utility Policy Gradient, which uses Monte Carlo rollouts to estimate the utility of full-episode returns. Distributional MCTS [42, 44], discussed in Section 3.5, also targets ESR.

Deep MORL. Deep multi-objective reinforcement learning extends the algorithms above to high-dimensional state and action spaces using neural function approximators. Mossalam et al. [65] combine the Optimistic Linear Support framework of Roijers et al. [84] with deep Q-networks, and compute the CCS by solving a sequence of scalarised MDPs. Abels et al. [1] consider the setting where user preferences change at execution time, and propose a single-DQN architecture that conditions the value estimate on the current weight vector, together with a diverse experience replay buffer that stores transitions across weight vectors. Xu et al. [114] combine a multi-objective extension of PPO with an evolutionary algorithm to construct a continuous approximation of the Pareto front for continuous-control tasks.

3.5 Multi-Objective Monte Carlo Tree Search

Wang and Sebag [106] present the first extension of MCTS to Multi-Objective MCTS (MOMCTS). A global *Pareto archive* is maintained, with each tree node storing a point estimate of the vector-valued return. Action selection consists of perturbing value estimates at each node by a UCB-style confidence bound and

selecting the perturbed point that leads to the largest hypervolume when combined with the Pareto archive. Perez et al. [74, 75], Perez-Liebana et al. [76] develop and refine hypervolume-based MOMCTS by instead storing a Pareto front at each node, and using the hypervolume indicator directly in a UCB-style search policy.

Chen and Liu [23] apply the Pareto-UCB rule [29] (Section 3.1) for selecting actions, and, like Wang and Sebag [106], store a point estimate of the vector-valued return at each node, constructing a Pareto front over the actions. Section 3.5.1 discusses why storing a point estimate at each node is insufficient to reconstruct an optimal solution set.

Xu and Li [115] take a different approach, using the Chebyshev scalarisation of Steuer and Choo [97] to select actions during both planning and execution. The motivation is that the Chebyshev function, unlike linear scalarisation, can recover Pareto-optimal solutions in non-convex regions of the front. However, Sections 5.5.2 and 5.5.3 show that selecting actions locally under a Chebyshev utility is not guaranteed to yield a globally optimal policy, which is why this thesis assumes linear utilities.

All of the above MOMCTS algorithms target the SER criterion. Hayes et al. [42, 44] instead address the ESR criterion, and propose Distributional MCTS. At each chance node, Distributional MCTS maintains an approximate posterior distribution over the utility of the returns rather than an expectation, and uses Thompson sampling for action selection.

The MOMCTS algorithms discussed in this section are motivated by scenarios similar to that of Section 1.2: search is run as an offline planning phase, and a single policy is extracted at execution time, either in the *unknown weights scenario* (Section 2.6) or a close variant. In contrast to this thesis, they take a Pareto rather than a utility approach, aiming to compute Pareto fronts, and assume deterministic environments. They are also motivated largely empirically, rather than grounded in theory. Distributional MCTS is the exception: it targets the ESR criterion under a known utility, and so considers the known weights scenario.

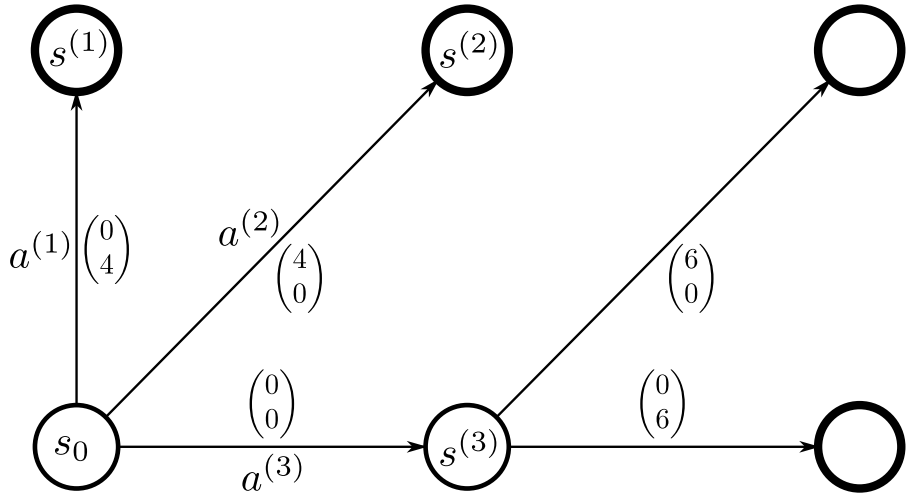


Figure 3.1: An example MOMDP to highlight why using point estimates at each node in MOMCTS is insufficient. Only the relevant labels and rewards are given. The optimal Pareto front at s_0 is $\{(6,0),(0,6)\}$, and both values are achieved by taking $a^{(3)}$. A single vector at $s^{(3)}$ cannot represent both, so $a^{(1)}$ or $a^{(2)}$ appears undominated at the root.

3.5.1 Multi-Objective Monte Carlo Tree Search with Point Estimates

Some prior work stores a point estimate (a single vector) at each node [106, 23]. To see why this is insufficient, consider the MOMDP in Figure 3.1. The optimal Pareto front at the initial state s_0 is $\{(6,0),(0,6)\}$, and both values are obtained by taking $a^{(3)}$. If each decision node stores a single vector, computed as a sample average, then the Q-values for $a^{(1)}$, $a^{(2)}$ and $a^{(3)}$ are $(0,4)$, $(4,0)$ and $(6\lambda, 6(1-\lambda))$ respectively, for some $\lambda \in [0,1]$. The computed Pareto front at the root is therefore drawn from $\{(0,4),(4,0),(6\lambda,6(1-\lambda))\}$, and whatever the value of λ, one of $(0,4)$ or $(4,0)$ remains undominated. The computed front is thus incorrect, and $a^{(1)}$ or $a^{(2)}$ may be taken at execution time. Note that $a^{(3)}$ is optimal at s_0 for every weight vector, so the failure arises because a single vector cannot represent both of the values achievable from $s^{(3)}$.

This counterexample relies on the values being updated by sample averages. More generally, and informally, a single vector cannot carry the information held by the set of values achievable under an undominated set of policies (Definition 2.6.7).

No matter how the vector is computed, there will be some MOMDP in which the Pareto front or convex hull obtained from it is suboptimal.

4

Monte Carlo Tree Search with Boltzmann Exploration

Contents

4.1 Introduction and Motivation 67
4.1.1 UCT 68
4.1.2 MENTS 70
4.1.3 Simple Regret and Consistency 72
4.2 Boltzmann Search 73
4.2.1 Boltzmann Tree Search 75
4.2.2 Decaying ENtropy Tree Search 79
4.2.3 Stochastic Search Policies 81
4.3 Theoretical MDPs 87
4.3.1 The D-Chain Problem 87
4.3.2 The Entropy Trap 89
4.4 Empirical Results 92
4.4.1 Environments 93
4.4.2 Evaluation Procedure 95
4.4.3 Results and Discussion 97
4.5 Theoretical Results 107
4.5.1 Notation 108
4.5.2 Preliminaries 110
4.5.3 Consistency Results 113
4.5.4 The Entropy Trap 121
4.6 Summary 125

This chapter considers MCTS algorithms for planning in single-objective environ-

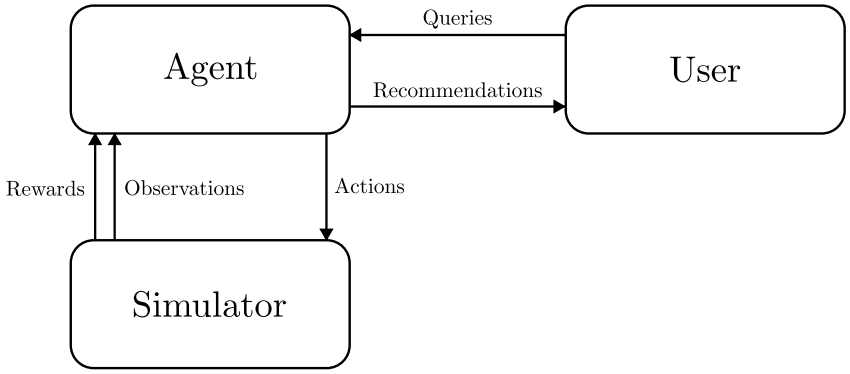


Figure 4.1: Exploration setting in reinforcement learning (edited from Figure 2.5), where the agent is only assessed on the recommendations that it provides, and is not penalised for considering poor actions in the simulator. This setting therefore places greater emphasis on exploration than exploitation.

ments, with the optional use of entropy as a second objective for exploration. The discussion focuses on the exploration setting for reinforcement learning (see Section 2.3 and Figure 4.1), where the agent is assessed purely on the recommendations it makes, rather than what actions it explored in simulation.

In maximum entropy reinforcement learning (Section 2.3.1), entropy is added directly to the expected return, resulting in a regularised objective function. In contrast, the preceding work in maximum entropy inverse reinforcement learning — which aims to learn a reward function or imitate a policy from a dataset of trajectories sampled from an expert — uses the principle of maximum entropy to disambiguate between multiple reward functions and policies that could replicate the expert trajectories [117, 116].

While the maximum entropy objective (Definition 2.3.7) yields a unique optimal policy, it is only guaranteed to be optimal for the standard objective (Definition 2.3.3) for a sufficiently small temperature parameter α [39, 70]. This motivates the investigation of **Q1.1 - Entropy** in this chapter, where it is further demonstrated that the temperature parameter may have to be set to a prohibitively small value for the objectives to align, negating the benefit of entropy-based exploration. Additionally, this chapter lays the foundations for **Q1.2 - Multi-Objective Exploration** in Chapter 6.

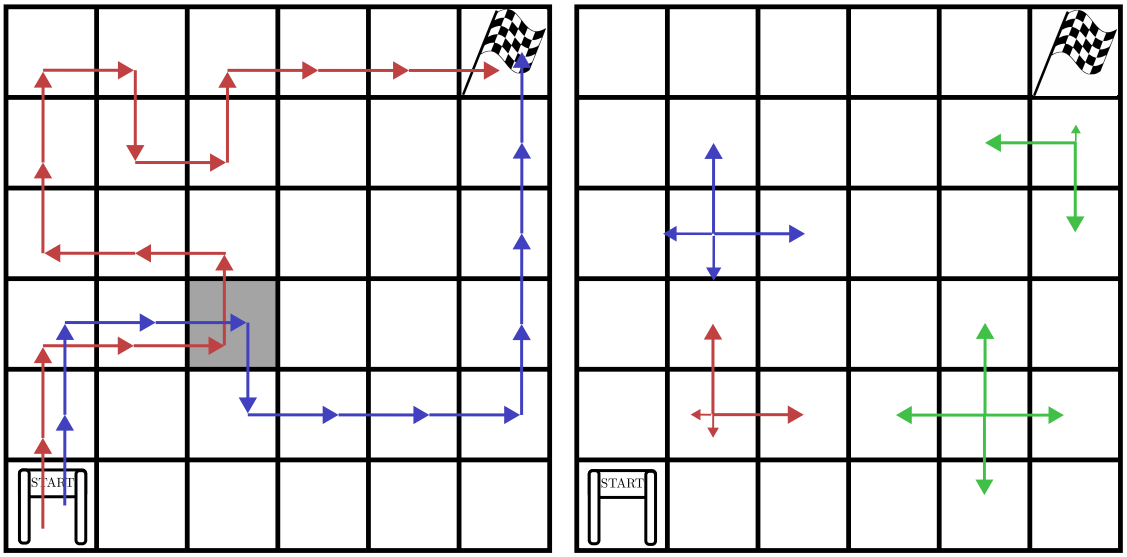
The remainder of this chapter is organised as follows:

- Section 4.1 discusses limitations of existing MCTS algorithms in the exploration setting of reinforcement learning. *Simple regret* is defined in the context of sequential decision-making, to theoretically analyse the performance of MCTS algorithms in the exploration setting.
- Section 4.2 defines the *Boltzmann Tree Search* (BTS) and *Decaying ENtropy Tree Search* (DENTS) algorithms.
- Section 4.2.3 discusses some properties of using a stochastic search policy in MCTS, and examines how the *Alias Method* can be used to address **Q3.1 - Complexity**.
- Section 4.3 introduces two theoretical MDPs, which are used to empirically demonstrate the limitations of the existing MCTS algorithms.
- Section 4.4 presents experimental results on gridworld environments and the game of Go, and gives an empirical answer to **Q1.1 - Entropy**.
- Section 4.5 presents the main theoretical analysis and proofs. Convergence properties of MENTS, BTS and DENTS are proven to provide a theoretical answer to **Q1.1 - Entropy**.

4.1 Introduction and Motivation

UCT has been used with great success, but there are environments in which it performs poorly. This section considers a gridworld shortest path problem, which is used to discuss the behaviour of UCT and MENTS in the exploration setting, and highlight some limitations of these algorithms.

In this gridworld, the agent starts at the origin $(0, 0)$, and must reach the goal state at $(6, 6)$. Each timestep, the agent may move deterministically in any cardinal direction, $\{\texttt{UP}, \texttt{RIGHT}, \texttt{DOWN}, \texttt{LEFT}\}$, as long as it remains in the grid. Each action incurs a cost of 1 (or a reward of -1), so the cumulative cost of a path is equal to its length. The time horizon is chosen to be 36 (six times the width of the grid) so that the horizon is long enough for exploration. The optimal policy will always select



(a) Some possible paths from start to finish.

(b) Optimal policy with the maximum entropy objective for varying temperatures.

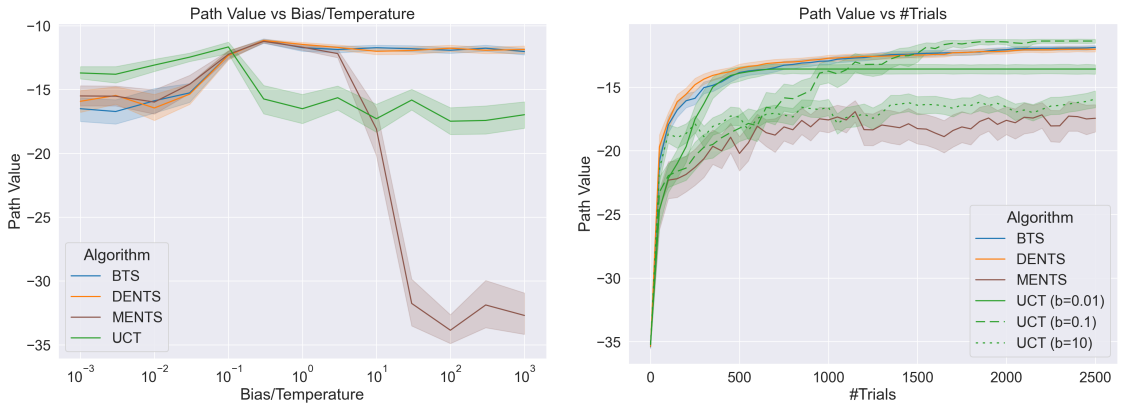
Figure 4.2: An example gridworld shortest path problem. (a) Shows two paths that UCT may have explored, where the blue path is shorter, but the red path takes one of the optimal actions at the (shaded) state $(2, 2)$. (b) Illustrates optimal probability distributions at four states in the maximum entropy objective, with varying temperatures. In red, the temperature is low and in blue the temperature is larger. In green, the temperature is large enough such that gathering entropy "reward" outweighs reaching the goal.

from the actions `UP` and `RIGHT`. The problem, with example paths, is illustrated in Figure 4.2. For reference, a uniformly random policy achieves a value of -35.1.

While UCT and MENTS can easily solve this problem given enough trials and appropriately set parameters, it is used with a limited number of trials to illustrate the behaviour of the algorithms in the exploration setting. In Section 4.3, MDPs are constructed adversarially to expose limitations of UCT and MENTS, and Section 4.4 considers more complex gridworlds — including terminal states, sparse rewards and stochastic transition distributions — that are harder to solve in practice.

4.1.1 UCT

The UCT algorithm (Section 2.4.3) is designed in the traditional reinforcement learning setting described in Section 2.3, where the UCT agent aims to minimise the cumulative regret. Thus UCT makes a trade-off between exploration and exploitation during its trials, and frequently chooses the same action to exploit,



(a) Reward of best path, after 2500 trials, for a range of bias/temperature parameters.

(b) Reward of best path relative to the number of trials. ($\alpha = 10$ and $b_{\text{PUCT}} = 0.01, 0.1, 10$).

Figure 4.3: Results on the example gridworld problem for a variety of bias and temperature parameters. For comparison, the Boltzmann Tree Search (BTS) and Decaying ENtropy Tree Search (DENTS) algorithms defined in Section 4.2 are also shown. (a) Each algorithm is run for 2500 trials for a range of bias and temperature parameters, and (b) shows the reward of the recommended best path as a function of the number of trials. In both plots each data point is averaged over 25 repeated experiments. In (a), UCT explores little for small bias parameters, and for larger ones is still exploring at this budget. Taking three of these bias parameters, (b) shows the same behaviour over trials: at $b = 0.01$ UCT converges quickly to a suboptimal solution, while at $b = 10$ progress is slow, as many actions are still chosen to exploit rather than explore. For MENTS, (a) shows that the *misalignment threshold* (Definition 4.1.1) is at most $\alpha = 10$. At $\alpha = 10$, (b) shows that MENTS consistently reaches the goal but prioritises entropy enough to take a scenic route, placing it in the blue case of Figure 4.2b; for reference, a random policy has an expected value of -35.1.

which can result in it getting stuck in local optima, particularly when rewards are sparse or not informative.

In many domains UCT does provide a good trade-off between exploration and exploitation. These tend to be domains where rewards are dense and informative, so that the best action acquires the best value estimate early in the search. Exploiting that estimate is then effective local exploration, concentrating trials on the later timesteps of the paths that currently look most promising. UCT struggles when the optimal action instead retains a worse value estimate throughout the initial exploration, because it then overcommits to a suboptimal action. Sparse or uninformative rewards are one such case, where the optimal action may appear worse by chance early on. A heuristic learned by function approximation that

assigns a low value to the optimal action is another. The remainder of this section considers the sparse and uninformative case.

A *sparse reward* is where the reward signal is infrequent, that is, most actions give a reward of zero. In the shortest path example, the reward is dense but uninformative, as the immediate cost of taking any action is the same.

Suppose that UCT has been run for many trials, and the best paths found by UCT so far are shown in red and blue in Figure 4.2a. The two paths coincide until the shaded state at $(2, 2)$, where UCT has the Q-value estimates $\bar{Q}_{\text{UCT}}((2, 2), \texttt{DOWN}) \approx -8$ and $\bar{Q}_{\text{UCT}}((2, 2), \texttt{UP}) \approx -12$. When a UCT trial reaches the shaded state at $(2, 2)$, it will frequently select the action `DOWN` over `UP` to exploit the best path it has found so far, rather than exploring the optimal actions `UP` and `RIGHT`.

In theory, UCT selects every action infinitely often (that is, $N(s, a) \to \infty$ for all reachable s, a) and will converge to the optimal values and policy. In practice, UCT shifts its focus from exploration to exploitation throughout the search, which is determined by the confidence intervals and the bias parameter b_{UCT} from Equation (2.45). This is illustrated in Figure 4.3, where for small bias parameters UCT quickly exploits a suboptimal solution and for large bias parameters UCT is still exploring and its value estimates have not yet converged.

Although UCT had to be limited to a small number of trials in this initial example, the same behaviour is visible at scale in the empirical results of Section 4.4. The results on DeterministicGridWorld in Figure 4.12 show UCT settling on a suboptimal path across all map sizes, with its performance flattening after the initial trials rather than continuing to improve. Its concentration of trials therefore acts as local search around an early estimate, and does not recover the optimal path within the planning budget.

4.1.2 MENTS

One can argue that when using the maximum entropy objective (Definition 2.3.7), it is possible to recover the standard objective (Definition 2.3.3) by setting the

temperature parameter to zero, or an infinitesimally small value. However, in practice, setting α to a tiny value will nullify the exploration advantages of using entropy. Considering the other extreme, when a very large temperature is used, entropy becomes the dominant objective in the maximum entropy objective, and the optimal soft policy will be (almost) uniform.

The optimal soft policy for a large temperature may be different from the optimal policy for the standard objective. In cases such as this, it can be said that the maximum entropy objective is *misaligned* with the standard objective.

Definition 4.1.1. *The policy induced by the optimal soft Q-values is* $\psi^*_{\text{sft}}(s) = \arg\max_{a'} Q^*_{\text{sft}}(s, a')$. *The maximum entropy objective is* misaligned *when the policy* $\psi^*_{\text{sft}}(s) \neq \pi^*(s)$, *where* π^* *is the optimal standard policy (Definition 2.3.4). The* misalignment threshold *is the smallest temperature parameter at which the maximum entropy objective is misaligned.*

In Figure 4.2b, the optimal soft policy for the maximum entropy objective is shown for varying temperature parameters. For small temperature values ($\alpha < 1$, illustrated in red), the optimal soft policy is close to the optimal standard policy, and for large temperature values ($\alpha > 100$, illustrated in green) the optimal soft policy is nearly uniform, except at states adjacent to the goal, where the agent will avoid reaching the goal to obtain more entropy "reward". At intermediate temperatures (illustrated in blue), the agent still reaches the goal, but prioritises entropy enough that, with high probability, it takes a longer route than necessary.

When using the maximum entropy objective, the temperature parameter often needs to be carefully tuned, large enough to gain an exploration benefit, but small enough to avoid misalignment issues. In this example problem, the misalignment threshold is at most $\alpha = 10$, as can be seen in Figure 4.3a. Moreover, because the entropy of a policy depends on the MDP, the temperature typically needs to be tuned for each MDP. In Section 4.3, an MDP is constructed where the misalignment threshold is prohibitively small for the maximum entropy objective to be effective, and Section 4.4 considers more complex gridworlds where the misalignment threshold is low enough to significantly affect performance.

Input: An MDP $\mathcal{M}$.

- For each round $m = 1, 2, \ldots$:
  1. the agent produces a search policy π^m to follow;
  2. the environment samples a trajectory $\tau \sim \pi^m$ (including rewards $r_t = R(s_t, a_t)$ for each s_t, a_t pair in τ);
  3. the agent produces a recommendation policy ψ^m;
  4. if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 4.4: The procedure of an exploration planning problem for MDPs, where ψ^m is the recommendation policy the agent produces after m trajectories are sampled using the exploration policy π^m.

4.1.3 Simple Regret and Consistency

This section generalises the concept of simple regret (Definition 2.1.2) to MDPs and introduces the concept of consistency for MCTS algorithms. *Simple regret* is used to analyse the algorithms introduced in this chapter. *Consistency* refers to the correctness of an MCTS algorithm in the limit — informally, a consistent algorithm will eventually find the optimal policy given enough trials. The term originates from the MCTS literature, to describe whether modifications to UCT preserve its asymptotic behaviour [10, 15, 5].

The *simple regret* of a policy is the difference between the value of the policy and optimal value of the policy (Equation (4.1)). Whilst *cumulative regret* is used to analyse algorithms in the standard reinforcement learning setting, simple regret is more appropriate for the exploration setting, where the agent is assessed only on its recommendations and not penalised for exploring poor actions in simulation.

Definition 4.1.2. *The* simple regret *of a policy ψ at state $s \in \mathcal{S}$ is the difference between the value of the policy and the optimal value at that state:*

$$\texttt{sim_regr}(s, \psi) = V^*(s) - V^{\psi}(s). \tag{4.1}$$

By definition, the optimal policy achieves a simple regret of zero. In this thesis an MCTS algorithm is considered *consistent* if its expected simple regret tends to

zero. Informally, a consistent algorithm will eventually output an optimal policy if run indefinitely, and is expected to improve its recommendation with more trials.

Definition 4.1.3. *An agent, that produces recommendation policies $\psi^1, \psi^2, \ldots$ is said to be* consistent *if $\mathbb{E}[\texttt{sim_regr}(s, \psi^m)] \to 0$ as $m \to \infty$.*

Note that the simple regret (Equation (4.1)) is a random variable, as it depends on the recommendation policy ψ, which itself depends on the random trajectories that are sampled. Hence the expected simple regret is used in Definition 4.1.3 to define consistency, and is the main value of interest in the theoretical analysis in Section 4.5.

Bubeck et al. [17] show that upper bounds on cumulative regret lead to both upper and lower bounds on simple regret. Returning to the discussion around UCT and MENTS, using this result, UCT is consistent with respect to Definition 4.1.3 because its upper bound on cumulative regret [52] leads to an upper bound on simple regret that tends to zero. However, MENTS is only consistent for a sufficiently small temperature parameter that depends on the MDP [70].

4.2 Boltzmann Search

This section presents Boltzmann Tree Search (BTS) and Decaying ENtropy Tree Search (DENTS). Following the discussion from Section 4.1, the design aims of these algorithms are as follows:

- to be as simple to define and implement as UCT and MENTS[1];
- to be able to explore effectively when rewards are sparse or uninformative;
- to be able to exploit when necessary, in large or stochastic domains;
- to be consistent using parameters that do not depend on the MDP.

[1] i.e. each decision node operates on a multi-armed bandit problem

These aims are motivated by the exploration setting of reinforcement learning in practice, that is, with a finite planning budget: the agent plans in a simulator, and is evaluated only on the recommendation it makes once that budget is exhausted (Section 2.3 and Figure 4.1).

Simple regret plays a different role: it is used to analyse the algorithms, not to motivate or design them. It gives theoretical guarantees about the behaviour of BTS and DENTS in the limit (Section 4.5), whereas an algorithm built solely to minimise simple regret need not perform well in practice.

The first aim is included so that the algorithms introduced here are easy to adopt in existing MCTS frameworks. The second is satisfied by MENTS, whose entropy bonus provides exploration even when rewards are sparse or uninformative, but not by UCT, which tends to exploit (Figure 4.3, and the gridworld results in Section 4.4.3). The third is satisfied by UCT, and by MENTS only when its temperature is small. The fourth is satisfied by UCT, but not by MENTS, whose consistency requires a temperature that cannot be chosen independently of the MDP [70].

Exploitation is included as an aim for two reasons. First, it concentrates exploration on the later timesteps of each trial, which becomes important in larger environments. Second, in the offline planning setting considered in this thesis (Section 1.2), the recommended policy is read from the search tree (Equation (4.36)), and any state outside the tree is handled by selecting actions uniformly at random. A broad but shallow tree therefore yields a policy that soon leaves the tree and acts uniformly, so in stochastic domains exploitation helps to ensure that the different outcomes of the same actions have been explored.

The FrozenLake environment (Section 4.4.1.1) is an example of a stochastic environment, where exploitation is beneficial for performance in practice (Section 4.4.3.1).

In Section 4.2.1 BTS is defined, which uses a Boltzmann search policy, similar to MENTS, but using the standard objective (Definition 2.3.3) and Bellman backups rather than the soft Bellman backups used in MENTS. Section 4.2.2 defines DENTS

which re-introduces entropy as an exploration bonus while maintaining consistency. Section 4.2.3 discusses the use of stochastic search policies in MCTS further, highlighting that mixture policies can be used to incorporate prior knowledge from $\pi_{\text{heuristic}}$ (Section 4.2.3.1) and how the *Alias Method* can be used to arrive at a more efficient MCTS algorithm addressing **Q3.1 - Complexity** (Section 4.2.3.3).

4.2.1 Boltzmann Tree Search

Boltzmann search has been used previously in heuristic search algorithms such as RTDP [8] and THTS [47]. This section presents *Boltzmann Tree Search* (BTS), which uses a Boltzmann search policy with decayed temperature similar to RTDP, but includes an additional uniform exploration term similar to MENTS [113].

BTS therefore has two exploration mechanisms: the Boltzmann search policy, controlled by the temperature α_{BTS}, and the uniform term, controlled by ϵ_{BTS}. They explore in different ways. Boltzmann exploration is directed, in that it is centred on the current Q-value estimates and so favours actions that are currently competitive. Uniform exploration is undirected, and continues to sample actions that the current estimates rate poorly. Retaining both allows the balance between them to be tuned for the domain at hand.

The Boltzmann temperature is typically decayed towards zero, so that the search policy can exploit and eventually select greedily (Theorem 4.2.2). There are two reasons for keeping the uniform exploration term from MENTS. The first is that it continues to sample all actions once the Boltzmann term has stopped exploring, which guarantees that an optimal policy is recommended in the limit (Theorem 4.2.1). The second is that it makes ϵ-greedy search policies available as a limiting case.

There are likewise two reasons for keeping the Boltzmann term. In practice, it is often useful to concentrate search on the actions that already appear competitive, rather than spreading trials evenly across all actions. In theory, holding the temperature above a positive lower bound gives a faster rate of convergence (Theorem 4.2.3).

Both exploration methods are sufficient to guarantee that an optimal policy is eventually recommended. In the experimental Section 4.4 the Boltzmann temperature is decayed so that the search policy can tend towards a greedy policy for exploitation if necessary, and the uniform exploration is kept so that actions are still explored when the Boltzmann policy is greedy.

Algorithm Definition The BTS algorithm uses the variables $\hat{V}_{\text{BTS}}$ and $\hat{Q}_{\text{BTS}}$ for value estimates, $N(s)$ and $N(s,a)$ for visit counts, and the function $\alpha_{\text{BTS}}(x) \geq 0$ for the search temperature. The temperature used at each decision node s is computed with respect to the visit count $N(s)$ as $\alpha_{\text{BTS}}(N(s))$.

The BTS search policy uses the flag `normalised_Q` to optionally normalise Q-values, and incorporates prior knowledge from $\hat{Q}_{\text{heuristic}}$ weighted by $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$ pseudo-visits:

$$\hat{Q}^{\text{bias}}_{\text{BTS}}(s,a) = \frac{N(s,a)\hat{Q}_{\text{BTS}}(s,a) + N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{4.2}$$

$$\hat{Q}^{\text{norm}}_{\text{BTS}}(s,a) = \begin{cases} \frac{\hat{Q}^{\text{bias}}_{\text{BTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{bias}}_{\text{BTS}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}^{\text{bias}}_{\text{BTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{bias}}_{\text{BTS}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}^{\text{bias}}_{\text{BTS}}(s,a) & \text{otherwise.} \end{cases} \tag{4.3}$$

The search policy of BTS is similar to MENTS (Section 2.4.5), but using a variable temperature and (the normalised and biased) Bellman value estimates rather than soft value estimates.

$$\pi_{\text{BTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{BTS}}))\rho_{\text{BTS}}(a|s) + \frac{\lambda(s,\epsilon_{\text{BTS}})}{|\mathcal{A}|}, \tag{4.4}$$

$$\rho_{\text{BTS}}(a|s) \propto \exp\left(\frac{\hat{Q}^{\text{norm}}_{\text{BTS}}(s,a)}{\alpha_{\text{BTS}}(N(s))}\right), \tag{4.5}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right), \tag{4.6}$$

where $\epsilon_{\text{BTS}} \in [0,\infty)$ is an exploration parameter, and a temperature of $\alpha_{\text{BTS}}(N(s)) = 0$ is taken to mean that ρ_{BTS} selects greedily with respect to $\hat{Q}^{\text{norm}}_{\text{BTS}}$.

For any new decision nodes s added to the tree, the `init_v` function initialises the visit count to $N(s) = 0$ and the value estimate to $\hat{V}_{\text{BTS}}(s) = \hat{V}_{\text{heuristic}}(s)$. For

any new chance nodes (s, a) added to the tree, the `init_q` function initialises the visit count to $N(s, a) = 0$ and the Q-value estimate to $\hat{Q}_{\text{BTS}}(s, a) = \hat{Q}_{\text{heuristic}}(s, a)$.

BTS also allows the value estimates to be biased by $\hat{V}_{\text{heuristic}}$ with a weighting of $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ pseudo-visits. When `single_node_expansion` is True, $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ is typically set to one, weighting the heuristic value estimate as a single pseudo-trial. When `single_node_expansion` is False, $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ is set to zero, as the trajectory already captures the rollout information.

Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{BTS}}$ be the trajectory sampled during the selection phase. The `backup_v` and `backup_q` functions then update the value estimates for $t = h - 1, \ldots, 0$:

$$\hat{Q}_{\text{BTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{BTS}}(s') \right), \tag{4.7}$$

$$\hat{V}_{\text{BTS}}(s_t) \leftarrow \frac{N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}} \hat{V}_{\text{heuristic}}(s_t) + N(s_t) \max_{a \in \mathcal{A}} \hat{Q}_{\text{BTS}}(s_t, a)}{N(s_t) + N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}}. \tag{4.8}$$

When BTS needs to recommend a policy, it can use its Q-value estimates, or the visit counts:

$$\psi_{\text{BTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{BTS}}(s, a), \tag{4.9}$$

$$\texttt{mv}_{\text{BTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{4.10}$$

Average Returns The BTS search policy can still be used with average returns. In *Boltzmann Tree Search with Average Returns* (AR-BTS) the Bellman value estimates are replaced with average returns $\bar{V}_{\text{AR-BTS}}$ and $\bar{Q}_{\text{AR-BTS}}$. Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$ the average returns are updated for $t = h - 1, \ldots, 0$:

$$\bar{Q}_{\text{AR-BTS}}(s_t, a_t) \leftarrow \frac{N(s_t, a_t) - 1}{N(s_t, a_t)} \left(\bar{Q}_{\text{AR-BTS}}(s_t, a_t) + \frac{\hat{V}_{\text{h}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t, a_t) - 1} \right), \tag{4.11}$$

$$\bar{V}_{\text{AR-BTS}}(s_t) \leftarrow \frac{N(s_t) - 1}{N(s_t)} \left(\bar{V}_{\text{AR-BTS}}(s_t) + \frac{\hat{V}_{\text{h}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t) - 1} \right), \tag{4.12}$$

where $\hat{V}_{\text{h}}$ is a shorthand for $\hat{V}_{\text{heuristic}}$.

The corresponding definitions of $\bar{Q}^{\text{norm}}_{\text{AR-BTS}}$, $\pi_{\text{AR-BTS}}$, $\psi_{\text{AR-BTS}}$, $\texttt{mv}_{\text{BTS}}$ and so on are similar to the definitions for BTS and are omitted for brevity. The complete equations for all algorithms can be found in Appendix A.1.

Summary of Theoretical Results In Section 4.5 it is shown that the BTS recommendation policy will eventually converge to the optimal policy.

Theorem 4.2.1. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{BTS}} > 0$, the expected simple regret of ψ_{BTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \to 0$.*

Additionally, if the temperature function is allowed to tend to zero, then the search policy will also converge to the optimal policy, and the most visited recommendation policy will also converge to the optimal policy as a result.

Theorem 4.2.2. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{BTS}} > 0$ and $\alpha_{\text{BTS}}(x) \to 0$, then $\pi_{\text{BTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{BTS}})] \to 0$.*

Conversely, if the temperature has a strictly positive lower bound (and does not converge to zero), then the BTS recommendation policy will converge to the optimal policy exponentially fast. However, note that neither the search policy nor the most-visited recommendation policy is guaranteed to converge to the optimal policy in this case.

Theorem 4.2.3. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{BTS}}(x) \geq L > 0$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \leq C \exp(-kn)$ after running n trials of BTS.*

For AR-BTS, the temperature function always needs to be restricted to functions that tend to zero. Intuitively, the average return estimates converge only if the search policy converges, and the search policy converges only if the temperature tends to zero.

Theorem 4.2.4. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-BTS}} > 0$ and $\alpha_{\text{AR-BTS}}(x) \to 0$, then $\pi_{\text{AR-BTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-BTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-BTS}})] \to 0$.*

4.2.2 Decaying ENtropy Tree Search

Algorithm Definition *Decaying ENtropy Tree Search* (DENTS) extends the BTS algorithm by adding *entropy estimates* as an exploration bonus. DENTS uses Bellman value estimates $\hat{V}_{\text{DNTS}}$ and $\hat{Q}_{\text{DNTS}}$ similarly to BTS, but also uses entropy value estimates, $\bar{\mathcal{H}}_{V,\text{DNTS}}$ and $\bar{\mathcal{H}}_{Q,\text{DNTS}}$, which are Monte Carlo estimates of the entropy of the search policy rooted from the relevant node. By keeping a separate estimate for entropy, DENTS is able to use entropy in its search policy, and discard it for recommendations.

DENTS uses parameters that correspond to the BTS parameters: `normalised_Q`, $N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$, $\alpha_{\text{DNTS}}(x) \geq 0$ and ϵ_{DNTS}. Additionally, the entropy terms are weighted by a non-negative function $\beta_{\text{DNTS}}(x) \geq 0$ during the selection phase. Typically, β_{DNTS} is chosen such that $\beta_{\text{DNTS}}(x) \to 0$ as $x \to \infty$, so that the entropy weighting diminishes over time, allowing deeper parts of the tree to be explored[2].

The Q-values used during search are computed as follows:

$$\hat{Q}^{\text{sft}}_{\text{DNTS}}(s,a) = \hat{Q}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a), \tag{4.13}$$

$$\hat{Q}^{\text{sft,bias}}_{\text{DNTS}}(s,a) = \frac{N(s,a)\hat{Q}^{\text{sft}}_{\text{DNTS}}(s,a) + N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{4.14}$$

$$\hat{Q}^{\text{sft,norm}}_{\text{DNTS}}(s,a) = \begin{cases} \frac{\hat{Q}^{\text{sft,bias}}_{\text{DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{DNTS}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{DNTS}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}^{\text{sft,bias}}_{\text{DNTS}}(s,a) & \text{otherwise.} \end{cases} \tag{4.15}$$

The DENTS search policy is similar to BTS:

$$\pi_{\text{DNTS}}(a|s) = (1 - \lambda(s,\epsilon_{\text{DNTS}}))\rho_{\text{DNTS}}(a|s) + \frac{\lambda(s,\epsilon_{\text{DNTS}})}{|\mathcal{A}|}, \tag{4.16}$$

$$\rho_{\text{DNTS}}(a|s) \propto \exp\left(\frac{\hat{Q}^{\text{sft,norm}}_{\text{DNTS}}(s,a)}{\alpha_{\text{DNTS}}(N(s))}\right), \tag{4.17}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{4.18}$$

[2]In practice, it is also advisable to choose β_{DNTS} such that $\frac{\beta_{\text{DNTS}}(x)}{\alpha_{\text{DNTS}}(x)} \to 0$, so that the entropy weighting decays faster than the temperature. Otherwise, even as the temperature vanishes, the entropy bonus may still dominate the Q-values in the Boltzmann exponent, skewing the search policy away from the optimal action.

where $\epsilon_{\text{DNTS}} \in [0, \infty)$ is an exploration parameter, and a temperature of $\alpha_{\text{DNTS}}(N(s)) = 0$ is taken to mean that ρ_{DNTS} selects greedily with respect to $\hat{Q}^{\text{sft,norm}}_{\text{DNTS}}$.

The `init_v` and `init_q` functions are also similar to BTS, setting: $N(s) = 0$, $N(s,a) = 0$, $\hat{V}_{\text{DNTS}}(s) = \hat{V}_{\text{heuristic}}(s)$ and $\hat{Q}_{\text{DNTS}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$. Additionally, the entropy value estimates are initialised to zero: $\bar{\mathcal{H}}_{V,\text{DNTS}}(s) = 0$ and $\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a) = 0$.

Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{DNTS}}$, the `backup_q` and `backup_v` functions update the entropy value estimates for $t = h-1, \ldots, 0$:

$$\bar{\mathcal{H}}_{Q,\text{DNTS}}(s_t, a_t) \leftarrow \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \bar{\mathcal{H}}_{V,\text{DNTS}}(s'), \tag{4.19}$$

$$\bar{\mathcal{H}}_{V,\text{DNTS}}(s_t) \leftarrow \mathcal{H}(\pi_{\text{DNTS}}(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi_{\text{DNTS}}(a|s_t) \bar{\mathcal{H}}_{Q,\text{DNTS}}(s_t, a), \tag{4.20}$$

where $\mathcal{H}$ is the Shannon entropy function. The Bellman value estimates are updated similarly to BTS, for $t = h-1, \ldots, 0$:

$$\hat{Q}_{\text{DNTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{DNTS}}(s') \right), \tag{4.21}$$

$$\hat{V}_{\text{DNTS}}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}_{\text{DNTS}} \hat{V}_{\text{heuristic}}(s_t) + N(s_t) \max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}(s_t, a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{DNTS}}}. \tag{4.22}$$

The recommendation policies for DENTS are also defined identically to BTS:

$$\psi_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}(s, a), \tag{4.23}$$

$$\texttt{mv}_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a), \tag{4.24}$$

Average Returns In *Decaying ENtropy Tree Search with Average Returns* (AR-DENTS) the Bellman value estimates are replaced with average returns $\bar{V}_{\text{AR-DNTS}}$ and $\bar{Q}_{\text{AR-DNTS}}$. For brevity and to avoid repetition, the equations are omitted but can be found in Appendix A.1.

Summary of Theoretical Results Corresponding theoretical results hold for DENTS and AR-DENTS to those stated for BTS. It is additionally required that $\beta_{\text{DNTS}} \to 0$, so that the entropy bonus vanishes and the search policy is not skewed away from the optimal action. Proofs of the theorems below are given in Section 4.5.

Theorem 4.2.5. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, the expected simple regret of ψ_{DNTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \to 0$.*

Theorem 4.2.6. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$, then $\pi_{\text{DNTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{DNTS}})] \to 0$.*

Theorem 4.2.7. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{DNTS}}(x) \geq L > 0$ and $U \geq \beta_{\text{DNTS}}(x)$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \leq C \exp(-kn)$ after running n trials of DENTS.*

Theorem 4.2.8. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-DNTS}} > 0$, $\alpha_{\text{AR-DNTS}}(x) \to 0$ and $\beta_{\text{AR-DNTS}}(x) \to 0$, then $\pi_{\text{AR-DNTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-DNTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-DNTS}})] \to 0$.*

4.2.3 Stochastic Search Policies: Prior Knowledge and Computational Efficiency

This subsection discusses two properties of using a stochastic search policy in MCTS. First, the prior policy $\pi_{\text{heuristic}}$ can be incorporated into the search policy as a mixture policy, allowing prior knowledge to guide the selection phase. Second, the *Alias Method* [105, 104] is used to derive a more efficient MCTS algorithm by trading off policy accuracy for computational speed, providing an answer to **Q3.1 - Complexity**.

4.2.3.1 Incorporating Prior Knowledge Through Mixture Policies

In MENTS, BTS and DENTS, a uniform distribution is mixed with the Boltzmann distribution to form the search policy, using a logarithmically decaying weight that gradually transitions towards the Boltzmann distribution as the number of visits increases. The prior policy $\pi_{\text{heuristic}}$ can be incorporated in the same way: by mixing it into the algorithm's search policy π_{ALG} with a decaying weight, prior knowledge guides exploration early in the search while leaving the asymptotic behaviour unchanged.

The mixture policy π_{mix} is defined by:

$$\pi_{\text{mix}}(a|s) = (1 - \lambda(s, \epsilon_{\text{mix}}))\pi_{\text{ALG}}(a|s) + \lambda(s, \epsilon_{\text{mix}})\pi_{\text{heuristic}}(a|s), \tag{4.25}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{4.26}$$

where $\epsilon_{\text{mix}} \in [0, \infty)$ controls the weight given to the prior policy in the mixture. The weight given to $\pi_{\text{heuristic}}$ decays with the visit count, eventually reaching zero, so that the behaviour of the algorithm is unchanged in the limit.

4.2.3.2 The Alias Method

This subsection briefly covers the *Alias Method* [105, 104], which is used to sample from a categorical distribution in constant time, with a linear cost for preprocessing.

Let $f : \{C_1, \ldots, C_m\} \to [0, 1]$ be the probability mass function of a categorical distribution with m categories. Suppose a sample $c \sim f$ is required.

The naive method to sample from f takes $O(m)$ time, where a continuous uniform random value is sampled, $u \sim \text{Uniform}(0, 1)$, and f is linearly scanned until a probability mass of u has been passed.

The Alias Method, however, uses an $O(m)$ pre-processing step to construct an *alias table*, which is a table consisting of m entries, each accounting for $\frac{1}{m}$ of the probability mass of f. Each entry takes the form $(c_0, c_1, \texttt{thrsh})$, where $c_0, c_1 \in \{C_1, \ldots, C_m\}$ are categories and $\texttt{thrsh} \in [0, 1]$ gives the ratio of the $\frac{1}{m}$ probability mass to assign to c_0 and c_1. That is, a mass of $\frac{\texttt{thrsh}}{m}$ is assigned to c_0 and $\frac{1-\texttt{thrsh}}{m}$ is assigned to c_1.

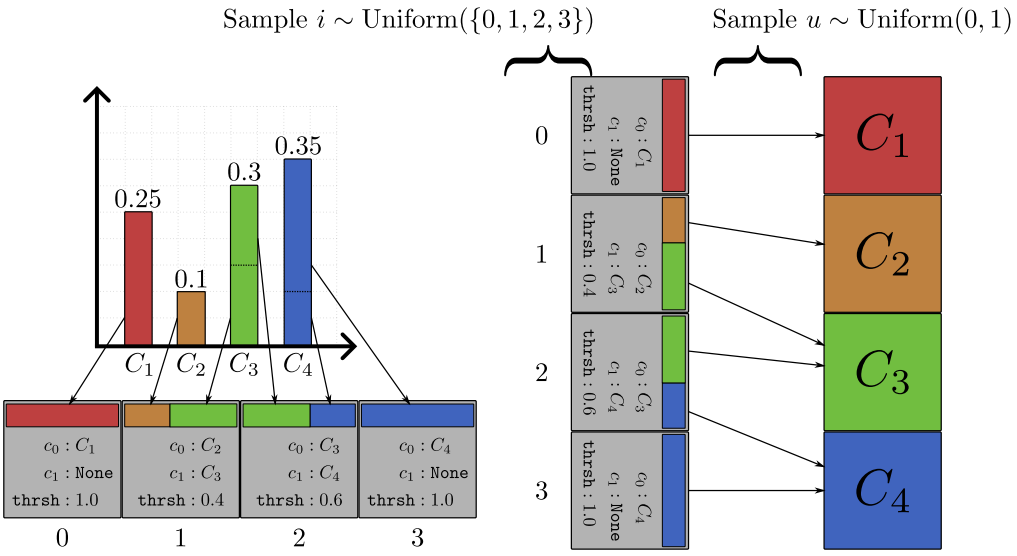


Figure 4.5: Example of building and sampling from an alias table, on a distribution with four categories. Left: shows how the probability mass is split into the four entries of the table. Right: shows how two random variables can be used to sample from the table in $O(1)$ time, where i is used to index into the table, and u is used to compare with `thrsh` and decide which of the two arrows from the entry to follow.

Sampling from the alias table can be performed in constant time by sampling two random numbers. The first number is a random integer index into the table, and the second number is a random continuous uniform value between 0 and 1. If the entry $(c_0, c_1, \texttt{thrsh})$ is sampled and the second number is less than or equal to `thrsh`, then c_0 is returned, otherwise c_1 is returned. Figure 4.5 illustrates an example of constructing an alias table and sampling from it.

Vose [104] proves that an alias table can always be constructed from an arbitrary categorical distribution, such that the probability of sampling any category from the alias table is identical to the probability of sampling it from the original probability mass function.

4.2.3.3 A More Efficient MCTS

The discussion in this section assumes that `single_node_expansion` is true, and that `transposition_table` is false. Additionally, the runtime complexities of `init_q` and `sample_outcome` are assumed to be $O(1)$[3]. In this case, the runtime

[3]This is typically the case when using a simulator. If given an arbitrary MDP, the Alias Method can be used with some preprocessing cost to sample outcomes in $O(1)$ time.

of one MCTS trial consists of the following time costs:

Selection Phase: The `select_action` function is called at up to $H_{\texttt{THTS++}}$ many decision nodes.

Heuristic Step: The `init_v` function is called once to initialise the value of the new decision node.

Backup Phase: The `backup_v` and `backup_q` functions are called at up to $H_{\texttt{THTS++}}$ many decision nodes and chance nodes.

Hence, the worst case complexity of an MCTS trial in `THTS++` is:

$$O(H_{\texttt{THTS++}} \cdot (O(\texttt{select_action}) + O(\texttt{backup_v}) + O(\texttt{backup_q})) + O(\texttt{init_v})). \tag{4.27}$$

When a rollout is used in `init_v`, its complexity is $O(H_{\texttt{THTS++}} \cdot A)$, where $A = |\mathcal{A}|$ is the number of actions. Unfortunately this is the dominating term in the runtime complexity of an MCTS trial. However, when using function approximation in `init_v`, i.e. $\hat{V}_{\text{heuristic}} = \tilde{V}$, and $\pi_{\text{heuristic}} = \tilde{\pi}$, the runtime complexity of `init_v` is $O(A)$, as $\pi_{\text{heuristic}}$ is evaluated for each action in $\mathcal{A}$ and the results cached. The rest of this subsection will assume that `init_v` runs in $O(A)$ time using function approximation.

Otherwise, `select_action` is typically the dominating factor in Equation (4.27), taking $O(A)$ time. For example in UCT (Section 2.4.3), the `select_action` function must compute a UCB score for each action in $\mathcal{A}$, taking $O(A)$ time.

However, if one is using a stochastic search policy and willing to use a slightly outdated search policy during selection, the Alias Method can be used as follows:

- In `select_action`, actions are sampled from an alias table stored in the decision node;
- In `init_v`, an initial alias table is constructed, with an additional $O(A)$ preprocessing cost, maintaining the $O(A)$ time complexity;

- In `backup_v`, the alias table is updated every A visits to the decision node, with an additional $O(A)$ time cost every A visits, giving only an amortised $O(1)$ time complexity on top of any backup equations used in `backup_v`.

The only remaining significant time costs are in `backup_v` and `backup_q`.

Average Return Backups. It is well known that the incremental updates for computing the average returns used in UCT (Equations (2.48) and (2.49)), AR-BTS and AR-DENTS have a constant $O(1)$ complexity.

Bellman `backup_v`. In BTS and DENTS, the `backup_v` function (Equations (4.8) and (4.22)) computes a maximum over actions in $\mathcal{A}$. Because the Q-value of only one action a_t is changed between the selection and backup phases, the maximum can be computed more efficiently using a max heap. `backup_v` updates the Q-value[4] corresponding to a_t in the max heap, in $O(\log(A))$ time, and then the maximum value can be read from the top of the heap.

Bellman `backup_q`. In the algorithms considered in this thesis so far, the Q-value backups are all either Bellman or average return backups. Bellman Q-value backups can be computed incrementally in $O(1)$ time. Consider the general form of the Bellman update at chance node (s_t, a_t), using the value estimates $\hat{V}$ and $\hat{Q}$:

$$\hat{Q}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}(s') \right). \tag{4.28}$$

Because the value of only one child node, s_{t+1}, is changed between the selection and backup phases, Equation (4.28) can be computed incrementally in $O(1)$ time as:

$$\hat{Q}(s_t, a_t) \leftarrow \begin{cases} R(s_t, a_t) + \hat{V}(s_{t+1}) & \text{if } N(s_t, a_t) = 1, \\ R(s_t, a_t) + S(s_t, a_t) + \frac{N(s_{t+1})}{N(s_t, a_t)} \hat{V}(s_{t+1}) & \text{otherwise,} \end{cases} \tag{4.29}$$

$$S(s_t, a_t) = \frac{N^{\mathrm{prev}}(s_t, a_t)}{N(s_t, a_t)} \left(Q(s_t, a_t) - R(s_t, a_t) - \frac{N^{\mathrm{prev}}(s_{t+1})}{N^{\mathrm{prev}}(s_t, a_t)} \hat{V}^{\mathrm{prev}}(s_{t+1}) \right), \tag{4.30}$$

[4]Note that in an arbitrary MDP, the Q-value may decrease due to stochasticity in the MDP. If the MDP is deterministic, computing the Q-values will only increase, and the maximum can be computed in $O(1)$ time.

where S computes the corrected average of $\hat{V}(s')$ for $s' \in \text{Succ}(s_t, a_t) - \{s_{t+1}\}$, $\hat{V}^{\text{prev}}(s_{t+1})$ is the previous value of $\hat{V}(s_{t+1})$, and $N^{\text{prev}}(s_{t+1}) = N(s_{t+1}) - 1$, $N^{\text{prev}}(s_t, a_t) = N(s_t, a_t) - 1$ are the previous values of $N(s_{t+1})$, $N(s_t, a_t)$.

MENTS `backup_v`. Using the same observation as in the previous paragraph, the `backup_v` function in MENTS (Equation (2.58)) can be computed in $O(1)$ time. However, this is difficult to implement in a numerically stable way, because the *correction term*[5] involves products of exponentials.

DENTS Entropy `backup_v`. When the Alias Method is used with DENTS, the search policy π_{DNTS} is updated every A visits. On trials where the search policy is not updated, the same observation can be used again, correcting the value estimates using only the updated value of the child node whose value changed.

When the Alias Method is not used, the search policy π_{DNTS} may change on each trial, so Equation (4.20) needs to be recomputed, taking $O(A)$ time.

Summary of Complexities. The complexities of the different components of an MCTS trial are summarised in Table 4.1. To answer **Q3.1 - Complexity**in the affirmative, when the Alias Method is used with the most efficient implementations of `backup_v` and `backup_q`, the complexity of running a trial of MENTS can be improved to $O(H_{\texttt{THTS++}} + A)$. The complexity of running a trial of BTS and DENTS can be improved to $O(H_{\texttt{THTS++}} \log(A) + A)$, and the complexity of running a trial of AR-BTS and AR-DENTS can be improved to $O(H_{\texttt{THTS++}} + A)$.

[5] e.g. $S(s_t, a_t)$ in Equation (4.29)

Algorithm	`select_action`	`backup_v`	`init_v`	Trial Complexity
UCT	$O(A)$	$O(1)$	$O(1)$	$O(H_{\texttt{THTS++}} \cdot A)$
PUCT	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
MENTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
MENTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$
BTS	$O(A)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
BTS+AM	$O(1)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot \log(A) + A)$
AR-BTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
AR-BTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$
DENTS	$O(A)$	$O(A)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
DENTS+AM	$O(1)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot \log(A) + A)$
AR-DENTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
AR-DENTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$

Table 4.1: Summary of complexities for different MCTS algorithms. Complexities for each of the significant components are shown for each algorithm. AM is short for Alias Method. `backup_q` can be implemented in $O(1)$ time for all algorithms in this table so is omitted. Finally, UCT does not use $\pi_{\text{heuristic}}$, so it can implement `init_v` in $O(1)$ by only computing the function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$.

4.3 Theoretical MDPs

This section builds on the discussion from Section 4.1 to construct adversarial MDPs that expose the limitations of UCT and the maximum entropy objective.

Coquelin and Munos [25] introduce the *D-chain problem*, which is a theoretical MDP for which UCT requires exponentially many trials to find the optimal policy. MENTS is able to solve the problem in a small number of trials due to its maximum entropy objective. However, Section 4.3.1 demonstrates that changing a single reward in the MDP can shift the misalignment threshold to a prohibitively small value, rendering the maximum entropy objective ineffective.

In Section 4.3.2, the D-chain problem is extended to include an *entropy trap*, to create an MDP for which both UCT and MENTS cannot solve in any feasible time.

4.3.1 The D-Chain Problem

The *D-chain problem* consists of a chain of D states with two possible actions and is depicted in Figure 4.6. If the action $a^L_{\mathcal{M}}$ is taken from state $s^i_{\mathcal{M}}$, then an immediate reward of $R(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-i}{D}$ is collected and the trial ends. If the action $a^R_{\mathcal{M}}$ is taken the agent continues along the chain without any reward, until the

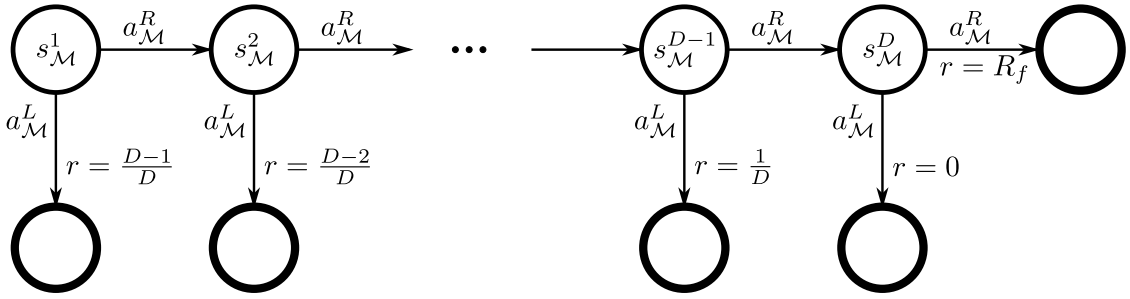


Figure 4.6: An illustration of the *(modified) D-chain problem*, where $s^1_{\mathcal{M}}$ is the starting state and transitions are deterministic. The action $a^L_{\mathcal{M}}$ from state $s^i_{\mathcal{M}}$ results in an immediate reward of $\frac{D-i}{D}$ and the trial ends. Taking action $a^R_{\mathcal{M}}$ to the end of the chain results in a reward of R_f. Where $R_f = 1$ in the original D-chain, and $R_f = \frac{1}{2}$ in the modified D-chain.

end of the chain is reached where the agent receives a reward of $R(s^D_{\mathcal{M}}, a^R_{\mathcal{M}}) = 1$. By inspection, the optimal policy is $\pi^*(s) = a^R_{\mathcal{M}}$, to follow the chain to the end and collect the maximum reward of one.

To intuitively understand why UCT will struggle on this MDP, the reward function can be decomposed into a dense and sparse component:

$$R_{\text{dense}}(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-i}{D}, \tag{4.31}$$

$$R_{\text{dense}}(s^i_{\mathcal{M}}, a^R_{\mathcal{M}}) = 0, \tag{4.32}$$

$$R_{\text{sparse}}(s^i_{\mathcal{M}}, a) = \mathbb{1}[i = D] \cdot \mathbb{1}[a = a^R_{\mathcal{M}}], \tag{4.33}$$

$$R(s, a) = R_{\text{dense}}(s, a) + R_{\text{sparse}}(s, a). \tag{4.34}$$

The two components of the rewards can be seen to encourage the agent to perform two different behaviours. Intuitively, R_{dense} encourages the agent to leave the chain as soon as possible, while R_{sparse} rewards the agent for staying on the chain until the end. Since UCT balances exploration and exploitation, it will frequently select $a^L_{\mathcal{M}}$ to exploit the immediate reward of $\frac{D-1}{D}$, rather than exploring to find the reward of one.

Compounding this, once a trial of UCT reaches the unit reward for the first time, the value estimates of $\bar{V}_{\text{UCT}}(s^i_{\mathcal{M}})$ still need many more trials to converge and recommend the optimal policy. This phenomenon can be seen in Figure 4.7, where UCT never recommends the optimal policy, while MaxUCT (Section 3.3) that uses the same search policy with Bellman backups will recommend the optimal policy once one trial reaches the unit reward at the end of the chain.

Coquelin and Munos [25] formally prove that UCT will suffer a regret of $\Omega((\exp \circ \exp \circ \ldots \circ \exp)(1))$, where $\circ$ denotes functional composition, and there are D many composed exponential functions.

From the perspective of practical single-objective reinforcement learning, this issue may appear to be one of poor reward shaping. However, rewards with conflicting objectives similar to this naturally arise in the context of multi-objective reinforcement learning. An example of this will be seen in the *Deep Sea Treasure* problem in Chapter 5, where the scalarised reward combines a dense time cost with a sparse treasure reward, yielding a structure similar to Equation (4.34).

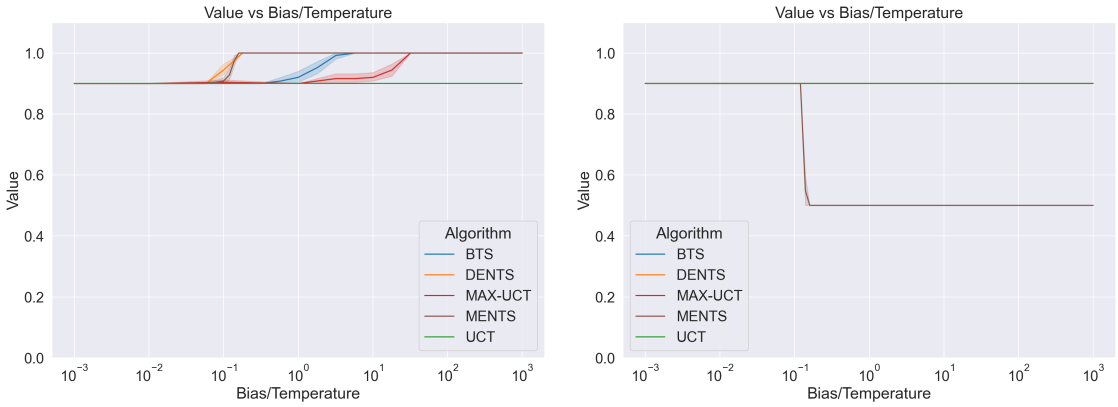
Introducing entropy as an exploration bonus naturally encourages the agent to explore along the chain, because once the agent leaves the chain it can no longer collect a bonus entropy reward. As can be seen in Figure 4.7, the D-chain problem is an example of an MDP where the standard and maximum-entropy objectives are well aligned.

To illustrate that in maximum entropy methods the temperature parameter requires careful per-MDP tuning, consider the *modified D-chain problem*, where the sparse reward at the end of the chain is replaced with $R(s^D_{\mathcal{M}}, a^R_{\mathcal{M}}) = \frac{1}{2}$. In this case the optimal policy takes the immediate reward of $R(s^1_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-1}{D}$ from the initial state, and as illustrated in Figure 4.7, the maximum entropy objective is misaligned with the standard objective for a temperature $\alpha > \frac{1}{5}$.

4.3.2 The Entropy Trap

In the *D-chain with Entropy Trap problem* (or *Entropy Trap problem* for short), an additional chain of states is appended to the end of the D-chain, designed to exploit the misalignment between the maximum entropy and standard objectives. At the end of the D-chain, the agent must choose between collecting the sparse reward of one, or entering the *entropy chain* where no reward is given but the agent can accumulate entropy "reward". The Entropy Trap problem is depicted in Figure 4.8.

At each step of the entropy chain the agent still has the option of the two actions $a^L_{\mathcal{M}}$ and $a^R_{\mathcal{M}}$. The optimal maximum entropy policy is uniform, taking either action



(a) D-chain problem. **(b)** Modified D-chain problem.

Figure 4.7: Results on the D-chain problem (a) and the modified D-chain problem (b), for a variety of parameters and with $D = 10$. Each algorithm was run for 5000 trials, and results are averaged over 25 repeated experiments. For BTS the temperature function is set to a constant value, and in DENTS the temperature function and entropy weighting function are set to the same constant value.

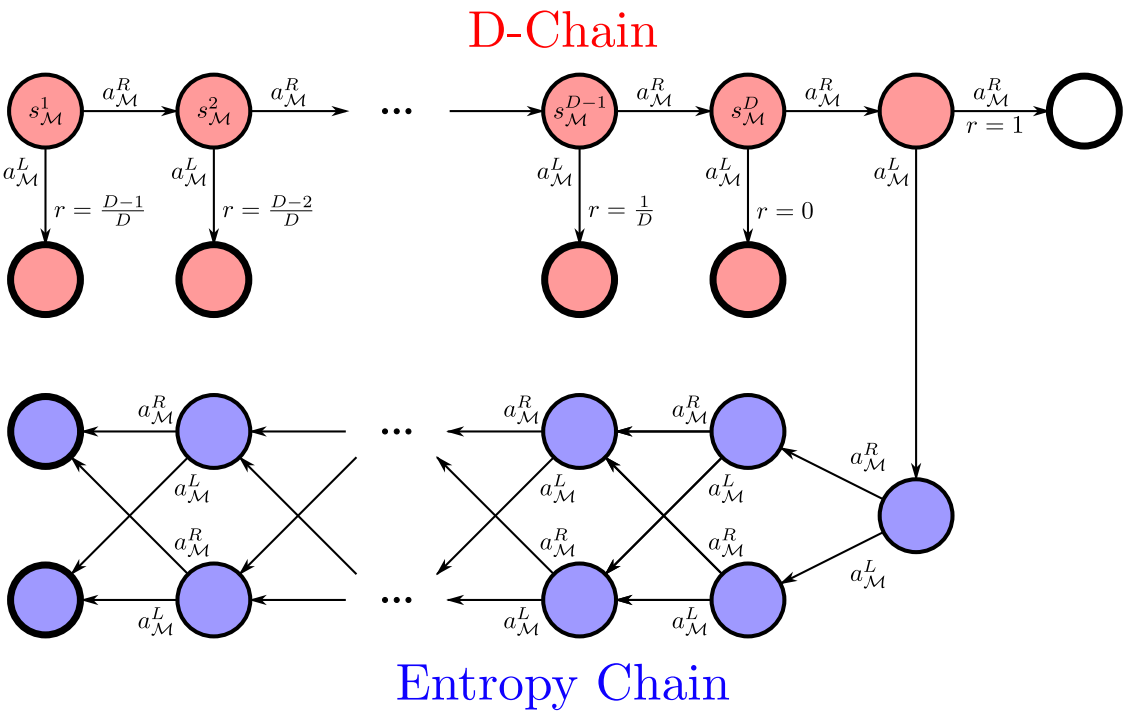


Figure 4.8: An illustration of the *Entropy Trap problem*. In red, the original D-chain problem (Figure 4.6) is labelled as the UCT Gauntlet, and the additional states added for the Entropy Trap are in blue. If a reward is not specified for a transition it is zero.

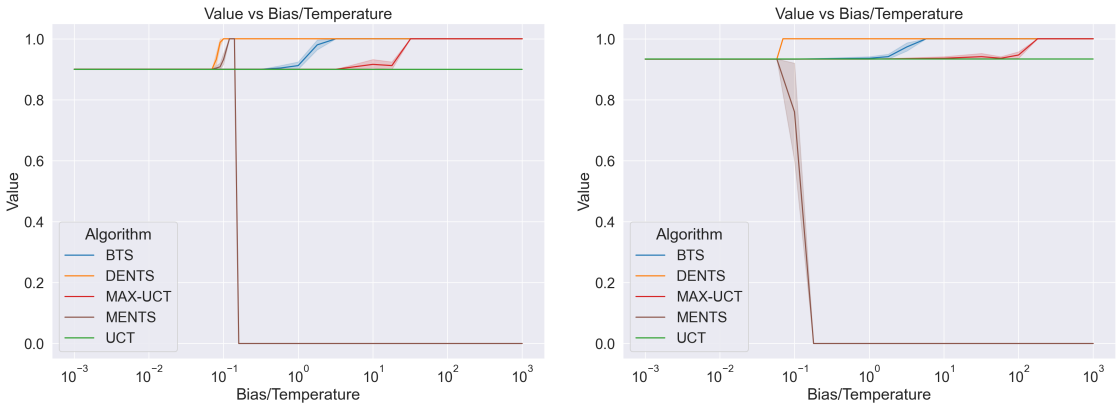
with probability $\frac{1}{2}$ with an entropy of $\log(2)$ per step. If there are E steps in the entropy chain and a temperature of α is used in the maximum entropy objective, then the maximum entropy bonus that can be collected from the chain is $\alpha E \log(2)$. Therefore, $\frac{1}{E\log(2)}$ is an upper bound on the misalignment threshold for the entropy trap, as if $\alpha > \frac{1}{E\log(2)}$, a maximum entropy agent will select the entropy chain over the reward of one at the end of the D-chain.

The entropy trap problem can therefore be made arbitrarily difficult for UCT by increasing D, while the misalignment threshold can be made arbitrarily small by increasing E. Theorem 4.3.1 uses the case when $D = E$ to show that MENTS must either suffer from misalignment or require an exponential number of trials with respect to the size of the state space.

Theorem 4.3.1. *For any value of α_{MENTS}, there exists an MDP such that either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy.*

More precisely, after n trials of MENTS, $\Pr(\psi_{\text{MENTS}} \neq \pi^*) > \frac{1}{2}$ whenever $n \leq 2^{D-1}$. Moreover, either $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \not\to 0$, or there exist constants $c, k > 0$ such that $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \geq c\left(1 - \frac{n}{k^{|S|}}\right)$.

Results on the Entropy Trap problem are given in Figure 4.9 for $(D, E) = (10, 10)$ and $(D, E) = (15, 25)$. In the $(D, E) = (15, 25)$ case, 500,000 trials were run per parameter setting. The misalignment threshold is bounded above by $\frac{1}{25\log(2)} \approx 0.06$, which is prohibitively small for effective exploration. Below this threshold, no more than 1000 of the 500,000 trials reached the third state of the D-chain, and typically fewer than 100. While BTS and MaxUCT are able to produce the optimal policy given a sufficiently large temperature/bias parameter, increasing D further would eventually prevent them from doing so within a reasonable number of trials. DENTS, by contrast, is able to recommend the optimal policy across a wider range of settings since its Bellman value estimates are unaffected by the entropy chain. For BTS and DENTS, each of α_{BTS}, α_{DNTS} and β_{DNTS} is set to a constant value. While this leads to the majority of trials in DENTS entering the entropy chain, this can be resolved by using a function that tends to zero for β_{DNTS}.



(a) $(D, E) = (10, 10)$, 5000 trials. **(b)** $(D, E) = (15, 25)$, 500,000 trials.

Figure 4.9: Results on the Entropy Trap problem. Each algorithm was run with a variety of temperature and bias parameters, averaged over 25 repeated experiments. For BTS the temperature function is set to a constant, and in DENTS the temperature and entropy weighting functions are set to the same constant. All other exploration parameters ϵ and the heuristic value function $\hat{V}_{\text{heuristic}}$ are set to zero. (a) MENTS produces the optimal policy for a small range of temperatures, and BTS and MaxUCT do so for sufficiently large temperature and bias parameters. (b) At the misalignment threshold MENTS recommendations drop to zero due to the entropy trap, while DENTS recommends the optimal policy using its Bellman value estimates.

4.4 Empirical Results

This section presents the empirical results for this chapter. For the gridworld environments the following baseline algorithms are used: UCT (Section 2.4.3), MaxUCT (Section 3.3), MENTS (Section 2.4.5), RENTS (Section 3.3), TENTS (Section 3.3) and H-MCTS (Section 3.3).

MENTS, RENTS and TENTS are used as baseline algorithms from the maximum entropy literature. H-MCTS is used as a baseline MCTS algorithm designed for the simple regret objective.

This section is organised as follows:

- Section 4.4.1 describes the MDP environments used in the experiments;
- Section 4.4.2 describes the evaluation setup used;
- Section 4.4.3 presents the results on gridworld environments and the game of Go;

- Appendix B provides additional details on these experiments, including the parameters used by each algorithm, and how they were selected.

4.4.1 Environments

Gridworld environments are used to investigate the behaviour of the search algorithms in controlled, solvable settings. Three gridworld environments are used: a heavily stochastic environment with sparse rewards (FrozenLake), a deterministic environment with sparse rewards (DeterministicGridWorld), and a mildly stochastic environment with dense rewards (the Sailing Problem).

4.4.1.1 FrozenLake

FrozenLake [101] is a gridworld environment where the agent aims to reach a goal state while avoiding holes that it could fall into. The agent can attempt to move in any cardinal direction at each time step. At each step, there is an equal probability that the agent moves in the intended direction or slips and moves in a perpendicular direction. For example, if the agent tries to move down, then there is a probability of $\frac{1}{3}$ that the agent moves down, moves left or moves right. If the agent moves or slips into a wall, then the agent remains in the same location.

If the agent reaches the goal state, then a reward of one is received. If the agent falls into a hole, then the trial ends and zero reward is received. Consequently, the value of a policy π at a state s is the probability that the goal is reached under π.

Figure 4.10 illustrates the environment. A range of maps of different sizes is used in the experiments, from 4x4 to 4x16, which are specified in Appendix B.1.

4.4.1.2 DeterministicGridWorld

DeterministicGridWorld (DGW) is used to refer to the deterministic version of FrozenLake. The environment is the same as FrozenLake, but the agent cannot slip and always moves in its intended direction. To differentiate between paths of different lengths a discount factor of γ is used, so that shorter paths gain more reward than longer ones. That is, if the agent reaches a goal after t timesteps,

Figure 4.10: An illustration of the FrozenLake environment, reproduced from the Gymnasium documentation [101].

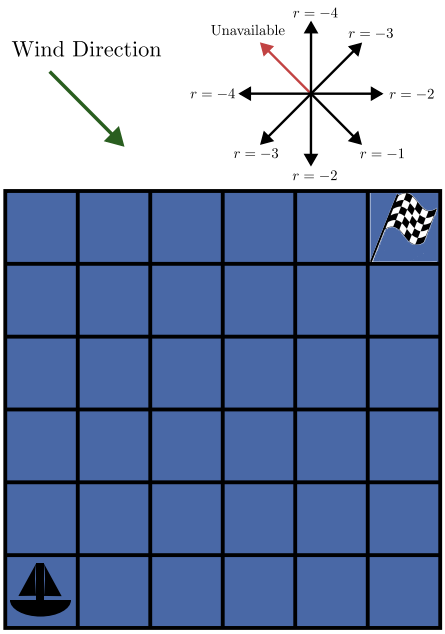
then a reward of γ^t is received. Maps of sizes 8x8 to 8x32, defined in Appendix B.1, are used in the experiments for this environment.

4.4.1.3 Sailing Problem

The *Sailing Problem* has often been used to evaluate MCTS algorithms [73, 51, 99, 32]. It is another gridworld environment with one goal location, at the opposite corner to the starting location of the agent. The agent has 8 different actions, where it can travel in any cardinal or semi-cardinal direction. In each state of the MDP, the wind is blowing in a given direction and will stochastically change after every transition. The agent cannot sail directly into the wind. Actions and wind directions can take values in $\{0, 1, \ldots, 7\}$, with a value of 0 representing North/up, 2 representing East/right, 4 representing South/down and 6 representing West/left. The remaining numbers represent the inter-cardinal directions. The cost of each action depends on the *tack*, the angle between the direction of the agent's travel and the wind. Figure 4.11 illustrates the problem and gives the wind transition probabilities for the Sailing problem.

4.4.1.4 Go

For a more challenging domain, a round-robin tournament was run on the game of Go, which has widely motivated the development of MCTS methods [36, 92, 93]. *Area scoring* is used to score the games, with a *komi* (a score handicap for black) of 7.5.



(a) 6x6 Sailing Problem map.

$$\begin{pmatrix} 0.4 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 \\ 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.4 & 0.2 & 0.4 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 & 0.4 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 & 0.4 \\ 0.4 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 \end{pmatrix}$$

(b) Wind transition probabilities.

Figure 4.11: (4.11a) illustration of the Sailing Problem with a 6x6 map, where the wind direction is currently South-East, and example costs of each action. (4.11b) The wind transition probabilities. The (i, j)th element of the matrix denotes the probability that the wind changes from direction i to direction j, where 0 denotes North/up, 1 denotes North-East/up-right, and so on.

The methods described in this chapter can be adapted for two-player games by using a discount factor in the MDP. The value of a policy π with a discount factor γ is defined as:

$$V^{\pi,\gamma}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} \gamma^{i-t} r_i \middle| s_t = s\right]. \tag{4.35}$$

If the two players are using the policies $\pi^{(0)}$ and $\pi^{(1)}$, and π is the policy that interleaves $\pi^{(0)}$ at even timesteps with $\pi^{(1)}$ at odd timesteps, then using a discount factor of $\gamma = -1$ is sufficient to handle two-player games where a reward of $+1$ is given when a player wins[6].

4.4.2 Evaluation Procedure

4.4.2.1 Gridworld Environments

Consider an algorithm with search tree $\mathcal{T}$, which provides a partial recommendation policy ψ_{ALG}. The recommendation policy is made complete by using a uniformly

[6]The score of -1 for losing a game is accounted for by the discount factor $\gamma = -1$.

random policy for states and actions that are outside of the tree $\mathcal{T}$ as follows:

$$\psi(a|s) = \begin{cases} \psi_{\text{ALG}}(a|s) & \text{if } s \in \mathcal{T}, \\ \frac{1}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{4.36}$$

Using the completed policy ψ, a Monte Carlo estimate of $V^{\psi}(s_0)$ is computed by sampling a number of trajectories from ψ and averaging the returns. Unless otherwise stated, each Monte Carlo estimate consists of 1024 trials, and results are averaged over 25 repeated experiments.

The BayesOpt package [61] was used to tune the parameters used by each algorithm for the gridworld environments. Appendix B.2 gives details on the algorithm parameters used and how they were tuned.

4.4.2.2 Go

In the Go experiments, each algorithm was given 5 seconds of compute time per move and allowed to use 32 search threads and access to a single Nvidia V100 GPU on a shared compute cluster. Additionally, each algorithm has access to a value network $\tilde{V}$ and prior policy network $\tilde{\pi}$ provided by the open-source library KataGo [111]. $\tilde{V}$ is used for the value initialisation function $\hat{V}_{\text{heuristic}}$, and $\tilde{\pi}$ is mixed into the search policy as described in Section 4.2.3.

For all algorithms, the *most visited* recommendation policy is used to recommend actions (e.g. Equation 4.24), as it is more robust than using value estimates directly: a largely unvisited action may receive a noisy neural network evaluation shortly before the recommendation is made, which the most visited policy avoids by using the visit statistics instead.

Algorithm parameters were tuned using round-robin tournaments on 9x9 Go boards, details are given in Appendix B.3.2.

The PUCT algorithm is used as a baseline, as described in AlphaGo Zero [93]. For maximum entropy baselines, AR-MENTS, AR-RENTS and AR-TENTS (Appendix B.3.1) are used. These are variants of MENTS, RENTS and TENTS using average return backups. It is experimentally shown in Appendix B.3.3 that the average returns algorithms decisively beat the Bellman versions in this domain. The Alias Method (Section 4.2.3) is used for all algorithms using stochastic search policies.

4.4.3 Results and Discussion

The empirical results are presented in this section and are organised as follows:

- Section 4.4.3.1 gives results on the gridworld environments;
- Section 4.4.3.2 gives additional results on the gridworld environments where the temperature and bias parameters are varied;
- Section 4.4.3.3 gives results from the Go round-robin;

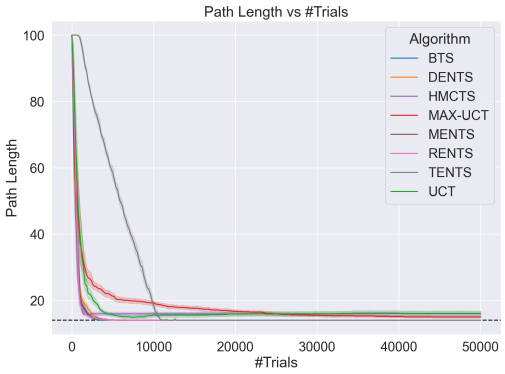
4.4.3.1 Gridworld Results

In each of the following figures, results are given for a range of map sizes in the three gridworld environments. Algorithm parameters were tuned on the smallest map of each environment (Appendix B.2.3) and then used across all map sizes, to investigate how well each algorithm generalises to larger state spaces.

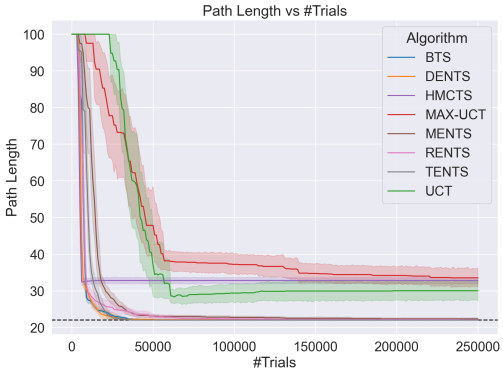
DeterministicGridWorld Results. Results for DGW are given in Figure 4.12 for a range of map sizes. The heuristic value functions $\hat{V}_{\text{heuristic}}$ and $\hat{Q}_{\text{heuristic}}$ are optimistically set to a constant value of one and the bias term $N^{\hat{Q}_{\text{heuristic}}}$ is set to one.

As discussed in Section 4.1, UCT-based algorithms converge to a local optimum due to greater focus on exploitation, which is visible across all map sizes. UCT and MaxUCT both take many trials to find any solution on the larger maps. Two factors contribute: the high bias parameters selected (Table B.2), and the fact that every action is tried once before any action is selected a second time (Section 2.4.3). The latter causes a broad initial search before exploiting, whereas a stochastic search policy such as that of BTS can reach the goal early by chance while exploring throughout its search. H-MCTS finds a solution and converges faster, with a smaller bias parameter.

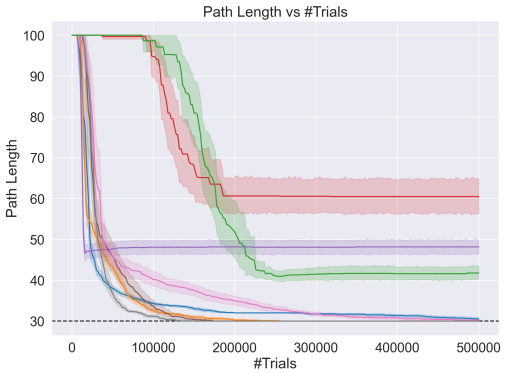
The stochastic search policies all tend towards the optimal path across all map sizes. However, particularly in the larger maps, 8x24 and 8x32, the path recommended by BTS improves rapidly initially, but without the benefit of entropy exploration it is slower to converge to the optimal path.



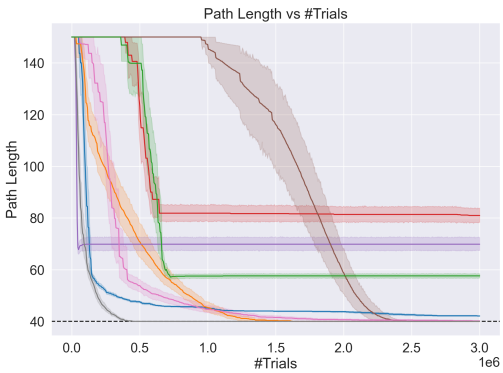
(a) 8x8 DeterministicGridWorld.



(b) 8x16 DeterministicGridWorld.



(c) 8x24 DeterministicGridWorld.

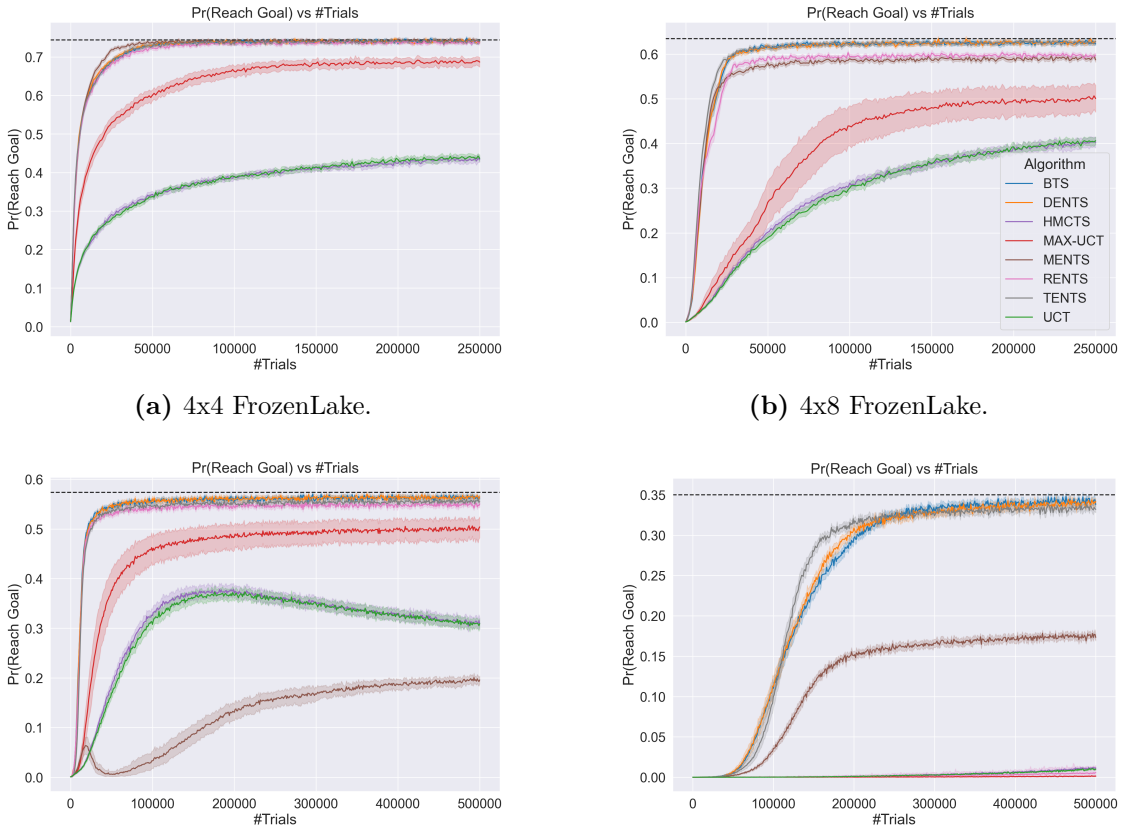


(d) 8x32 DeterministicGridWorld.

Figure 4.12: Results on DeterministicGridWorld for a range of map sizes. The map definitions are given in Figure B.1.

The tuned temperature parameter α_{MENTS} is below the misalignment threshold for all maps, as MENTS converges to an optimal policy, however the 8x32 map contains some larger regions without any holes where MENTS can gather entropy rewards, leading to slow convergence. DENTS is able to improve on this by using a larger search temperature (i.e. $\alpha_{\text{DNTS}}(x)$) than MENTS, but with a lower entropy coefficient (Table B.2). This allows DENTS to converge to the optimal path faster than MENTS, particularly on the 8x32 map.

RENTS uses *relative entropy* instead of Shannon entropy, which performs a KL regularization between the search policy at each timestep and the previous timestep, which in practice leads to sampling long sequences of the same action. This strategy is effective on smaller maps, but becomes less effective on larger maps where the optimal path is more complex.



(a) 4x4 FrozenLake.

(b) 4x8 FrozenLake.

(c) 4x12 FrozenLake.

(d) 4x16 FrozenLake.

Figure 4.13: Results on FrozenLake for a range of map sizes. The map definitions are given in Figure B.1.

Finally, TENTS uses *Tsallis entropy*, corresponding to a *sparsemax* distribution for the search policy. The sparsemax distribution concentrates probability mass on a subset of actions rather than distributing it more uniformly as softmax does. In combination with the optimistic heuristic, this produces behaviour resembling breadth-first search, leading to more thorough exploration on larger maps but slower initial progress, as visible in the 8x8 map.

FrozenLake Results. Results for FrozenLake are given in Figure 4.13 for a range of map sizes. The heuristic value functions $\hat{V}_{\text{heuristic}}$ and $\hat{Q}_{\text{heuristic}}$ are again optimistically set to a constant value of one and the bias term $N^{\hat{Q}_{\text{heuristic}}}$ is set to one.

Because FrozenLake is a stochastic environment, there is a greater emphasis on exploitation in this experiment. Exploitation is necessary to ensure that the

recommendation policy (Equation 4.36) remains within the search tree as much as possible, since states outside the tree fall back to a uniformly random policy.

BTS and DENTS perform well across all map sizes in this experiment, demonstrating their ability to balance exploration and exploitation when necessary. In particular, TENTS was the only other algorithm able to effectively solve the 4x16 map.

UCT-based algorithms can still be seen to converge to a local optimum, with MaxUCT significantly outperforming UCT, likely because MaxUCT uses Bellman backups rather than average returns, which propagate value information more directly through the tree and allow for more focused exploration before committing to exploitation. UCT and H-MCTS performed similarly on every map, and all three UCT-based algorithms made little or no progress on the 4x16 map.

Recall that parameters were optimised using the 4x4 map and reused for all maps. The temperature selected for MENTS was misaligned on the 4x12 and 4x16 maps. TENTS selected a similar temperature but was not misaligned, which suggests that the misalignment threshold is higher for Tsallis entropy than for Shannon entropy on these maps. RENTS selected a much lower temperature, and behaved as it did in the DGW environments, preferring long sequences of the same action. This was relatively successful until the 4x16 map, where RENTS made little or no progress, although its temperature may also have been above the misalignment threshold for that map.

The 4x12 map displays two non-monotonic trends, both of which are consistent across runs. Because the maps were randomly generated, this one happened to have the lowest ratio of holes to area, with one hole directly next to the goal and another to the north of it.

The first trend is for MENTS, which makes some initial progress, because the map is open and the goal is easy to reach. Being open, the map also contains regions where a large entropy reward can be collected. Performance therefore dips once such a region has been explored enough for the entropy reward to accumulate, at which point MENTS prioritises that reward over reaching the goal.

It converges to a misaligned, high-entropy policy that reaches the goal with lower probability than the optimal policy.

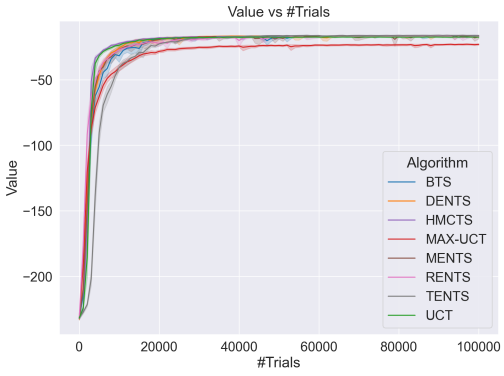
The second trend is that the value of the recommendations from UCT and H-MCTS declines beyond 150,000 trials. MaxUCT does not follow this trend, and differs from UCT only in its backups, so the cause must lie in estimating values with sample averages. At a state with one good action and several bad ones, the sample average remains low for a long time, so a state whose actions are all of moderate value may be preferred instead. The states next to the goal have exactly this structure, because of the adjacent holes. A likely explanation is therefore that before 150,000 trials the recommendation policy leaves the tree and falls back to acting uniformly at random (Equation 4.36) before reaching the goal, and that beyond 150,000 trials UCT instead commits to actions near the goal that perform worse than the uniformly random fallback they replace. Committing to a single action removes the chance of selecting the good action that acting at random retains.

Sailing Results. The Sailing Problem provides a mildly stochastic environment with a dense reward signal, in contrast to the sparse rewards of DGW and FrozenLake. The stochasticity in this environment affects only the reward received at each step, through the wind direction, rather than the position of the agent.

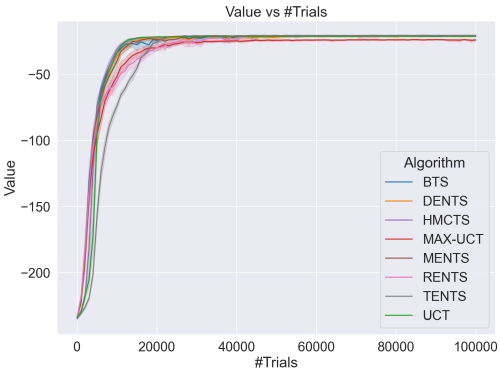
The UCT-based algorithms are more competitive on this environment, although they again converge to a local optimum, consistent with the pattern observed in the other environments.

Boltzmann-based algorithms generally outperform UCT across both initial wind directions and all map sizes.

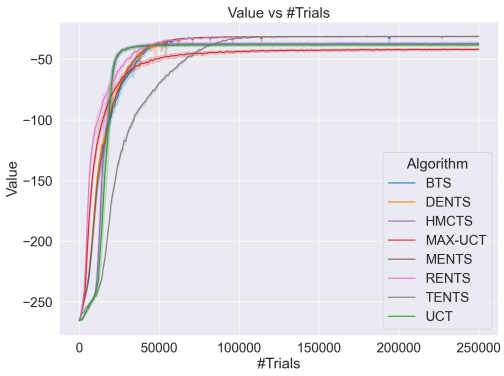
An interesting asymmetry can be seen in the performance of RENTS between the 16x16 North map and the 8x16 South-East map. The wind transition probabilities are asymmetric (Figure 4.11), with transitions towards North being more likely. Because the agent must travel North-East to reach the goal, when the wind is North facing, the agent incurs less cost, and, specifically is less likely to penalise



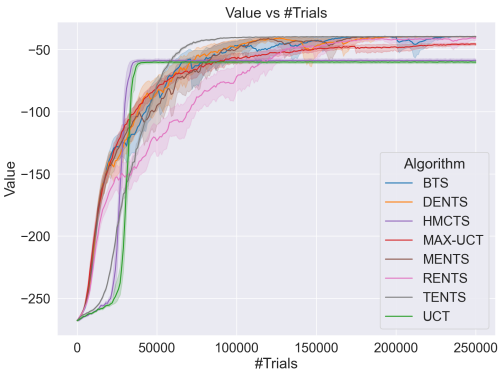
(a) 8x8 Sailing (North).



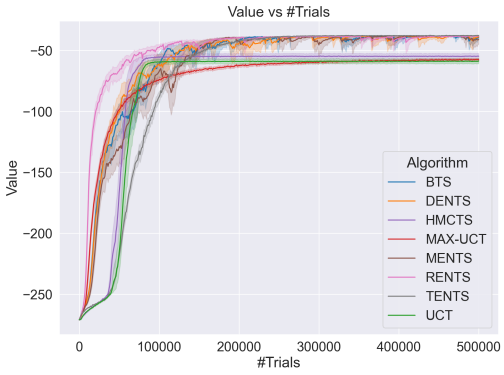
(b) 8x8 Sailing (South-East).



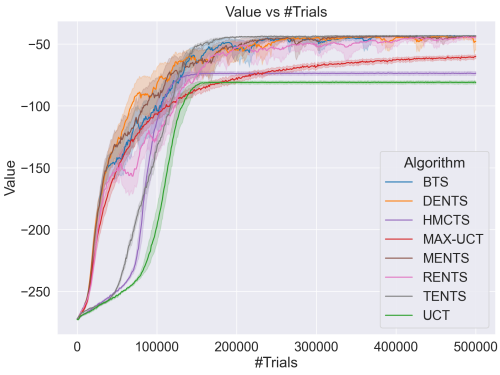
(c) 8x16 Sailing (North).



(d) 8x16 Sailing (South-East).



(e) 16x16 Sailing (North).



(f) 16x16 Sailing (South-East).

Figure 4.14: Results on the Sailing Problem for three map sizes and two initial wind directions. (Left) Initial wind direction is North. (Right) Initial wind direction is South-East. Map sizes of 8x8, 8x16 and 16x16 correspond to the first, second and third rows respectively.

(a) $\epsilon_{\text{ALG}} = 0$. **(b)** $\epsilon_{\text{ALG}} = 1$.

Figure 4.15: DeterministicGridWorld results with a range of temperature parameters on the 8x8 map (Figure B.1). Each algorithm was run for 100,000 trials per parameter setting. Values are estimated with 256 evaluation trials and results are averaged over 10 repeated experiments. For each algorithm ALG, (a) sets the exploration parameter $\epsilon_{\text{ALG}} = 0$ and (b) sets $\epsilon_{\text{ALG}} = 1$. UCT and MaxUCT do not have an ϵ parameter and appear only in (a).

long sequences of North-East actions. In contrast, when the wind is initially South-East facing, the agent is more likely to encounter states where the actions need to adapt to the wind.

The temperatures used for MENTS and TENTS remained below the misalignment threshold on all of the maps, and both performed well, converging to the optimal policy. As in DGW, the TENTS search policy often made a slow start, behaving somewhat like a breadth-first search.

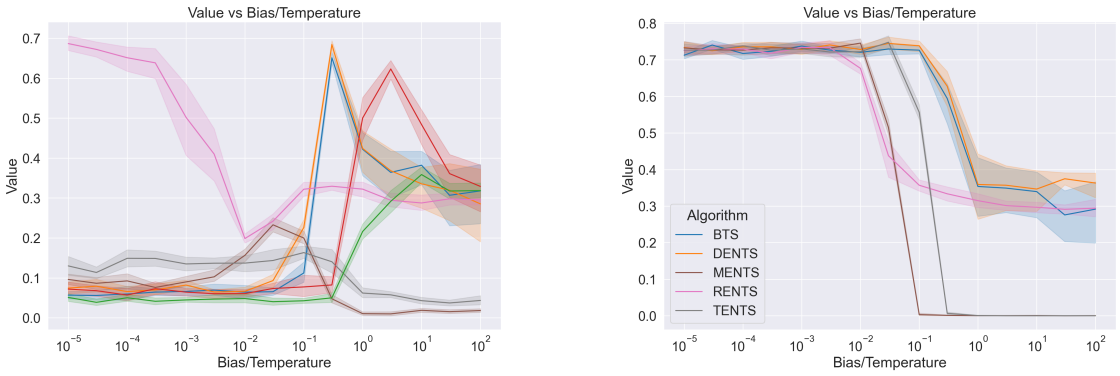
4.4.3.2 Parameter Sensitivity in Gridworld Environments

This experiment investigates how the temperature parameter affects algorithm performance in practice, and whether the entropy misalignment problem identified in Section 4.1 manifests empirically in gridworld environments.

Figures 4.15 and 4.16 show results where each algorithm is run for 100,000 trials for a range of parameter settings. Values are estimated with 256 Monte Carlo trials and results are averaged over 10 repeated experiments.

For BTS and DENTS, the temperature and entropy coefficient functions are set to constant values,

$$\alpha_{\text{DNTS}}(x) = \beta_{\text{DNTS}}(x) = \alpha_{\text{const}}, \tag{4.37}$$



(a) $\epsilon_{\text{ALG}} = 0$. **(b)** $\epsilon_{\text{ALG}} = 1$.

Figure 4.16: FrozenLake results with a range of temperature parameters on the 4x4 map (Figure B.1). The same setup is used as in Figure 4.15. Colours in (4.16a) match Figure 4.15a; the legend is omitted for clarity.

with α_{const} varied across the range shown.

As shown in Figures 4.15 and 4.16, when the temperature parameter of MENTS or TENTS is sufficiently large, the value of the recommended policy falls to zero, consistent with the misalignment behaviour predicted theoretically.

Moreover, in Figure 4.16 it can be seen that the most effective temperature for exploration is around the misalignment threshold. This is consistent with Figure 4.13 where MENTS suffered from misalignment on the larger maps. The latter is explained by the larger maps containing more open spaces without holes, providing greater opportunity to gather entropy rewards and effectively lowering the misalignment threshold.

Empirically, RENTS does not exhibit entropy misalignment. This can be understood by again noting that the relative entropy objective encourages the search policy to repeat the same action across consecutive timesteps within a trial, which in the 8x8 DGW and 4x4 FrozenLake maps is an effective strategy. For example, in the 8x8 DGW map (Figure B.1) there is a path around the edge of the map from the starting state to the goal, where the optimal path involves sustained movement in a single direction.

RENTS also performs well at low temperature, where the search policy is nearly greedy, even when $\epsilon_{\text{RENTS}} = 0$. The RENTS regularisation will still bias early trials towards sustained sequences of the same action.

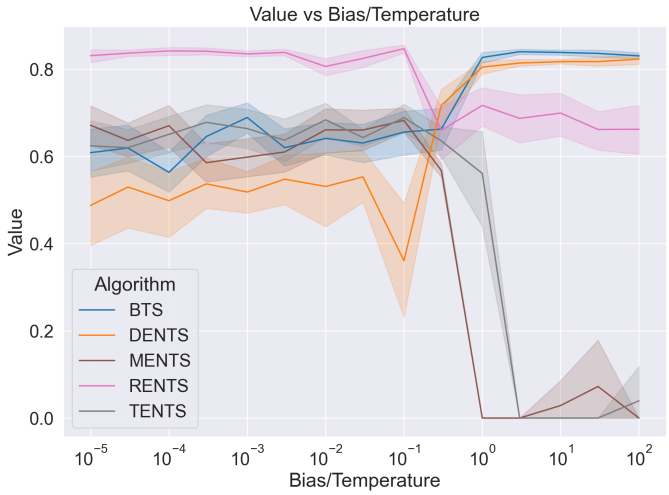


Figure 4.17: A repetition of Figure 4.15a using decaying temperatures for MENTS, RENTS and TENTS. For all algorithms ALG, the temperature function $\alpha_{\text{ALG}}(x) = \frac{\alpha_{\text{init}}}{\sqrt{1+x/10}}$ is used.

A natural question is whether replacing the fixed temperature in MENTS and TENTS with a decaying schedule, analogous to that used in BTS and DENTS, would resolve the misalignment problem. However, recall the maximum entropy objective (Definition 2.3.7):

$$J_{\text{sft}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} r_i + \alpha\mathcal{H}(\pi(\cdot|s_i))\right]. \tag{4.38}$$

Using a decaying temperature in MENTS and TENTS will in effect give each term in the entropy sum of Equation (4.38) its own coefficient, so the objective is still misaligned because of large temperatures deeper in the tree. Experimentally this is confirmed to persist in Figure 4.17, where the temperature is set to the function

$$\alpha_{\text{ALG}}(x) = \frac{\alpha_{\text{init}}}{\sqrt{1+x/10}}, \tag{4.39}$$

for $\text{ALG} \in \{\text{MENTS}, \text{RENTS}, \text{TENTS}\}$.

DENTS overcomes this issue via its entropy values which estimate the entire entropy sum of Equation (4.38), which can be decayed independently at each node by reducing $\beta_{\text{DNTS}}(x)$.

Black \White	PUCT	AR-M	AR-R	AR-T	AR-B	AR-D	Trials/move
PUCT	–	33–17	27–23	42–8	17–33	15–35	1054
AR-MENTS	12–48	–	13–37	38–12	10–40	12–38	4851
AR-RENTS	20–30	24–26	–	39–11	18–32	14–36	3672
AR-TENTS	8–42	11–39	9–41	–	6–44	10–40	5206
AR-BTS	25–25	35–15	31–19	34–16	–	15–35	5375
AR-DENTS	23–27	36–14	29–21	36–14	15–35	–	4677

Table 4.2: Results for the Go round-robin tournament. The first column gives the agent playing as black. The final column gives the average trials run per move across the entire round-robin. In the top row, algorithm names are abbreviated for space.

4.4.3.3 Go Results

Results of the round-robin are summarised in Table 4.2, where each algorithm played 50 games as black and 50 as white. AR-BTS ran the most trials per move and won all of its matches with the exception of AR-DENTS, with which it drew. The Alias Method allowed the Boltzmann search algorithms to run significantly more trials per move than PUCT. BTS and DENTS were able to beat PUCT with results of 57–43 and 58–42 respectively. Using entropy did not seem to have much benefit in these experiments, as can be witnessed by MENTS only beating TENTS, and DENTS drawing 50–50 with BTS. This is likely because the additional exploration provided by entropy is vastly outweighed by utilising the information contained in the neural networks $\tilde{V}$ and $\tilde{\pi}$. Interestingly RENTS had the best performance out of the prior works, losing 43–57 to PUCT, and the use of relative entropy appears to take advantage of a heuristic for Go that the RAVE [37] algorithm used: the value of a move is typically unaffected by other moves on the board, so if a move is assigned high probability but not selected at a given timestep, the relative entropy biases the policy towards selecting it in subsequent timesteps.

To validate the strength of the `THTS++` PUCT agent, it was played directly against KataGo [111] for 100 games with a budget of 1600 trials per move, winning 61–39 in 9x9 Go and losing 35–65 in 19x19 Go, suggesting it is strong enough to provide a meaningful comparison for the other algorithms. AR-BTS was also played against KataGo under the same conditions for 50 games and lost 31–19, confirming that emphasising more exploration within a fixed trial budget

is outweighed by the information contained in the neural networks. In Go, the Boltzmann search algorithms gain their advantage primarily through the Alias Method and the ability to run more trials per move. Note that the KataGo networks were not fine-tuned, so the Boltzmann algorithms used networks originally trained for PUCT with average returns.

4.5 Theoretical Results

This section provides the theoretical analysis and proofs for this chapter. It is structured as follows:

1. First, Section 4.5.1 defines additional notation so that *MCTS processes* can be reasoned about with respect to the number of trials run;

2. Second, Section 4.5.2 establishes preliminary results, lemmas and definitions used in subsequent proofs;

3. Third, Section 4.5.3 proves the consistency results for (AR-)BTS and (AR-)DENTS from Section 4.2 (Theorems 4.2.1, 4.2.2, 4.2.4, 4.2.5, 4.2.6, 4.2.8);

4. Finally, Section 4.5.4 proves Theorem 4.3.1, showing that the maximum entropy objective is either misaligned or requires an exponential number of trials in the Entropy Trap problem.

Some of the particularly verbose arguments are left to Appendix D so that they do not interrupt the flow of the main arguments. In particular, Theorems 4.2.3 and 4.2.7, which show exponential simple regret bounds on BTS and DENTS.

The following assumptions are made for this section, which do not affect the asymptotic behaviour of the algorithms but simplify the proofs and presentation:

- the unique optimal policy assumption and the unique state assumption remain (Assumptions 2.3.1 and 2.4.1);

- `single_node_expansion` is set to false and `transposition_table` is set to false;
- heuristic values are set to zero, $\hat{V}_{\text{heuristic}}(s) = 0$ and $\hat{Q}_{\text{heuristic}}(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$;
- the length of all trajectories and the planning horizon $H_{\texttt{THTS++}}$ are equal to the horizon H of the MDP;
- the pseudo-visit biasing terms $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{AR-DNTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{AR-DNTS}}^{\hat{Q}_{\text{heuristic}}}$, from Section 4.2 are all set to zero.

Finally, note that mathematically (AR-)BTS is a special case of (AR-)DENTS, with $\beta_{\text{DNTS}}(x) = 0$ and $\beta_{\text{AR-DNTS}}(x) = 0$. Results will be proven about (AR-)DENTS, and the corresponding results for BTS follow as a direct consequence.

4.5.1 Notation

Because of the unique state assumption (Section 2.4.1), any state $s_t \in \tau = (s_0, a_0, r_0, \ldots, s_{H-1}, a_{H-1}, r_{H-1}, s_H)$ may only appear at timestep t in any trajectory. Throughout this chapter, s_t and a_t are used to denote an arbitrary state and action at an arbitrary timestep t.

When making arguments that apply to multiple algorithms, the general policies π and ψ are used. When making arguments about specific algorithms, subscripts are used, e.g. π_{BTS}.

To reason about MCTS processes over multiple trials, the existing notation is extended with superscripts to track both the number of trials run and the number of times each node has been visited.

Most variables are denoted with a superscript n to denote that it is the value used on the nth trial of MCTS. For example:

- If an algorithm uses a search policy π and outputs a recommendation policy ψ, then the search policy followed on the nth trial is denoted π^n, and the recommendation policy after n trials is denoted ψ^n.

- When it is necessary to reason about multiple trajectories, the superscript notation will also be used. The trajectory sampled in the nth trial is $\tau^n = (s_0^n, a_0^n, r_0^n, \ldots, s_{H-1}^n, a_{H-1}^n, r_{H-1}^n, s_H^n)$. If it is sampled from a search policy π^n, then $\tau^n \sim \pi^n$.

- The search tree after n trials is denoted $\mathcal{T}^n$, the initial search tree is $\mathcal{T}^0 = \{s_0\}$, and $\mathcal{T}^n = \mathcal{T}^{n-1} \cup \{s \in \tau^n\} \cup \{(s,a) \in \tau^n\}$.

Let $T^n(s_t)$ and $T^n(s_t, a_t)$ denote the sets of trial indices on which s_t was visited and action a_t was selected respectively, in the first n trials, where t is the timestep at which the node appears in the trajectory:

$$T^n(s_t) = \{i | 1 \leq i \leq n, s_t^i = s_t\} \tag{4.40}$$

$$T^n(s_t, a_t) = \{i | 1 \leq i \leq n, s_t^i = s_t, a_t^i = a_t\}. \tag{4.41}$$

The visit counts after n trials are defined as follows, with $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$:

$$N^n(s_t) = \sum_{i=1}^{n} \mathbb{1}[s_t^i = s_t] = |T^n(s_t)|, \tag{4.42}$$

$$N^n(s_t, a_t) = \sum_{i=1}^{n} \mathbb{1}[s_t^i = s_t \cap a_t^i = a_t] = |T^n(s_t, a_t)|. \tag{4.43}$$

Under the unique state assumption, the visit counts can also be expressed in the following equivalent forms, which are more convenient for the proofs:

$$N^n(s_t, a_t) = \sum_{i \in T^n(s_t)} \mathbb{1}[a_t^i = a_t], \tag{4.44}$$

$$N^n(s_{t+1}) = \sum_{i \in T^n(s_t, a_t)} \mathbb{1}[s_{t+1}^i = s_{t+1}] = |T^n(s_t, a_t)|. \tag{4.45}$$

The key observation is that $N^n(s_t, a_t)$ can be computed as a sum over the $N^n(s_t)$ trials on which s_t was visited, and $N^n(s_{t+1})$ as a sum over the $N^n(s_t, a_t)$ trials on which a_t was selected from s_t.

Value estimates are the one exception to the superscript notation to denote the number of trials. Superscripts for value estimates denote the number of times that the value has been backed up. To differentiate this counting with respect to backups, instead of with respect to trials, a superscript with brackets will be

used. For example, the backups for DENTS value estimates would be written as follows. Let $\tau^n \sim \pi^n_{\text{DNTS}}$ be a trajectory sampled from the DENTS search policy on the nth trial. Let $\tau^n = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$. The backups for $t = h-1, \ldots, 0$ are then:

$$\hat{Q}^{(N^n(s_t,a_t))}_{\text{DNTS}}(s_t, a_t) = R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N^n(s')}{N^n(s_t, a_t)} \hat{V}^{(N^n(s'))}_{\text{DNTS}}(s') \right), \tag{4.46}$$

$$\hat{V}^{(N^n(s_t))}_{\text{DNTS}}(s_t) = \max_{a \in \mathcal{A}} \hat{Q}^{(N^n(s_t,a))}_{\text{DNTS}}(s_t, a), \tag{4.47}$$

$$\bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{DNTS}}(s_t, a_t) = \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \bar{\mathcal{H}}^{(N^n(s'))}_{V,\text{DNTS}}(s'), \tag{4.48}$$

$$\bar{\mathcal{H}}^{(N^n(s_t))}_{V,\text{DNTS}}(s_t) = \mathcal{H}(\pi^n_{\text{DNTS}}(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi^n_{\text{DNTS}}(a_t|s_t) \bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{DNTS}}(s_t, a_t). \tag{4.49}$$

For reference, all of the algorithms considered in this Section are written out in full with this additional notation in Appendix D.1.

4.5.2 Preliminaries

This subsection introduces definitions and preliminary lemmas that are used in the proofs of Sections 4.5.3 and 4.5.4.

The proofs in this section make use of the *immediate simple regret*, which measures the regret incurred by taking a single action step rather than following the full policy. Working with immediate simple regret simplifies the proofs, since bounds and convergence results transfer between the two notions via Lemma 4.5.1.

Definition 4.5.1. *The* immediate simple regret *is defined as:*

$$\texttt{imm_regr}(s_t, \psi^n) = V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^*(s_t, a_t)]. \tag{4.50}$$

The following lemma shows that asymptotic bounds and convergence results transfer between immediate and simple regret, so it suffices to prove results about immediate simple regret.

Lemma 4.5.1. $\mathbb{E}\left[\texttt{sim_regr}(s_t, \psi^n)\right] = O(f(n))$ *for all* $s_t \in \mathcal{S}_0$ *if and only if* $\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n)\right] = O(f(n))$ *for all* $s_t \in \mathcal{S}_0$. *Consequently,* $\mathbb{E}\left[\texttt{sim_regr}(s_t, \psi^n)\right] \to 0$ *if and only if* $\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n)\right] \to 0$.

Proof. Firstly, the simple regret can be written recursively in terms of the immediate simple regret:

$$\begin{aligned}
\texttt{sim_regr}(s_t, \psi^n) =& V^*(s_t) - V^{\psi^n}(s_t) && (4.51)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^{\psi^n}(s_t, a_t)] && (4.52)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[R(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[V^{\psi^n}(s_{t+1})]] && (4.53)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}\Big[Q^*(s_t, a_t) - \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[V^*(s_{t+1})]\\
&+ \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[V^{\psi^n}(s_{t+1})]\Big] && (4.54)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^*(s_t, a_t)]\\
&+ \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[V^*(s_{t+1}) - V^{\psi^n}(s_{t+1})] && (4.55)\\
=& \texttt{imm_regr}(s_t, \psi^n)\\
&+ \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[\texttt{sim_regr}(s_{t+1}, \psi^n)], && (4.56)
\end{aligned}$$

where line (4.54) uses $R(s_t, a_t) = Q^*(s_t, a_t) - \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[V^*(s_{t+1})]$, a rearrangement of the Bellman optimality equation for $Q^*(s_t, a_t)$.

Immediately, this shows that $\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n)\right] \leq \mathbb{E}\left[\texttt{sim_regr}(s_t, \psi^n)\right]$ and hence if $\mathbb{E}\left[\texttt{sim_regr}(s_t, \psi^n)\right] = O(f(n))$ then $\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n)\right] = O(f(n))$.

If $\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n)\right] = O(f(n))$ then by induction over $t = H, \ldots, 0$, using the relation in Equation (4.56):

$$\begin{aligned}
\mathbb{E}\left[\texttt{sim_regr}(s_t, \psi^n)\right] &= \mathbb{E}\left[O(f(n)) + \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t, a_t)}[O(f(n))]\right] && (4.57)\\
&= O(f(n)). && (4.58)
\end{aligned}$$

The base case is $t = H$: since s_H is a terminal state $\texttt{sim_regr}(s_H, \psi^n) = \texttt{imm_regr}(s_H, \psi^n) = 0$. □

The DKW inequality (Theorem 2.5.1) is used to show that the empirical transition probabilities converge in probability to the true transition probabilities.

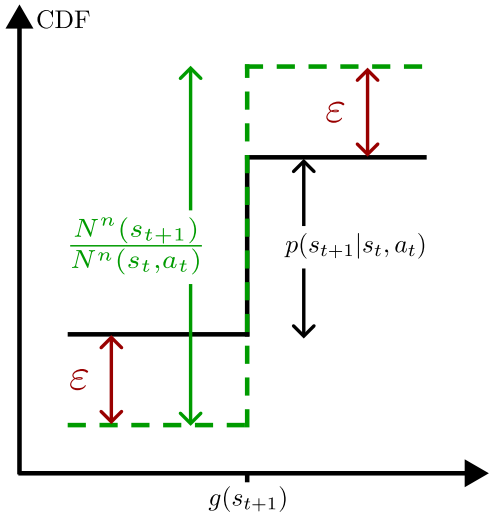


Figure 4.18: Intuition behind bounding the empirical transition probabilities to the true transition probabilities. The proof of Corollary 4.5.1 constructs cumulative distribution functions (CDFs) that can be used with Theorem 2.5.1 (the DKW inequality). g is a mapping from states to integers used by the construction. This figure depicts the CDF corresponding to the true transition probabilities (solid black line) and the CDF corresponding to the empirical transition probabilities (dashed green line) at the point $g(s_{t+1})$. It shows that if the two CDFs are within ε of each other, then $p(s_{t+1}|s_t,a_t)$ and the empirical probability $\frac{N^n(s_{t+1})}{N^n(s_t,a_t)}$ must be within 2ε of each other.

Corollary 4.5.1. *For all* $(s_t,a_t) \in \mathcal{S}\times\mathcal{A}$ *and for all* $\varepsilon > 0$*:*

$$\Pr\left(\max_{s_{t+1}\in \text{Succ}(s_t,a_t)} \left|\frac{N^n(s_{t+1})}{N^n(s_t,a_t)} - p(s_{t+1}|s_t,a_t)\right| > 2\varepsilon\right) \leq 2\exp\left(-2\varepsilon^2 N^n(s_t,a_t)\right), \tag{4.59}$$

as $N^n(s_t,a_t)\to\infty$*. Consequently* $\frac{N^n(s_{t+1})}{N^n(s_t,a_t)} \xrightarrow{p} p(s_{t+1}|s_t,a_t)$ *as* $N^n(s_t,a_t)\to\infty$*.*

Figure 4.18 gives the intuition behind how the DKW inequality leads to this corollary, which bounds the true transition probabilities against the empirical ones. It also shows where the 2ε term comes from.

Proof. Let $m = N^n(s_t,a_t)$ and let g be an arbitrary bijection from $\mathcal{S}$ to $\mathbb{N}_{\mathcal{S}} = \{1,2,3,\cdots,|\mathcal{S}|\}$. Consider the following true and empirical probability mass functions for $i \in \mathbb{N}_{\mathcal{S}}$:

$$f(i) = p(g^{-1}(i) \mid s_t,a_t), \tag{4.60}$$

$$f_m(i) = \frac{N^n(g^{-1}(i))}{m}, \tag{4.61}$$

with corresponding cumulative distribution functions F and F_m respectively. For any $0 < \delta < 1$ it is true that $f(i) = F(i+\delta) - F(i-\delta)$ and $f_m(i) = F_m(i+\delta) - F_m(i-\delta)$ by construction, so by the triangle inequality:

$$|f(i) - f_m(i)| \leq |F(i+\delta) - F_m(i+\delta)| + |F(i-\delta) - F_m(i-\delta)|. \tag{4.62}$$

Hence the following implication holds (depicted at one point in Figure 4.18):

$$\sup_{i \in \mathbb{N}_{\mathcal{S}}} |F(i) - F_m(i)| \leq \varepsilon \implies \sup_{i \in \mathbb{N}_{\mathcal{S}}} |f(i) - f_m(i)| \leq 2\varepsilon. \tag{4.63}$$

Taking the contrapositive and using the DKW inequality (Theorem 2.5.1) then gives the result:

$$\begin{aligned}
&\Pr\left(\max_{s_{t+1} \in \mathrm{Succ}(s_t, a_t)} \left| \frac{N^n(s_{t+1})}{N^n(s_t, a_t)} - p(s_{t+1}|s_t, a_t) \right| > 2\varepsilon \right) && (4.64)\\
=&\Pr\left(\sup_{i \in \mathbb{N}_{\mathcal{S}}} |f(i) - f_m(i)| > 2\varepsilon \right) && (4.65)\\
\leq&\Pr\left(\sup_{i \in \mathbb{N}_{\mathcal{S}}} |F(i) - F_m(i)| > \varepsilon \right) && (4.66)\\
\leq&2\exp\left(-2\varepsilon^2 m\right) && (4.67)
\end{aligned}$$

□

4.5.3 Consistency Results

This subsection proves the consistency results for (AR-)DENTS. The proof of Theorem 4.2.8 is given in full, as it uses all of the preliminary lemmas. The proofs of the remaining theorems are given as outlines, highlighting only where they differ.

Theorems 4.2.1, 4.2.2 and 4.2.4 for (AR-)BTS follow directly from Theorems 4.2.5, 4.2.6 and 4.2.8 respectively, by setting $\beta_{\text{DNTS}}(x) = 0$ and $\beta_{\text{AR-DNTS}}(x) = 0$.

Theorem 4.2.8. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-DNTS}} > 0$, $\alpha_{\text{AR-DNTS}}(x) \to 0$ and $\beta_{\text{AR-DNTS}}(x) \to 0$, then $\pi_{\text{AR-DNTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-DNTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-DNTS}})] \to 0$.*

Proof. Firstly by Corollary 4.5.2 (of Lemma 4.5.2), with the assumption that $\epsilon_{\text{AR-DNTS}} > 0$, all visit counts $N^n(s_t)$ and $N^n(s_t, a_t)$ will tend to infinity almost surely as $n \to \infty$.

Then the (Q-)value estimates and search policy will converge almost surely to their optimal counterparts. This follows by induction on the timestep t. For the base case $t = H$ at the planning horizon, $\hat{V}^{(N^n(s_H))}_{\text{AR-DNTS}}(s_H) = V^*(s_H) = 0$ by definition. The inductive steps proceed as follows, for each $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$:

1. Assume that $\hat{V}^{(N^n(s_{t+1}))}_{\text{AR-DNTS}}(s_{t+1}) \xrightarrow{p} V^*(s_{t+1})$;
2. By Lemma 4.5.3, it follows that $\hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ and consequently $\psi^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;
3. Letting $\dot{Q}^n(s_t, a_t) = \hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) + \beta_{\text{AR-DNTS}}(N^n(s_t))\bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{AR-DNTS}}(s_t, a_t)$, the assumption that $\beta_{\text{AR-DNTS}}(x) \to 0$ ensures that $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$. Then using Lemma 4.5.4, with the assumption that $\alpha_{\text{AR-DNTS}}(x) \to 0$, it follows that $\pi^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;
4. Then by Lemma 4.5.5 it follows that $\frac{N^n(s_t,a_t)}{N^n(s_t)} \xrightarrow{p} \pi^*(a_t|s_t)$ and consequently $\texttt{mv}^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;
5. By Lemma 4.5.7, $\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) \xrightarrow{p} V^*(s_t)$.

The immediate regret (Definition 4.5.1) then satisfies:

$$\begin{aligned}
\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n_{\text{AR-DNTS}})\right] &= \sum_{a\in\mathcal{A}} \left(V^*(s_t) - Q^*(s_t, a)\right) \psi^n_{\text{AR-DNTS}}(a|s_t) && (4.68)\\
&\xrightarrow{p} \sum_{a\in\mathcal{A}} \left(V^*(s_t) - Q^*(s_t, a)\right) \pi^*(a|s_t) && (4.69)\\
&= \mathbb{E}\left[\texttt{imm_regr}(s_t, \pi^*)\right] && (4.70)\\
&= 0, && (4.71)
\end{aligned}$$

and similarly

$$\mathbb{E}\left[\texttt{imm_regr}(s_t, \texttt{mv}^n_{\text{AR-DNTS}})\right] \to 0. \tag{4.72}$$

The result follows by Lemma 4.5.1 to lift the convergence of the immediate regret to convergence of the simple regret. □

The proof of Theorem 4.2.5 proceeds similarly, but since only the recommendation policy ψ_{DNTS} is required to converge, steps 3 and 4 are unnecessary.

Theorem 4.2.5. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, the expected simple regret of ψ_{DNTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \to 0$.*

Proof outline. Proof proceeds similarly to the proof of Theorem 4.2.8, where the induction steps 3 and 4 are unnecessary, and hence the conditions $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$ are not required. Induction step 5 uses Lemma 4.5.6 instead of Lemma 4.5.7. □

The proof of Theorem 4.2.6 is nearly identical to that of Theorem 4.2.8, the only difference being that Lemma 4.5.6 is used in place of Lemma 4.5.7.

Theorem 4.2.6. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$, then $\pi_{\text{DNTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{DNTS}})] \to 0$.*

Proof outline. Proof proceeds similarly to the proof of Theorem 4.2.8. □

The following lemma considers a bandit setting at a single decision node, and shows that the logarithmically decaying exploration term ensures every action is selected infinitely often almost surely.

Lemma 4.5.2. *Let ρ^m be an arbitrary policy and consider the following search policy used on the mth round of a bandit problem:*

$$\pi^m(a|s) = (1 - \lambda(m))\rho^m(a) + \lambda(m)\frac{1}{|\mathcal{A}|}, \tag{4.73}$$

$$\lambda(m) = \min(1, \epsilon/\log(e + m)), \tag{4.74}$$

where $\epsilon > 0$, $s \in \mathcal{S}$ and $a \in \mathcal{A}$. Let $a^m \sim \pi^m$ be the mth independently sampled action, and let $M^m(a)$ be the number of times action a was selected in the first m rounds, i.e. $M^m(a) = \sum_{i=1}^{m} \mathbb{1}[a^i = a]$. Then $M^m(a) \xrightarrow{as} \infty$ as $m \to \infty$ for all $a \in \mathcal{A}$.

Proof. Suppose that the result does not hold and there exists some $b \in \mathcal{A}$ such that:

$$\Pr\left(M^m(b) \to \infty\right) < 1. \tag{4.75}$$

If $M^m(b) \not\to \infty$, then it is logically equivalent to say that there exists some round $\ell \in \mathbb{N}$ from which b is never sampled again (with $\ell \geq \exp(\epsilon + 1)$). That is:

$$\Pr\left(\bigcap_{m=\ell}^{\infty} a^m \neq b\right) = \Pr\left(M^m(b) \not\to \infty\right) > 0. \tag{4.76}$$

However, from Equation (4.73) action b is sampled with probability at least:

$$\Pr\left(a^m = b\right) \geq \frac{\lambda(m)}{|\mathcal{A}|} = \frac{\epsilon}{|\mathcal{A}| \log(e + m)}. \tag{4.77}$$

So the probability of never sampling action b again after the first $\ell - 1$ rounds is at most:

$$\begin{aligned}
\Pr\left(\bigcap_{m=\ell}^{\infty} a^m \neq b\right) &= \lim_{k \to \infty} \prod_{m=\ell}^{k} \Pr(a^m \neq b) && (4.78)\\
&\leq \lim_{k \to \infty} \prod_{m=\ell}^{k} \left(1 - \frac{\epsilon}{|\mathcal{A}| \log(e + k)}\right) && (4.79)\\
&\leq \lim_{k \to \infty} \left(1 - \frac{\epsilon}{|\mathcal{A}| \log(e + k)}\right)^{k - \ell} && (4.80)\\
&= 0, && (4.81)
\end{aligned}$$

which is in contradiction to inequality (4.76) that was assumed.

Inequality (4.80) holds because the factors in the product are increasing with respect to m. To see why the final equality (4.81) holds, consider the simpler function $f(x) = \left(1 - \frac{1}{\log(x)}\right)^x$. Taking logarithms and applying L'Hôpital's rule multiple times gives $\lim_{x \to \infty} \log f(x) = -\infty$, so $\lim_{x \to \infty} f(x) = 0$.

□

As a consequence of Lemma 4.5.2, every reachable state (and state-action pair) will be visited infinitely often in any MCTS process using a search policy of the form given in Equation (4.73).

The following corollary lifts Lemma 4.5.2 to the full tree search setting, where the search policy is indexed by the number of trials n, but the lemma is applied at each node with respect to its visit count $m = N^n(s_t)$.

Corollary 4.5.2. *For all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$, any MCTS process using a search policy of the following form on the nth trial:*

$$\pi^n(a_t|s_t) = (1 - \lambda(s_t, \epsilon))\rho^n(a_t|s_t) + \lambda(s_t, \epsilon)\frac{1}{|\mathcal{A}|}, \tag{4.82}$$

$$\lambda(s_t, \epsilon) = \min(1, \epsilon/\log(e + N^n(s_t))), \tag{4.83}$$

where ρ^n is an arbitrary policy and $\epsilon > 0$. All reachable states and state-action pairs will be visited infinitely often. That is $N^n(s_t) \overset{as}{\to} \infty$ and $N^n(s_t, a_t) \overset{as}{\to} \infty$ as $n \to \infty$.

Proof outline. The result follows by induction over Lemma 4.5.2. At $t = 0$ the result holds directly from the lemma with $m = n$. For the inductive step, if $N^n(s_t, a_t) \overset{as}{\to} \infty$ then $N^n(s_{t+1}) \overset{as}{\to} \infty$ since s_{t+1} is sampled from $p(\cdot|s_t, a_t)$ with positive probability, then $N^n(s_{t+1}, a_{t+1}) \overset{as}{\to} \infty$ by applying Lemma 4.5.2 with $m = N^n(s_{t+1})$. □

The following lemma shows that if the value estimates converge to the optimal values, then the Q-value estimates computed via a sample-weighted average of child node values also converge to the optimal Q-values.

Lemma 4.5.3. *For all $s_t \in \mathcal{S}_0$, $a_t \in \mathcal{A}$, if $N^n(s_t, a_t) \to \infty$ and $\dot{V}^{(N^n(s'))}(s') \overset{p}{\to} V^*(s')$ for all $s' \in \text{Succ}(s_t, a_t)$, then the Q-values computed by:*

$$\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) = R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \dot{V}^{(N^n(s'))}(s'), \tag{4.84}$$

converge to the optimal Q-values: $\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) \overset{p}{\to} Q^*(s_t, a_t)$.

Proof outline. By Corollary 4.5.1, $\frac{N^n(s')}{N^n(s_t,a_t)} \xrightarrow{p} p(s'|s_t,a_t)$ as $N^n(s_t,a_t) \to \infty$. Then:

$$\dot{Q}^{(N^n(s_t,a_t))}(s_t,a_t) = R(s_t,a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t,a_t)} \dot{V}^{(N^n(s'))}(s') \tag{4.85}$$

$$\xrightarrow{p} R(s_t,a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} p(s'|s_t,a_t) V^*(s') \tag{4.86}$$

$$= Q^*(s_t,a_t). \tag{4.87}$$

□

For Boltzmann search policies, if the Q-value estimates converge to the optimal Q-values and the temperature tends to zero, then the search policy converges to the optimal policy.

Lemma 4.5.4. *If a search policy of the following form is used in MCTS:*

$$\pi^n(a_t|s_t) = (1 - \lambda(s_t,\epsilon))\rho^n(a_t|s_t) + \lambda(s_t,\epsilon)\frac{1}{|\mathcal{A}|}, \tag{4.88}$$

$$\rho^n(a_t|s_t) \propto \begin{cases} \exp\left(\frac{\dot{Q}^n(s_t,a_t)}{\alpha(N^n(s_t))}\right) & \text{if } \alpha(N^n(s_t)) > 0, \\ \mathbb{1}[\dot{Q}^n(s_t,a_t) = \max_{a \in \mathcal{A}} \dot{Q}^n(s_t,a)] & \text{if } \alpha(N^n(s_t)) = 0, \end{cases} \tag{4.89}$$

$$\lambda(s_t,\epsilon) = \min(1, \epsilon/\log(e + N^n(s_t))), \tag{4.90}$$

where $\epsilon > 0$, $\alpha(x) \to 0$ *and* $\dot{Q}^n(s_t,a_t) \xrightarrow{p} Q^*(s_t,a_t)$ *as* $n \to \infty$ *for all* $s_t \in \mathcal{S}_0$ *and* $a_t \in \mathcal{A}$. *Then* $\pi^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$ *as* $n \to \infty$.

Proof outline. From Corollary 4.5.2, $N^n(s_t) \xrightarrow{as} \infty$ as $n \to \infty$, and hence $\lambda(s_t,\epsilon) \xrightarrow{as} 0$, so $\pi^n(a_t|s_t) - \rho^n(a_t|s_t) \xrightarrow{as} 0$.

It is known that a Boltzmann distribution converges to the one-hot distribution over the argmax as the temperature tends to zero: if $\alpha(N^n(s_t)) \to 0$ and $\dot{Q}^n(s_t,a_t) \xrightarrow{p} Q^*(s_t,a_t)$, then $\rho^n(a_t|s_t) \xrightarrow{p} \mathbb{1}[a_t = \arg\max_{a \in \mathcal{A}} Q^*(s_t,a)] = \pi^*(a_t|s_t)$.

Combining these two results gives $\pi^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$. □

The following lemma shows that if the search policy converges to a limiting policy under which one action is strictly preferred at every state, then the most-visited recommendation policy converges to that uniquely preferred action.

Lemma 4.5.5. *Suppose* $\pi^n \xrightarrow{as} \pi_{\lim}$ *as* $n \to \infty$*, where* $\pi_{\lim}$ *is a policy maximised by some action* a_t^**. That is,* $\pi_{\lim}(a_t^*|s_t) > \pi_{\lim}(a_t|s_t)$ *for all* $a_t \in \mathcal{A} - \{a_t^*\}$ *and* $s_t \in \mathcal{S}_0$*. Then the empirical action frequencies converge almost surely to the limiting policy:*

$$\frac{N^n(s_t, a_t)}{N^n(s_t)} \xrightarrow{as} \pi_{\lim}(a_t|s_t). \tag{4.91}$$

Consequently, since $\pi_{\lim}$ *has a unique maximising action* $a_t^* = \arg\max_{a \in \mathcal{A}} \pi_{\lim}(a|s_t)$ *at every state, the most-visited recommendation policy converges to always recommending* a_t^**.*

Proof.

$$\begin{aligned}
\frac{N^n(s_t, a_t)}{N^n(s_t)} &= \sum_{i \in T^n(s_t)} \frac{\mathbb{1}[a_t^i = a_t]}{N^n(s_t)} && (4.92)\\
&\xrightarrow{as} \lim_{n \to \infty} \sum_{i \in T^n(s_t)} \frac{\pi^i(a_t|s_t)}{N^n(s_t)} && (4.93)\\
&= \pi_{\lim}(a_t|s_t), && (4.94)
\end{aligned}$$

where line (4.92) uses Equation (4.44). The almost sure convergence follows from Kolmogorov's criterion[7] [33]. The final equality uses Corollary 4.5.2 and Stolz-Cesàro's theorem: since $\pi^n(a_t|s_t) \xrightarrow{as} \pi_{\lim}(a_t|s_t)$ by assumption, the running average converges to $\pi_{\lim}(a_t|s_t)$.

Since $\frac{N^n(s_t,a_t)}{N^n(s_t)} \xrightarrow{as} \pi_{\lim}(a_t|s_t)$ and $\pi_{\lim}$ has a unique maximising action a_t^* at every state, it follows that $\arg\max_{a \in \mathcal{A}} N^n(s_t, a) \xrightarrow{as} a_t^*$, so the most-visited recommendation policy (e.g. Equation (4.24)) converges to always recommending a_t^*. □

[7] A form of the Strong Law of Large Numbers for independent but *not* necessarily identically distributed random variables.

The next two lemmas show that the state value estimates converge to the optimal values, for DENTS and AR-DENTS respectively, given that the Q-value estimates have already converged.

For DENTS, the state value is computed as the maximum over Q-values, so convergence follows directly.

Lemma 4.5.6. *If* $N^n(s_t) \xrightarrow{as} \infty$ *and* $\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ *for all* $s_t \in \mathcal{S}_0$ *and* $a_t \in \mathcal{A}$*, then* $\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) \xrightarrow{p} V^*(s_t)$.

Proof outline. From the definition of the DENTS state value backup (Equation (D.8)), it follows that:

$$\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) = \max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) \tag{4.95}$$

$$\xrightarrow{p} \max_{a \in \mathcal{A}} Q^*(s_t, a) \tag{4.96}$$

$$= V^*(s_t). \tag{4.97}$$

□

For AR-DENTS, the state value is computed as a visit-count weighted average of Q-values, so convergence additionally requires the search policy to converge.

Lemma 4.5.7. *If* $N^n(s_t) \xrightarrow{as} \infty$, ${\pi_{\text{AR-DNTS}}}^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$ *and* $\hat{Q}_{\text{AR-DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ *for all* $s_t \in \mathcal{S}_0$ *and* $a_t \in \mathcal{A}$*, then* $\hat{V}_{\text{AR-DNTS}}^{(N^n(s_t))}(s_t) \xrightarrow{p} V^*(s_t)$.

Proof outline. An equivalent equation, computed using child Q-values and visit counts, for the AR-DENTS state value backup is:

$$\hat{V}_{\text{AR-DNTS}}^{(N^n(s_t))}(s_t) = \sum_{a \in \mathcal{A}} \frac{N^n(s_t, a)}{N^n(s_t)} \hat{Q}_{\text{AR-DNTS}}^{(N^n(s_t,a))}(s_t, a). \tag{4.98}$$

By Lemma 4.5.5, $\frac{N^n(s_t,a)}{N^n(s_t)} \xrightarrow{p} \pi^*(a|s_t)$ as $N^n(s_t) \to \infty$. It follows that:

$$\hat{V}_{\text{AR-DNTS}}^{(N^n(s_t))}(s_t) \xrightarrow{p} \sum_{a \in \mathcal{A}} \pi^*(a|s_t) Q^*(s_t, a) \tag{4.99}$$

$$= V^*(s_t). \tag{4.100}$$

□

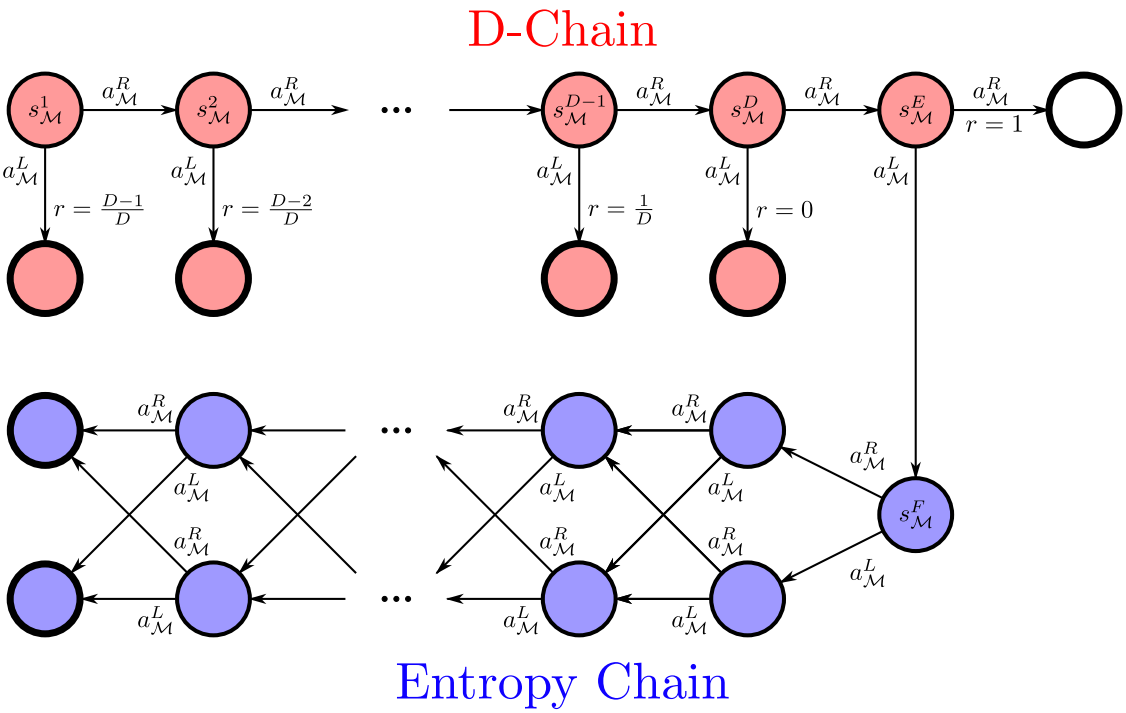


Figure 4.19: (Repeated). An illustration of the *Entropy Trap problem*, with additional state labels added for reference in the proof below.

4.5.4 The Entropy Trap

This subsection provides the proof of Theorem 4.3.1, that informally says that for any temperature either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy. The core of the argument reduces to bounding the *hitting time* of the state $s^E_{\mathcal{M}}$ — the number of trials required for MENTS to first reach the end of the D-chain portion of the Entropy Trap MDP. For reference, the Entropy Trap MDP is repeated in Figure 4.19, with additional state labels added for the proof.

Theorem 4.3.1. *For any value of α_{MENTS}, there exists an MDP such that either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy.*

More precisely, after n trials of MENTS, $\Pr(\psi_{\text{MENTS}} \neq \pi^*) > \frac{1}{2}$ *whenever* $n \leq 2^{D-1}$. *Moreover, either* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \not\to 0$, *or there exist constants* $c, k > 0$ *such that* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \geq c\left(1 - \frac{n}{k^{|\mathcal{S}|}}\right)$.

Proof outline. Proof is by construction. Consider the Entropy Trap MDP defined in Figure 4.19, which is parameterised by D the length of the D-chain, and K, half the number of states in the entropy trap. In this proof, the case where $K = D$ is considered.

Note that for clarity, the full superscript notation established in Section 4.5.1 is dropped in this proof outline to avoid confusion with the Entropy Trap state notation $s^i_{\mathcal{M}}$.

Two cases need to be considered: when the temperature is sufficiently high for the maximum entropy objective to be misaligned; and, when the temperature is lower than this threshold.

Case 1: $\alpha_{\text{MENTS}} > \frac{1}{\log(2)D}$ **(Maximum entropy objective is misaligned)**

When $\alpha_{\text{MENTS}} > \frac{1}{\log(2)D}$, the optimal soft value of the state $s^F_{\mathcal{M}}$ is greater than one, and so action $a^L_{\mathcal{M}}$ is preferred over $a^R_{\mathcal{M}}$ at state $s^E_{\mathcal{M}}$. Let ϕ be a uniform policy. Then:

$$
\begin{aligned}
Q^*_{\text{sft}}(s^E_{\mathcal{M}}, a^L_{\mathcal{M}}) &= V^*_{\text{sft}}(s^F_{\mathcal{M}}) && (4.101)\\
&= 0 + \alpha_{\text{MENTS}} \cdot \mathcal{H}(\phi) && (4.102)\\
&= \alpha_{\text{MENTS}} \cdot \log(2)D && (4.103)\\
&> 1 && (4.104)\\
&= Q^*_{\text{sft}}(s^E_{\mathcal{M}}, a^R_{\mathcal{M}}). && (4.105)
\end{aligned}
$$

Hence the maximum entropy objective is misaligned: $\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \to 1 > 0$.

Case 2: $\alpha_{\text{MENTS}} \leq \frac{1}{\log(2)D}$

In this case, the temperature is sufficiently low that the MENTS search policy is nearly greedy. Hence MENTS will have a small probability of ever having a trial hitting state $s^E_{\mathcal{M}}$. For now, assume that $\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0$, and $N(s^E_{\mathcal{M}}) = 0$.

First consider the composed and simplified soft backup on the D-chain states for $1 \leq i \leq D$:

$$\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leftarrow \alpha_{\text{MENTS}} \log \left(\exp\left(\frac{D-i}{D\alpha_{\text{MENTS}}} \right) + \exp\left(\frac{\hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})}{\alpha_{\text{MENTS}}} \right) \right) \tag{4.106}$$

$$\leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}}) \right) + \alpha_{\text{MENTS}} \log(2) \tag{4.107}$$

$$\leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}}) \right) + \frac{1}{D} \tag{4.108}$$

where inequality (4.107) uses the property of log-sum-exp functions that $\alpha \log \sum_{j=1}^{\ell} \exp(x_j/\alpha) \leq \max_j(x_j) + \alpha \log(\ell)$.

Then, by induction on $i = D, D-1, \ldots, 1$, it holds that $\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leq \frac{D-(i-1)}{D}$:

$$\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}}) \right) + \frac{1}{D} \tag{4.109}$$

$$= \frac{D-i}{D} + \frac{1}{D} \tag{4.110}$$

$$= \frac{D-(i-1)}{D}. \tag{4.111}$$

Hence, it must be the case that $\hat{Q}_{\text{MENTS}}(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) > \hat{Q}_{\text{MENTS}}(s^i_{\mathcal{M}}, a^R_{\mathcal{M}})$ and $\pi_{\text{MENTS}}(a^R_{\mathcal{M}}|s^i_{\mathcal{M}}) \leq \frac{1}{2}$ for all $1 \leq i \leq D$.

Let $Y(n)$ be the event that the state $s^E_{\mathcal{M}}$ is visited in the nth trial. Additionally, let $Z(n)$ be the event that $s^E_{\mathcal{M}}$ has not been visited on any of the first n trials. That is, $Z(n) = \bigcap_{j=1}^{n} \neg Y(j)$. Additionally note that $Z(n)$ implies that $\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0$ and so:

$$\Pr(Y(n)|Z(n-1)) = \Pr(Y(n)|\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0) \tag{4.112}$$

$$\leq 2^{-D}. \tag{4.113}$$

Hence, the probability that $s^E_{\mathcal{M}}$ is not visited in the first n trials can be bounded below inductively:

$$\begin{aligned}
\Pr(Z(n)) &= \Pr\left(\neg Y(n) \cap Z(n-1)\right) && (4.114)\\
&= \Pr\left(\neg Y(n) | Z(n-1)\right) \Pr(Z(n-1)) && (4.115)\\
&\geq (1 - 2^{-D}) \Pr(Z(n-1)) && (4.116)\\
&\geq \dots && (4.117)\\
&\geq (1 - 2^{-D})^n \Pr(Z(0)) && (4.118)\\
&\geq (1 - n2^{-D}), && (4.119)
\end{aligned}$$

where the last line uses Bernoulli's inequality and $\Pr(Z(0)) = 1$. $Z(0)$ is trivially true, as no trials have been run yet.

From the argument above, $Z(n)$ implies that $\hat{Q}_{\text{MENTS}}(s^1_{\mathcal{M}}, a^R_{\mathcal{M}}) \leq \frac{D-1}{D} = \hat{Q}_{\text{MENTS}}(s^1_{\mathcal{M}}, a^L_{\mathcal{M}})$ and so $\psi_{\text{MENTS}}(s^1_{\mathcal{M}}) = a^L_{\mathcal{M}} \neq a^R_{\mathcal{M}} = \pi^*(s^1_{\mathcal{M}})$. Hence, $\Pr(\psi_{\text{MENTS}} \neq \pi^*) = \Pr(Z(n)) \geq (1 - n2^{-D})$, which is greater than $\frac{1}{2}$ if $n \leq 2^{D-1}$.

For the second part of the result, a simple regret of at least $\frac{D-1}{D}$ is incurred with probability at least $\Pr(Z(n))$. Hence, the expected simple regret can be bounded below as follows:

$$\begin{aligned}
\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] &\geq \left(1 - \frac{D-1}{D}\right) \Pr(Z(n)) && (4.120)\\
&\geq \frac{1}{D}\left(1 - n2^{-D}\right). && (4.121)
\end{aligned}$$

Then $|\mathcal{S}| = 4D + 3 \leq 5D$ for $D \geq 3$ and so substituting $D \geq \frac{|\mathcal{S}|}{5}$ gives the result with $c = \frac{1}{D}$ and $k = 2^{1/5}$:

$$\begin{aligned}
\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] &\geq \frac{1}{D}\left(1 - n2^{-D}\right) && (4.122)\\
&\geq \frac{1}{D}\left(1 - \frac{n}{\left(2^{\frac{1}{5}}\right)^{|\mathcal{S}|}}\right). && (4.123)
\end{aligned}$$

□

4.6 Summary

This chapter investigated the use of entropy-based exploration in MCTS for the exploration setting of reinforcement learning, addressing **Q1.1 - Entropy** and laying the groundwork for **Q1.2 - Multi-Objective Exploration** in Chapter 6.

The central finding is that the maximum entropy objective can be misaligned with the standard objective: for any fixed temperature, there exists an MDP in which MENTS either fails to recommend the optimal policy or requires an exponential number of trials to do so (Theorem 4.3.1). This was demonstrated both theoretically, through the D-chain and Entropy Trap MDPs, and experimentally in the gridworld and Go experiments of Section 4.4.

The BTS and DENTS algorithms were introduced (Section 4.2) as alternative MCTS algorithms using Boltzmann exploration, where DENTS uses entropy as an auxiliary exploration bonus rather than a primary objective. By maintaining separate Bellman value estimates and entropy estimates, DENTS is able to use entropy to guide exploration during search whilst discarding it for recommendations. Both algorithms were shown to be consistent under mild conditions (Theorems 4.2.1, 4.2.5 and 4.2.8), and BTS and DENTS empirically outperformed UCT and MENTS across the gridworld environments, demonstrating their ability to balance exploration and exploitation effectively in both deterministic and stochastic settings.

The stochastic search policies of BTS and DENTS also enable use of the Alias Method (Section 4.2.3), which significantly reduces the per-trial computational cost and allowed BTS and DENTS to run substantially more trials per move than PUCT in the Go experiments, addressing **Q3.1 - Complexity**.

The focus now shifts to the multi-objective setting. Chapter 5 extends the MCTS framework to MOMDPs using convex hull value sets, and the question of how to incorporate entropy-based exploration in the multi-objective setting is revisited in Chapter 6.

5

Convex Hull Monte Carlo Tree Search

Contents

5.1 Introduction and Motivation 128
5.1.1 Consistent Action Selection 129
5.1.2 Contextual Tree Search 131
5.2 Contextual Zooming for Trees 132
5.2.1 Algorithm Definition 132
5.2.2 Differences from Contextual Zooming 135
5.3 Convex Hull Monte Carlo Tree Search 136
5.3.1 Common Framework 137
5.3.2 Convex Hull UCT 138
5.3.3 Convex Hull BTS 138
5.3.4 Convex Hull CZT 138
5.3.5 Prior Search Policies 139
5.3.6 Optimisations 140
5.3.7 Entropy and Convex Hulls 143
5.4 Empirical Results 144
5.4.1 Environments 144
5.4.2 Evaluation Procedure 147
5.4.3 Multi-Objective Gridworld Results 148
5.4.4 Scalability Results 152
5.5 Theoretical Discussion 155
5.5.1 Notation 156
5.5.2 Consistent Action Selection and the Linear Utility Assumption . . 157
5.5.3 When Local Optimality Implies Global Optimality 161
5.5.4 Convergence of CHMCTS 163
5.6 Summary 164

This chapter considers multi-objective Monte Carlo Tree Search (MOMCTS) for the unknown weights scenario (Section 2.6 and Figure 2.10) with linear utilities, where the agent must plan for an unknown weight vector $\mathbf{w}$ over the objectives. Primarily, this chapter aims to address **Q2 - Stochastic Multi-Objective Domains**.

Two approaches are considered in this chapter, the first following the same design paradigm as UCT where an algorithm for CMAB problems (Section 2.1.3) is applied at each decision node; the second is to maintain a convex hull value set at each node in the search tree, representing the set of achievable expected returns across all possible weightings over the objectives which is updated using convex hull backups (Section 2.6.1).

This chapter is organised as follows:

- Section 5.1 introduces the *consistent action selection* problem, and motivates *contextual tree search* as the approach taken in this chapter to resolve it;
- Section 5.2 introduces *Contextual Zooming for Trees* (CZT), a MOMCTS algorithm following the same design paradigm as UCT;
- Section 5.3 generalises prior work in MOMCTS to stochastic environments through *convex hull backups*;
- Section 5.4 presents empirical results on Deep Sea Treasure environments and a range of multi-objective environments from the Multi-Objective Gymnasium library [34];
- Section 5.5 discusses and justifies the assumption of a linear utility function, and outlines convergence proofs for CHMCTS.

5.1 Introduction and Motivation

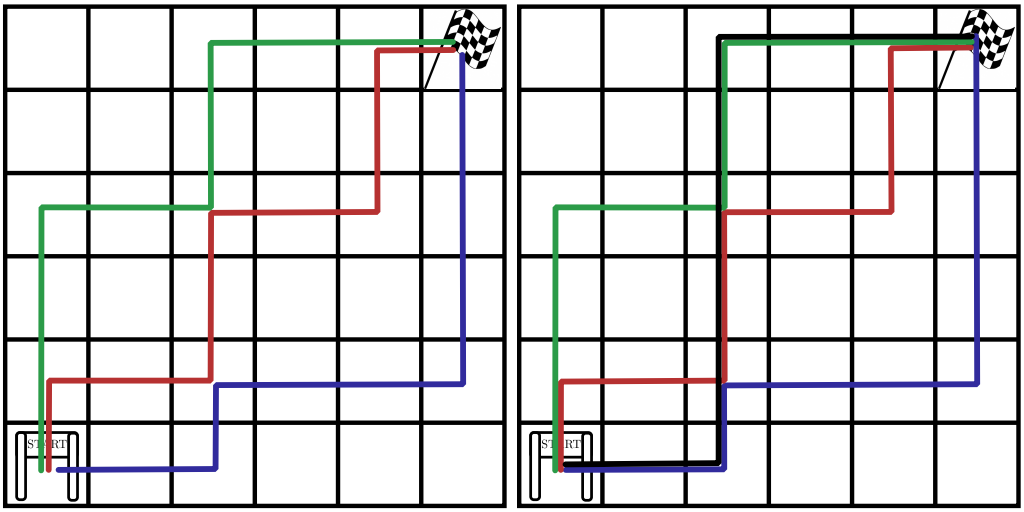


Figure 5.1: Example of locally optimal actions that are not globally optimal (adapted from Van Moffaert and Nowé [103]). (a) The three globally optimal paths in green, red and blue. (b) The black path interleaves actions from the green, red and blue paths where actions are locally optimal at each step, but is a globally suboptimal path.

As set out in Section 1.2, the MOMCTS algorithms in this thesis are used in the *planning phase* of the *unknown weights scenario* (Section 2.6): the weight vector is unknown while planning, so the aim is to produce a solution set of policies for the initial state of the MOMDP, covering all possible preferences.

First, Section 5.1.1 highlights that actions need to be selected consistently to follow a policy from the initial state during the *execution phase*. Then Section 5.1.2 extends the argument to the *planning phase* to arrive at the *contextual tree search* approach taken in this thesis: a weight vector is sampled at random at the start of each trial, and used to select actions consistently throughout it.

5.1.1 Consistent Action Selection

During the *execution phase* the agent follows a single policy. Given a set of multi-objective Q-values (for example a convex hull value set, Definition 2.6.11), how can such a policy be extracted? In the single-objective case the solution is simple: given the optimal Q-values for an MDP, an optimal policy can be extracted by taking an argmax. However, as discussed by Van Moffaert and Nowé [103], the

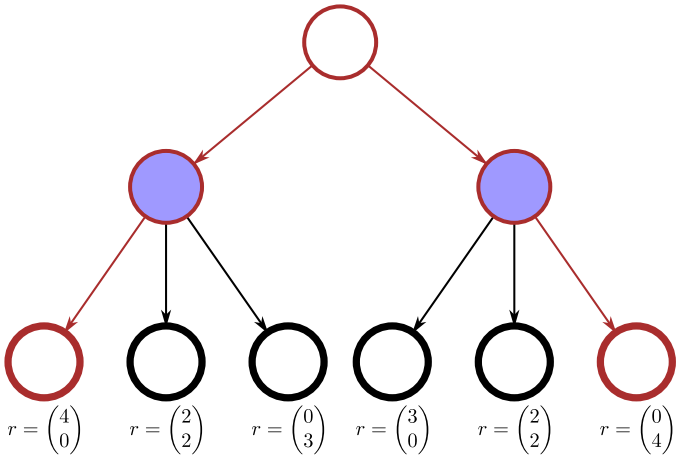


Figure 5.2: A motivating example MOMDP for contextual tree search. The convex hull value set at the root consists of $\{(4,0),(0,4)\}$, following the red paths. The optimal values at the root node require specific actions at the child nodes (blue), while at the child nodes all actions are *locally optimal*.

question is not easily resolved in the multi-objective case. They compute and track Pareto-optimal policies using an adaptation of Q-learning and, critically, highlight that even if an action is *locally optimal*, it may not be *globally optimal*. An action is *locally optimal* if it maximises the utility achievable from the current state, whereas a *globally optimal* policy is any policy in the undominated set of policies (Definition 2.6.7). To demonstrate this disparity, they use an example with three optimal paths (Figure 5.1), where the black path alternates between multiple optimal paths, taking only locally optimal actions at each step, but fails to follow any of the three globally optimal paths.

This is an issue for MOMCTS algorithms, because MCTS algorithms make local decisions at each decision node, and fundamentally assume that a globally optimal solution (i.e. an undominated policy from the initial state) can be found by making local decisions.

Section 5.5 discusses the problem more formally: Theorem 5.5.1 shows that selecting locally optimal actions leads to globally optimal policies if and only if the utility is linear, which is why this thesis assumes a linear utility.

5.1.2 Contextual Tree Search

While Section 5.1.1 discussed the consistent action selection problem in the context of the execution phase, the problem is still relevant during the planning phase.

Consider the MOMDP illustrated in Figure 5.2, with two objectives. The convex hull value set at the initial state of the MOMDP is $\{(4,0),(0,4)\}$: a reward $(4,0)$ is only reachable via the left action, and $(0,4)$ only via the right action. Under a linear utility, the optimal action at the root node therefore depends entirely on the weight vector $\mathbf{w}$: if $w_1 > w_2$ then the left action is optimal, and if $w_2 > w_1$ then the right action is optimal.

Once the agent has taken the left action from the root node, it has implicitly committed to optimising towards the first objective (i.e. weights with $w_1 > w_2$) because the rewards available from the right action are no longer reachable. Although all subsequent actions are *locally optimal* (i.e. they lead to a value in the local convex hull value set), only the action leading to $(4,0)$ is consistent with the commitment implicitly made by the first action.

Prior MOMCTS work (Section 3.5) typically aims to compute a full Pareto front at all nodes, for example at the blue states in Figure 5.2. Suppose the MOMDP of Figure 5.2 had a longer horizon, where rewards past the second action are small. Computing the Pareto front at all nodes, in particular at the blue states, overprioritises exploration in parts of the MOMDP that are suboptimal from the initial state, leading to search policies that select actions inconsistently within each trial, optimising for no particular objective.

The approach taken in this thesis is to consider a *contextual tree search*, where a context weight vector $\mathbf{w}$ is sampled at the start of each trial and used to guide action selection throughout that trial. For MORL with linear utilities, this is sufficient to resolve the consistent action selection problem, ensuring that all actions within a trial are consistent with a single objective.

In the `THTS++` framework, the context weight $\mathbf{w}$ is sampled during the `sample_context` function (Section 2.4.2) and passed to all subsequent steps of

the trial, including the search policy and backup functions. To reflect this, the context weight is added as a parameter to the policy notation:

Definition 5.1.1. *A* contextual policy $\pi(a|s;\mathbf{w})$ *is a policy that may depend on the context weight vector* $\mathbf{w} \in \Delta^D$*, where* $\pi(a|s;\mathbf{w})$ *gives the probability of selecting action* a *at state* s *under weight* $\mathbf{w}$*.*

A policy π *is* context-independent *if* $\pi(a|s;\mathbf{w}) = \pi(a|s;\mathbf{w}')$ *for all* $\mathbf{w}, \mathbf{w}' \in \Delta^D$ *and all* $s \in \mathcal{S}$*,* $a \in \mathcal{A}$*. Otherwise,* π *is* context-dependent*.*

5.2 Contextual Zooming for Trees

A common design paradigm in MCTS is to adapt a MAB algorithm to run at each decision node of the search tree, treating the node as a (non-stationary) bandit problem. UCT follows this paradigm by running UCB at each node (Section 2.4.3). Section 5.1.2 posed MOMCTS as a contextual problem, in which a weight vector is sampled at the start of each trial, so a natural question is whether a contextual bandit algorithm (Section 2.1.3) can be run at each node in the same way. This section explores that approach.

The *Contextual Zooming* (CZ) algorithm [50, 95] is a generalisation of the UCB algorithm to contextual bandits (Section 2.1.3). Following the same methodology, *Contextual Zooming for Trees* (CZT) runs the CZ algorithm at each decision node of the search tree.

CZT is presented in this section in the context of MOMCTS, where the context is the weight vector $\mathbf{w}$ sampled at the start of each trial. Although CZ, and by extension CZT, is not motivated by the exploration setting this thesis focuses on, it is introduced as a natural contextual analogue of UCT, and as a contextual search policy baseline for MOMCTS, where no such baseline currently exists.

5.2.1 Algorithm Definition

Contextual Zooming operates over a similarity space (defined in Section 2.1.3), which in the case of CZT is the weight-action space $\Delta^D \times \mathcal{A}$. The following metric

is defined over the weight-action space for use in CZT:

$$d_{\text{czt}}((\mathbf{w}, a), (\mathbf{w}', a')) = \|\mathbf{w} - \mathbf{w}'\|_2 + C_{\text{czt}} \mathbb{1}\left[a \neq a'\right], \tag{5.1}$$

where $C_{\text{czt}} > \sqrt{2}$ is a large constant such that the distance between any two actions is always greater than the distance between any two weights.

CZ maintains a set of *active balls* that cover the weight-action space. In CZT, let $A(s, a)$ denote the set of active balls for action a at decision node s, where a ball with centre $(\mathbf{w}, a)$ and radius r is defined as $\texttt{ball}((\mathbf{w}, a), r) = \{(\mathbf{w}', a') \in \Delta^D \times \mathcal{A} \mid d((\mathbf{w}, a), (\mathbf{w}', a')) < r\}$. Let $A(s)$ be the set of all active balls at decision node s, i.e. $A(s) = \cup_{a \in \mathcal{A}} A(s, a)$.

Each set of active balls is initialised to a single ball, using the centroid of the simplex $\frac{1}{D}\mathbf{1}$, where $\mathbf{1}$ is the vector of all ones. Specifically,

$$A^0(s, a) = \left\{ \texttt{ball}\left(\left(\frac{1}{D}\mathbf{1}, a \right), \sqrt{\frac{D-1}{D}} \right) \right\}. \tag{5.2}$$

Each ball $B \in A(s, a)$ has an associated visit count $M(B)$ and average return value estimate $\bar{\mathbf{Q}}_{\texttt{CZT}}(B)$. The visit count and value estimates are initialised to $M(B) = 0$ and $\bar{\mathbf{Q}}_{\texttt{CZT}}(B) = \mathbf{0}$ respectively.

Two rules are applied to run contextual zooming:

- the *selection rule*, to select a ball (and action) from the set of active balls at the current decision node, as part of the `select_action` function;
- the *activation rule*, which optionally adds a ball with smaller radius to the set of active balls, as part of the `backup_v` function.

Selection Rule Let $\mathbf{w}$ be the context weight sampled for the nth trial, let $r(B)$ denote the radius of a ball $B \in A(s, a)$. The *domain* of B in CZ consists of the portion of the ball not covered by any smaller active ball. Specifically,

$$\texttt{dom}(B) = B - \left(\bigcup_{B' \in A(s,a): r(B') < r(B)} B' \right). \tag{5.3}$$

The *confidence radius* $\texttt{conf}(B)$ of ball B is:

$$\texttt{conf}(B) = b_{\texttt{CZT}}\sqrt{\frac{\log(N(s))}{1+M(B)}}, \tag{5.4}$$

where $b_{\texttt{CZT}} > 0$ is a UCB style bias parameter.

Let $\texttt{relevant}(s, a; \mathbf{w})$ denote the set of *relevant* balls at a decision node s for action a. A ball is relevant if $(\mathbf{w}, a)$ is in the domain of the ball. Specifically,

$$\texttt{relevant}(s, a; \mathbf{w}) = \{B \in A(s, a) \mid (\mathbf{w}, a) \in \texttt{dom}(B)\}, \tag{5.5}$$

$$\texttt{relevant}(s; \mathbf{w}) = \cup_{a \in \mathcal{A}} \texttt{relevant}(s, a; \mathbf{w}). \tag{5.6}$$

Contextual Zooming defines a confidence interval term, called the *pre-index*, around each ball, similar to UCB's confidence interval term:

$$I^{\texttt{pre}}_{\texttt{CZT}}(B; \mathbf{w}) = \mathbf{w}^\top \bar{\mathbf{Q}}_{\texttt{CZT}}(B) + r(B) + \texttt{conf}(B). \tag{5.7}$$

The full *index* of a ball B allows a newly activated ball with few visits to inherit an optimistic value estimate from nearby well-explored balls, ensuring it remains eligible for selection. Specifically,

$$I_{\texttt{CZT}}(B; \mathbf{w}) = r(B) + \min_{B' \in A(s)} \left(I^{\texttt{pre}}_{\texttt{CZT}}(B'; \mathbf{w}) + d(B, B')\right), \tag{5.8}$$

where $d(B, B')$ is the distance between the centres of balls B and B' according to the metric d. In CZT the constant C_{czt} is set to a large enough value such that balls with a different action will never minimise the expression, so it suffices to consider only balls with the same action as B in Equation (5.8).

To select a ball, the `select_ball` function chooses the ball from the set of relevant balls with the highest index:

$$\texttt{select_ball}(s, \mathbf{w}) = \operatorname*{arg\,max}_{B \in \texttt{relevant}(s; \mathbf{w})} I_{\texttt{CZT}}(B). \tag{5.9}$$

If $\texttt{select_ball}(s, \mathbf{w}) = \texttt{ball}((\mathbf{w}_B, a_B), r_B)$, then the CZT selection policy returns $\pi_{\texttt{CZT}}(s; \mathbf{w}) = a_B$.

Backups and Activation Rule Let $\boldsymbol{\tau}_{:h} \sim \pi_{\texttt{CZT}}$ be the trajectory sampled in the selection phase of CZT. Let B_t be the ball selected at the decision node s_t.

The backup functions update the ball selection counts and the multi-objective average returns for $t = h-1, \ldots, 0$:

$$M(B_t) \leftarrow M(B_t) + 1, \tag{5.10}$$

$$\bar{\mathbf{Q}}_{\texttt{CZT}}(B_t) \leftarrow \frac{M(B_t)-1}{M(B_t)} \bar{\mathbf{Q}}_{\texttt{CZT}}(B_t) + \frac{\hat{\mathbf{V}}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} \mathbf{r}_i}{M(B_t)}, \tag{5.11}$$

where $\hat{\mathbf{V}}_{\text{heuristic}}(s_h)$ is the (multi-objective) heuristic value function.

The *activation rule* introduces a new ball with smaller radius when the following conditions are met:

$$M(B_t) > M_{\min} \quad \text{and} \quad \texttt{conf}(B_t) \leq r(B_t), \tag{5.12}$$

where $M_{\min}$ is a minimum selection threshold, and when both conditions are met, a new ball $B_{\text{new}} = \texttt{ball}((\mathbf{w}, a_t), \frac{r(B_t)}{2})$ is added to the set of active balls $A(s,a)$.

Recommendation Policy To recommend an action in CZT, the ball with maximum linear utility from the set of active balls is selected. Specifically,

$$\psi_{\texttt{CZT}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \max_{B \in \texttt{relevant}(s,a;\mathbf{w})} \mathbf{w}^\top \bar{\mathbf{Q}}_{\texttt{CZT}}(B). \tag{5.13}$$

5.2.2 Differences from Contextual Zooming

Section 5.2.1 described the CZT algorithm. The main adaptations from CZ are summarised below:

1. In CZT, the set of active balls is specialised to a set for each action. In CZ, a single set of active balls is used across the entire *similarity space* (defined in Section 2.1.3).

2. CZ assumes that the algorithm is run for a known number of rounds T, and uses this value instead of the current number of visits at the decision node $N(s)$ in the confidence radii (Equation 5.4).

3. The activation rule of CZ does not have a hard lower bound on the number of ball visits required to add a new child ball. CZT makes this adaptation to account for item 2, which results in smaller confidence radii.

4. In CZT, all balls contain exactly one action, whereas in CZ with a general continuous space of arms, balls cover many arms. After a ball is selected in CZ, an arm is sampled uniformly randomly from the domain of the selected ball.

CZ [50, 95] extends its regret analysis from a fixed number of rounds T to an unknown number of rounds using the *doubling trick* [11]. The doubling trick runs an independent instance of the algorithm for T rounds, then $2T$ rounds, then $4T$ rounds and so on. In practice, however, discarding all accumulated data at each doubling interval is undesirable. Instead, in CZT the number of visits $N(s)$ at the current node is used directly in place of T, which avoids the need for the doubling trick at the cost of losing regret guarantees.

5.3 Convex Hull Monte Carlo Tree Search

This section introduces *Convex Hull Monte Carlo Tree Search* (CHMCTS), which generalises prior work in MOMCTS (Section 3.5) to stochastic environments by maintaining *convex hull value sets* at each node in the search tree and updating them using convex hull backups (Section 2.6.1). When the MOMDP is deterministic and a search policy from prior work is used, CHMCTS reduces to the prior MOMCTS algorithms discussed in Chapter 3.

The common `THTS++` framework and backup equations shared by all CHMCTS algorithms are given first. The search policies for each algorithm are then defined. Two practical optimisations are discussed in Section 5.3.6, and the section concludes with a brief discussion on the relationship between entropy and convex hull value sets.

5.3.1 Common Framework

Let ALG denote any of the CHMCTS algorithms defined in this section. At each decision node s, a convex hull value set $\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)$ is maintained, and at each chance node (s, a), a convex hull Q-value set $\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s, a)$ is maintained. The specific form of the search policy varies between algorithms and is defined in the subsections below.

For any new decision node s added to the tree, the `init_v` function initialises the value set estimate as $\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s) \leftarrow \left\{\hat{\mathbf{V}}_{\text{heuristic}}(s)\right\}$. Similarly, at new chance nodes (s, a), the `init_q` function initialises the Q-value set estimate as $\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s, a) \leftarrow \left\{\hat{\mathbf{Q}}_{\text{heuristic}}(s, a)\right\}$.

Given a context weight $\mathbf{w} \in \Delta^D$ and a trajectory $\tau_{:h} = (s_0,\ a_0,\ \mathbf{r}_0,\ \ldots,\ s_{h-1},\ a_{h-1},\ \mathbf{r}_{h-1},\ s_h) \sim \pi_{\text{ALG}}(\cdot|\cdot; \mathbf{w})$, the value set estimates are updated for $t = h-1, \ldots, 0$ during the backup phase:

$$\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s_t) \leftarrow \texttt{cvx_prune}\left(\bigcup_{a \in \mathcal{A}} \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s_t, a)\right), \tag{5.14}$$

$$\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s_t, a_t) \leftarrow \mathbf{R}(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s'). \tag{5.15}$$

The `cvx_prune` operation, defined in Equation (2.73) and repeated here for convenience, retains only those vectors in a set $\boldsymbol{\mathcal{V}}$ that are optimal for at least one weight vector:

$$\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}) = \{\mathbf{v} \in \boldsymbol{\mathcal{V}} \mid \exists \mathbf{w} \in \Delta^D.\ \forall \mathbf{v}' \in \boldsymbol{\mathcal{V}} - \{\mathbf{v}\}.\ \mathbf{w}^\top \mathbf{v} > \mathbf{w}^\top \mathbf{v}'\}. \tag{5.16}$$

Since the context weight is passed to policies, it is useful to define *contextual value estimates*, which extract a scalar value from a convex hull value set for a given weight $\mathbf{w} \in \Delta^D$:

$$\hat{V}^{\mathbf{w}}_{\text{ALG}}(s) = \max_{\mathbf{v} \in \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)} \mathbf{w}^\top \mathbf{v}, \tag{5.17}$$

$$\hat{Q}^{\mathbf{w}}_{\text{ALG}}(s, a) = \max_{\mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a)} \mathbf{w}^\top \mathbf{q}. \tag{5.18}$$

The recommendation policy selects the action with the highest contextual Q-value under the given weight:

$$\psi_{\text{ALG}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}}_{\text{ALG}}(s, a). \tag{5.19}$$

5.3.2 Convex Hull UCT

Another natural extension of UCT to the contextual multi-objective setting is to replace the scalar Q-value estimates (used in Section 2.4.3) with contextual value estimates from Equations (5.17) and (5.18). The resulting algorithm, *Convex Hull UCT* (CH-UCT), uses the following search policy:

$$\pi_{\text{CH-UCT}}(s;\mathbf{w}) = \arg\max_{a\in\mathcal{A}} \left[\hat{Q}^{\mathbf{w}}_{\text{CH-UCT}}(s,a) + c_{\text{CH-UCT}}\sqrt{\frac{\log(N(s))}{N(s,a)}}\right], \tag{5.20}$$

where $c_{\text{CH-UCT}} > 0$ is a bias parameter controlling exploration. The search policy is context-dependent (Definition 5.1.1), since the contextual Q-value $\hat{Q}^{\mathbf{w}}_{\text{CH-UCT}}(s,a)$ depends on $\mathbf{w}$, ensuring that action selection is consistent with the sampled objective throughout each trial.

5.3.3 Convex Hull BTS

Convex Hull BTS (CH-BTS) uses the BTS search policy (Section 4.2.1) within the CHMCTS framework, using contextual value estimates in place of the scalar Bellman value estimates:

$$\rho_{\text{CH-BTS}}(a|s;\mathbf{w}) \propto \exp\left(\frac{\hat{Q}^{\mathbf{w}}_{\text{CH-BTS}}(s,a)}{\alpha_{\text{CH-BTS}}(N(s))}\right), \tag{5.21}$$

$$\pi_{\text{CH-BTS}}(a|s;\mathbf{w}) = (1-\lambda(s,\epsilon_{\text{CH-BTS}}))\rho_{\text{CH-BTS}}(a|s;\mathbf{w}) + \frac{\lambda(s,\epsilon_{\text{CH-BTS}})}{|\mathcal{A}|}, \tag{5.22}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e+N(s))}\right), \tag{5.23}$$

where $\alpha_{\text{CH-BTS}}(x) > 0$ is a temperature function and $\epsilon_{\text{CH-BTS}} \in [0,\infty)$ is an exploration parameter. As in BTS, the temperature function controls the trade-off between exploration and exploitation, and the exploration parameter ensures that all actions are eventually explored.

5.3.4 Convex Hull CZT

In single-objective MCTS, Bellman backups often outperform average return backups in the exploration setting (Chapter 4), since they propagate value information more directly through the tree by bootstrapping estimates from successor states rather

than averaging over full trajectories. Convex hull backups can be viewed as the multi-objective analogue of Bellman backups: rather than maintaining a scalar average return at each node, they maintain a set of achievable value vectors that more accurately represents the range of outcomes reachable from that node. The CZT ball value estimates $\bar{\mathbf{Q}}_{\texttt{CZT}}(B)$ provide a coarser representation, averaging over a region of weight space rather than capturing the full structure of achievable values.

Since the CHMCTS backups (Equations (5.14) and (5.15)) are independent of the search policy, they can be combined with any search policy, including CZT. The resulting algorithm, *Convex Hull CZT* (CH-CZT), runs the CZT algorithm but maintains additional convex hull value sets at each node and uses the recommendation policy from Equation (5.19) in place of the CZT recommendation policy. This combines the contextual exploration of CZT with the richer value representation of CHMCTS.

5.3.5 Prior Search Policies

The MOMCTS algorithms discussed in Section 3.5 propose a number of search policies that could also be used within the CHMCTS framework. In general, prior work uses search policies of the following form:

$$\pi_{\text{ALG}}(a|s;\mathbf{w}) = \arg\max_{a\in\mathcal{A}} \left[\zeta_{\text{ALG}}(\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a);\mathbf{w}) + c_{\text{ALG}}\sqrt{\frac{\log(N(s))}{N(s,a)}}\right], \tag{5.24}$$

where ζ_{ALG} is a scalarisation function that maps a convex hull Q-value set to a scalar, and $c_{\text{ALG}} > 0$ is a bias parameter. Notably, when ζ_{ALG} does not depend on $\mathbf{w}$, the search policy is context-independent, which leads to linear cumulative contextual regret [69].

Hypervolume CHMCTS (H-CHMCTS). Perez et al. [74] use the *hypervolume indicator* (Definition 2.6.14) as the scalarisation function ζ_{ALG} in the search policy. That is, $\zeta_{\text{H-CHMCTS}}(\boldsymbol{\mathcal{V}};\mathbf{w}) = \text{HV}(\boldsymbol{\mathcal{V}};\mathbf{r}_{\text{ref}})$, where $\mathbf{r}_{\text{ref}}$ is reference point for the hypervolume indicator and is a parameter of the algorithm.

Chebyshev CHMCTS (C-CHMCTS). Xu and Li [115] use the *Chebyshev utility* (Definition 2.6.13) as a scalarisation function for the search policy. So the scalarisation function used for C-CHMCTS is:

$$\zeta_{\text{C-CHMCTS}}(\boldsymbol{\mathcal{V}}; \mathbf{w}, \mathbf{r}_{\text{uto}}) = \max_{\mathbf{v} \in \boldsymbol{\mathcal{V}}} u_{\text{cheby}}(\mathbf{v}; \mathbf{w}, \mathbf{r}_{\text{uto}}), \tag{5.25}$$

where $\mathbf{r}_{\text{uto}}$ is a *utopian point* satisfying $\mathbf{r}_{\text{uto}} \succeq_P \mathbf{v}$ for all $\mathbf{v} \in \boldsymbol{\mathcal{V}}$ and is a parameter of the algorithm.

Pareto CHMCTS (P-CHMCTS). The notable exception to this form is Pareto MCTS [23], which uses a Pareto UCB algorithm [29, 30] at each decision node, and does not fit directly into the template of Equation (5.24). *Pareto CHMCTS* (P-CHMCTS) is the algorithm that results from running Pareto UCB at each decision node within the CHMCTS framework.

Pareto UCB adds a UCB-style bonus to each vector in the value set:

$$\hat{\boldsymbol{\mathcal{V}}}^{\texttt{conf}}_{\text{P-CHMCTS}}(s) = \texttt{cvx_prune}\left(\hat{\boldsymbol{\mathcal{V}}}^{U}_{\text{P-CHMCTS}}(s)\right), \tag{5.26}$$

$$\hat{\boldsymbol{\mathcal{V}}}^{U}_{\text{P-CHMCTS}}(s) = \left\{(\mathbf{q} + U_{\text{ucb}}(s,a)\mathbf{1}, a) \mid a \in \mathcal{A}, \mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{P-CHMCTS}}(s,a)\right\}, \tag{5.27}$$

$$U_{\text{ucb}}(s,a) = b_{\text{P-CHMCTS}} \sqrt{\frac{\log(N(s))}{N(s,a)}}. \tag{5.28}$$

where $b_{\text{P-CHMCTS}} > 0$ is a bias parameter and where the notation $(\mathbf{q}, a)$ tags each vector with its corresponding action to allow the action to be recovered after the `cvx_prune` operation.

The search policy samples a point uniformly randomly from the *confidence value set*, and selects the corresponding action. That is:

$$\pi_{\text{P-CHMCTS}}(a|s; \mathbf{w}) = \sum_{(\mathbf{v}, a') \in \hat{\boldsymbol{\mathcal{V}}}^{\texttt{conf}}_{\text{P-CHMCTS}}(s)} \frac{\mathbb{1}[a = a']}{|\hat{\boldsymbol{\mathcal{V}}}^{\texttt{conf}}_{\text{P-CHMCTS}}(s)|}. \tag{5.29}$$

5.3.6 Optimisations

Two practical optimisations are used in the implementation of CHMCTS: *solved labelling*, which avoids wasting computation on well-explored parts of the search tree; and *size pruning*, which limits the number of points stored in each convex hull value set.

Solved Labelling. In planning, *labelling* [14] is used to mark nodes as solved to avoid wasting computation on exploring parts of the tree that are already solved. A node is marked as solved when its value estimate has converged to the true value, which requires knowledge of the true transition distribution. In the setting considered here, the true transition distribution $p(\cdot|s,a)$ is unknown, so a probabilistic analogue for labelling is used instead.

Each chance node (s,a) is assigned an *unsolved value* $\texttt{unsolved}(s,a) \in [0,1]$, where $\texttt{unsolved}(s,a) = 0$ indicates that the node is solved and $\texttt{unsolved}(s,a) = 1$ indicates that it is unsolved. The unsolved value of a node is an upper bound on the probability that the node's value estimate has not yet converged.

The unsolved value of a decision node is computed as the maximum unsolved value of its children:

$$\texttt{unsolved}(s) = \max_{a \in \mathcal{A}} \texttt{unsolved}(s,a). \tag{5.30}$$

The unsolved values are updated during the backpropagation phase of THTS++ and used during the selection phase to filter out actions that have already been solved, to a tolerance of $\texttt{unsolved}_{\text{tol}}$, with high probability. That is, at a decision node s, any action a with $\texttt{unsolved}(s,a) \leq \texttt{unsolved}_{\text{tol}}$ is filtered out before selecting an action. The remainder of this section describes how the unsolved values are propagated through chance nodes, that is, how the value of $\texttt{unsolved}(s,a)$ is computed.

The unsolved value of a chance node (s,a) is computed as an upper bound on the expected unsolved values of its successor states, using the empirical transition distribution $\frac{N(s')}{N(s,a)}$ as a proxy for the true distribution $p(s'|s,a)$. Since the true distribution is unknown, the bound must account for the estimation error in the empirical distribution. Let $\delta_{\text{unsolved}} > 0$ be a confidence parameter. The unsolved value $\texttt{unsolved}(s,a)$ is defined as an upper bound satisfying:

$$\Pr\Big(\mathbb{E}_{s' \sim p(\cdot|s,a)}[\texttt{unsolved}(s')] \leq \texttt{unsolved}(s,a)\Big) \geq 1 - \delta_{\text{unsolved}}. \tag{5.31}$$

In words, with probability at least $1 - \delta_{\text{unsolved}}$, the expected unsolved value of the children of chance node (s,a) is bounded by the upper bound $\texttt{unsolved}(s,a)$.

To derive this bound, the expected unsolved value is decomposed into contributions from observed and unobserved successor states:

$$\mathbb{E}[\texttt{unsolved}(s')] = \sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} p(s'|s,a)\texttt{unsolved}(s') \tag{5.32}$$

$$+ \sum_{s' \notin \text{Succ}_{\text{obs}}(s,a)} p(s'|s,a)\texttt{unsolved}(s'), \tag{5.33}$$

where $\text{Succ}_{\text{obs}}(s,a) = \{s' \in \text{Succ}(s,a) \mid N(s') > 0\}$ is the set of successor states observed so far.

For the observed successors, Corollary 4.5.1 of the DKW inequality (Theorem 2.5.1) gives with probability at least $1 - \frac{\delta_{\text{unsolved}}}{2}$ for all $s' \in \text{Succ}_{\text{obs}}(s,a)$:

$$p(s'|s,a) \leq \frac{N(s')}{N(s,a)} + 2\sqrt{\frac{\log(4/\delta_{\text{unsolved}})}{2N(s,a)}}, \tag{5.34}$$

so the contribution of observed successors in (5.32) is bounded by:

$$\sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} p(s'|s,a)\texttt{unsolved}(s') \tag{5.35}$$

$$\leq \sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} \left(\frac{N(s')}{N(s,a)} + 2\sqrt{\frac{\log(4/\delta_{\text{unsolved}})}{2N(s,a)}} \right) \texttt{unsolved}(s'). \tag{5.36}$$

The *Good-Turing estimate* (Section 2.5) for the missing mass is:

$$\texttt{missing_mass}_{\text{GT}}(s,a) = \sum_{s' \in \text{Succ}(s,a)} \frac{\mathbb{1}\left[N(s') = 1\right]}{N(s,a)}. \tag{5.37}$$

For the unobserved successors, since $\texttt{unsolved}(s') \leq 1$ for all s', the contribution of unobserved successors (5.33) is bounded by the missing mass using Theorem 2.5.2 with probability at least $1 - \frac{\delta_{\text{unsolved}}}{2}$:

$$\sum_{s' \notin \text{Succ}_{\text{obs}}(s,a)} p(s'|s,a) \leq \texttt{missing_mass}_{\text{GT}}(s,a) + \sqrt{\frac{\log(2/\delta_{\text{unsolved}})}{N(s,a)}}. \tag{5.38}$$

Combining these bounds via the union bound, the following assignment of $\texttt{unsolved}(s,a)$ satisfies (5.31):

$$\texttt{unsolved}(s,a) \leftarrow \sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} \left(\frac{N(s')}{N(s,a)} + 2\sqrt{\frac{\log(4/\delta_{\text{unsolved}})}{2N(s,a)}} \right) \texttt{unsolved}(s')$$

$$+ \texttt{missing_mass}_{\text{GT}}(s,a) + \sqrt{\frac{\log(2/\delta_{\text{unsolved}})}{N(s,a)}}. \tag{5.39}$$

When the `unsolved` labelling is used in CHMCTS, $\texttt{unsolved}_{\text{tol}}$ and δ_{unsolved} are parameters of the algorithm. An action a at state s is marked as solved when $\texttt{unsolved}(s, a) \leq \texttt{unsolved}_{\text{tol}}$, at which point it is no longer selected in subsequent trials.

Convex Hull Value Set Size Pruning. As the search progresses, the convex hull value sets at each node may grow rapidly, increasing the computational cost of the `cvx_prune` operation and the convex hull backups. To limit this cost, a maximum size $K_{\max}$ is imposed on each convex hull value set. When a value set exceeds this limit after a backup, points are pruned using the *crowding distance* from the NSGA-II algorithm [28].

The crowding distance of a point $\mathbf{v}$ in a set $\boldsymbol{\mathcal{V}}$ measures how isolated $\mathbf{v}$ is from its neighbours in objective space. For each objective dimension $i = 1, \ldots, D$, the points in $\boldsymbol{\mathcal{V}}$ are sorted by their ith coordinate. The boundary points (those with the smallest and largest values in dimension i) are assigned an infinite crowding distance and are always retained. For interior points, the crowding distance contribution from dimension i is:

$$\text{cd}_i(\mathbf{v}) = \frac{v_i^+ - v_i^-}{\max_{\mathbf{v}' \in \boldsymbol{\mathcal{V}}} v'_i - \min_{\mathbf{v}' \in \boldsymbol{\mathcal{V}}} v'_i}, \tag{5.40}$$

where v_i^+ and v_i^- are the ith coordinates of the nearest neighbours of $\mathbf{v}$ above and below in dimension i respectively. The total crowding distance is $\text{cd}(\mathbf{v}) = \sum_{i=1}^{D} \text{cd}_i(\mathbf{v})$. When the value set exceeds $K_{\max}$ points, the $K_{\max}$ points with the largest crowding distances are retained, preserving the spread of the convex hull.

5.3.7 Entropy and Convex Hulls

Convex hull value sets represent the set of achievable expected returns in *objective space*, storing value vectors attainable by different policies. Entropy, by contrast, is a property of a policy rather than a value vector: it measures the uncertainty of the action distribution induced by a policy at each state.

A key observation is that a single point on the convex hull does not correspond to a unique policy. Since the contextual policies considered in this chapter depend

on the weight vector $\mathbf{w}$, a given value vector $\mathbf{v} \in \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)$ may be optimal for many different weight vectors, each inducing a different policy $\pi(\cdot|s;\mathbf{w})$ with a different entropy.

As a result, there is no principled way to associate a single entropy value with a point on the convex hull. This is the fundamental reason why the entropy-based exploration used in DENTS (Chapter 4) does not extend naturally to the multi-objective setting with convex hull value sets, and motivates the simplex map approach introduced in Chapter 6.

5.4 Empirical Results

This section presents the empirical results for this chapter. It is structured similarly to Section 4.4 in Chapter 4, as follows:

- Section 5.4.1 describes the MOMDP environments used in the experiments;
- Section 5.4.2 describes the evaluation setup used;
- Section 5.4.3 presents the results on the MOMDP environments;
- Section 5.4.4 presents scalability results on the *Deep Sea Treasure* environment;
- Appendix C provides the algorithm parameters used and how they were chosen.

The implementation of Convex Hull based algorithms used the QHull library [6] for convex hull operations where possible, falling back to the linear programming method to compute `cvx_prune` (Equation 2.73) of Roijers et al. [84] for geometrically degenerate cases.

5.4.1 Environments

Standard gridworld environments from *MO-Gymnasium* [34] are used to investigate the behaviour of the search algorithms.

Resource-Gathering An agent explores a gridworld, and aims to collect a piece of gold and a diamond and return to the starting location. There are two enemies on the map, which will attack and kill the agent with a probability of $\frac{1}{10}$ when the agent enters the enemy's cell. If killed the agent is reset to the starting location. There are three rewards, a cost of -1 for being killed, a reward of 1 for returning with the goal, and a reward of 1 for returning with the diamond.

Four-Room The agent must navigate from a starting location to a goal location, with obstructing walls in the map. There are three rewards, corresponding to three collectibles contained in the map.

Fruit-Tree-7 An MOMDP consisting of a binary tree of depth 7, where each leaf node is a terminal state giving a random 6-dimensional reward vector.

Breakable-Bottles A 1-dimensional gridworld, where the agent can move between the home state and a bottle bank. The agent can pick up bottles, and carry at most two bottles at a time. If the agent is carrying two bottles at once, there is a probability of $\frac{1}{10}$ that one of the bottles is dropped. Once a bottle is dropped, it cannot be picked up again. There are three rewards, a time cost of -1 per timestep, a reward of 25 for delivering a bottle and an additional cost of -1 for dropping a bottle. The trial ends when the agent has delivered two bottles.

For each environment, a horizon of $H = 100$ was used where applicable.

Deep Sea Treasure In *Deep Sea Treasure* (DST) environment, illustrated in Figure 5.3, the agent starts in the top left corner and can move in any cardinal direction. DST has two rewards: first a time cost of -1 per step and second a *treasure* reward for reaching one of the goal states where the magnitude of the reward is specified by the map.

While MO-Gymnasium provides a *Deep Sea Treasure* environment, a custom `C++` implementation of *Deep Sea Treasure* is used for the following reasons:

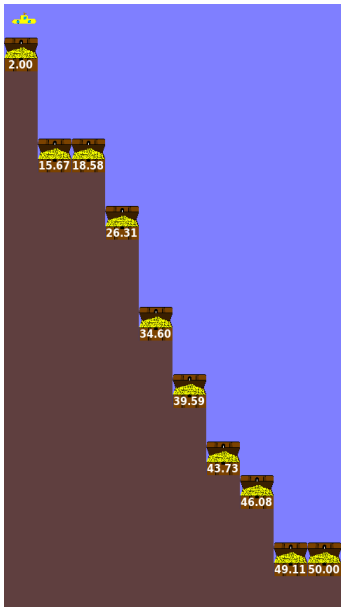


Figure 5.3: The Deep Sea Treasure environment, $\texttt{DST}(10, p_{\texttt{swept}})$. Illustration is via [19].

- provides a more efficient implementation, enabling the scalability experiments in Section 5.4.4;
- provides access to a list of states and transition probabilities between states, so that CHVI (Section 2.6.1) can be used as a baseline;
- allows the transition model to include stochasticity.

To introduce a stochastic component to DST, there is a $p_{\texttt{swept}}$ probability of the agent being swept by a current at each timestep. When swept, the agent is moved in an additional random direction before the agent makes their own move.

To enable scalability experiments, a range of map sizes are generated using the MORLGlue library [102], parameterised by the width of the gridworld. Let $\texttt{DST}(W, p_{\texttt{swept}})$ denote a DST environment with width W and probability of being swept $p_{\texttt{swept}}$. For each map of width W, if $(x_{\text{treasure}}, y_{\text{treasure}})$ is the coordinate of furthest treasure, then the maximum treasure value and planning horizon are both set to $2 \times (x_{\text{treasure}} + y_{\text{treasure}})$, so that both rewards are given a consistent scale.

In Section 5.4.3 the $\texttt{DST}(10, 0)$, $\texttt{DST}(10, \frac{1}{40})$ and $\texttt{DST}(10, \frac{1}{5})$ environments are used. In Section 5.4.4 the environments $\texttt{DST}(W, 0)$, $\texttt{DST}(W, \frac{1}{40})$ and $\texttt{DST}(W, \frac{1}{5})$ are used, with W ranging from 10 to 100.

5.4.2 Evaluation Procedure

As discussed in Sections 1.2 and 2.6, MOMCTS is used during the planning phase in the unknown weights scenario, building a search tree from the initial state. To evaluate the performance of the algorithms, two metrics are used, both computed at the initial state. The *Expected Utility Metric* (EUM, Definition 2.6.15) estimates the utility of the recommended solution set under a uniformly random weight vector. The *Hypervolume* (Definition 2.6.14) measures the volume of objective space dominated by that set, and can therefore indicate how wide a range of values is achievable.

The EUM is the metric of primary interest, as it estimates the utility that the agent would actually achieve with no prior knowledge of the objective weights. Both metrics are commonly used to assess the quality of recommended solution sets, so both are reported here. Neither is perfect. EUM assumes that all weightings over the objectives are equally likely, which need not hold in a given application. While the hypervolume respects domination — if every policy in a set Π_1 is dominated by a policy in another set Π_2, then the hypervolume for Π_2 is greater than that for Π_1 — the converse does not hold, so a larger hypervolume does not imply a better solution set.

The rest of this subsection specifies how the solution set of recommendation policies is defined given a search tree, and how the EUM is approximated via sampling.

Given a context $\mathbf{w}$ and search tree $\mathcal{T}$ that defines a partial recommendation policy ψ_{ALG}, the policy is completed as in Section 4.4.2:

$$\texttt{complete}(\psi_{\text{ALG}})(a|s;\mathbf{w}) = \begin{cases} \psi_{\text{ALG}}(a|s;\mathbf{w}) & \text{if } s \in \mathcal{T}, \\ \frac{1}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{5.41}$$

The convex hull of recommendation policies can then be written as

$$\Psi_{\text{ALG}} = \{\texttt{complete}(\psi_{\text{ALG}})(\cdot|\cdot;\mathbf{w}) \mid \mathbf{w} \in \Delta^D\}. \tag{5.42}$$

Following Blank et al. [13], let $E(\Delta^D)$ be an evenly spaced set of points in

Δ^D. Then $\text{EUM}(\Psi_{\text{ALG}})$ can be approximated as:

$$\text{EUM}(\Psi_{\text{ALG}}) \approx \frac{1}{|E(\Delta^D)|} \sum_{\mathbf{w} \in E(\Delta^D)} \mathbf{w}^\top \hat{\mathbf{V}}^{\boldsymbol{\tau}(\mathbf{w})}, \tag{5.43}$$

where $\boldsymbol{\tau}(\mathbf{w}) \sim \texttt{complete}(\psi_{\text{ALG}})$ is a sampled trajectory for the point $\mathbf{w} \in \Delta^D$, and $\hat{\mathbf{V}}^{\boldsymbol{\tau}(\mathbf{w})}$ is the cumulative return from the trajectory. That is, if the trajectory is $\boldsymbol{\tau}(\mathbf{w}) = (s_0, a_0, \mathbf{r}_0, ..., s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h)$, then $\hat{\mathbf{V}}^{\boldsymbol{\tau}(\mathbf{w})} = \sum_{i=0}^{h-1} \mathbf{r}_i$.

For CZT, to compute a hypervolume, an approximate convex hull value set is computed, using the $\bar{\mathbf{Q}}_{\texttt{CZT}}$ value estimates at each node. Specifically, the following set of points is used at the root node:

$$\texttt{cvx_prune}\left(\{\bar{\mathbf{Q}}_{\texttt{CZT}}(B) \mid B \in \texttt{relevant}(s_0; \mathbf{w})\}\right). \tag{5.44}$$

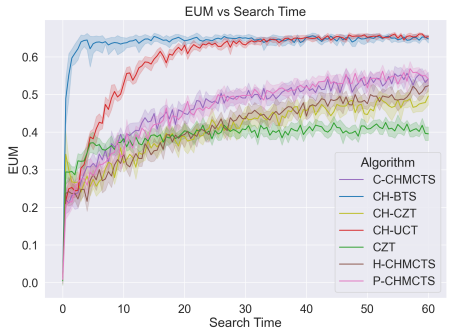
Each algorithm is given 60 seconds of search time, except for DST environments where 15 seconds are given. Each algorithm uses 16 threads for computation, and the EUM is approximated using $|E(\Delta^D)| = 128$ evenly spaced weight vectors, with one trajectory sampled per weight vector.

As in Section 4.4, the BayesOpt package [61] is used to tune the parameters of all algorithms. Full details on the parameter tuning of multi-objective algorithms are given in Appendix C.

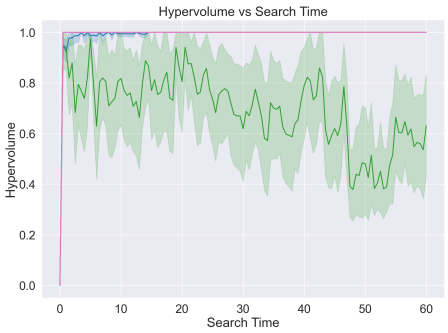
5.4.3 Multi-Objective Gridworld Results

Gymnasium Environments. Results on the MO-Gymnasium environments are given in Figure 5.4, where both EUM and hypervolume are reported against the elapsed search time. Multiple hypervolume plots show non-monotonic trends, for which there are two explanations, one for CZT and one for the CHMCTS algorithms.

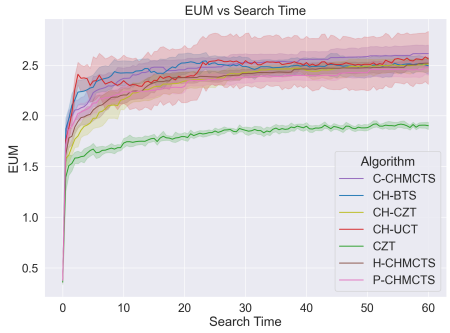
The hypervolume for CZT is approximated using the set of value estimates from the relevant balls (Equation (5.44)). The individual estimates are computed using average returns (Equation (5.11)), so they may decrease. The set of relevant balls (Equation (5.6)) also changes as the *activation rule* (Section 5.2.1) adds new balls, each starting with a fresh average return. Either effect can lower the hypervolume.



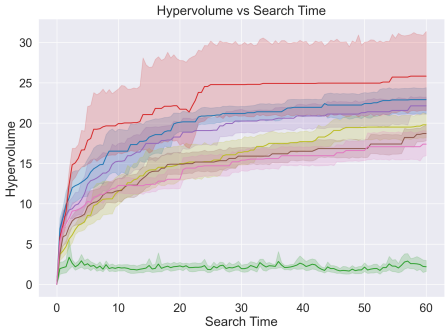
(a) Resource-Gathering (EUM).



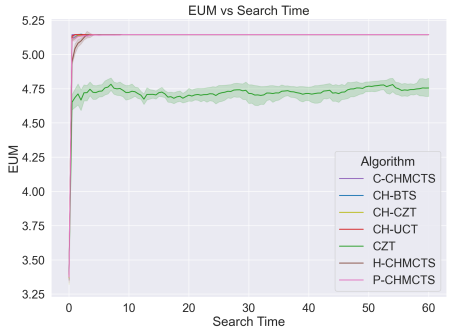
(b) Resource-Gathering (HV).



(c) Four-Room (EUM).

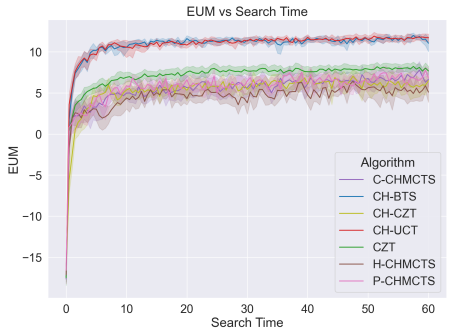


(d) Four-Room (HV).

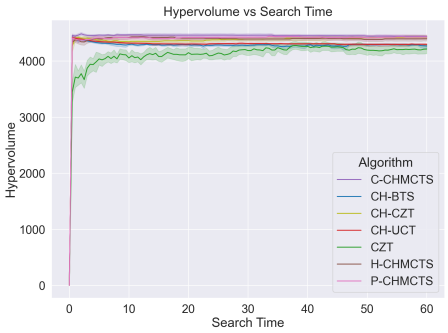


(e) Fruit-Tree-7 (EUM).

(f) Fruit-Tree-7 (HV).



(g) Breakable-Bottles (EUM).



(h) Breakable-Bottles (HV).

Figure 5.4: Results on the MO-Gymnasium environments. (Left) Expected Utility Metric (EUM). (Right) Hypervolume (HV). Each row shows the results for a different environment: Resource-Gathering, Four-Room, Fruit-Tree-7 and Breakable-Bottles respectively.

The hypervolume for CHMCTS algorithms would grow monotonically were it not for the size pruning of convex hull value sets, which uses the crowding distance (Equation (5.40)) and is described in Section 5.3.6. When the value sets are pruned to a maximum of $K_{\max} = 15$ points, the convex hull backups (Equations (5.14) and (5.15)) are no longer guaranteed to converge, and the hypervolume may decrease on any backup that prunes points (for example in Figures 5.4f and 5.4h).

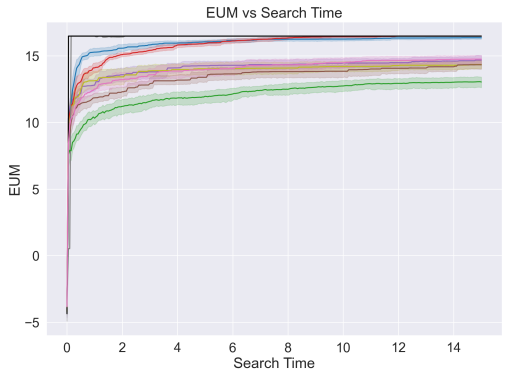
On all four environments, CZT is outperformed by at least one CHMCTS algorithm, if not all of them. This is consistent with the exploitation bias of UCT-based algorithms discussed in Chapter 4, where UCT-style algorithms tend to converge to a local optimum.

On the Resource-Gathering and Breakable-Bottles environments, CH-UCT and CH-BTS — which use contextual search policies — clearly outperform all other algorithms with respect to EUM. These environments have larger state spaces and more complex reward structures, where the benefit of contextual search policies is most pronounced.

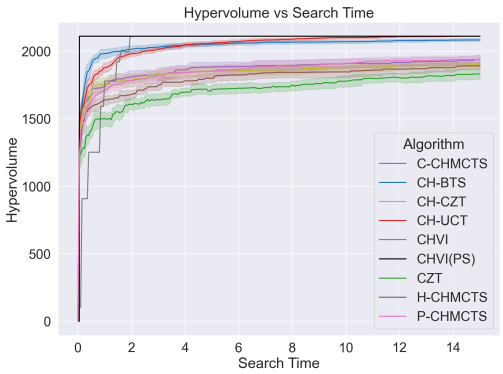
Fruit-Tree-7 consists of a binary tree of decisions, and so contains only $2^8 - 1 = 255$ states. These can be explored exhaustively in a small number of trials, and all CHMCTS algorithms solve the environment.

The Four-Room environment is larger, but its deterministic nature and lack of a time cost reward mean that a single meandering path collecting many collectibles is sufficient to achieve a near-optimal solution for all weight vectors simultaneously. A context-independent policy can therefore perform comparably to a contextual one, so all CHMCTS algorithms achieve a similar EUM, although the contextual search policies find value sets with larger hypervolumes.

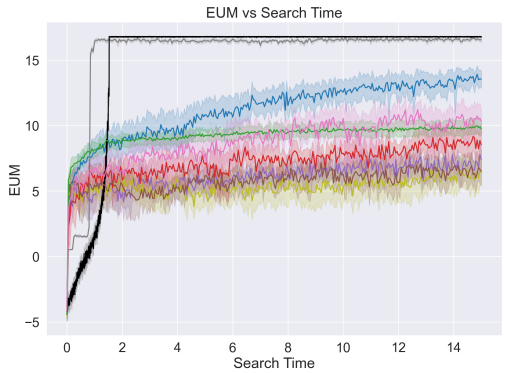
Deep Sea Treasure Environments. For the DST environments, CHVI (Section 2.6.1) and CHVI with Prioritised Sweeping (CHVI(PS)) are used as baseline algorithms, where Prioritised Sweeping [98] performs Bellman backups from timestep $t = H$ down to $t = 0$. To provide a fair comparison, the implementation of convex hull backups is shared between CHVI and the CHMCTS algorithms. Hence



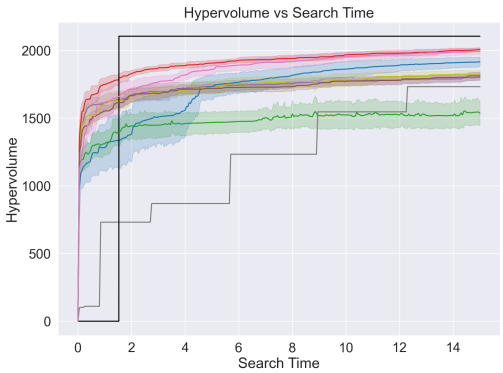
(a) DST$(10, 0)$ (EUM).



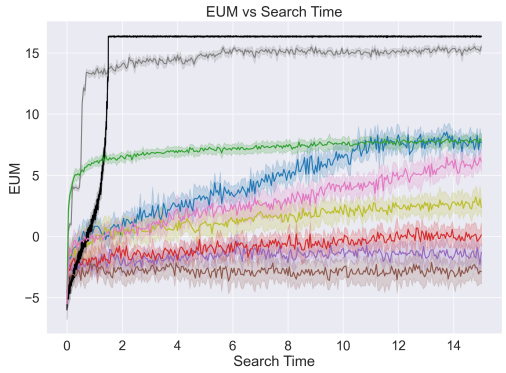
(b) DST$(10, 0)$ (HV).



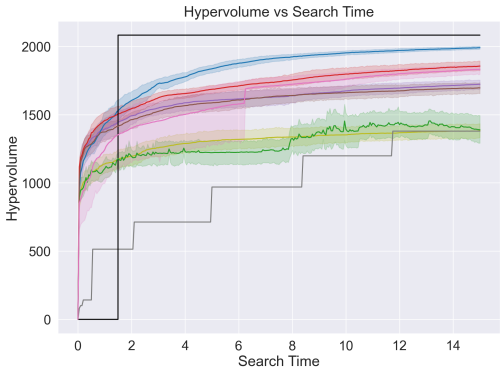
(c) DST$(10, \frac{1}{40})$ (EUM).



(d) DST$(10, \frac{1}{40})$ (HV).

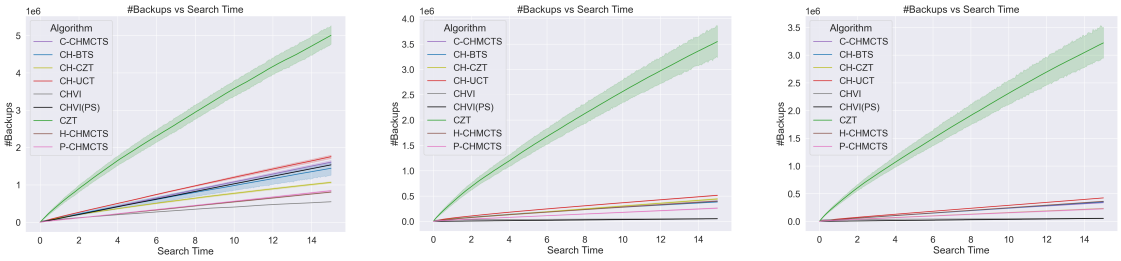


(e) DST$(10, \frac{1}{5})$ (EUM).



(f) DST$(10, \frac{1}{5})$ (HV).

Figure 5.5: Results on the Deep Sea Treasure environments. (Left) Expected Utility Metric (EUM). (Right) Hypervolume (HV). Each row shows the results for DST$(10, 0)$, DST$(10, \frac{1}{40})$ and DST$(10, \frac{1}{5})$ respectively.



(a) $\texttt{DST}(10, 0)$. **(b)** $\texttt{DST}(10, \frac{1}{40})$. **(c)** $\texttt{DST}(10, \frac{1}{5})$.

Figure 5.6: Number of backups performed on the Deep Sea Treasure environments for $\texttt{DST}(10, 0)$, $\texttt{DST}(10, \frac{1}{40})$ and $\texttt{DST}(10, \frac{1}{5})$ respectively.

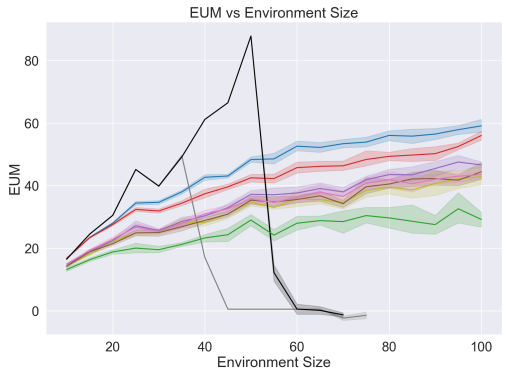
CHVI uses the same value of $K_{\max}$ for the size pruning optimisation (Section 5.3.6) as the CHMCTS algorithms (Appendix C.2), and so does not necessarily compute the exact convex hulls.

Results are given in Figures 5.5 and 5.6. On the deterministic environment $\texttt{DST}(10, 0)$, CH-BTS and CH-UCT performed best among the CHMCTS algorithms. On the stochastic environments, CH-BTS and P-CHMCTS performed better than the other CHMCTS algorithms.

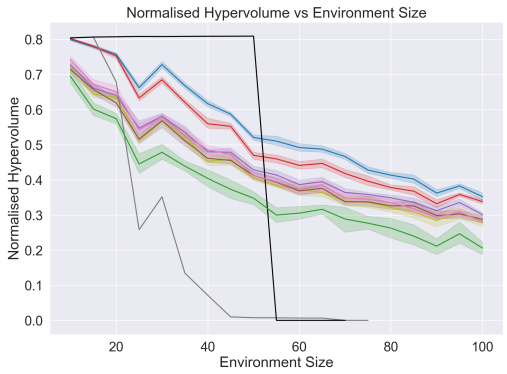
However, CHMCTS algorithms generally struggled in the stochastic environments. As shown in Figure 5.6, CHMCTS is only able to perform approximately one third as many backups in the stochastic environments as in the deterministic one, due to the growth of intermediate convex hull value sets and the additional complexity of taking expectations over them (particularly the addition operation in Equation (2.81)). With low stochasticity ($\texttt{DST}(10, \frac{1}{40})$), despite the performance penalty, CH-BTS is able to outperform the remaining MOMCTS algorithms, although it remains far short of the value computed by CHVI. With higher stochasticity ($\texttt{DST}(10, \frac{1}{5})$), CHMCTS is unable to run a sufficient number of trials within the time budget to perform well. CZT, which limits the rate of ball additions, does not suffer from this issue and maintains a competitive number of trials across all three environments.

5.4.4 Scalability Results

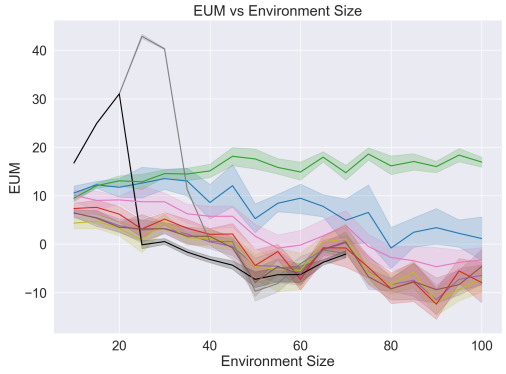
Figure 5.7 gives results on randomly generated Deep Sea Treasure environments of increasing size, as described in Section 5.4.1. Each algorithm is given 15 seconds of



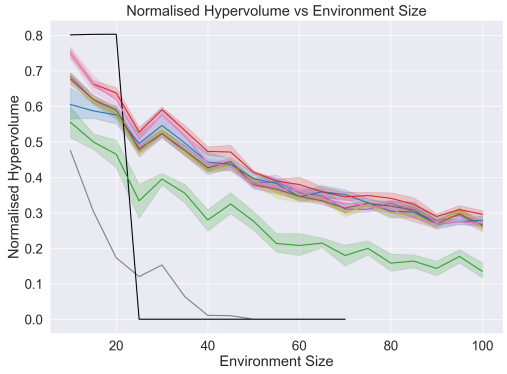
(a) DST$(W, 0)$ (EUM).



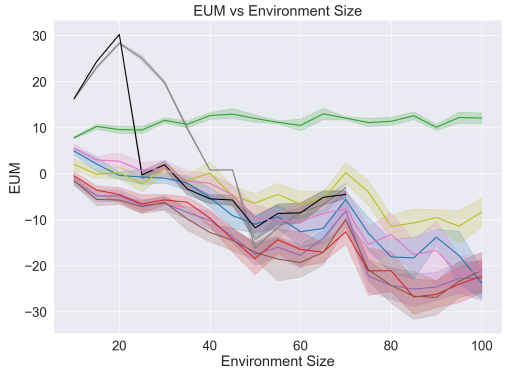
(b) DST$(W, 0)$ (NHV).



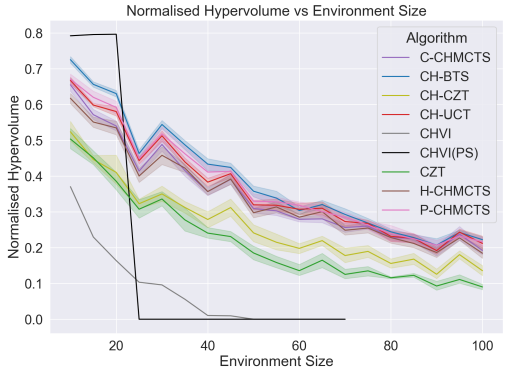
(c) DST$(W, \frac{1}{40})$ (EUM).



(d) DST$(W, \frac{1}{40})$ (NHV).



(e) DST$(W, \frac{1}{5})$ (EUM).



(f) DST$(W, \frac{1}{5})$ (NHV).

Figure 5.7: Scalability results on the Deep Sea Treasure environments. (Left) Expected Utility Metric (EUM). (Right) Normalised Hypervolume (NHV). Each row shows results for DST$(W, 0)$, DST$(W, \frac{1}{40})$ and DST$(W, \frac{1}{5})$ respectively, for widths W ranging from 10 to 100. Each algorithm was allowed up to 16 GB of memory; the curves for CHVI and CHVI(PS) are truncated where memory was exhausted.

planning time on every environment, regardless of its size, before being evaluated on its recommendation policy as outlined in Section 5.4.2. The *normalised hypervolume* (NHV) is used in this experiment, where values for both objectives are normalised to the range $[0, 1]$ before computing the hypervolume, so that results are comparable across environments of different scales.

The best achievable EUM increases with the size of the environment, because the maximum treasure reward is set to twice the number of steps needed to reach it (Section 5.4.1), so the treasure reward outgrows the time cost of collecting it. The random generation of the environments introduces some non-monotonicity, visible as a kink at $W = 30$ in the CHVI plot (Figure 5.7a). The dip in EUM for CHVI and CHVI(PS) is expected, since the planning budget is the same for all environment sizes.

The CHVI(PS) curves suggest that the best achievable NHV is consistently around 0.8. This is clearest for $p_{\texttt{swept}} = 0$, where it holds across all environment sizes. In the stochastic environments CHVI(PS) only converges for environment sizes up to and including $W = 20$, so the 0.8 value can only be confirmed over that range. The downward trends of NHV for all algorithms are expected, given the fixed planning budget, and the kinks in the NHV plots (Figures 5.7b, 5.7d and 5.7f) have the same cause as the kink in EUM (Figure 5.7a).

On the deterministic environment $\texttt{DST}(W, 0)$, CHVI is able to solve environments up to a width of approximately $W = 50$, beyond which the state space has grown too large for CHVI to converge within the time budget. At $W = 80$, CHVI exhausts the 16 GB memory limit. CH-BTS maintains strong performance across all environment sizes, with an NHV consistently above 0.4. Its EUM and NHV start close to optimal for small environments, but are outperformed by CHVI at intermediate sizes. Above $W = 50$, the tree search algorithms still produce a policy where CHVI and CHVI(PS) cannot, and their NHV degrades gradually rather than dropping sharply.

On both stochastic environments, all convex hull-based algorithms struggle. The performance of CHVI(PS) drops sharply after $W = 20$, while CHVI degrades more

gradually. This is consistent with the results of Section 5.4.3, where stochasticity increases the number of points in the convex hull value sets, and hence the computational cost of convex hull backups.

CZT performs best on the larger stochastic environments, maintaining its performance where convex hull-based algorithms are limited by the computational complexity discussed in Section 5.4.3. However, across all environments CZT tends to exploit, converging to a locally optimal solution that collects nearby treasure quickly without exploring enough of the environment to discover higher-value options further from the starting state.

This exploitation is advantageous under EUM at large environment sizes with a limited compute budget. CZT settles on a policy that collects treasure near the starting state and then fills out the search tree around it, much as exploitation was beneficial in the stochastic FrozenLake experiments of Section 4.4.3.1. The larger hypervolumes found by the CHMCTS algorithms suggest that they do discover larger treasure values in the stochastic environments, but their search trees are not filled out for all of the stochastic outcomes, so the recommendation policy is more likely to leave the tree and fall back to acting uniformly at random (Equation (5.41)), which lowers their EUM. It should be noted, however, that no algorithm achieves an EUM that increases with the environment size, despite the best achievable EUM doing so. Even CZT, which performs best at the larger sizes, only maintains an EUM of between 10 and 20 throughout.

5.5 Theoretical Discussion

This section revisits the consistent action selection problem of Section 5.1.1 more precisely, and gives theoretical results for CHMCTS and MOMCTS. It is structured as follows:

- Section 5.5.1 gives some additional notation needed to precisely discuss consistent action selection and to state and prove Theorem 5.5.1;

- Section 5.5.2 discusses consistent action selection, and why the assumption of linear utilities is made throughout this thesis;
- Section 5.5.3 shows that locally optimal action selection yields globally optimal policies if and only if the utility function is linear (Theorem 5.5.1);
- Section 5.5.4 outlines how the convergence of CHMCTS can be established.

5.5.1 Notation

Because this thesis assumes a linear utility throughout, some additional notation is needed to discuss the problem of consistent action selection (Section 5.1.1) precisely.

The discussion in this section uses the *Chebyshev utility* as an example, because it is non-linear, and commonly used as a utility where the undominated values can recover the Pareto front (Definition 2.6.12) [97]. For reference, Equation (2.77) (from Definition 2.6.13) is repeated here:

$$u_{\text{cheby}}(\mathbf{v}; \mathbf{w}, \mathbf{r}) = -\max_{i=1,\dots,D} w_i |v_i - r_i|, \tag{5.45}$$

where $\mathbf{r}$ is a utopian (reference) point.

Consider an arbitrary MOMDP $\mathcal{M}$, with a valid set of policies Π. Recall that $U(\Pi; u)$ is the undominated set of policies with respect to utility function u (Definition 2.6.7). As is typical in multi-objective reinforcement learning, this set is taken with respect to the initial state[1] [43]. In this discussion it is useful to consider the undominated set of policies from any given state s:

$$U(s, \Pi; u) = \left\{ \pi \in \Pi \;\middle|\; \exists \mathbf{w} \in \Delta^D . \forall \pi' \in \Pi : u(\mathbf{V}^{\pi}(s; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s; 0); \mathbf{w}) \right\}. \tag{5.46}$$

Then the sets of optimal values, with respect to utility u, at state s can be defined as:

$$\boldsymbol{\mathcal{V}}^*_u(s) = \{\mathbf{V}^{\pi}(s) \mid \pi \in U(s, \Pi; u)\}, \tag{5.47}$$

$$\boldsymbol{\mathcal{Q}}^*_u(s, a) = \{\mathbf{Q}^{\pi}(s, a) \mid \pi \in U(s, \Pi; u)\}. \tag{5.48}$$

Then a *locally optimal policy* for weight $\mathbf{w}$, with respect to utility u, selects actions greedily at each timestep using the optimal Q-value sets:

$$\pi^{\mathbf{w}}_{\text{local},u}(s) \in \arg\max_{a \in \mathcal{A}} \max_{\mathbf{q} \in \mathcal{Q}^*_u(s,a)} u(\mathbf{q}; \mathbf{w}). \tag{5.49}$$

And finally, a policy $\pi \in \Pi$ is said to be *utility-optimal* for weight $\mathbf{w} \in \Delta^D$ if

$$u(\mathbf{V}^{\pi}(s_0); \mathbf{w}) = \max_{\pi' \in \Pi} u(\mathbf{V}^{\pi'}(s_0); \mathbf{w}). \tag{5.50}$$

5.5.2 Consistent Action Selection and the Linear Utility Assumption

MCTS methods are built on the assumption that making decisions locally at each decision node can lead to a globally optimal solution. In particular, they typically rely on two core components:

- each node stores a reward-to-go estimate;
- actions are selected using those local estimates.

As mentioned in Section 5.1, this is sound for single-objective problems.

In MOMCTS the two components become:

- each node stores a set of vector-valued reward-to-go estimates, such as a convex hull value set in CHMCTS (Section 5.3);
- actions are selected by maximising the utility over those estimates.

However, for these to be sound, they require that $\pi^{\mathbf{w}}_{\text{local},u}$ is utility-optimal for $\mathbf{w}$. Theorem 5.5.1 shows that this holds for every MOMDP if and only if the utility u is linear.

To see why, consider taking the maximum linear utility over the Q-value backup used in CHMCTS (Equation (5.15)):

$$\begin{aligned} &\max \left\{ u_{\text{lin}}(\mathbf{q}; \mathbf{w}) \mid \mathbf{q} \in \hat{\mathcal{Q}}_{\text{ALG}}(s_t, a_t) \right\} \\ =&u_{\text{lin}}\left(\mathbf{R}(s_t, a_t); \mathbf{w}\right) + \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \max \left\{ u_{\text{lin}}(\mathbf{v}; \mathbf{w}) \mid \mathbf{v} \in \hat{\mathcal{V}}_{\text{ALG}}(s') \right\}. \end{aligned} \tag{5.51}$$

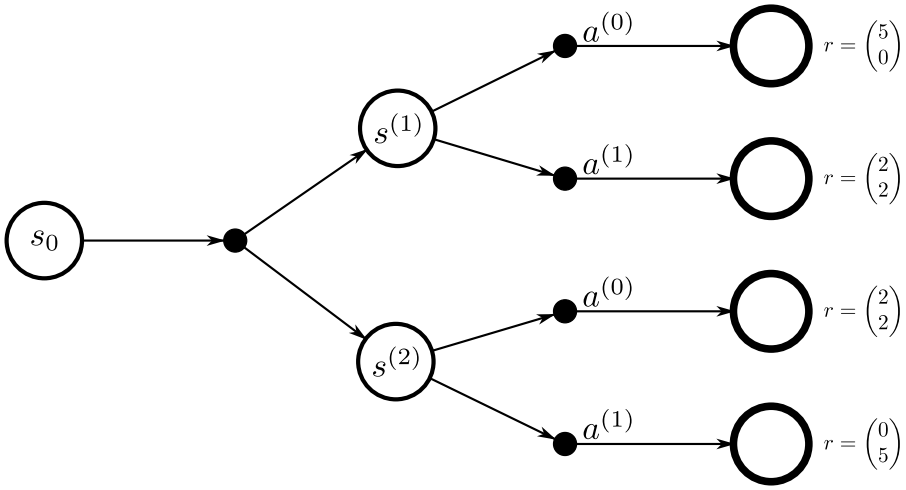


Figure 5.8: A stochastic MOMDP where making locally optimal decisions does not yield globally optimal policies for *Chebyshev* utilities. At the first (black) chance node from s_0 there is a $\frac{1}{2}$ probability of each of the outcomes $s^{(1)}$ and $s^{(2)}$.

The distributive property of the linear utility in Equation (5.51) does not hold for a general utility. The right-hand side maximises over each successor's value set independently, which is what allows actions to be selected locally; under a non-linear utility the values chosen at different successors may instead have to be coordinated. In other words, the second component above (selecting actions by maximising the utility locally) requires utilities to be linear for the algorithm to be sound. Theorem 5.5.1 shows this more formally.

Figure 5.8 gives a concrete example of how this breaks down: under the Chebyshev utility, locally optimal actions in this MOMDP do not yield a globally optimal policy. It is also the MOMDP used to prove the "only if" direction of Theorem 5.5.1. The Pareto front at s_0 consists of the points $\{(1, 3.5), (2.5, 2.5), (3.5, 1)\}$. Achieving the value $(2.5, 2.5)$ requires the rewards $(5, 0)$ and $(0, 5)$ to be selected from the states $s^{(1)}$ and $s^{(2)}$ respectively.

The locally optimal policy using the Chebyshev utility with $\mathbf{w} = (0.5, 0.5)$ and $\mathbf{r} = (6, 6)$ will always achieve a value of $(2, 2)$, which is not utility-optimal, as it is dominated by the value of $(2.5, 2.5)$. The rewards $(5, 0)$ and $(0, 5)$ prioritise opposite

[1] The undominated set is typically taken with respect to an initial state distribution, which in the context of this thesis is a fixed initial state.

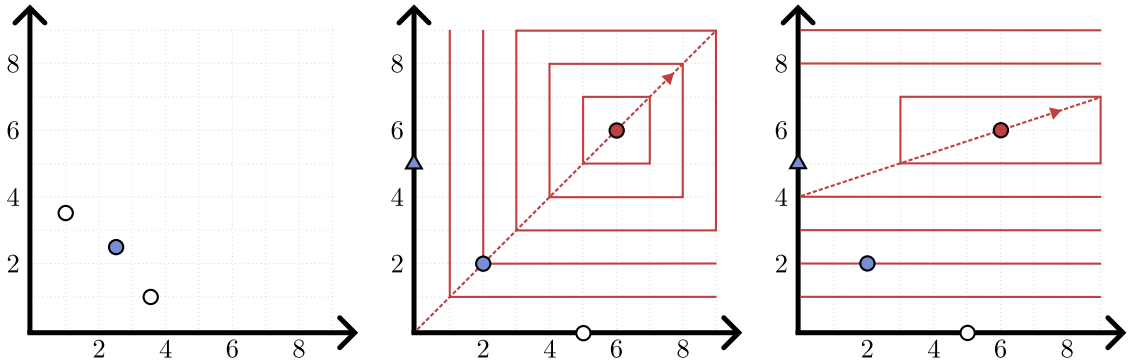


Figure 5.9: Pareto fronts for the MOMDP from Figure 5.8 with contours of the Chebyshev utility. The left plot shows the Pareto front at the initial state s_0, with the problematic point of $(2.5, 2.5)$ highlighted in blue. The middle and right plots show the Pareto fronts from $s^{(1)}$ and $s^{(2)}$ simultaneously: the circles form the Pareto front at $s^{(1)}$, and blue points form the Pareto front at $s^{(2)}$. The middle plot shows the pyramid-like contours of the Chebyshev utility (Equation 5.45) with $\mathbf{w} = (0.5, 0.5)$ and $\mathbf{r} = (6, 6)$, and the right plot with $\mathbf{w} = (0.25, 0.75)$ and $\mathbf{r} = (6, 6)$. The dashed red lines run along the vector $(1/w_1, 1/w_2)$ and pass through the corner points of the contours. For the value of $(2.5, 2.5)$ to be achievable by a policy of the form of Equation (5.49) that selects greedily with respect to the Chebyshev utility would require a value of $\mathbf{w}$ and $\mathbf{r}$ that assigns a higher value to the blue triangle and white circle than the blue circle, which geometrically is not possible.

objectives, so neither can be chosen in isolation: the agent must commit to a target value at the initial state, and then select actions consistently with that commitment.

The Pareto fronts at the states s_0, $s^{(1)}$ and $s^{(2)}$ are depicted in Figure 5.9, along with contours of the Chebyshev utility, to highlight that it cannot simultaneously assign a higher utility to $(5, 0)$ and $(0, 5)$ than to $(2, 2)$, which would be needed to achieve the value of $(2.5, 2.5)$.

In fact, there is no choice of $\mathbf{w}$ and $\mathbf{r}$ for which $\pi^{\mathbf{w}}_{\text{local},u_{\text{cheby}}}$ achieves the value $(2.5, 2.5)$. This is not shown here, as it is stronger than what the proof of Theorem 5.5.1 requires.

5.5.2.1 Multi-Objective Monte Carlo Tree Search with Non-Linear Utilities

While Theorem 5.5.1 rules out any guarantee for MOMCTS algorithms that use the two components above on arbitrary MOMDPs, this does not mean that MCTS cannot be applied successfully when the utility is non-linear.

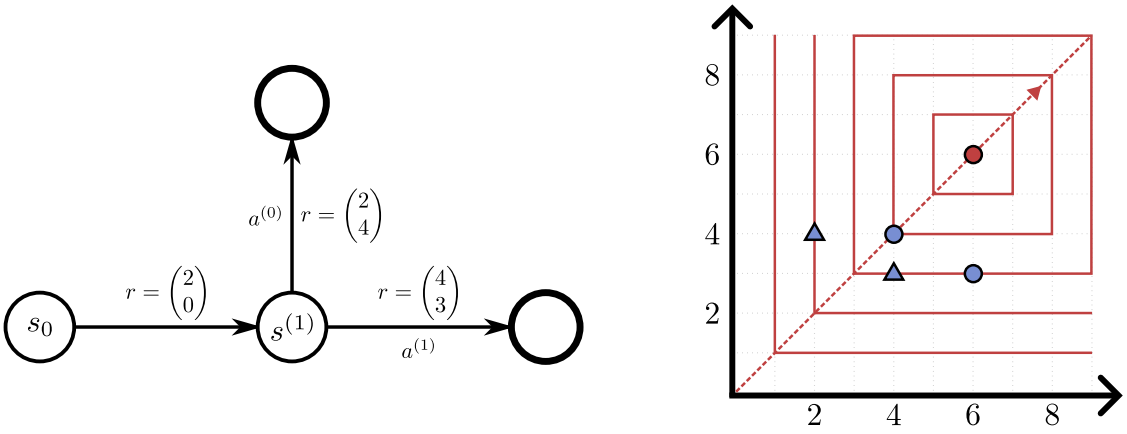


Figure 5.10: The consistent action selection problem in a deterministic MOMDP with Chebyshev utilities. Left: A deterministic MOMDP to highlight that using Chebyshev utilities over Pareto fronts still exhibits a consistent action selection problem. Right: circles form the Pareto front at s_0 and triangles the local Pareto front at $s^{(1)}$. The contours of the Chebyshev utility with $\mathbf{w} = (0.5, 0.5)$ and $\mathbf{r} = (6, 6)$ are also shown. The return of $(4, 4)$ is optimal for this utility and weight, but action $a^{(1)}$ is preferred at state $s^{(1)}$ leading to a suboptimal return of $(6, 3)$.

As the example MOMDP from Figure 5.8 highlights, the best action at a state can depend on outcomes that did not occur: changing the rewards available at $s^{(2)}$ can change which action is optimal at $s^{(1)}$, as they are the two possible outcomes from the transition from the initial state. In terms of MCTS, a trial only ever samples one of the two branches, so the optimal action can depend on a branch that the trial never visits. Hence, to soundly use MOMCTS with Chebyshev utilities, the MOMDP must at least be assumed to be deterministic. More generally, an MOMCTS algorithm using non-linear utilities needs to restrict the types of MOMDP it can soundly solve.

As an example, the remainder of this section will consider using the Chebyshev utility (Equation (5.45)) for deterministic MOMDPs. Figure 5.10 highlights that if Chebyshev utilities are used to select actions from Pareto fronts during execution, then there is still an issue with consistent action selection (Section 5.1.1).

In this case, actions can be selected consistently by instead making decisions over the value set $\mathbf{R}_{:t-1} + \hat{\mathcal{Q}}_{\text{ALG}}(s_t, a_t)$, where $\mathbf{R}_{:t-1} = \sum_{i=0}^{t-1} \mathbf{R}(s_i, a_i)$ is the reward accrued so far in the trial. The accrued reward is needed because a non-linear utility

depends on the entire return, and the utility does not distribute over the sum of cumulative returns.

This adaptation changes the second of the two components above: the accrued reward is tracked through the trial, making the decision *non-local*[2].

Hayes et al. [42, 44] change the first component in a different way, storing a distribution over returns at each node rather than a value set. This gives an MOMCTS algorithm for the expected scalarised returns criterion (Section 3.4), where the utility is applied to the return of a single execution rather than to the expected return.

Finally, Theorem 5.5.1 rules out a guarantee rather than good performance. The MOMDP of Figure 5.8 is constructed specifically to break local decision-making, however non-linear utilities such as the Chebyshev utility have been used successfully in practice. Using MOMCTS with a non-linear utility may therefore still be effective, but it comes without the guarantees of the linear case.

5.5.3 When Local Optimality Implies Global Optimality

This section formalises the claim that following locally optimal actions gives a globally optimal policy if and only if the utility is linear. The "only if" direction is the stronger statement: even when the value sets of the undominated policies are known exactly, the locally optimal policy that selects actions by maximising the utility over those value sets is not guaranteed to be optimal for that utility.

Theorem 5.5.1. *The policy* $\pi^{\mathbf{w}}_{\text{local},u}$ *is utility-optimal, that is:*

$$u(\mathbf{V}^{\pi^{\mathbf{w}}_{\text{local},u}}(s_0); \mathbf{w}) = \max_{\pi \in \Pi} u(\mathbf{V}^{\pi}(s_0); \mathbf{w}), \tag{5.52}$$

for every MOMDP $\mathcal{M}$ *and every* $\mathbf{w} \in \Delta^D$ *if and only if* $u(\cdot; \mathbf{w})$ *is linear in its first argument for every* $\mathbf{w} \in \Delta^D$.

Proof outline. **If direction.**

[2] Alternatively, the accrued reward could be stored in the node (changing the other component), but would not be compatible with transposition tables (Section 2.4.2), as the accrued reward depends on the path taken to reach the node.

Follows from the convergence of Convex Hull Value Iteration [7].

Only if direction.

The contrapositive is established by exhibiting a single MOMDP $\boldsymbol{\mathcal{M}}$, utility function u and weight vector $\mathbf{w}$ where the locally optimal policy $\pi^{\mathbf{w}}_{\text{local},u}$ is not utility-optimal.

Let $\boldsymbol{\mathcal{M}}$ be the MOMDP in Figure 5.8 and u be the Chebyshev utility (Equation (5.45)) with reference point $\mathbf{r} = (6, 6)$:

$$u(\mathbf{v}; \mathbf{w}) = u_{\text{cheby}}(\mathbf{v}; \mathbf{w}, \mathbf{r}) = -\max_i w_i \left| v_i - r_i \right|. \tag{5.53}$$

The four paths through the MOMDP yield rewards of:

$$s_0 \to s^{(1)} \to a^{(0)} : (5, 0), \qquad s_0 \to s^{(1)} \to a^{(1)} : (2, 2),$$
$$s_0 \to s^{(2)} \to a^{(0)} : (2, 2), \qquad s_0 \to s^{(2)} \to a^{(1)} : (0, 5).$$

Fix the weight vector $\mathbf{w} = (\frac{1}{2}, \frac{1}{2})$. By inspection, there are four stationary policies yielding expected values at s_0 of $(3.5, 1)$, $(2.5, 2.5)$, $(2, 2)$, and $(1, 3.5)$, with the policy achieving $(2.5, 2.5)$ being utility-optimal for $\mathbf{w}$:

$$u((3.5, 1); \mathbf{w}) = -2.5, \quad u((2.5, 2.5); \mathbf{w}) = -1.75, \quad u((1, 3.5); \mathbf{w}) = -2.5.$$

(The value $(2, 2)$ is Pareto-dominated and is omitted.) The utility-optimal policy selects $a^{(0)}$ at $s^{(1)}$ (receiving $(5, 0)$) and $a^{(1)}$ at $s^{(2)}$ (receiving $(0, 5)$), giving expected value $(2.5, 2.5)$ and utility -1.75.

Now consider $\pi^{\mathbf{w}}_{\text{local},u}$. The Pareto fronts at the successor states are singletons,

$$\boldsymbol{\mathcal{Q}}^*_u(s^{(1)}, a^{(0)}) = \{(5, 0)\}, \qquad \boldsymbol{\mathcal{Q}}^*_u(s^{(1)}, a^{(1)}) = \{(2, 2)\},$$
$$\boldsymbol{\mathcal{Q}}^*_u(s^{(2)}, a^{(0)}) = \{(2, 2)\}, \qquad \boldsymbol{\mathcal{Q}}^*_u(s^{(2)}, a^{(1)}) = \{(0, 5)\},$$

and the Chebyshev utilities are $u((5, 0); \mathbf{w}) = u((0, 5); \mathbf{w}) = -3$ and $u((2, 2); \mathbf{w}) = -2$. Hence $\pi^{\mathbf{w}}_{\text{local},u}$ selects $a^{(1)}$ at $s^{(1)}$ and $a^{(0)}$ at $s^{(2)}$, both yielding $(2, 2)$. The expected value at s_0 is

$$\mathbf{V}^{\pi^{\mathbf{w}}_{\text{local},u}}(s_0) = \tfrac{1}{2}(2, 2) + \tfrac{1}{2}(2, 2) = (2, 2),$$

with utility $u((2, 2); \mathbf{w}) = -2 < -1.75$. Therefore $\pi^{\mathbf{w}}_{\text{local},u}$ is not utility-optimal. □

5.5.4 Convergence of CHMCTS

This section outlines how the convergence of CHMCTS can be proven. First, to reason about the performance of a contextual tree search algorithm, the notion simple regret is generalised to *contextual simple regret*, which is the expected regret with respect to utility over a uniformly random weight vector.

Definition 5.5.1. *The* (linear) contextual simple regret *of a recommendation policy* ψ *at state* $s \in \mathcal{S}$ *is:*

$$\texttt{ctx_sim_regr}(s, \psi) = \mathbb{E}_{\mathbf{w} \sim U(\Delta^D)} \left[\max_{\pi'} \mathbf{w}^\top \mathbf{V}^{\pi'}(s) - \mathbf{w}^\top \mathbf{V}^{\psi(\cdot;\mathbf{w})}(s) \right], \tag{5.54}$$

where $\mathbf{w} \sim U(\Delta^D)$ *denotes a weight vector sampled uniformly from the unit* $(D-1)$*-dimensional simplex.*

As a first step, if all states are visited infinitely often, then by the convergence of Convex Hull Value Iteration [7] the convex hull value sets converge to the convex coverage sets.

Theorem 5.5.2. *For any MOMDP* $\boldsymbol{\mathcal{M}}$*, if* $N(s) \xrightarrow{p} \infty$ *and* $N(s,a) \xrightarrow{p} \infty$ *for all* $s \in \mathcal{S}$ *and* $a \in \mathcal{A}$*, then the convex hull value sets, computed by Equations* (5.14) *and* (5.15)*, satisfy* $\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s) \xrightarrow{p} \boldsymbol{\mathcal{V}}^*_{u_{lin}}(s)$ *and* $\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a) \xrightarrow{p} \boldsymbol{\mathcal{Q}}^*_{u_{lin}}(s,a)$ *for all* $s \in \mathcal{S}$ *and* $a \in \mathcal{A}$*, where* $\boldsymbol{\mathcal{V}}^*_{u_{lin}}(s)$ *and* $\boldsymbol{\mathcal{Q}}^*_{u_{lin}}(s,a)$ *are the optimal value sets for the linear utility function* u_{lin}*.*

Proof outline. Follows from Corollary 4.5.1 and the convergence of Convex Hull Value Iteration [7]. □

While Lemma 4.5.2 provides the conditions for $N(s) \xrightarrow{p} \infty$ and $N(s,a) \xrightarrow{p} \infty$ for CH-BTS, the remaining CHMCTS algorithms need a corresponding lemma to show that all reachable states are visited infinitely often for UCT-based policies:

Lemma 5.5.1. *Let* ρ^m *be an arbitrary policy and consider the following search policy used on the* m*th round of a bandit problem:*

$$\pi^m(\mathbf{w}) = \arg\max_{a \in \mathcal{A}} \dot{Q}(a) + b \sqrt{\frac{\log(m)}{M^m(a)}}, \tag{5.55}$$

where $b > 0$ is a bias parameter, $M^m(a)$ is the number of times a was selected in the first m rounds, and $\dot{Q}$ is an arbitrary function with $\dot{Q}(a) \leq Q_{\max}$ for all $a \in \mathcal{A}$. Then $M^m(a) \to \infty$ as $m \to \infty$ for all $a \in \mathcal{A}$.

Proof outline. By contradiction: if $M^m(a)$ remained bounded while every other action's count grew, the $b\sqrt{\log(m)/M^m(a)}$ term would diverge and force a to be selected, contradicting boundedness of $M^m(a)$. □

Given Lemmas 4.5.2 and 5.5.1 and Theorem 5.5.2 it follows that for all CHMCTS algorithms defined in Section 5.3 the convex hull value sets converge to the optimal value sets. Hence, by Theorem 5.5.1, any CHMCTS algorithm ALG satisfies $\mathbb{E}\left[\texttt{ctx_sim_regr}(s, \psi_{\text{ALG}})\right] \to 0$ as the number of trials $n \to \infty$, mirroring Theorem 4.2.1 for BTS in the single-objective case. CH-BTS also admits analogues of Theorems 4.2.2 and 4.2.3, again by using Lemma 4.5.2 and Theorems 5.5.2 and 5.5.1.

5.6 Summary

This chapter extended the MCTS framework to the multi-objective unknown weights scenario with linear utilities, introducing two approaches for contextual planning in MOMDPs.

The consistent action selection problem was identified as a fundamental challenge in MOMCTS (Section 5.1): actions taken within a trial should be consistent with each other (that is, all consistent with a single weight vector), otherwise the search policy optimises for no particular objective. Contextual tree search was proposed as a solution during the planning phase, where a weight vector $\mathbf{w}$ is sampled at the start of each trial and used to guide all subsequent action selections. It was also shown (Theorem 5.5.1) that locally optimal action selection yields globally optimal policies if and only if the utility function is linear, confirming that the assumption of linear utility is necessary for an MOMCTS algorithm to guarantee convergence, at least for stochastic environments, addressing **Q2 - Stochastic Multi-Objective Domains** theoretically.

Contextual Zooming for Trees (CZT) was introduced (Section 5.2) as a natural contextual analogue of UCT, applying the Contextual Zooming algorithm at each decision node. *Convex Hull MCTS* (CHMCTS) was introduced (Section 5.3) as a generalisation of prior MOMCTS work to stochastic environments, maintaining convex hull value sets at each node and updating them via convex hull backups. The prior MOMCTS algorithms of Chapter 3 were recovered as special cases of the CHMCTS framework by substituting appropriate context-independent search policies.

Empirically, CHMCTS algorithms generally outperformed CZT on deterministic and near-deterministic environments, where the convex hull value sets provide an accurate representation of achievable returns. However, in stochastic environments the computational cost of the convex hull backups grows rapidly, reducing the number of trials that can be run within a fixed time budget and limiting performance. CZT avoids this issue but exhibits the exploitation bias of UCT-style algorithms, converging to locally optimal solutions. This leaves **Q2 - Stochastic Multi-Objective Domains** unsatisfactorily addressed empirically. However, **Q3.2 - Multi-Objective Scalability** is addressed in part by the scalability results in Section 5.4.4.

Two limitations of the convex hull approach motivate the work of Chapter 6. First, convex hull value sets become computationally expensive in stochastic environments, suggesting that a hierarchical representation may be more scalable. Second, convex hull value sets do not naturally support entropy-based exploration, making it difficult to utilise entropy as an exploration bonus. Both limitations are addressed by the simplex map approach introduced in Chapter 6.

6

Simplex Maps for Multi-Objective Monte Carlo Tree Search

Contents

6.1 Introduction and Motivation 168
6.1.1 Limitations of CHMCTS and CZT 168
6.1.2 Motivation for Working in Weight Space 169
6.1.3 Adaptive Subdivisions of the Weight Space 170
6.2 Simplex Maps 171
6.2.1 Interface 171
6.2.2 Implementation 173
6.2.3 Complexity Discussion 178
6.3 Simplex Maps for Tree Search 179
6.4 Empirical Results 181
6.4.1 Evaluation Procedure 182
6.4.2 Multi-Objective Gridworld Results 182
6.4.3 Scalability Results 185
6.5 Summary 187

This chapter introduces *simplex maps*, an adaptive data structure for maintaining value and entropy estimates as a function of the weight vector $\mathbf{w} \in \Delta^D$ over the unit simplex. Simplex maps address two limitations of the algorithms considered in Chapter 5: the computational cost of maintaining convex hull value sets in stochastic environments, and the inability to naturally support entropy-based exploration (Chapter 4). By partitioning the weight space into a hierarchical and adaptive

triangulation, simplex maps provide a more scalable representation: values can be looked up faster, and the number of stored values grows only where the weight space requires finer resolution, addressing **Q3.2 - Multi-Objective Scalability** and **Q2 - Stochastic Multi-Objective Domains** empirically. Per-vertex entropy estimates allow entropy-based exploration to be lifted to the multi-objective setting, addressing **Q1.2 - Multi-Objective Exploration**.

The remainder of this chapter is organised as follows:

- Section 6.1 motivates the simplex map approach, drawing on the empirical results of Chapter 5;
- Section 6.2 defines the simplex map data structure, describing its interface (Section 6.2.1), implementation (Section 6.2.2) and a brief discussion on the computational complexity of the operations (Section 6.2.3);
- Section 6.3 introduces MOMCTS algorithms using simplex maps as a value representation, generalising BTS and DENTS to the multi-objective setting;
- Section 6.4 presents empirical results using the same experimental setup as Chapter 5, with CZT and CHMCTS algorithms as baselines.

6.1 Introduction and Motivation

This section discusses some of the limitations of the algorithms introduced in Chapter 5, and motivates the use of simplex maps as a more scalable alternative that supports entropy-based exploration.

6.1.1 Limitations of CHMCTS and CZT

The empirical results of Chapter 5 (Section 5.4) highlight a failure case for CHMCTS, where it is outperformed by CZT in the stochastic DST environment.

While convex hull value sets provide an accurate representation of achievable returns encountered during the search — which is beneficial in the exploration setting — they become computationally expensive in stochastic environments. The

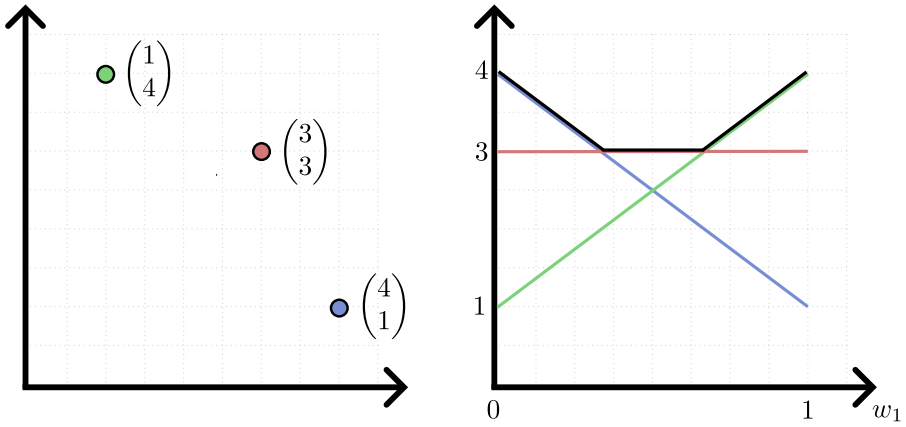


Figure 6.1: (Left) A convex hull value set $\mathcal{V}$ and (right) its corresponding value curve in weight space. Each point in objective space corresponds to a line in weight space (with corresponding colours). In bold the maximum linear utility $\max_{\mathbf{v}\in\mathcal{V}} \mathbf{w}^\top \mathbf{v}$ as a function of the weight vector $\mathbf{w} = (w_1, 1 - w_1) \in \Delta^D$ is shown, i.e. the *value curve*.

number of points in the intermediate convex hull value sets can grow rapidly, causing a significant increase in the computational cost of each backup and a corresponding reduction in the number of trials that can be run within a fixed time budget.

In contrast, CZT avoids this issue: by limiting the rate of ball additions via the M_{min} parameter, it can run a comparable number of trials per unit time in stochastic environments. Despite this advantage, CZT still exhibits the exploration limitations similar to those described in Chapter 4: it tends to exploit aggressively and does not explore sufficiently, in the same way that UCT can get stuck in local optima in the single-objective setting. While CZT outperforms CHMCTS algorithms on the stochastic DST environment, it still recommends suboptimal actions compared to CHVI (Figure 5.5).

CZT also suffers from the undesirable property that it will always add new balls to the cover, regardless of whether further refinement is warranted. This was addressed in practice by introducing a minimum visit threshold M_{min} before a new ball is created (Section 5.2), but this is an ad hoc fix rather than a principled solution.

6.1.2 Motivation for Working in Weight Space

Convex hull value sets operate in *objective space*, representing the set of achievable expected return vectors. An alternative perspective is to consider the *value curve*

in weight space: the function $\mathbf{w} \mapsto \max_{\mathbf{v} \in \boldsymbol{\mathcal{V}}} \mathbf{w}^\top \mathbf{v}$ that maps each weight vector to the maximum linear utility achievable under that weight. This value curve is a piecewise-linear, convex function over Δ^D, and is illustrated in Figure 6.1. Maintaining an estimate of this function directly over the weight space, rather than maintaining the full convex hull value set, suggests a more scalable approach.

Working in weight space has led to more scalable approaches in the prior literature. For example, Lizotte et al. [57] use piecewise linear splines as an alternative to convex hulls, yielding a more efficient value iteration algorithm, although this approach does not extend straightforwardly to higher dimensions [58].

Following the argument made in Section 5.3.7, working in objective space also does not naturally support entropy-based exploration, and moving to weight space provides a natural way to support entropy-based exploration.

While CZT does operate in weight space, each ball covers a region of the weight space and maintains a single value estimate for that entire region. As such, they cannot naturally store an entropy estimate associated with a single weight vector.

6.1.3 Adaptive Subdivisions of the Weight Space

This chapter investigates the use of an *adaptive subdivision* of the weight space as an alternative representation. Rather than covering the weight space with balls, the simplex Δ^D is triangulated into a set of simplices, each associated with a vertex at which a value estimate and entropy estimate are maintained. The triangulation is refined adaptively during the search, subdividing regions of the weight space where the value function varies across the region and leaving regions where the value is uniform intact.

Adaptive subdivisions have been used in a range of settings. *Hierarchical Optimistic Optimisation* (HOO) [16] is an algorithm for continuous-armed bandit problems and uses a tree of hierarchically nested sets to adaptively search a continuous domain, refining the cover in regions of high estimated value. Unlike the approach of this chapter, HOO focuses on identifying a single region of high value rather than estimating the value function across the entire domain.

In numerical analysis, the finite element method [118] is a method for solving partial differential equations by discretising the domain into a mesh and solving the equation on each element. *Adaptive* finite element methods [67] alternate between solving a partial differential equation over a mesh and refining the mesh in regions where the solution is not yet well resolved, a paradigm widely used in fluid dynamics and structural mechanics. The simplex map data structure introduced in this chapter draws on similar principles, adapting the resolution of the weight space cover to the demands of the search.

A substantial body of work has studied adaptive mesh refinement strategies for triangulations. In particular, *longest edge bisection* (LEB) [79, 80] is a popular refinement strategy that splits the longest edge of a simplex into two equal length edges. It has been shown in two and three dimensions that iterative LEB of a unit simplex produces a finite set of similarity classes of simplices [85, 72, 3]. For higher dimensions the LEB refinement is conjectured to produce finitely many similarity classes, but a proof remains an open problem [88].

6.2 Simplex Maps

This section defines the *simplex map* data structure. Section 6.2.1 describes the interface that a simplex map provides, Section 6.2.2 describes the implementation, and Section 6.2.3 discusses the complexity of simplex maps operations compared to CZT and CHMCTS.

6.2.1 Interface

A simplex map $\boldsymbol{\mathcal{G}}_{\text{SM}} = (\boldsymbol{\mathcal{V}}_{\text{SM}}, \boldsymbol{\mathcal{E}}_{\text{SM}}, \boldsymbol{\mathcal{T}}_{\text{SM}}, \mathbf{f}_{\text{SM}})$ consists of a set of vertices $\boldsymbol{\mathcal{V}}_{\text{SM}} \subseteq \Delta^D$, a set of undirected edges $\boldsymbol{\mathcal{E}}_{\text{SM}} \subseteq \boldsymbol{\mathcal{V}}_{\text{SM}} \times \boldsymbol{\mathcal{V}}_{\text{SM}}$, a binary tree of simplices $\boldsymbol{\mathcal{T}}_{\text{SM}}$ and a function $\mathbf{f}_{\text{SM}} : \boldsymbol{\mathcal{V}}_{\text{SM}} \to \mathbb{R}^D$ mapping vertices to value estimates. The root node of $\boldsymbol{\mathcal{T}}_{\text{SM}}$ is the unit simplex and the tree is formed using longest edge bisections so that the leaf nodes of the tree form a triangulation of the unit simplex. Each simplex in $\boldsymbol{\mathcal{T}}_{\text{SM}}$ is represented by its set of D vertices from $\boldsymbol{\mathcal{V}}_{\text{SM}}$, and a simplex is

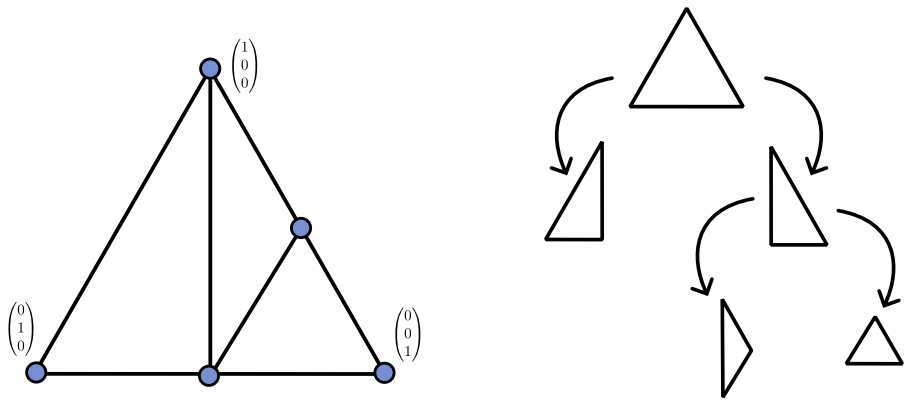


Figure 6.2: Illustration of a simplex map $\boldsymbol{\mathcal{G}}_{\mathrm{SM}} = (\boldsymbol{\mathcal{V}}_{\mathrm{SM}}, \boldsymbol{\mathcal{E}}_{\mathrm{SM}}, \boldsymbol{\mathcal{T}}_{\mathrm{SM}}, \mathbf{f}_{\mathrm{SM}})$ with $D = 3$. (Left) The graph of vertices $(\boldsymbol{\mathcal{V}}_{\mathrm{SM}}, \boldsymbol{\mathcal{E}}_{\mathrm{SM}})$ is shown. The vertices corresponding to the unit vectors are highlighted to emphasise that simplex map diagrams are using *barycentric coordinates*. (Right) The corresponding binary tree of simplices $\boldsymbol{\mathcal{T}}_{\mathrm{SM}}$.

identified with its vertex set throughout, since a simplex is uniquely determined by its vertices. Figure 6.2 illustrates a simplex map with $D = 3$.

`find_containing_simplex`$(\boldsymbol{\mathcal{G}}_{\mathrm{SM}}, \mathbf{w})$**:** Given a weight vector $\mathbf{w} \in \Delta^D$, returns the leaf node in the binary tree of simplices $\boldsymbol{\mathcal{T}}_{\mathrm{SM}}$ that contains $\mathbf{w}$.

`find_closest_vertex`$(\boldsymbol{\mathcal{G}}_{\mathrm{SM}}, \mathbf{w})$**:** Given a weight vector $\mathbf{w} \in \Delta^D$, finds the (approximately) closest vertex in the simplex map $\mathbf{w}_{\mathrm{SM}} \in \boldsymbol{\mathcal{V}}_{\mathrm{SM}}$ and returns the corresponding value estimate $\mathbf{f}_{\mathrm{SM}}(\mathbf{w}_{\mathrm{SM}})$.

`push`$(\boldsymbol{\mathcal{G}}_{\mathrm{SM}}, \mathbf{w}_{\mathrm{SM}})$**:** Given a vertex $\mathbf{w}_{\mathrm{SM}} \in \boldsymbol{\mathcal{V}}_{\mathrm{SM}}$, attempts to share its value estimate with its neighbours in the vertex graph via breadth-first search. A neighbour $\mathbf{w}'_{\mathrm{SM}} \in \boldsymbol{\mathcal{V}}_{\mathrm{SM}}$ accepts the value from $\mathbf{w}_{\mathrm{SM}}$ if it is an improvement under $\mathbf{w}'_{\mathrm{SM}}$'s own weight vector:

$$\mathbf{f}_{\mathrm{SM}}(\mathbf{w}'_{\mathrm{SM}}) \leftarrow \mathbf{f}_{\mathrm{SM}}(\mathbf{w}_{\mathrm{SM}}) \quad \text{if } {\mathbf{w}'_{\mathrm{SM}}}^{\top} \mathbf{f}_{\mathrm{SM}}(\mathbf{w}_{\mathrm{SM}}) > {\mathbf{w}'_{\mathrm{SM}}}^{\top} \mathbf{f}_{\mathrm{SM}}(\mathbf{w}'_{\mathrm{SM}}). \tag{6.1}$$

If the value estimate at $\mathbf{w}'_{\mathrm{SM}}$ is updated, then the breadth-first search is continued from $\mathbf{w}'_{\mathrm{SM}}$, up to a maximum push radius $\texttt{push_radius}_{\mathrm{SM}}$.

`maybe_split`$(\boldsymbol{\mathcal{G}}_{\mathrm{SM}}, \boldsymbol{\mathcal{S}}_{\mathrm{SM}})$**:** Given a leaf simplex $\boldsymbol{\mathcal{S}}_{\mathrm{SM}} \in \boldsymbol{\mathcal{T}}_{\mathrm{SM}}$, performs a longest edge bisection and adds the two new simplices in the tree.

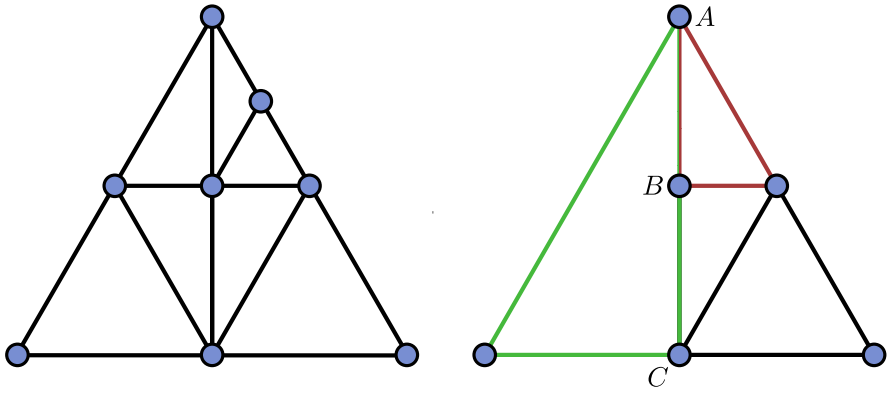


Figure 6.3: Examples of conforming and non-conforming triangulations. (Left) A conforming triangulation of the unit simplex. (Right) A non-conforming triangulation of the unit simplex. The green simplex is non-conforming because its intersection with the adjacent red simplex is the line segment AB, which is an edge of the red simplex but only part of the edge AC of the green simplex.

Together with direct writes to the value estimates $\mathbf{f}_{\text{SM}}$, this interface is sufficient to define MOMCTS algorithms using simplex maps. In particular, if the value estimates $\mathbf{f}_{\text{SM}}$ of a simplex map $\boldsymbol{\mathcal{G}}_{\text{SM}}$ represent multi-objective Q-values at a chance node (s, a), then the following can be used as a contextual value estimate:

$$\hat{Q}^{\mathbf{w}}_{\text{SM}}(s,a) = \mathbf{w}^\top \mathbf{f}_{\text{SM}}(\mathbf{w}_{\text{SM}}) \quad \text{where } \mathbf{w}_{\text{SM}} = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w}). \quad (6.2)$$

6.2.2 Implementation

Further details on the implementation of simplex maps are given in this section. Some implementation details are simpler in two dimensions than in the general D-dimensional case; $D = 3$ is used for examples throughout.

Conforming triangulations are described first, followed by the initialisation and implementation of each operation.

An additional auxiliary structure, called the *bipartite triangulation graph* (BTG), is used in the implementation but not defined explicitly to avoid unnecessary complexity. The BTG consists of edges of $\boldsymbol{\mathcal{E}}_{\text{SM}}$ on one side and the leaf simplices of $\boldsymbol{\mathcal{T}}_{\text{SM}}$ on the other. The BTG is used to determine adjacency between simplices and to support the conformity checks described below.

Conforming Triangulations. Two simplices are *adjacent* if they share at least one point. A simplex is *conforming* if, for every adjacent simplex, the intersection of the two is *exactly* a point, edge or face of the simplex itself. A triangulation is *conforming* if every simplex in it is conforming. Figure 6.3 illustrates a conforming and a non-conforming triangulation.

It has been shown that recursive longest edge bisection, in a general D-dimensions, produces a conforming triangulation within a finite number of steps [78, 79, 80].

The implementation of the `find_closest_vertex` operation assumes that the nearest vertex to any point $\mathbf{w}$ lies either in the simplex containing $\mathbf{w}$ or in an adjacent simplex. This assumption is supported by the conjectured result that LEB refinement produces a finite number of similarity classes in higher dimensions [88].

The BTG can be used to determine if a simplex in $\boldsymbol{\mathcal{T}}_{\text{SM}}$ is conforming by checking if the number of connections in the BTG is equal to the number of edges of a simplex. For example, in Figure 6.3, the non-conforming simplex is highlighted in green. It has 3 edges, but is adjacent to 4 edges of the vertex graph $\boldsymbol{\mathcal{E}}_{\text{SM}}$.

The simplex map is implemented to be *eventually conforming*, meaning that a non-conforming triangulation is tolerated temporarily, and after every operation described in Section 6.2.1, a largest non-conforming simplex is identified and the `maybe_split` operation is performed on it.

Initialisation. The vertex set is initialised to the vertices of the unit simplex Δ^D, which are the standard basis vectors of $\mathbb{R}^D$, giving $\boldsymbol{\mathcal{V}}_{\text{SM}} = \{\mathbf{e}_1, \ldots, \mathbf{e}_D\}$. The edge set $\boldsymbol{\mathcal{E}}_{\text{SM}}$ is initialised to every unique pair of these vertices, and the binary tree of simplices $\boldsymbol{\mathcal{T}}_{\text{SM}}$ is initialised to a single root node, the unit simplex Δ^D. The value estimates $\mathbf{f}_{\text{SM}}(\mathbf{w}_{\text{SM}})$ are initialised uniformly for all $\mathbf{w}_{\text{SM}} \in \boldsymbol{\mathcal{V}}_{\text{SM}}$, to zero unless stated otherwise.

`find_containing_simplex`$(\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w})$: To find the simplex in the triangulation that contains a weight vector $\mathbf{w} \in \Delta^D$, the triangulation is traversed by the binary tree $\boldsymbol{\mathcal{T}}_{\text{SM}}$ using a sequence of halfspace checks at each internal node.

`find_closest_vertex`($\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w}$)**:** In the case of $D = 2$, the closest vertex can be found exactly by returning the closest endpoint of the line segment (1-simplex) containing $\mathbf{w}$.

For the general case of $D > 2$, the closest vertex can be found approximately. First the simplex containing $\mathbf{w}$ is found using `find_containing_simplex`. Then the *k-ring* (k-step neighbourhood) of simplices is checked for the closest vertex to $\mathbf{w}$.

k is technically a parameter used in the simplex map implementation, but it is always set to 1 in this thesis. The assumption is made that the nearest vertex to $\mathbf{w}$ is either on the simplex containing $\mathbf{w}$ or an adjacent simplex. This assumption is supported by the conjecture that LEB refinement produces finitely many similarity classes [88], which would enforce that the diameter of adjacent simplices is bounded, and ensure that the nearest vertex cannot lie far from the containing simplex.

Figure 6.4 illustrates that for $D = 3$, setting $k = 0$ is insufficient and that a neighbouring triangle may contain the closest vertex to $\mathbf{w}$. Also illustrated for $D = 3$ is that $k = 1$ is insufficient to find the closest vertex if the triangulation is non-conforming.

`push`**:** The description of `push` given in Section 6.2.1 is sufficient to implement it, and Figure 6.5 illustrates the operation.

The function $\mathbf{f}_{\text{SM}}$ is implemented as a hash map from vertices to value estimates, which an algorithm can update directly.

`maybe_split`($\boldsymbol{\mathcal{G}}_{\text{SM}}, \boldsymbol{\mathcal{S}}_{\text{SM}}$)**:** The `maybe_split` operation is controlled by three parameters $\texttt{split_min_radius}_{\text{SM}}$, $\texttt{split_min_delta}_{\text{SM}}$ and $\texttt{split_min_visit}_{\text{SM}}$, which determine the conditions under which a simplex is split. Let $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}})$ be a counter for the number of times that `maybe_split` has been called on the simplex $\boldsymbol{\mathcal{S}}_{\text{SM}}$.

The following steps are performed to determine if the simplex should be split:

- If the simplex $\boldsymbol{\mathcal{S}}_{\text{SM}}$ is non-conforming, then the simplex is split as described below;

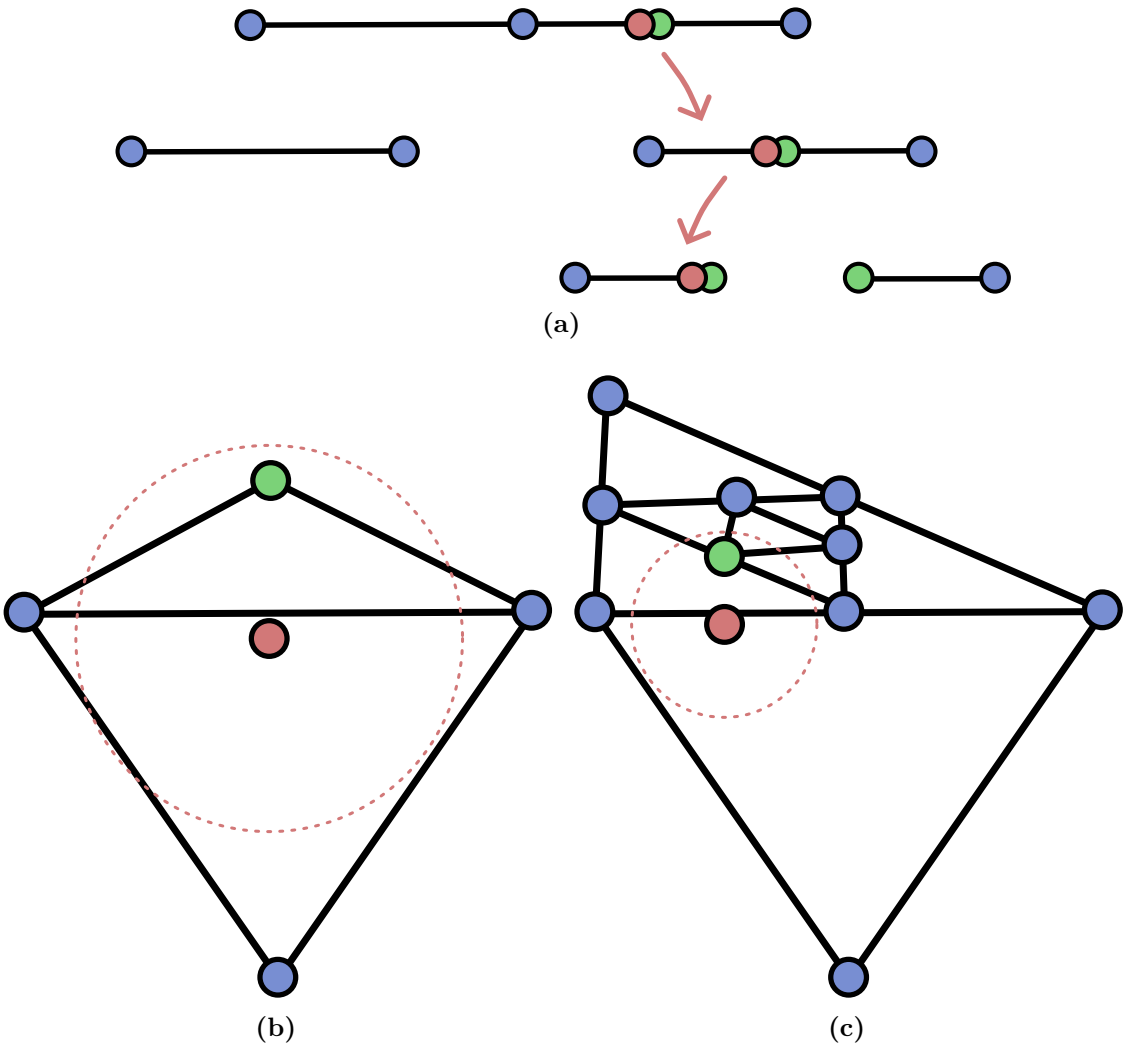


Figure 6.4: Closest vertex lookup examples. In all subfigures, the weight vector $\mathbf{w}$ is shown in red, and the closest vertex in the triangulation is shown in green. (a) Finding the closest vertex in $D = 2$ corresponds to traversing the binary tree and returning the closest vertex. (b) Part of a triangulation where the closest vertex is in a neighbouring simplex. (c) Example where a triangulation is non-conforming and the closest vertex is not on an extreme point of a neighbouring simplex[1].

- If the radius of the simplex $\boldsymbol{\mathcal{S}}_{\text{SM}}$ is less than $\texttt{split_min_radius}_{\text{SM}}$, then the simplex is not split;
- If $\max_{\mathbf{w}_{\text{SM}}, \mathbf{w}'_{\text{SM}} \in \boldsymbol{\mathcal{S}}_{\text{SM}}} \|\mathbf{f}_{\text{SM}}(\mathbf{w}_{\text{SM}}) - \mathbf{f}_{\text{SM}}(\mathbf{w}'_{\text{SM}})\|_2 < \texttt{split_min_delta}_{\text{SM}}$, then the counter $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}})$ is reset to 0, and the simplex is not split;
- $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}})$ is incremented by 1, and if $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}}) < \texttt{split_min_visit}_{\text{SM}}$, then the simplex is not split;

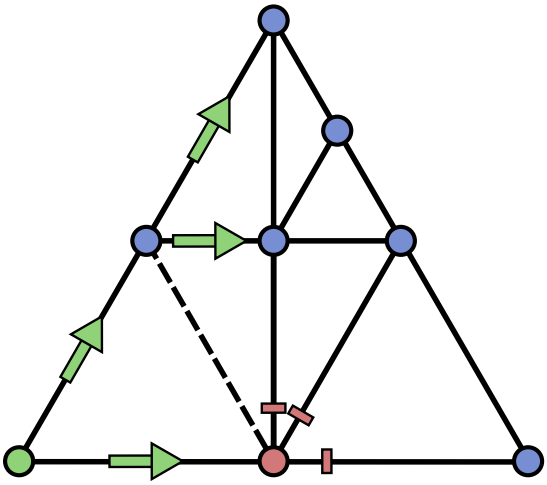


Figure 6.5: Illustration of the `push` operation of a simplex map, with $\texttt{push_radius}_{\text{SM}} = 2$. The green vertex is pushing its value estimate, and the green arrows indicate the direction of propagation. The red vertex does not accept or propagate the value, shown by the red barriers. Nothing is propagated along the dashed edge, as it joins two vertices at the same depth of the breadth first search from the green vertex.

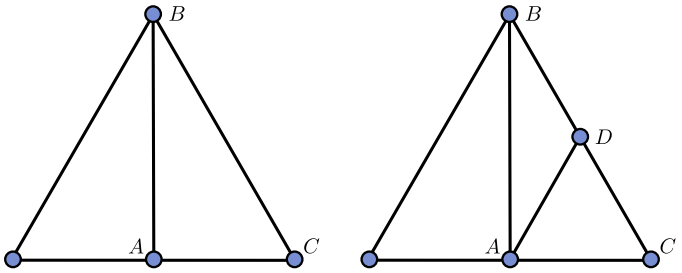


Figure 6.6: Example of a successful `maybe_split` operation on the simplex ABC in the simplex map $\mathcal{G}_{\text{SM}} = (\boldsymbol{\mathcal{V}}_{\text{SM}}, \boldsymbol{\mathcal{E}}_{\text{SM}}, \boldsymbol{\mathcal{T}}_{\text{SM}}, \mathbf{f}_{\text{SM}})$. The longest edge of the simplex, BC, is bisected, and the new vertex D is added to $\boldsymbol{\mathcal{V}}_{\text{SM}}$. The edge BC is removed from $\boldsymbol{\mathcal{E}}_{\text{SM}}$, and the edges AD, BD and CD are added to $\boldsymbol{\mathcal{E}}_{\text{SM}}$. The two new simplices ABD and ACD are added as children of ABC in $\boldsymbol{\mathcal{T}}_{\text{SM}}$, and $\mathbf{f}_{\text{SM}}$ is initialised at the new vertex D using Equation (6.6). (Left) The vertex graph before the split. (Right) The vertex graph after the split.

- Otherwise, the simplex is split as described below.

Let $\boldsymbol{\mathcal{S}}_{\text{SM}} = \{\mathbf{w}_{\text{SM}}^{(1)}, \mathbf{w}_{\text{SM}}^{(2)}, \mathbf{w}_{\text{SM}}^{(3)}, \ldots, \mathbf{w}_{\text{SM}}^{(D)}\}$, where without loss of generality $(\mathbf{w}_{\text{SM}}^{(1)}, \mathbf{w}_{\text{SM}}^{(2)})$ is the longest edge. The new vertex $\mathbf{w}_{\text{SM}}^{(\text{mid})} = \frac{1}{2}(\mathbf{w}_{\text{SM}}^{(1)} + \mathbf{w}_{\text{SM}}^{(2)})$ is added to $\boldsymbol{\mathcal{V}}_{\text{SM}}$, the edge $(\mathbf{w}_{\text{SM}}^{(1)}, \mathbf{w}_{\text{SM}}^{(2)})$ is removed from $\boldsymbol{\mathcal{E}}_{\text{SM}}$ and edges to connect the new vertex are added:

$$\boldsymbol{\mathcal{E}}_{\text{SM}} \leftarrow \left(\boldsymbol{\mathcal{E}}_{\text{SM}} - \{(\mathbf{w}_{\text{SM}}^{(1)}, \mathbf{w}_{\text{SM}}^{(2)})\}\right) \cup \{(\mathbf{w}_{\text{SM}}, \mathbf{w}_{\text{SM}}^{(\text{mid})}) \mid \mathbf{w}_{\text{SM}} \in \boldsymbol{\mathcal{S}}_{\text{SM}}\}. \tag{6.3}$$

The following simplices are added as the children of $\boldsymbol{\mathcal{S}}_{\mathrm{SM}}$ in the binary tree $\boldsymbol{\mathcal{T}}_{\mathrm{SM}}$:

$$\boldsymbol{\mathcal{S}}^{(1)}_{\mathrm{SM}} = \{\mathbf{w}^{(1)}_{\mathrm{SM}}, \mathbf{w}^{(\mathrm{mid})}_{\mathrm{SM}}, \mathbf{w}^{(3)}_{\mathrm{SM}}, \mathbf{w}^{(4)}_{\mathrm{SM}}, \ldots, \mathbf{w}^{(D)}_{\mathrm{SM}}\}, \tag{6.4}$$

$$\boldsymbol{\mathcal{S}}^{(2)}_{\mathrm{SM}} = \{\mathbf{w}^{(2)}_{\mathrm{SM}}, \mathbf{w}^{(\mathrm{mid})}_{\mathrm{SM}}, \mathbf{w}^{(3)}_{\mathrm{SM}}, \mathbf{w}^{(4)}_{\mathrm{SM}}, \ldots, \mathbf{w}^{(D)}_{\mathrm{SM}}\}. \tag{6.5}$$

Finally, the new vertex $\mathbf{w}^{(\mathrm{mid})}_{\mathrm{SM}}$ inherits its value estimate from the values of the edge that was split:

$$\mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(\mathrm{mid})}_{\mathrm{SM}}) \leftarrow \begin{cases} \mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(1)}_{\mathrm{SM}}) & \text{if } \mathtt{N}_{\mathrm{SM}}(\mathbf{w}^{(2)}_{\mathrm{SM}}) = 0, \\ \mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(2)}_{\mathrm{SM}}) & \text{if } \mathtt{N}_{\mathrm{SM}}(\mathbf{w}^{(1)}_{\mathrm{SM}}) = 0, \\ \mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(1)}_{\mathrm{SM}}) & \text{if } {\mathbf{w}^{(\mathrm{mid})}_{\mathrm{SM}}}^{\top} \mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(1)}_{\mathrm{SM}}) > {\mathbf{w}^{(\mathrm{mid})}_{\mathrm{SM}}}^{\top} \mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(2)}_{\mathrm{SM}}), \\ \mathbf{f}_{\mathrm{SM}}(\mathbf{w}^{(2)}_{\mathrm{SM}}) & \text{otherwise,} \end{cases} \tag{6.6}$$

where $\mathtt{N}_{\mathrm{SM}}(\mathbf{w})$ is the number of times that $\mathbf{f}_{\mathrm{SM}}(\mathbf{w})$ has been updated for the vertex weight $\mathbf{w} \in \boldsymbol{\mathcal{V}}_{\mathrm{SM}}$.

Figure 6.6 illustrates a successful `maybe_split`$(\boldsymbol{\mathcal{G}}_{\mathrm{SM}}, \boldsymbol{\mathcal{S}}_{\mathrm{SM}})$ operation. The BTG is updated accordingly; the details are omitted as they do not affect the algorithm semantics.

6.2.3 Complexity Discussion

The computational cost of simplex map operations is briefly compared to those of CHMCTS and CZT as follows.

For CHMCTS, using K_{CH} to denote the number of points in a convex hull value set, taking the union of two value sets during the $\hat{\boldsymbol{\mathcal{V}}}_{\mathrm{ALG}}$ backup (Equation 5.14) costs $\Omega(K_{\mathrm{CH}})$. However, computing the $\hat{\boldsymbol{\mathcal{Q}}}_{\mathrm{ALG}}$ backup (Equation 5.15) requires adding two value sets, which involves computing a Minkowski sum of $\Omega(K^2_{\mathrm{CH}})$ candidate points before pruning. This provides intuition for why CHMCTS becomes significantly slower in stochastic environments, where intermediate value sets grow large.

For CZT, the cost of selection scales with the number of active balls via Equation (5.8), which grows unboundedly over time.

For simplex maps, under the assumption that LEB refinement produces finitely many similarity classes, the maximum number of edges incident to any vertex is $O(2^D)$, and the BTG has a similar bound on its degree. Simplex lookups via the

binary tree cost roughly $O(D \log(K_{\text{SM}}))$, where K_{SM} is the number of vertices and the factor of D comes from the halfspace checks at each internal node.

Therefore, for an MOMDP where the number of achievable values grows large, i.e. $K_{\text{CH}} \gg 2^D$, the computational complexity of CHMCTS will dominate the complexity of simplex map operations. Conversely, when an MOMDP has a small number of achievable values, simplex maps may introduce additional computational overhead over CHMCTS, as the order of vertices in the simplex map will still be $\Omega(2^D)$.

6.3 Simplex Maps for Tree Search

This section defines the DENTS with Simplex Maps (SM-DENTS) algorithm. SM-DENTS is a variant of DENTS that uses simplex maps to maintain the value and entropy estimates. The algorithm uses parameters $\alpha_{\text{SM-D}}$, $\beta_{\text{SM-D}}$, and $\epsilon_{\text{SM-D}}$ corresponding to the DENTS parameters. SM-DENTS also follows the contextual tree search approach (Section 5.1.2), where a context weight $\mathbf{w}$ is randomly sampled and used throughout the trial.

Each decision node s and chance node (s, a) keeps an associated simplex map

$$\boldsymbol{\mathcal{G}}_{\text{SM}}(s) = (\boldsymbol{\mathcal{V}}_{\text{SM}}(s), \boldsymbol{\mathcal{E}}_{\text{SM}}(s), \boldsymbol{\mathcal{T}}_{\text{SM}}(s), \hat{\mathbf{V}}_{\text{SM-D}}(s;\cdot), \bar{\mathcal{H}}_{V,\text{SM-D}}(s;\cdot)) \tag{6.7}$$

$$\boldsymbol{\mathcal{G}}_{\text{SM}}(s,a) = (\boldsymbol{\mathcal{V}}_{\text{SM}}(s,a), \boldsymbol{\mathcal{E}}_{\text{SM}}(s,a), \boldsymbol{\mathcal{T}}_{\text{SM}}(s,a), \hat{\mathbf{Q}}_{\text{SM-D}}(s,a;\cdot), \bar{\mathcal{H}}_{Q,\text{SM-D}}(s,a;\cdot)), \tag{6.8}$$

where $\hat{\mathbf{V}}_{\text{SM-D}}$ and $\hat{\mathbf{Q}}_{\text{SM-D}}$ are the value and Q-value estimate functions for each map, and $\bar{\mathcal{H}}_{V,\text{SM-D}}(s;\cdot)$ and $\bar{\mathcal{H}}_{Q,\text{SM-D}}(s,a;\cdot)$ are the corresponding entropy estimate functions, where SM-DENTS is abbreviated to SM-D in subscripts. The simplex maps are initialised as described in Section 6.2.2, and the value estimates $\hat{\mathbf{V}}_{\text{SM-D}}(s;\cdot)$ and $\hat{\mathbf{Q}}_{\text{SM-D}}(s,a;\cdot)$ are initialised to $\hat{\mathbf{V}}_{\text{heuristic}}(s)$ and $\hat{\mathbf{Q}}_{\text{heuristic}}(s,a)$ for each vertex in $\boldsymbol{\mathcal{V}}_{\text{SM}}(s)$ and $\boldsymbol{\mathcal{V}}_{\text{SM}}(s,a)$ respectively.

Two further pieces of notation are convenient for defining SM-DENTS. First, the context weight $\mathbf{w}$ sampled for a trial is generally not itself a vertex of the simplex maps, so the closest vertex in each map is often looked up:

$$\mathbf{w}_{\text{SM}}(s) = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s), \mathbf{w}), \tag{6.9}$$

$$\mathbf{w}_{\text{SM}}(s,a) = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s,a), \mathbf{w}). \tag{6.10}$$

Second, following Equation (6.2), contextual value estimates for an arbitrary weight $\mathbf{w}' \in \Delta^D$ are obtained by using the linear utility from the closest vertex in the simplex map:

$$\hat{V}^{\mathbf{w}'}_{\text{SM-D}}(s) = \mathbf{w}'^{\top} \hat{\mathbf{V}}_{\text{SM-D}}(s; \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s), \mathbf{w}')), \tag{6.11}$$

$$\hat{Q}^{\mathbf{w}'}_{\text{SM-D}}(s, a) = \mathbf{w}'^{\top} \hat{\mathbf{Q}}_{\text{SM-D}}(s, a; \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s, a), \mathbf{w}')). \tag{6.12}$$

Search Policy The search policy is defined similarly to DENTS (Section 4.2.2), given a context weight $\mathbf{w} \in \Delta^D$, using the contextual value and entropy estimates:

$$\begin{aligned} \hat{Q}^{\mathbf{w},\text{sft}}_{\text{SM-D}}(s, a) &= \hat{Q}^{\mathbf{w}}_{\text{SM-D}}(s, a) \\ &\quad + \beta_{\text{SM-D}}(N(s)) \bar{\mathcal{H}}_{Q,\text{SM-D}}(s, a; \mathbf{w}_{\text{SM}}(s, a)), \end{aligned} \tag{6.13}$$

$$\pi_{\text{SM-D}}(a|s; \mathbf{w}) = (1 - \lambda(s, \epsilon_{\text{SM-D}})) \rho_{\text{SM-D}}(a|s; \mathbf{w}) + \frac{\lambda(s, \epsilon_{\text{SM-D}})}{|\mathcal{A}|}, \tag{6.14}$$

$$\rho_{\text{SM-D}}(a|s; \mathbf{w}) \propto \exp\left(\frac{\hat{Q}^{\mathbf{w},\text{sft}}_{\text{SM-D}}(s, a)}{\alpha_{\text{SM-D}}(N(s))}\right), \tag{6.15}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{6.16}$$

Backups Given a context weight $\mathbf{w} \in \Delta^D$ and a trajectory $\tau_{:h} = (s_0, a_0, \mathbf{r}_0, \ldots, s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h) \sim \pi_{\text{SM-D}}(\cdot|\cdot; \mathbf{w})$, the `backup_q` and `backup_v` functions update the multi-objective and entropy value estimates for $t = h-1, \ldots, 0$:

$$\bar{\mathcal{H}}_{Q,\text{SM-D}}(s_t, a_t; \mathbf{w}_{\text{SM}}(s_t, a_t)) \leftarrow \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \bar{\mathcal{H}}_{V,\text{SM-D}}(s'; \mathbf{w}_{\text{SM}}(s')), \tag{6.17}$$

$$\begin{aligned} \bar{\mathcal{H}}_{V,\text{SM-D}}(s_t; \mathbf{w}_{\text{SM}}(s_t)) &\leftarrow \mathcal{H}(\pi_{\text{SM-D}}(\cdot|s_t; \mathbf{w}_{\text{SM}}(s_t))) \\ &\quad + \sum_{a \in \mathcal{A}} \pi_{\text{SM-D}}(a|s_t; \mathbf{w}_{\text{SM}}(s_t)) \bar{\mathcal{H}}_{Q,\text{SM-D}}(s_t, a; \mathbf{w}_{\text{SM}}(s_t, a)), \end{aligned} \tag{6.18}$$

$$\begin{aligned} \hat{\mathbf{Q}}_{\text{SM-D}}(s_t, a_t; \mathbf{w}_{\text{SM}}(s_t, a_t)) &\leftarrow \mathbf{R}(s_t, a_t) \\ &\quad + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{\mathbf{V}}_{\text{SM-D}}(s'; \mathbf{w}_{\text{SM}}(s'))\right), \end{aligned} \tag{6.19}$$

$$\hat{a}_t = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}_{\text{SM}}(s_t)}_{\text{SM-D}}(s_t, a), \tag{6.20}$$

$$\hat{\mathbf{V}}_{\text{SM-D}}(s_t; \mathbf{w}_{\text{SM}}(s_t)) \leftarrow \hat{\mathbf{Q}}_{\text{SM-D}}(s_t, \hat{a}_t; \mathbf{w}_{\text{SM}}(s_t, \hat{a}_t)). \tag{6.21}$$

Two points are worth noting about these backups:

- The value estimates are updated only at the closest vertices $\mathbf{w}_{\text{SM}}(s_t)$ and $\mathbf{w}_{\text{SM}}(s_t, a_t)$, looked up from the simplex maps $\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t)$ and $\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t, a_t)$ in Equations (6.9) and (6.10), rather than at the context $\mathbf{w}$ used for selection.

- Since a maximum is not well-defined over vectors, Equation (6.20) computes the action $\hat{a}_t$ that maximises the contextual Q-value with respect to $\mathbf{w}_{\text{SM}}(s_t)$, and Equation (6.21) uses that action to update the value estimate.

Once the Q-value estimates are updated by Equations (6.17) and (6.19), the new values at $\mathbf{w}_{\text{SM}}(s_t, a_t)$ are propagated across the simplex map by $\texttt{push}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t, a_t), \mathbf{w}_{\text{SM}}(s_t, a_t))$. A simplex is then split if necessary by calling:

$$\texttt{maybe_split}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t, a_t), \texttt{find_containing_simplex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t, a_t), \mathbf{w})). \tag{6.22}$$

Likewise, once the value estimates are updated by Equations (6.18) and (6.21), the new values at $\mathbf{w}_{\text{SM}}(s_t)$ are propagated by $\texttt{push}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t), \mathbf{w}_{\text{SM}}(s_t))$, followed by:

$$\texttt{maybe_split}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t), \texttt{find_containing_simplex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t), \mathbf{w})). \tag{6.23}$$

Recommendation Policy The recommendation policy is defined to follow the maximal contextual Q-values:

$$\psi_{\text{SM-D}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}}_{\text{SM-D}}(s, a). \tag{6.24}$$

BTS with Simplex Maps BTS with Simplex Maps (SM-BTS) is defined by setting the entropy coefficient function $\beta_{\text{SM-D}}(x) = 0$.

6.4 Empirical Results

This section presents the empirical results for this chapter. The experimental setup follows that of Section 5.4 in Chapter 5, so only the differences are given here:

- The MOMDP environments are the same as those described in Section 5.4.1;

- Section 6.4.1 gives additional details on the evaluation setup used;

- Section 6.4.2 presents the results on those environments;
- Section 6.4.3 presents scalability results on the Deep Sea Treasure environment;
- Appendix C provides the algorithm parameters used and how they were chosen.

Results for CZT, CH-BTS and CH-UCT are repeated from Section 5.4 as baselines for SM-BTS and SM-DENTS.

6.4.1 Evaluation Procedure

The evaluation setup used for this chapter is identical to that of Section 5.4.2. However, like CZT, the SM-BTS and SM-DENTS algorithms do not operate directly on convex hull value sets, so a value set must be specified in order to compute the hypervolume metric.

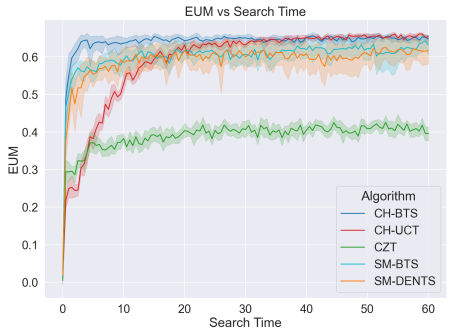
As was done for CZT in Section 5.4.2, an approximate convex hull value set is constructed from the values stored in the simplex map at the root node. Given the simplex map $\boldsymbol{\mathcal{G}}_{\text{SM}}(s_0)$ defined in Equation (6.7), the following set of points is used:

$$\texttt{cvx_prune}\left(\left\{\hat{\mathbf{V}}_{\text{SM-D}}(s_0;\mathbf{w}) \mid \mathbf{w} \in \boldsymbol{\mathcal{V}}_{\text{SM}}(s_0), \mathtt{N}_{\text{SM}}(\mathbf{w}) > 0\right\}\right), \tag{6.25}$$

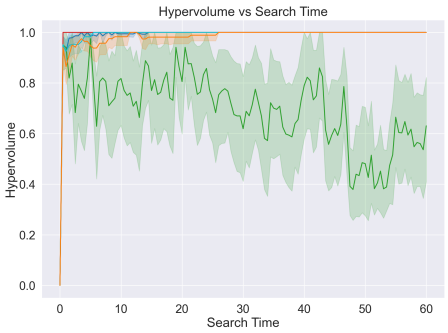
where $\mathtt{N}_{\text{SM}}(\mathbf{w})$ is the number of times that the value estimate at the vertex $\mathbf{w}$ of the simplex map has been updated.

6.4.2 Multi-Objective Gridworld Results

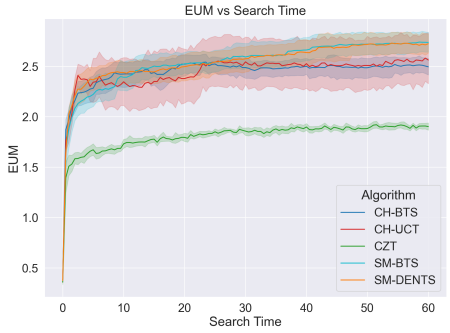
Gymnasium Environments. Results on the MO-Gymnasium environments are given in Figure 6.7. SM-BTS and SM-DENTS perform comparably to the CHMCTS algorithms across all four environments, though generally converging slightly more slowly. On Fruit-Tree-7, the hypervolume curves for the simplex map algorithms are higher than those of the CHMCTS algorithms. This is because size pruning limits each convex hull value set to $K_{\max} = 15$ points (Section 5.3.6),



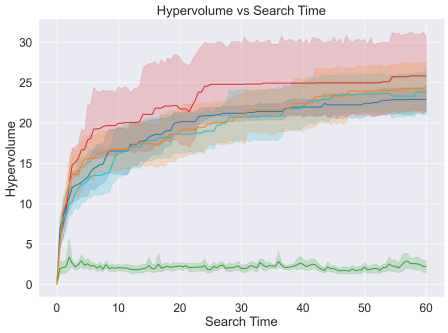
(a) Resource-Gathering (EUM).



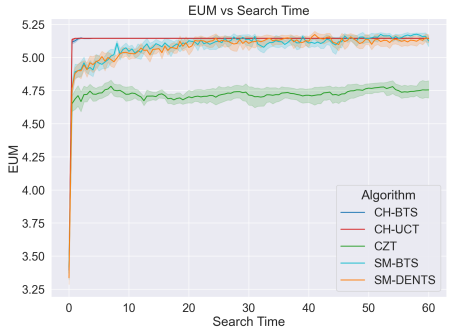
(b) Resource-Gathering (HV).



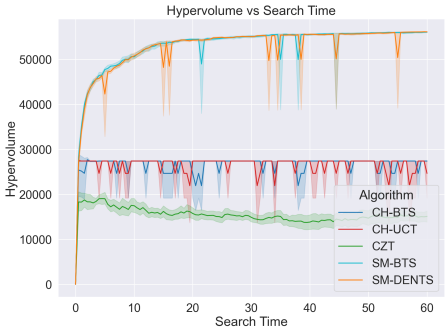
(c) Four-Room (EUM).



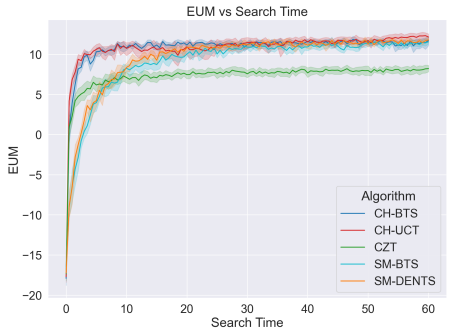
(d) Four-Room (HV).



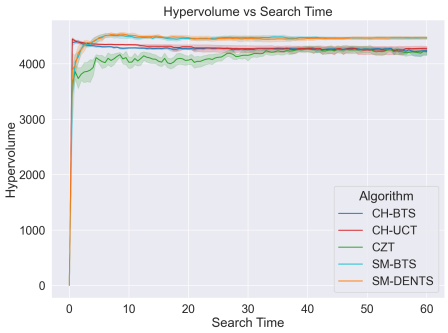
(e) Fruit-Tree-7 (EUM).



(f) Fruit-Tree-7 (HV).

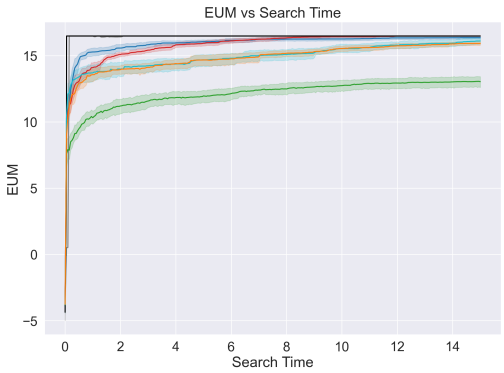


(g) Breakable-Bottles (EUM).

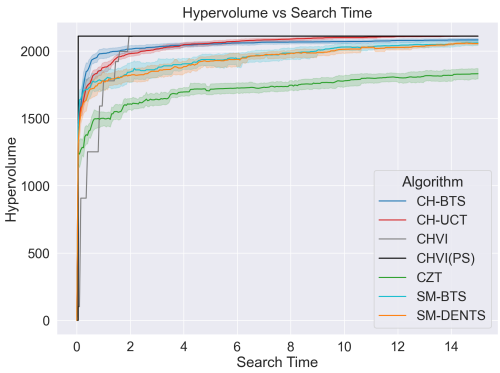


(h) Breakable-Bottles (HV).

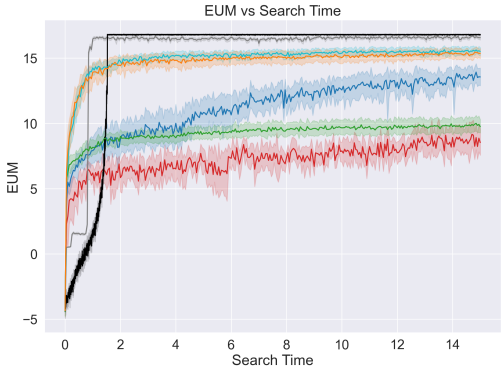
Figure 6.7: Results on the MO-Gymnasium environments. (Left) Expected Utility Metric (EUM). (Right) Hypervolume (HV). Each row shows the results for a different environment: Resource-Gathering, Four-Room, Fruit-Tree-7 and Breakable-Bottles respectively.



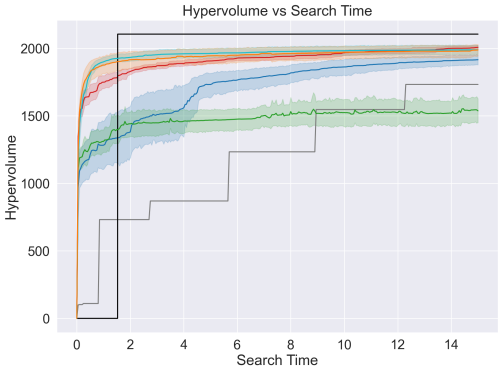
(a) DST$(10, 0)$ (EUM).



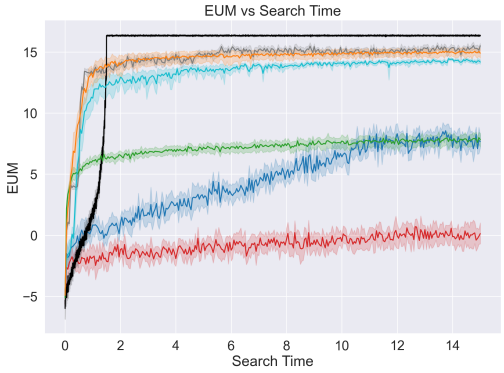
(b) DST$(10, 0)$ (HV).



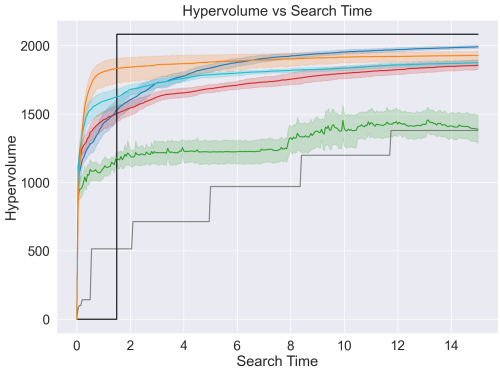
(c) DST$(10, \frac{1}{40})$ (EUM).



(d) DST$(10, \frac{1}{40})$ (HV).



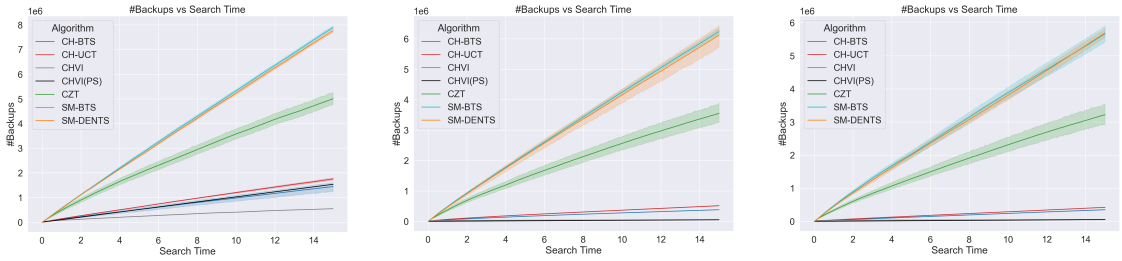
(e) DST$(10, \frac{1}{5})$ (EUM).



(f) DST$(10, \frac{1}{5})$ (HV).

Figure 6.8: Results on the Deep Sea Treasure environments. (Left) Expected Utility Metric (EUM). (Right) Hypervolume (HV). Each row shows the results for DST$(10, 0)$, DST$(10, \frac{1}{40})$ and DST$(10, \frac{1}{5})$ respectively.

which limits the hypervolume that the CHMCTS algorithms can achieve, whereas simplex maps are not subject to such a constraint.



(a) $\texttt{DST}(10, 0)$. **(b)** $\texttt{DST}(10, \frac{1}{40})$. **(c)** $\texttt{DST}(10, \frac{1}{5})$.

Figure 6.9: Number of backups performed on the Deep Sea Treasure environments for $\texttt{DST}(10, 0)$, $\texttt{DST}(10, \frac{1}{40})$ and $\texttt{DST}(10, \frac{1}{5})$ respectively.

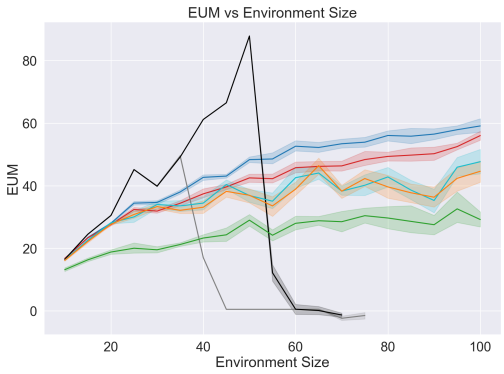
The results for CZT, CH-BTS and CH-UCT are repeated from Figure 5.4, so the non-monotonic trends in their hypervolume curves are explained in Section 5.4.3.

Deep Sea Treasure Environments. Results on the Deep Sea Treasure environments are given in Figures 6.8 and 6.9. On the deterministic environment $\texttt{DST}(10, 0)$, simplex map algorithms perform similarly to the CHMCTS algorithms. However, on both stochastic environments, $\texttt{DST}(10, \frac{1}{40})$ and $\texttt{DST}(10, \frac{1}{5})$, the EUM of SM-BTS and SM-DENTS is significantly higher than all algorithms from Chapter 5, matching the performance of CHVI (Figure 6.8e). As shown in Figure 6.9, simplex map algorithms are able to perform substantially more backups than convex hull-based algorithms within the same time budget, which accounts for their improved performance in stochastic environments.

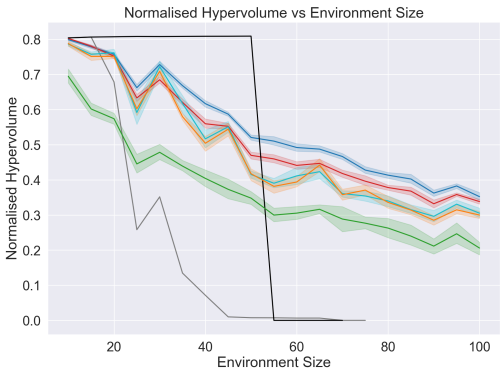
6.4.3 Scalability Results

Figure 6.10 repeats the scalability experiment of Section 5.4.4 with the simplex map algorithms added. The experimental setup, use of normalised hypervolume (NHV), and the trends in the CHVI, CZT and CHMCTS curves are as described there.

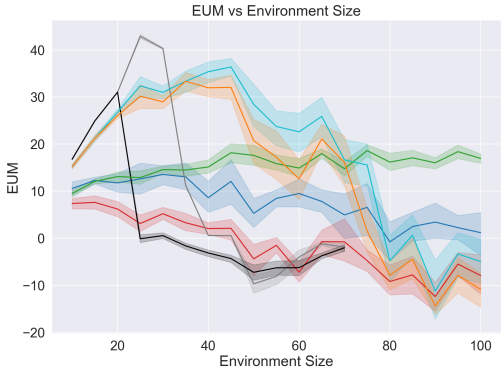
On the deterministic environment $\texttt{DST}(W, 0)$, the simplex map algorithms perform similarly to the CHMCTS algorithms, though slightly below them on both EUM and NHV across all environment sizes.



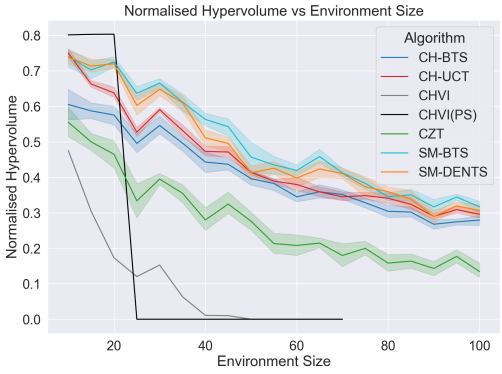
(a) $\texttt{DST}(W, 0)$ (EUM).



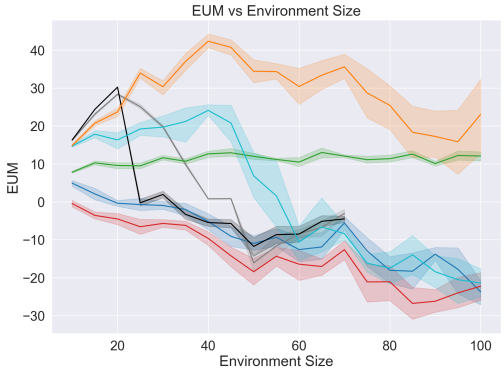
(b) $\texttt{DST}(W, 0)$ (NHV).



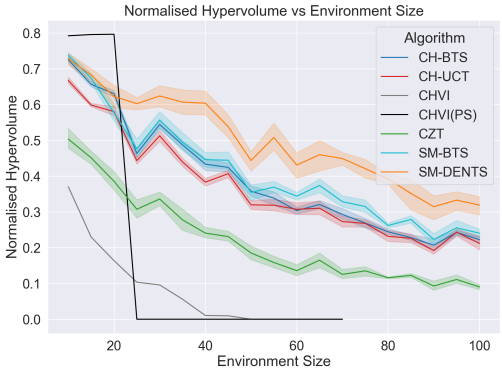
(c) $\texttt{DST}(W, \frac{1}{40})$ (EUM).



(d) $\texttt{DST}(W, \frac{1}{40})$ (NHV).



(e) $\texttt{DST}(W, \frac{1}{5})$ (EUM).



(f) $\texttt{DST}(W, \frac{1}{5})$ (NHV).

Figure 6.10: Scalability results on the Deep Sea Treasure environments. (Left) Expected Utility Metric (EUM). (Right) Normalised Hypervolume (NHV). Each row shows the results for $\texttt{DST}(W, 0)$, $\texttt{DST}(W, \frac{1}{40})$ and $\texttt{DST}(W, \frac{1}{5})$ respectively, for a range of widths W from 10 to 100. Each algorithm was allowed up to 16 GB of memory; the curves for CHVI and CHVI(PS) are truncated where memory was exhausted.

In contrast to Figure 5.7, SM-BTS and SM-DENTS improve in EUM in the stochastic environments as the environment size increases, before declining once the fixed planning budget becomes the limiting factor. On the mildly stochastic environment $\texttt{DST}(W, \frac{1}{40})$, both maintain good performance up to $W = 40$.

On the heavily stochastic environment $\texttt{DST}(W, \frac{1}{5})$, SM-DENTS achieves near-optimal EUM for environment sizes up to $W = 20$ (beyond which CHVI and CHVI(PS) degrade), continues to improve until around $W = 40$, plateaus until $W = 70$, and then degrades to performance comparable to CZT at $W = 100$. This represents a substantial improvement over all CHMCTS algorithms, which are unable to run a sufficient number of trials to make progress in stochastic environments at these scales.

SM-BTS does not match this performance. Its temperature is tuned so low that the search policy is close to greedy, leaving the uniform exploration parameter, set to more than twice the value used by SM-DENTS (Table C.3), as its main source of exploration. SM-DENTS instead obtains exploration from the entropy bonus, which allows it to exploit more later in the search. As discussed in Section 4.4.3.1, exploitation is still necessary in these stochastic environments so that the recommendation policy does not leave the tree and fall back to acting uniformly at random (Equation (5.41)).

6.5 Summary

This chapter introduced *simplex maps*, an adaptive data structure for maintaining value and entropy estimates as a function of the weight vector $\mathbf{w} \in \Delta^D$, and used them to define two MOMCTS algorithms: SM-BTS and SM-DENTS.

Simplex maps address the two limitations of CHMCTS identified in Chapter 5. By using a hierarchical representation the computational costs are generally lower (**Q3.1 - Complexity**), particularly in environments where the number of points in a convex hull value set is large, making simplex maps significantly more scalable in stochastic environments (**Q2 - Stochastic Multi-Objective**

Domains). Additionally, by utilising a collection of point estimates in weight space, simplex maps provide a principled way to associate entropy values with specific weight vectors, enabling entropy-based exploration to be lifted to the multi-objective setting via SM-DENTS (**Q1.2 - Multi-Objective Exploration**).

The simplex map data structure was defined (Section 6.2) through its interface of query, update, message passing and split operations, and its implementation via a vertex graph, a binary tree of simplices, and an eventually conforming triangulation maintained using longest edge bisection. The SM-BTS and SM-DENTS algorithms were defined (Section 6.3) as multi-objective extensions of BTS and DENTS respectively, using contextual value estimates derived from the simplex map in place of scalar Bellman value estimates.

Empirically, simplex map algorithms performed comparably to CHMCTS on deterministic environments and small MO-Gymnasium environments, while significantly outperforming all CHMCTS algorithms on the stochastic $\texttt{DST}(W, \frac{1}{5})$ environment, both in terms of EUM and the number of backups that could be performed within the time budget. The scalability results demonstrated that SM-DENTS maintains strong performance across a much wider range of map sizes than CHMCTS in heavily stochastic environments (**Q3.2 - Multi-Objective Scalability**).

7 Conclusion

Contents

7.1 Summary of Contributions 189
7.2 Future Work 191

This thesis has investigated MCTS algorithms for sequential decision-making problems, in both the single- and multi-objective settings. The main contributions of this thesis are summarised in Section 7.1 by revisiting the research questions of Chapter 1, and Section 7.2 discusses possible directions for future work.

7.1 Summary of Contributions

The research questions considered in this thesis (Section 1.3) are revisited in turn, and the corresponding contributions are summarised.

Q1 - Exploration. Chapter 4 considered how MCTS algorithms can prioritise exploration in the exploration setting of reinforcement learning, where the agent is assessed only on the recommendations it makes. UCT was shown to be biased towards exploitation in this setting, and the maximum entropy objective used by MENTS was shown to be misaligned with the standard objective of maximising expected return: for any fixed temperature there is an MDP in which MENTS

either fails to recommend the optimal action, or requires an exponential number of trials to do so (**Q1.1 - Entropy**). The BTS and DENTS algorithms were introduced to address this misalignment, and were empirically shown to outperform UCT, MENTS and PUCT on a range of gridworld environments and on the game of Go. The same principles were lifted to the multi-objective setting in Chapter 6 by combining DENTS with simplex maps to give SM-DENTS (**Q1.2 - Multi-Objective Exploration**).

Q2 - Stochastic Multi-Objective Domains. Chapter 5 considered the question of how to extend MOMCTS algorithms to stochastic environments. The consistent action selection problem was identified as a fundamental challenge in MOMCTS: actions taken within a single trial should be consistent with each other, otherwise the search policy optimises for no particular objective. Theorem 5.5.1 showed that locally optimal action selection yields globally optimal policies if and only if the utility function is linear, confirming that the assumption of linear utility is necessary for an MOMCTS algorithm to guarantee convergence in stochastic environments. CZT and CHMCTS were introduced as two approaches to MOMCTS in stochastic environments, and were empirically shown to outperform existing MOMCTS methods on a range of multi-objective environments. CHMCTS recovers prior MOMCTS algorithms as special cases of a common framework, generalising them to stochastic environments via convex hull backups.

Q3 - Scalability. Scalability was a recurring concern throughout this thesis. In the single-objective setting, the stochastic search policies of BTS and DENTS enable the use of the Alias Method (**Q3.1 - Complexity**), which significantly reduces the per-trial computational cost of MCTS. Empirically, this allowed BTS and DENTS to run substantially more trials per move than PUCT in the Go experiments. In the multi-objective setting, the simplex map was introduced in Chapter 6 as an adaptive data structure for maintaining value and entropy estimates as a function of the weight vector (**Q3.2 - Multi-Objective Scalability**). Simplex maps avoid the computational cost of convex hull backups in stochastic environments,

and were empirically shown to scale significantly better than CHMCTS on the stochastic Deep Sea Treasure environment.

7.2 Future Work

Several directions for future work remain.

Decaying Tsallis entropy. The DENTS algorithm of Chapter 4 uses Shannon entropy as an auxiliary exploration bonus, with a temperature schedule that decays the entropy term over time. The same idea could be applied to other entropy regularisers, such as the Tsallis entropy used by TENTS [27]. As TENTS outperformed MENTS on the gridworld experiments (Section 4.4.3.1), the resulting algorithm could inherit the sparsemax search policy of TENTS, whilst still recommending the optimal action under the standard objective of maximising expected return. The convergence analysis of DENTS in Chapter 4 should generalise to this setting with relatively minor modifications.

AlphaBTS and AlphaDENTS. The BTS and DENTS algorithms of Chapter 4 are tabular, and store a separate Bellman value and entropy estimate at each tree node. A natural extension is to combine BTS and DENTS with neural function approximators, in the style of AlphaZero [94]. This would allow BTS and DENTS to be applied to large-scale environments such as Atari and Go without a hand-crafted heuristic, and the entropy term in DENTS may help to address the exploration challenges that arise in self-improvement loops.

Heuristic biases in MOMCTS algorithms. The heuristic bias terms from Chapter 4 were not included in either of Chapters 5 and 6. Optimism under uncertainty is a well-established idea in reinforcement learning, and could be incorporated into the convex hull and simplex map algorithms to improve performance, particularly in larger environments.

Multi-objective AlphaZero. A natural combination of the contributions of this thesis is a multi-objective AlphaZero, in which a neural network is used to provide a prior policy and value function for an MOMCTS algorithm based on simplex maps or convex hulls, and the network is trained on the recommendations of the search. A challenge in this setting is that the policy network $\pi(a|s;\mathbf{w})$ is conditioned on the weight vector $\mathbf{w}$, and may degrade to the same policy for many weights $\mathbf{w}$ during training — a form of catastrophic forgetting. Ensuring that the policy is updated for many weight vectors $\mathbf{w}$ in the loss could help to address this.

Appendices

A

Additional Algorithm Details for Chapter 4

Contents

A.1 AR-BTS 195
A.2 AR-DENTS 197

This appendix gives the full equations for AR-BTS and AR-DENTS for reference. Each algorithm is specified minimally using the `THTS++` schema given in Section 2.4.2.

A.1 AR-BTS

Value Estimates and Parameters.

- $\bar{V}_{\text{AR-BTS}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{AR-BTS}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- $\alpha_{\text{AR-BTS}}(x) > 0$ - The search temperature function;
- $\epsilon_{\text{AR-BTS}} \in [0, \infty)$ - The uniform exploration parameter;

- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s,a)$ - The heuristic Q-value estimate for state s and action a;

Initialisation.

$$N(s) \leftarrow 0, \tag{A.1}$$

$$N(s,a) \leftarrow 0, \tag{A.2}$$

$$\bar{V}_{\text{AR-BTS}}(s) \leftarrow 0, \tag{A.3}$$

$$\bar{Q}_{\text{AR-BTS}}(s,a) \leftarrow 0. \tag{A.4}$$

Search Policy. Intermediate Q-values:

$$\bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) = \frac{N(s,a)\bar{Q}_{\text{AR-BTS}}(s,a) + N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{A.5}$$

$$\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a) = \begin{cases} \frac{\bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{A.6}$$

Search policy:

$$\pi_{\text{AR-BTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{AR-BTS}}))\rho_{\text{AR-BTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{AR-BTS}})}{|\mathcal{A}|}, \tag{A.7}$$

$$\rho_{\text{AR-BTS}}(a|s) \propto \exp\left(\frac{\bar{Q}_{\text{AR-BTS}}^{\text{norm}}(s,a)}{\alpha_{\text{AR-BTS}}(N(s))}\right), \tag{A.8}$$

$$\lambda(s, \epsilon_{\text{AR-BTS}}) = \min\left(1, \frac{\epsilon_{\text{AR-BTS}}}{\log(e + N(s))}\right). \tag{A.9}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{AR-BTS}}$:

$$\bar{Q}_{\text{AR-BTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \bar{V}_{\text{AR-BTS}}(s') \right), \tag{A.10}$$

$$\bar{V}_{\text{AR-BTS}}(s_t) \leftarrow \frac{N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}} \hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}} + \sum_{a \in \mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}} \bar{Q}_{\text{AR-BTS}}(s_t, a). \tag{A.11}$$

Recommendation Policy.

$$\psi_{\text{AR-BTS}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}_{\text{AR-BTS}}(s, a), \tag{A.12}$$

$$\texttt{mv}_{\text{AR-BTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{A.13}$$

A.2 AR-DENTS

Value Estimates and Parameters.

- $\bar{V}_{\text{AR-DNTS}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{AR-DNTS}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- $\alpha_{\text{AR-DNTS}}(x) > 0$ - The search temperature function;
- $\epsilon_{\text{AR-DNTS}} \in [0, \infty)$ - The uniform exploration parameter;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{AR-DNTS}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{AR-DNTS}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s, a)$ - The heuristic Q-value estimate for state s and action a;

Initialisation.

$$N(s) \leftarrow 0, \tag{A.14}$$

$$N(s,a) \leftarrow 0, \tag{A.15}$$

$$\bar{V}_{\text{AR-DNTS}}(s) \leftarrow 0, \tag{A.16}$$

$$\bar{Q}_{\text{AR-DNTS}}(s,a) \leftarrow 0. \tag{A.17}$$

Search Policy. Intermediate Q-values:

$$\hat{Q}^{\text{sft}}_{\text{AR-DNTS}}(s,a) = \bar{Q}_{\text{AR-DNTS}}(s,a) + \beta_{\text{AR-DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s,a), \tag{A.18}$$

$$\hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) = \frac{N(s,a)\hat{Q}^{\text{sft}}_{\text{AR-DNTS}}(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{AR-DNTS}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{AR-DNTS}}}, \tag{A.19}$$

$$\hat{Q}^{\text{sft,norm}}_{\text{AR-DNTS}}(s,a) = \begin{cases} \frac{\hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) & \text{otherwise.} \end{cases} \tag{A.20}$$

Search policy:

$$\pi_{\text{AR-DNTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{AR-DNTS}}))\rho_{\text{AR-DNTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{AR-DNTS}})}{|\mathcal{A}|}, \tag{A.21}$$

$$\rho_{\text{AR-DNTS}}(a|s) \propto \exp\left(\frac{\hat{Q}^{\text{sft,norm}}_{\text{AR-DNTS}}(s,a)}{\alpha_{\text{AR-DNTS}}(N(s))}\right), \tag{A.22}$$

$$\lambda(s, \epsilon_{\text{AR-DNTS}}) = \min\left(1, \frac{\epsilon_{\text{AR-DNTS}}}{\log(e + N(s))}\right). \tag{A.23}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{AR-DNTS}}$:

$$\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s_t, a_t) \leftarrow \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \bar{\mathcal{H}}_{V,\text{AR-DNTS}}(s'), \tag{A.24}$$

$$\begin{aligned} \bar{\mathcal{H}}_{V,\text{AR-DNTS}}(s_t) \leftarrow\ & \mathcal{H}(\pi_{\text{AR-DNTS}}(\cdot|s_t)) \\ & + \sum_{a\in\mathcal{A}} \pi_{\text{AR-DNTS}}(a|s_t)\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s_t, a), \end{aligned} \tag{A.25}$$

$$\bar{Q}_{\text{AR-DNTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \bar{V}_{\text{AR-DNTS}}(s')\right), \tag{A.26}$$

$$\begin{aligned} \bar{V}_{\text{AR-DNTS}}(s_t) \leftarrow\ & \frac{N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}} \hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}} \\ & + \sum_{a\in\mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}} \bar{Q}_{\text{AR-DNTS}}(s_t, a). \end{aligned} \tag{A.27}$$

Recommendation Policy.

$$\psi_{\text{AR-DNTS}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}_{\text{AR-DNTS}}(s, a), \tag{A.28}$$

$$\texttt{mv}_{\text{AR-DNTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{A.29}$$

B

Additional Experimental Details for Chapter 4

Contents

B.1 Additional DeterministicGridWorld, FrozenLake details 202
B.2 Parameter Details for Gridworld Experiments 202
B.2.1 Parameter Summary 202
B.2.2 Integration of Common Parameters into Prior Algorithms 205
B.2.3 Parameter Optimisation 207
B.3 Additional Details for Go Experiments 212
B.3.1 AR-MENTS, AR-RENTS and AR-TENTS 212
B.3.2 Go Round Robin Algorithm Parameter Selection 212
B.3.3 Average Returns vs Bellman Updates for Go 218

This appendix gives additional details for the experiments conducted in Chapter 4. It is structured as follows:

- Section B.1 gives additional details for the environments used in Section 4.4;
- Section B.2 gives details on the algorithm parameters used for the experiments in Section 4.4 and the optimisation process used to select them;
- Section B.3 gives additional details on the Go experiments, including algorithm parameter selection;

B.1 Additional DeterministicGridWorld, FrozenLake details

Figure B.1 shows the maps used in the DeterministicGridWorld and FrozenLake environments. The 4x4 and 8x8 maps are the standard maps from the Gymnasium library [101]. The remaining maps are randomly generated, where the starting position is always the top left corner, the goal position is always the bottom right corner and the remaining positions are randomly sampled with a $\frac{1}{5}$ probability of being a hole. Rejection sampling is used to ensure that there is always a path from the starting position to the goal.

B.2 Parameter Details for DeterministicGridWorld, FrozenLake and Sailing Experiments

This section is organised as follows:

- Section B.2.1 gives a list of shared and algorithm-specific parameters used for the experiments;
- Section B.2.2 describes how the shared parameters are integrated into prior algorithms, using UCT as an example;
- Section B.2.3 gives details on the parameter optimisation and the final algorithm parameters that were selected.

B.2.1 Parameter Summary

The following parameters were used in common by all algorithms considered in Section 4.4:

- `single_node_expansion` - a `THTS++` parameter that specifies whether the selection phase should end when a new decision node is added to the tree or not;

```
SFFF
FHFH
FFFH
HFFG
```
(a) 4x4.

```
SFFFHFFF
FHFFFFFF
FHFFHFFF
FFFFFFHG
```
(b) 4x8.

```
SFFHFFFFFFFH
FFFFFHFFFFFF
FFFFHFFFFFFF
FFFFFFFFFFHG
```
(c) 4x12.

```
SFFFFHFFFFFFFFF
HFFFFFFFFHFFHHFF
FFHHHFHFFFHHHFFF
FFHHFFFFFFFFHFFG
```
(d) 4x16.

```
SFFFFFFF
FFFFFFFF
FFFHFFFF
FFFFFHFF
FFFHFFFF
FHHFFFHF
FHFFHFHF
FFFHFFFG
```
(e) 8x8.

```
SFFFFFFFFFFFFFHF
FHHFFFFHFHHFFFHFF
FHFFHFFFFHFFFFFFF
FFFHFFFHFFFFFHHF
HFFHFHFFFFFFHHFF
FFFHFHFFHHFFFFFF
FFFFFFFFFFFFFFHF
FFFFFHFFHHFFHHFG
```
(f) 8x16.

```
SFFFFHFHFFFFFFHHFFFHHFFH
HFFHFFHFFFFFFFHFFFFFHFHF
FFFFFFHFFHHFFFFFFFFFHFFF
FFHHFFFFFFFFFFFFHFFHFFFH
FFFHFFFFFHFFHFFHFHFHFFFF
FFHFFFFFFFFFHFHFFFFFFFFF
HHFFFFFFFFFFFFFFHFFFHFFF
FFFFFFHFFFFFFFFFFHFFHHFG
```
(g) 8x24.

```
SFHFFFFHFFFFFFFFFHFHHFHFHFHHFHFF
SFHFFFFHFFFFFFFFFHFHHFHFHFHHFHFF
FFFFFFHFFFHHFFHFFFFFHFHFFFHFFFFFH
HFFFFFFFFFFFFFFFFFFHFFHHHFFFHFFFF
FFHHHFHFFHHFHFFFFHFFFFFFFFHFFHFF
FFFFHFFFFHFHHFHFFFFFHHFHFHFFFFFH
FHFHFFHHFFFHFFFHFFFFFFFFFFFFHFFH
FHFFFFHFHFFFFFHFFFFFFFFFFFFHFFFF
HFFFFHFFHHFFFHHFFFFFFFFFFFHFHFFG
```
(h) 8x32.

Figure B.1: Maps used for the DeterministicGridWorld and FrozenLake environments. `S` is the starting location for the agent, `F` represents floor that the agent can move too, `H` are holes that end the agents trial and `G` is the goal location.

- `transposition_table` - a `THTS++` parameter that specifies whether a transposition table should be used to store and re-use the decision nodes of the search tree when they are reached again via another path;
- `normalised_Q` - if Q-values should be normalised before being used in the search policy;
- `init_v` - the heuristic value function;
- `init_q` - the heuristic Q-value function;
- $N^{\hat{V}_{\text{heuristic}}}$ - the number of pseudo-visits assigned to `init_v` in backups;
- $N^{\hat{Q}_{\text{heuristic}}}$ - the number of pseudo-visits assigned to `init_q` to the Q-value estimate;

The following parameters are used by specific algorithms:

- ϵ - the (uniform) exploration parameter used by algorithms using a Boltzmann search policy, e.g. MENTS, BTS;
- α - the temperature parameter for algorithms using a maximum entropy objective, e.g. MENTS, RENTS, TENTS;
- $\alpha(x)$ - the temperature schedule for Boltzmann distributions in BTS and DENTS;
- $\beta(x)$ - the entropy weighting function for DENTS;
- b - the bias term used in UCB style policies, e.g. UCT, H-MCTS;
- `uct_budget` - the search budget for UCT between each round of Sequential Halving in H-MCTS;

To be compatible with parameter optimisation, the functional form of α and β need to be specified using tunable scalar parameters. Hence for the experiments in Section 4.4.3.1, the following forms were used:

$$\alpha(x) = \frac{\alpha_0}{\sqrt{1 + \alpha_{\text{decay}} x}} \tag{B.1}$$

$$\beta(x) = \begin{cases} \beta_c \left(1 - \frac{x}{\beta_{\text{thresh}}}\right) & \text{if } x \leq \beta_{\text{thresh}}, \\ 0 & \text{otherwise.} \end{cases} \tag{B.2}$$

α_0 specifies the initial temperature at $x = 0$, and α_{decay} specifies the rate of decay. β follows a linear schedule, starting at a value of β_c at $x = 0$ and reaches a value of zero at $x = \beta_{\text{thresh}}$.

These functions are chosen so that the entropy weighting becomes negligible, and more specifically so that $\frac{\beta(x)}{\alpha(x)}$ tends to zero as x tends to infinity.

B.2.2 Integration of Common Parameters into Prior Algorithms

Consider an algorithm with value estimates $\bar{V}(s)$ and $\bar{Q}(s, a)$. To use the $N^{\hat{Q}_{\text{heuristic}}}$ and `normalised_Q` parameters, the biased Q-values $\bar{Q}^{\text{norm}}(s, a)$ used in the search policy are given by:

$$\bar{Q}^{\text{bias}}(s, a) = \frac{N(s, a)\bar{Q}(s, a) + N^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s, a)}{N(s, a) + N^{\hat{Q}_{\text{heuristic}}}}, \tag{B.3}$$

$$\bar{Q}^{\text{norm}}(s, a) = \begin{cases} \frac{\bar{Q}^{\text{bias}}(s,a) - \min_{a \in \mathcal{A}} \bar{Q}^{\text{bias}}(s,a)}{\max_{a \in \mathcal{A}} \bar{Q}^{\text{bias}}(s,a) - \min_{a \in \mathcal{A}} \bar{Q}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}^{\text{bias}}(s, a) & \text{otherwise.} \end{cases} \tag{B.4}$$

All visit counts and value estimates are initialised to zero: $N(s) = 0$, $N(s, a) = 0$, $\bar{V}(s) = 0$ and $\bar{Q}(s, a) = 0$. While MCTS algorithms are often presented as initialising new leaf nodes directly with a visit and the heuristic value (Section 2.4.3), this biasing term can instead be incorporated into the backups, allowing the algorithm to be run with and without the `single_node_expansion` parameter.

Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$, the *full backups* for the

average returns $\bar{V}$ and $\bar{Q}$ including the biasing term for $t = h-1, \ldots, 0$ are given by:

$$\bar{Q}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s'|s_t, a_t)}{N(s_t, a_t)} \bar{V}(s') \right), \tag{B.5}$$

$$\bar{V}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}} \hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} + \sum_{a \in \mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} \bar{Q}(s_t, a). \tag{B.6}$$

In the main thesis the visit counts $N(s'|s, a)$ are briefly mentioned in Section 2.4.2, but are not used in the general presentation, which follows prior work. When using `transposition_table`, all backups need to be *full backups*, as discussed in Section 2.4.2 and argued by Wu [111].

B.2.2.1 UCT Implementation with Full Backups

As a worked example, the version of UCT used in experiments that utilises all of the parameters from Section B.2.1 is given. The algorithm is specified minimally using the `THTS++` schema given in Section 2.4.2.

Value Estimates and Parameters.

- $\bar{V}_{\text{UCT}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{UCT}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{UCT}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{UCT}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s, a)$ - The heuristic Q-value estimate for state s and action a;
- b_{UCT} - the bias parameter used in the UCT search policy;

Initialisation.

$$N(s) \leftarrow 0, \tag{B.7}$$

$$N(s,a) \leftarrow 0, \tag{B.8}$$

$$\bar{V}_{\mathrm{UCT}}(s) \leftarrow 0, \tag{B.9}$$

$$\bar{Q}_{\mathrm{UCT}}(s,a) \leftarrow 0. \tag{B.10}$$

Search Policy. Intermediate Q-values:

$$\bar{Q}^{\mathrm{bias}}_{\mathrm{UCT}}(s,a) = \frac{N(s,a)\bar{Q}_{\mathrm{UCT}}(s,a) + N_{\mathrm{UCT}}^{\hat{Q}_{\mathrm{heuristic}}}\hat{Q}_{\mathrm{heuristic}}(s,a)}{N(s,a) + N_{\mathrm{UCT}}^{\hat{Q}_{\mathrm{heuristic}}}}, \tag{B.11}$$

$$\bar{Q}^{\mathrm{norm}}_{\mathrm{UCT}}(s,a) = \begin{cases} \frac{\bar{Q}^{\mathrm{bias}}_{\mathrm{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\mathrm{bias}}_{\mathrm{UCT}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}^{\mathrm{bias}}_{\mathrm{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\mathrm{bias}}_{\mathrm{UCT}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}^{\mathrm{bias}}_{\mathrm{UCT}}(s,a) & \text{otherwise.} \end{cases} \tag{B.12}$$

Search policy:

$$\pi_{\mathrm{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}^{\mathrm{norm}}_{\mathrm{UCT}}(s,a) + b_{\mathrm{UCT}}\sqrt{\frac{\log(N(s))}{N(s,a)}}, \tag{B.13}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\mathrm{UCT}}$:

$$\bar{Q}_{\mathrm{UCT}}(s_t,a_t) \leftarrow R(s_t,a_t) + \sum_{s'\in \mathrm{Succ}(s_t,a_t)} \left(\frac{N(s'|s_t,a_t)}{N(s_t,a_t)} \bar{V}_{\mathrm{UCT}}(s') \right), \tag{B.14}$$

$$\bar{V}_{\mathrm{UCT}}(s_t) \leftarrow \frac{N^{\hat{V}_{\mathrm{heuristic}}}\hat{V}_{\mathrm{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\mathrm{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t,a)}{N(s_t) + N^{\hat{V}_{\mathrm{heuristic}}}} \bar{Q}_{\mathrm{UCT}}(s_t,a). \tag{B.15}$$

Recommendation Policy.

$$\psi_{\mathrm{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}_{\mathrm{UCT}}(s,a), \tag{B.16}$$

$$\mathtt{mv}_{\mathrm{UCT}}(s) = \arg\max_{a\in\mathcal{A}} N(s,a). \tag{B.17}$$

B.2.3 Parameter Optimisation

The parameters shared between all algorithms were set manually to the following values for all environments:

- `single_node_expansion` = False - the selection phase continues until the planning horizon or a terminal state is reached;

- `transposition_table` = True - transposition tables are used to store and re-use decision nodes when they are reached again via another path;
- `normalised_Q` = True - Q-values are linearly normalised before being used in the search policy;
- $N^{\hat{V}_{\text{heuristic}}} = 0$ - the number of pseudo-visits assigned to `init_v` in backups;
- $N^{\hat{Q}_{\text{heuristic}}} = 1$ - the number of pseudo-visits assigned to `init_q` to the Q-value estimate.

Using the principle of optimism in the face of uncertainty, the heuristic value functions were set to a trivial upper bound on the values that could be achieved in each environment. For the DeterministicGridWorld and FrozenLake environments, this meant the value functions were set to $\hat{V}_{\text{heuristic}}(s) = \hat{Q}_{\text{heuristic}}(s, a) = 1.0$ for all states s and actions a. For the Sailing environments, because all rewards are negative, the value functions were set to $\hat{V}_{\text{heuristic}}(s) = \hat{Q}_{\text{heuristic}}(s, a) = 0.0$ for all states s and actions a.

The remaining algorithm parameters were selected using Bayesian optimisation via the BayesOpt library [61]. BayesOpt was chosen as an optimisation package because it provides a C++ interface and integrates easily with the C++ implementation of `THTS++` used in this thesis.

Table B.1 gives the space searched over for each parameter, and if parameters were sampled using a linear or logarithmic scale.

Parameters were optimised using the 8x8 map for DeterministicGridWorld, the 4x4 map for FrozenLake (Figure B.1) and the 8x8 map for Sailing with both wind directions.

To evaluate the performance of a given set of parameter values, the algorithm was run for 50,000 trials, and 1024 trials used to evaluate the recommendation policy. At least 10 runs were performed for each set of parameters, and runs were repeated until the standard deviation across the runs was less than 0.025 for DeterministicGridWorld and FrozenLake, and less than 1.0 for Sailing.

Parameter	Minimum	Maximum	Scaling
ϵ	10^{-6}	1	Logarithmic
α	10^{-5}	10^{2}	Logarithmic
α_0	10^{-5}	10^{2}	Logarithmic
α_{decay}	10^{-5}	10^{2}	Logarithmic
β_c	10^{-5}	10^{2}	Logarithmic
β_{thresh}	10^{3}	10^{12}	Logarithmic
b	10^{-5}	10^{2}	Logarithmic
`uct_budget`	1	10^{5}	Linear

Table B.1: The parameter search space for gridworld experiments (Section 4.4.3.1). Recall that α is the temperature used for maximum entropy algorithms (e.g. MENTS), and α_0, α_{decay} β_c and β_{thresh} are parameters for $\alpha(x)$ and $\beta(x)$ used in BTS and DENTS. The search space for α_{decay} and β_{thresh} are selected such that the function spaces for $\alpha(x)$ and $\beta(x)$ include functions that are nearly constant for the duration of the search.

The selected algorithm parameters for DeterministicGridWorld, FrozenLake, Sailing(North) and Sailing(South-East) are given in Tables B.2, B.3, B.4 and B.5 respectively.

The parameter optimisation does make a significant difference in the performance of the algorithm. For example consider Figures 4.15 and 4.16 where the bias and temperature parameters are varied and have a significant effect on performance. It is additionally worth noting that because all parameters were tuned simultaneously that parameters that have relatively little effect on performance were set to somewhat arbitrary values. For example, the value of α_{decay} for BTS in Table B.2b has little effect on the performance of BTS, because the value of α_0 is set to a small value 1.00×10^{-5}, which already causes the Boltzmann term in the search policy to be nearly greedy (Equation (4.5)).

Algorithm	b	`uct_budget`
UCT	7.00	
MaxUCT	6.86	
H-MCTS	1.50	9.95×10^{4}

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	1.00	6.81×10^{-4}				
RENTS	1.00	1.34×10^{-4}				
TENTS	1.00	9.20×10^{-5}				
BTS	1.00		1.00×10^{-5}	3.16		
DENTS	0.744		4.04×10^{-3}	0.977	3.36×10^{-4}	10^{12}

(b) Boltzmann-based search algorithms.

Table B.2: Algorithm parameters used for DeterministicGridWorld experiments. Empty cells indicate the parameter is not used by that algorithm.

Algorithm	b	`uct_budget`
UCT	7.07	
MaxUCT	2.58	
H-MCTS	6.81	28

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	0.248	1.02×10^{-2}				
RENTS	0.432	2.89×10^{-3}				
TENTS	0.288	1.01×10^{-2}				
BTS	0.460		0.134	4.82×10^{-2}		
DENTS	0.444		0.264	0.200	9.17×10^{-4}	1.38×10^{4}

(b) Boltzmann-based search algorithms.

Table B.3: Algorithm parameters used for FrozenLake experiments. Empty cells indicate the parameter is not used by that algorithm.

Algorithm	b	`uct_budget`
UCT	9.48×10^{-2}	
MaxUCT	9.05×10^{-2}	
H-MCTS	8.10×10^{-2}	32

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	1.00	5.40×10^{-4}				
RENTS	0.319	1.57×10^{-4}				
TENTS	1.00	7.61×10^{-4}				
BTS	1.00		2.32×10^{-3}	1.16×10^{-3}		
DENTS	0.971		1.02×10^{-5}	0.813	1.14×10^{-3}	2.18×10^{4}

(b) Boltzmann-based search algorithms.

Table B.4: Algorithm parameters used for Sailing(North) experiments. Empty cells indicate the parameter is not used by that algorithm.

Algorithm	b	`uct_budget`
UCT	0.181	
MaxUCT	0.226	
H-MCTS	0.145	100

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	1.00	6.05×10^{-4}				
RENTS	1.00	1.00×10^{-5}				
TENTS	1.00	3.24×10^{-3}				
BTS	1.00		2.32×10^{-3}	1.17×10^{-3}		
DENTS	1.00		1.01×10^{-5}	9.91×10^{2}	1.01×10^{-5}	3.91×10^{4}

(b) Boltzmann-based search algorithms.

Table B.5: Algorithm parameters used for Sailing(South-East) experiments. Empty cells indicate the parameter is not used by that algorithm.

B.3 Additional Details for Go Experiments

These experiments were run with `single_node_expansion` turned on.

Because running Bayesian optimisation at the scale of the Go experiments would be prohibitively expensive, a round-robin tournament was used to select the algorithm parameters. This section gives the results of the round-robin tournaments.

For PUCT, the parameters used were copied from the reported values for KataGo and AlphaGo Zero [111, 93].

This section is organised as follows:

- Section B.3.1 outlines the implementation of the AR-MENTS, AR-RENTS and AR-TENTS algorithms;
- Section B.3.2 gives details on the round-robin tournament used to select the algorithm parameters for Go experiments;
- Section B.3.3 gives results for matches between the Bellman and average return versions of each algorithm.

B.3.1 AR-MENTS, AR-RENTS and AR-TENTS

To implement AR-MENTS the AR-DENTS (Section A) algorithm was used, but with $\beta(x) = \alpha(x) = \alpha_{\text{fix}}$. Implementing AR-RENTS and AR-TENTS is similar, replacing the use of Shannon entropy $\mathcal{H}$ in the backups by the corresponding relative entropy and Tsallis entropy functions. For AR-TENTS, the search policy uses the *sparsemax* function used in TENTS instead of the softmax function.

B.3.2 Go Round Robin Algorithm Parameter Selection

The following parameters were tuned for the BTS, DENTS, MENTS, RENTS and TENTS algorithms:

- ϵ - the exploration parameter used by algorithms using a Boltzmann search policy, e.g. MENTS, BTS;

Black \White	3	1	0.3	0.1	0.03
3	-	7-8	9-6	12-3	12-3
1	15-0	-	12-3	15-0	15-0
0.3	13-2	11-4	-	14-1	15-0
0.1	14-1	10-5	11-4	-	15-0
0.03	13-2	5-10	8-7	13-2	-

Table B.6: Round robin parameter selection for α_{init} for BTS. The value of 1.0 won all four of its matches so was selected.

- $\epsilon_{\tilde{\lambda}}$ - the weighting of the prior policy used by algorithms using a Boltzmann search policy, e.g. BTS;
- α - the temperature parameter for algorithms using a maximum entropy objective, e.g. MENTS, RENTS, TENTS;
- α_{init} - the initial temperature parameter for BTS and DENTS algorithms;
- β_{init} - the initial entropy coefficient for DENTS algorithms.

The temperature and entropy coefficient functions used in BTS and DENTS for Go experiments were of the form $\alpha(x) = \alpha_{\text{init}}/\sqrt{x}$ and $\beta(x) = \beta_{\text{init}}/\sqrt{x}$ respectively. For DENTS, only the entropy coefficient was tuned independently and used the parameters tuned for BTS.

Parameters are tuned one at a time, with a round-robin tournament used to select the best value for each parameter, keeping the remaining parameters fixed. The parameter that won the most matches is selected, with ties broken by the agent that won the most games. If the winning agent had the largest or smallest value, then another tournament is run adjusting the values accordingly.

These tuning tournaments were run using the game of 9x9 Go with a komi of 6.5, and giving each algorithm 2.5 seconds per move.

Initially the values of ϵ and $\epsilon_{\tilde{\lambda}}$ were set to 0.03 and 1.0 respectively and the search temperatures were tuned first. Tables B.6 and B.7 give results of tuning the initial search temperature α_{init} for BTS and AR-BTS respectively.

Black \White	3	1	0.3	0.1	0.03
3	-	3-12	4-11	3-12	1-14
1	14-1	-	8-7	9-6	7-8
0.3	14-1	8-7	-	7-8	11-4
0.1	15-0	11-4	9-6	-	9-6
0.03	14-1	8-7	4-11	8-7	-

Table B.7: Round robin parameter selection for α_{init} for BTS. The value of 0.1 won all four of its matches so was selected.

Black \White	10	5	3	2	1
10	-	5-10	3-12	1-14	3-12
5	14-1	-	12-3	11-4	11-4
3	15-0	15-0	-	12-3	12-3
2	15-0	12-3	12-3	-	10-5
1	15-0	14-1	12-3	9-6	-

Table B.8: Round robin parameter selection for $\epsilon_{\tilde{\lambda}}$ for BTS. The value of 2.0 won the most matches so was selected.

$\epsilon_{\tilde{\lambda}}$ was tuned next, using the selected values of α_{init} for BTS and AR-BTS. Tables B.8 and B.9 give results for the round-robin tournament to select the prior policy weight for BTS and AR-BTS.

Next the exploration parameter ϵ was tuned, using the selected values of α_{init} and $\epsilon_{\tilde{\lambda}}$ for BTS and AR-BTS. Tables B.10 and B.11 give results for the round-robin tournament to select the exploration parameter for BTS and AR-BTS. Although the lowest value of 0.001 was selected for AR-BTS, it is close enough to zero such that very few actions are randomly sampled. A round-robin with values between 10^{-3} and 10^{-5} was tried, but all matches resulted in draws.

To tune the MENTS, RENTS and TENTS algorithms, the values of ϵ, $\epsilon_{\tilde{\lambda}}$ selected for BTS were used, and the value of α varied. Similarly, to tune the AR-MENTS,

Black \White	3	2	1	0.75	0.5
3	-	14-1	9-6	13-2	14-1
2	12-3	-	12-3	15-0	10-5
1	12-3	13-2	-	11-4	13-2
0.75	11-4	10-5	12-3	-	14-1
0.5	11-4	13-2	9-6	7-8	-

Table B.9: Round robin parameter selection for $\epsilon_{\tilde{\lambda}}$ for AR-BTS. The value of 1.0 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	12-3	10-5	8-7	7-8
0.03	9-6	-	8-7	13-2	12-3
0.01	11-4	9-6	-	7-8	12-3
0.003	13-2	9-6	11-4	-	11-4
0.001	12-3	11-4	8-7	9-6	-

Table B.10: Round robin parameter selection for ϵ for BTS. The value of 0.003 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	12-3	14-1	12-3	13-2
0.03	15-0	-	11-4	14-1	14-1
0.01	15-0	11-4	-	13-2	13-2
0.003	15-0	14-0	14-1	-	13-2
0.001	14-1	14-1	14-1	15-0	-

Table B.11: Round robin parameter selection for ϵ for AR-BTS. The value of 0.001 won the most matches so was selected.

AR-RENTS and AR-TENTS algorithms, the values of ϵ, $\epsilon_{\tilde{\lambda}}$ selected for AR-BTS were used, and the value of α varied. Results of the round-robin tournaments to select the temperature parameter α are given in the following tables:

- MENTS - Table B.12;
- AR-MENTS - Table B.13;
- RENTS - Table B.14;
- AR-RENTS - Table B.15;
- TENTS - Table B.16;
- AR-TENTS - Table B.17.

Finally, for (AR-)DENTS, all parameters were copied from (AR-)BTS, and the entropy temperature β was tuned. Tables B.18 and B.19 give results for the round-robin tournament to select the initial entropy temperature β_{init} for DENTS and AR-DENTS respectively.

Black \White	3	1	0.3	0.1	0.03
3	-	9-6	9-6	11-4	10-5
1	11-4	-	11-4	9-6	10-5
0.3	9-6	7-8	-	11-4	6-9
0.1	4-11	4-11	0-15	-	4-11
0.03	2-13	2-13	0-15	0-15	-

Table B.12: Round robin parameter selection for α for MENTS. The value of 1.0 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	2-13	1-14	3-12	5-10
1	14-1	-	11-4	13-2	15-0
0.3	15-0	12-3	-	12-3	14-1
0.1	15-0	9-6	9-6	-	15-0
0.03	15-0	8-7	9-6	14-1	-

Table B.13: Round robin parameter selection for α for AR-MENTS. The value of 0.3 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	7-8	9-6	10-5	9-6
1	13-2	-	12-3	11-4	12-3
0.3	10-5	11-4	-	10-5	5-10
0.1	3-12	2-13	1-14	-	3-12
0.03	3-12	0-15	0-15	0-15	-

Table B.14: Round robin parameter selection for α for RENTS. The value of 1.0 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	4-11	2-13	9-6	3-12
1	15-0	-	12-3	13-2	13-2
0.3	15-0	13-2	-	14-1	15-0
0.1	15-0	10-5	13-2	-	15-0
0.03	13-2	13-2	12-3	14-1	-

Table B.15: Round robin parameter selection for α for AR-RENTS. The value of 0.3 won the most matches so was selected.

Black \White	300	100	30	10	3
300	-	3-12	5-10	7-8	9-6
100	6-9	-	9-6	12-3	12-3
30	8-7	7-8	-	9-6	11-4
10	0-15	2-13	2-13	-	6-9
3	1-14	1-14	2-13	5-10	-

Table B.16: Round robin parameter selection for α for TENTS. The value of 30.0 won the most matches so was selected.

Black \White	30	10	3	1	0.3
30	-	14-1	15-0	14-1	14-1
10	15-0	-	13-2	15-0	15-0
3	15-0	14-1	-	14-1	15-0
1	15-0	15-0	14-1	-	15-0
0.3	15-0	15-0	14-1	15-0	-

Table B.17: Round robin parameter selection for α for AR-TENTS. The value of 3.0 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	10-5	12-3	8-7	11-4
0.03	13-2	-	11-4	13-2	10-5
0.01	12-3	11-4	-	10-5	11-4
0.003	7-8	13-2	13-2	-	10-5
0.001	13-2	13-2	11-4	11-4	-

Table B.18: Round robin parameter selection for β_{init} for DENTS. The value of 0.3 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	11-4	12-3	12-3	14-1
1	13-2	-	11-4	13-2	13-2
0.3	14-1	13-2	-	13-2	14-1
0.1	12-3	12-3	12-3	-	11-4
0.03	12-3	13-2	13-2	14-1	-

Table B.19: Round robin parameter selection for β_{init} for AR-DENTS. The value of 0.3 won the most matches so was selected.

Black \White	BTS	AR-BTS
BTS	-	16-9
AR-BTS	22-3	-
Black \White	DENTS	AR-DENTS
DENTS	-	21-4
AR-DENTS	22-3	-
Black \White	MENTS	AR-MENTS
MENTS	-	0-25
AR-MENTS	25-0	-
Black \White	RENTS	AR-RENTS
RENTS	-	2-23
AR-RENTS	24-1	-
Black \White	TENTS	AR-TENTS
TENTS	-	8-17
AR-TENTS	23-2	-

Table B.20: Results for matches between the average return and Bellman versions of BTS, DENTS, MENTS, RENTS and TENTS.

B.3.3 Average Returns vs Bellman Updates for Go

After tuning all of the algorithms, each algorithm using Bellman updates played a match against its average returns counterpart, with results given in Table B.20. Across all matches, the average returns algorithms won each match by a significant margin.

C

Additional Experimental Details for Chapters 5 and 6

Contents

C.1 Parameter Summary **219**
C.2 Parameter Optimisation **222**

This appendix gives additional details on the algorithm parameters used for the experiments in Sections 5.4 and 6.4, and the optimisation process used to select them. The appendix is organised as follows:

- Section C.1 gives a list of shared and algorithm-specific parameters used for the experiments;
- Section C.2 gives details on the parameter optimisation and the final algorithm parameters that were selected.

C.1 Parameter Summary

In the experiments in Sections 5.4 and 6.4, when an algorithm used a temperature schedule $\alpha(x)$ or an entropy weighting function $\beta(x)$, the parametric forms of

Equations (B.1) and (B.2) were used, with parameters α_0, α_{decay}, β_c and β_{thresh}. This keeps all algorithms compatible with the parameter optimisation performed in Section C.2.

The following parameters were used by all algorithms considered in Sections 5.4 and 6.4:

- `single_node_expansion` - a `THTS++` parameter that specifies whether the selection phase should end when a new decision node is added to the tree;
- `transposition_table` - a `THTS++` parameter that specifies whether a transposition table should be used to store and re-use the decision nodes of the search tree when they are reached again via another path;
- `init_v` - the heuristic value function;
- `init_q` - the heuristic Q-value function;
- $\mathbf{r}_{\text{hv}}$ - the reference point $\mathbf{r}$ for computing the hypervolume metric (Definition 2.6.14).

The following parameters were used by all CHMCTS algorithms, and relate to the optimisations of Section 5.3.6:

- $K_{\max}$ - the maximum size of a convex hull value set, used in the size pruning optimisation;
- $\texttt{unsolved}_{\text{tol}}$ - the tolerance for marking a chance node as solved, which happens when $\texttt{unsolved}(s, a) \leq \texttt{unsolved}_{\text{tol}}$;
- δ_{unsolved} - the confidence parameter used when computing `unsolved` values from confidence bounds; smaller values give more pessimistic estimates.

The following parameters were used by both SM-BTS and SM-DENTS from Section 6.3, and control the simplex map operations outlined in Section 6.2.1:

- $\texttt{push_radius}_{\text{SM}}$ - the radius used in the `push` operation;

- `split_min_radius`$_{\text{SM}}$ - if a simplex has a smaller radius than `split_min_radius`$_{\text{SM}}$, then it is never further subdivided;
- `split_min_delta`$_{\text{SM}}$ - if value estimates at all vertices of a simplex are within `split_min_delta`$_{\text{SM}}$ of each other, then it is not subdivided by `maybe_split` operations;
- `split_min_visit`$_{\text{SM}}$ - the number of times a simplex map needs to be "visited", or updated, before it may be further subdivided.

The following parameters were each used by a single algorithm:

- b_{CZT} - the bias term used in CZT;
- $(M_{\text{min}})_{\text{CZT}}$ - the minimum number of visits to a state-action pair before it is considered for selection in CZT;
- $b_{\text{CH-UCT}}$ - the bias term used in CH-UCT;
- $\alpha_{\text{CH-BTS0}}$ - the initial temperature for CH-BTS;
- $\alpha_{\text{CH-BTSdecay}}$ - the decay rate for CH-BTS;
- $\epsilon_{\text{CH-BTS}}$ - the exploration parameter for CH-BTS;
- $b_{\text{CH-CZT}}$ - the bias term used in CH-CZT;
- $(M_{\text{min}})_{\text{CH-CZT}}$ - the minimum number of visits to a state-action pair before it is considered for selection in CH-CZT;
- $b_{\text{H-CHMCTS}}$ - the bias term used in H-CHMCTS;
- $\mathbf{r}_{\text{ref}}$ - the reference point used to compute hypervolumes in H-CHMCTS;
- $b_{\text{C-CHMCTS}}$ - the bias term used in C-CHMCTS;
- $\mathbf{r}_{\text{uto}}$ - the utopian point used for the Chebyshev utility in C-CHMCTS;
- $b_{\text{P-CHMCTS}}$ - the bias term used in P-CHMCTS;

- $\alpha_{\text{SM-BTS},0}$ - the initial temperature for SM-BTS;
- $\alpha_{\text{SM-BTS,decay}}$ - the decay rate for SM-BTS;
- $\epsilon_{\text{SM-BTS}}$ - the exploration parameter for SM-BTS;
- $\alpha_{\text{SM-DENTS},0}$ - the initial temperature for SM-DENTS;
- $\alpha_{\text{SM-DENTS,decay}}$ - the decay rate for SM-DENTS;
- $\beta_{\text{SM-DENTS},c}$ - the constant term for SM-DENTS;
- $\beta_{\text{SM-DENTS,thresh}}$ - the threshold for SM-DENTS;
- $\epsilon_{\text{SM-DENTS}}$ - the exploration parameter for SM-DENTS.

C.2 Parameter Optimisation

When an upper or lower bound was necessary, the trivial upper and lower bounds, $\mathbf{V}^{\max}$ and $\mathbf{V}^{\min}$, were used. These are computed by assuming that the maximum or minimum reward for each objective was received at each timestep:

$$V_i^{\max} = H \cdot \max_{s \in \mathcal{S}, a \in \mathcal{A}} R(s,a)_i, \tag{C.1}$$

$$V_i^{\min} = H \cdot \min_{s \in \mathcal{S}, a \in \mathcal{A}} R(s,a)_i. \tag{C.2}$$

In particular, when a hypervolume was computed, the lower bound was used as the reference point, and the upper bound was used as the utopian point for the Chebyshev utility:

$$\mathbf{r}_{\text{hv}} \leftarrow \mathbf{V}^{\min}, \tag{C.3}$$

$$\mathbf{r}_{\text{ref}} \leftarrow \mathbf{V}^{\min}, \tag{C.4}$$

$$\mathbf{r}_{\text{uto}} \leftarrow \mathbf{V}^{\max}. \tag{C.5}$$

As in Appendix B.2.3, the heuristic value functions were set to a trivial upper bound on the values that could be achieved in each environment. That is, for

each environment, the heuristic value functions were set to $\hat{\mathbf{V}}_{\text{heuristic}}(s) \leftarrow \mathbf{V}^{\max}$ and $\hat{\mathbf{Q}}_{\text{heuristic}}(s,a) \leftarrow \mathbf{V}^{\max}$.

The `THTS++` parameters shared by all algorithms were set to:

- `single_node_expansion` = False - the selection phase continues until the planning horizon or a terminal state is reached;
- `transposition_table` = True - transposition tables are used to store and re-use decision nodes when they are reached again via another path.

The parameters related to the optimisations of Section 5.3.6, used by all CHMCTS algorithms, were set to:

- $K_{\max} = 15$,
- $\texttt{unsolved}_{\text{tol}} = 0.1$,
- $\delta_{\text{unsolved}} = 0.05$.

The value of $K_{\max}$ was chosen by hand to be small enough to avoid the rapid growth in the cost of convex hull operations discussed in Section 5.3.6, while still retaining enough points for the value sets to be a useful approximation.

The values of $\texttt{unsolved}_{\text{tol}}$ and δ_{unsolved} were also chosen by hand. Small values of both parameters give more confidence that a node marked as solved has computed an optimal policy. However, the confidence terms in Equation (5.39) shrink at a rate of $O(1/\sqrt{N(s,a)})$, so values that are too small prevent any node from being labelled as solved within the compute budget. The values chosen gave a reasonable trade-off. For example, the missing mass confidence term alone falls below the tolerance, $\sqrt{\log(2/\delta_{\text{unsolved}})/N(s,a)} \leq \texttt{unsolved}_{\text{tol}}$, once $N(s,a) \geq \log(40)/0.01 \approx 369$, so solved labelling can only take effect in parts of the tree that have been visited at least a few hundred times.

The parameters for simplex maps, used by SM-BTS and SM-DENTS in Chapter 6, were set to the following:

- $\texttt{push_radius}_{\text{SM}} = 10$,

Parameter	Minimum	Maximum	Scaling
b_{CZT}	10^{-5}	10^{5}	Logarithmic
$(M_{\text{min}})_{\text{CZT}}$	1	1024	Linear
$b_{\text{CH-UCT}}$	10^{-5}	10^{5}	Logarithmic
$\alpha_{\text{CH-BTS0}}$	10^{-5}	10^{2}	Logarithmic
$\alpha_{\text{CH-BTSdecay}}$	10^{-3}	10^{2}	Logarithmic
$\epsilon_{\text{CH-BTS}}$	10^{-6}	1	Logarithmic
$b_{\text{CH-CZT}}$	10^{-5}	10^{5}	Logarithmic
$(M_{\text{min}})_{\text{CH-CZT}}$	1	1024	Linear
$b_{\text{H-CHMCTS}}$	10^{-5}	10^{5}	Logarithmic
$b_{\text{C-CHMCTS}}$	10^{-5}	10^{5}	Logarithmic
$b_{\text{P-CHMCTS}}$	10^{-5}	10^{5}	Logarithmic
$\alpha_{\text{SM-BTS,0}}$	10^{-5}	10^{2}	Logarithmic
$\alpha_{\text{SM-BTS,decay}}$	10^{-3}	10^{2}	Logarithmic
$\epsilon_{\text{SM-BTS}}$	10^{-6}	1	Logarithmic
$\alpha_{\text{SM-DENTS,0}}$	10^{-5}	10^{2}	Logarithmic
$\alpha_{\text{SM-DENTS,decay}}$	10^{-3}	10^{2}	Logarithmic
$\beta_{\text{SM-DENTS},c}$	10^{-6}	1	Logarithmic
$\beta_{\text{SM-DENTS,thresh}}$	10^{-5}	10^{2}	Logarithmic
$\epsilon_{\text{SM-DENTS}}$	10^{3}	10^{12}	Logarithmic

Table C.1: The parameter search space used in the MO-Gymnasium and Deep Sea Treasure experiments (Sections 5.4.3 and 6.4.2). Section C.1 lists each of the parameters optimised and how they are used in their respective algorithms.

- $\texttt{split_min_radius}_{\text{SM}} = 0.01$,
- $\texttt{split_min_delta}_{\text{SM}} = 0$,
- $\texttt{split_min_visit}_{\text{SM}} = 10$.

These values were also chosen by hand, as a reasonable trade-off between allowing nearby weight vectors to share value estimates and avoiding the triangulation becoming too fine-grained too quickly. As they already gave reasonable performance in preliminary runs, they were not included in the parameter optimisation below, which kept the size of the search space down.

As in Appendix B.2.3, the remaining algorithm parameters were optimised using the BayesOpt library. Table C.1 gives the search space for each parameter, and whether the parameter was sampled on a linear or logarithmic scale.

Table C.2 provides the selected parameters for the MO-Gymnasium environments (Figures 5.4 and 6.7), and Table C.3 provides the selected parameters for the Deep

Parameter	FruitTree	ResourceGathering	BreakableBottles	FourRoom
b_{CZT}	1.00×10^{3}	2.78	1.94	100
$(M_{\text{min}})_{\text{CZT}}$	123	11	10	65
$b_{\text{CH-UCT}}$	1.00×10^{3}	0.315	1.00	8.03×10^{-2}
$\alpha_{\text{CH-BTS0}}$	100	4.30×10^{-3}	1.75×10^{-4}	0.186
$\alpha_{\text{CH-BTSdecay}}$	100	12.0	9.82×10^{-3}	1.00×10^{-3}
$\epsilon_{\text{CH-BTS}}$	1.00	3.86×10^{-2}	1.00	0.150
$b_{\text{CH-CZT}}$	995	1.00×10^{3}	512	1.00×10^{3}
$(M_{\text{min}})_{\text{CH-CZT}}$	123	10	10	32
$b_{\text{H-CHMCTS}}$	1.00×10^{3}	22.4	1.00×10^{3}	1.00×10^{3}
$b_{\text{C-CHMCTS}}$	1.00×10^{3}	263	0.617	8.90
$b_{\text{P-CHMCTS}}$	1.00×10^{3}	1.00×10^{3}	1.00	1.00×10^{3}
$\alpha_{\text{SM-BTS,0}}$	0.893	1.55×10^{-3}	1.66×10^{-2}	8.30×10^{-2}
$\alpha_{\text{SM-BTS,decay}}$	3.20×10^{-2}	8.35	14.1	1.00×10^{-3}
$\epsilon_{\text{SM-BTS}}$	0.995	1.00×10^{-2}	0.500	7.44×10^{-2}
$\alpha_{\text{SM-DENTS,0}}$	1.00×10^{-5}	9.21×10^{-3}	1.67×10^{-2}	8.29×10^{-2}
$\alpha_{\text{SM-DENTS,decay}}$	2.91×10^{-3}	16.4	13.9	1.00×10^{-3}
$\beta_{\text{SM-DENTS},c}$	15.7	1.69×10^{-3}	5.00×10^{-2}	1.00×10^{-2}
$\beta_{\text{SM-DENTS,thresh}}$	1.00×10^{3}	3.74×10^{3}	5.00×10^{3}	1.00×10^{3}
$\epsilon_{\text{SM-DENTS}}$	1.00	1.00×10^{-2}	0.500	7.40×10^{-2}

Table C.2: Algorithm parameters for each algorithm on the MO-Gymnasium environments.

Sea Treasure environments (Figures 5.5 and 6.8). In the scaling experiments from Sections 5.4.4 and 6.4.3, each algorithm still used the same parameters selected in Table C.3 to avoid re-running optimisation for every environment size.

Parameter	$\texttt{DST}(W, 0)$	$\texttt{DST}(W, 1/40)$	$\texttt{DST}(W, 1/5)$
b_{CZT}	8.32	1.64	2.04
$M_{\min}(\text{CZT})$	43	55	150
$b_{\text{CH-UCT}}$	3.46	4.21	4.21
$\alpha_{\text{CH-BTS0}}$	8.99×10^{-4}	3.31×10^{-4}	1.00×10^{-5}
$\alpha_{\text{CH-BTSdecay}}$	12.6	99.9	4.51
$\epsilon_{\text{CH-BTS}}$	0.999	0.113	0.990
$b_{\text{CH-CZT}}$	99.6	201	5.81
$M_{\min}(\text{CH-CZT})$	95	222	40
$b_{\text{H-CHMCTS}}$	254	20.4	16.0
$b_{\text{C-CHMCTS}}$	1.23×10^{3}	1.58×10^{3}	1.09
$b_{\text{P-CHMCTS}}$	388	20.1	0.365
$\alpha_{\text{SM-BTS,0}}$	1.31×10^{-3}	1.86×10^{-3}	1.00×10^{-5}
$\alpha_{\text{SM-BTS,decay}}$	1.00×10^{-3}	99.9	1.00×10^{-3}
$\epsilon_{\text{SM-BTS}}$	0.693	0.138	0.859
$\alpha_{\text{SM-DENTS,0}}$	1.18×10^{-3}	1.01×10^{-5}	3.88×10^{-4}
$\alpha_{\text{SM-DENTS,decay}}$	1.60×10^{-3}	2.67×10^{-2}	9.40×10^{-2}
$\beta_{\text{SM-DENTS},c}$	0.233	0.425	1.83×10^{-2}
$\beta_{\text{SM-DENTS,thresh}}$	3.20×10^{3}	9.41×10^{3}	1.60×10^{3}
$\epsilon_{\text{SM-DENTS}}$	0.753	0.189	0.374

Table C.3: Algorithm parameters for each algorithm on the DST environments.

D

Additional Theoretical Results for Chapter 4

Contents

D.1 The DENTS Stochastic Process 227
D.1.1 The BTS process. 229
D.2 Exponential Convergence Results 229
D.2.1 Lemmas for Exponential Convergence Theorems 230
D.2.2 Inductive Steps for Exponential Convergence Theorems 232

This appendix provides the proofs for Theorems 4.2.3 and 4.2.7, the exponential convergence results for BTS and DENTS stated in Section 4.5. The DENTS stochastic process is first recalled in Section D.1, and the theoretical results are proven in Section D.2.

Note that Theorems 4.2.3 and 4.2.7 are not related to the AR-BTS and AR-DENTS versions, so are not considered in this appendix.

D.1 The DENTS Stochastic Process

This subsection recalls the *DENTS stochastic process*, with the notation and assumptions from Section 4.5 in-lined.

All value estimates are initialised to zero for all $s \in \mathcal{S}_0$ and $a \in \mathcal{A}$:

$$\hat{V}^{(0)}_{\text{DNTS}}(s) = 0, \quad \hat{Q}^{(0)}_{\text{DNTS}}(s,a) = 0, \quad \bar{\mathcal{H}}^{(0)}_{V,\text{DNTS}}(s) = 0, \quad \bar{\mathcal{H}}^{(0)}_{Q,\text{DNTS}}(s,a) = 0. \tag{D.1}$$

On the nth trial, a trajectory is sampled using the search policy $\tau^n \sim \pi^n_{\text{DNTS}}$. Where for all $(s,a) \in \mathcal{S}_0 \times \mathcal{A}$, the search policy is defined as:

$$\hat{Q}^{(N^n(s,a))}_{\text{DNTS,sft}}(s,a) = \hat{Q}^{(N^n(s,a))}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N^n(s))\bar{\mathcal{H}}^{(N^n(s,a))}_{Q,\text{DNTS}}(s,a), \tag{D.2}$$

$$\rho^n_{\text{DNTS}}(a|s) = \frac{\exp\left(\frac{\hat{Q}^{(N^n(s,a))}_{\text{DNTS,sft}}(s,a)}{\alpha_{\text{DNTS}}(N^n(s))}\right)}{\sum\limits_{a' \in \mathcal{A}} \exp\left(\frac{\hat{Q}^{(N^n(s,a'))}_{\text{DNTS,sft}}(s,a')}{\alpha_{\text{DNTS}}(N^n(s))}\right)}. \tag{D.3}$$

$$\lambda^n(s, \epsilon_{\text{DNTS}}) = \min\left(1, \frac{\epsilon_{\text{DNTS}}}{\log(e + N^{n-1}(s))}\right), \tag{D.4}$$

$$\pi^n_{\text{DNTS}}(a|s) = (1 - \lambda^n(s, \epsilon_{\text{DNTS}}))\rho^n_{\text{DNTS}}(a|s) + \frac{\lambda^n(s, \epsilon_{\text{DNTS}})}{|\mathcal{A}|}. \tag{D.5}$$

$$\tag{D.6}$$

Letting $\tau^n = (s^n_0, a^n_0, r^n_0, \ldots, s^n_{H-1}, a^n_{H-1}, r^n_{H-1}, s^n_H)$, the DENTS value and entropy estimates are computed as follows for $t = H-1, \ldots, 0$:

$$\hat{Q}^{(N^n(s_t,a_t))}_{\text{DNTS}}(s_t,a_t) = R(s_t,a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N^n(s')}{N^n(s_t,a_t)} \hat{V}^{(N^n(s'))}_{\text{DNTS}}(s')\right), \tag{D.7}$$

$$\hat{V}^{(N^n(s_t))}_{\text{DNTS}}(s_t) = \max_{a \in \mathcal{A}} \hat{Q}^{(N^n(s_t,a))}_{\text{DNTS}}(s_t,a), \tag{D.8}$$

$$\bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{DNTS}}(s_t,a_t) = \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t,a_t)} \bar{\mathcal{H}}^{(N^n(s'))}_{V,\text{DNTS}}(s'), \tag{D.9}$$

$$\bar{\mathcal{H}}^{(N^n(s_t))}_{V,\text{DNTS}}(s_t) = \mathcal{H}(\pi^n_{\text{DNTS}}(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi^n_{\text{DNTS}}(a_t|s_t)\bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{DNTS}}(s_t,a_t). \tag{D.10}$$

Recall that the visit counts are *defined* as a sum of indicator random variables using the sampled trajectories:

$$N^n(s_t) = \sum_{i=1}^{n} \mathbb{1}[s^i_t = s_t], \tag{D.11}$$

$$N^n(s_t,a_t) = \sum_{i=1}^{n} \mathbb{1}[s^i_t = s_t \cap a^i_t = a_t]. \tag{D.12}$$

And note that if $s \notin \tau^n$, then $N^n(s) = N^{n-1}(s)$, and similarly if $(s,a) \notin \tau^n$, then $N^n(s,a) = N^{n-1}(s,a)$. More specifically, this implies, for example, that if $s \notin \tau^n$ then:

$$\hat{V}^{(N^n(s))}_{\text{DNTS}}(s) = \hat{V}^{(N^{n-1}(s))}_{\text{DNTS}}(s). \tag{D.13}$$

Finally, the maximum value and most visited recommendation policies after n trials are respectively defined as:

$$\psi^n_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{(N^n(s,a))}_{\text{DNTS}}(s, a), \tag{D.14}$$

$$\texttt{mv}^n_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} N^n(s, a). \tag{D.15}$$

D.1.1 The BTS process.

The BTS process is a special case of the DENTS process when $\beta_{\text{DNTS}}(x) = 0$.

D.2 Exponential Convergence Results

This subsection provides the proofs of Theorems 4.2.3 and 4.2.7. Similar to Section 4.5, the main results are shown first, providing forward references to the lemmas used.

Theorem 4.2.7. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{DNTS}}(x) \geq L > 0$ and $U \geq \beta_{\text{DNTS}}(x)$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \leq C \exp(-kn)$ after running n trials of DENTS.*

Proof. The proof is broken down into the following steps:

- Lemma D.2.2 shows that under the assumptions that at any state, all actions have a strictly positive probability of being selected;
- Lemma D.2.3 establishes that, under the assumptions, that concentration inequalities with respect to $N^n(s_t, a_t)$ can be lifted to concentration inequalities with respect to $N^n(s_t)$;
- Lemma D.2.4 similarly establishes that concentration inequalities with respect to $N^n(s_{t+1})$ can be lifted to inequalities with respect to $N^n(s_t, a_t)$;
- Lemmas D.2.5 and D.2.6 provide the inductive steps used to show concentration inequalities on the value estimates of the DENTS process in Theorem D.2.2;

- Lemma D.2.7 then uses Theorem D.2.2 to provide exponential bounds on the immediate simple regret of the DENTS process;
- Finally, this theorem holds by using Lemma 4.5.1 to lift the immediate simple regret bounds from Lemma D.2.7 to bounds on the simple regret.

□

Theorem 4.2.3. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{BTS}}(x) \geq L > 0$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \leq C\exp(-kn)$ after running n trials of BTS.*

Proof. Follows from the proof of Theorem 4.2.7 with $\beta_{\text{DNTS}}(x) = 0$. □

The bounds shown in this appendix are *problem dependent*, where the constants may depend on the MDP $\mathcal{M}$.

Definition D.2.1. *The* suboptimality gap $\Delta_{\mathcal{M}}$ *of an MDP $\mathcal{M}$ is the smallest difference in optimal values between an optimal action and a suboptimal action, over the reachable states $\mathcal{S}_0$ (Definition 2.2.3). That is, let $a_s^* = \arg\max_a Q^*(s,a)$. The suboptimality gap is then:*

$$\Delta_{\mathcal{M}} = \min_{s \in \mathcal{S}_0} \left(Q^*(s, a_s^*) - \max_{a \in \mathcal{A} - \{a_s^*\}} Q^*(s,a) \right). \tag{D.16}$$

D.2.1 Lemmas for Exponential Convergence Theorems

Firstly, a standard consequence of the union bound will be useful: intersections over exponentially likely events are also exponentially likely (and conversely for unions over exponentially unlikely events). It is stated explicitly for reference.

Lemma D.2.1. *Let $A_1, ..., A_\ell$ be some events that each satisfy for $1 \leq i \leq \ell$ the inequality $\Pr(\neg A_i) \leq C_i \exp(-k_i)$, then:*

$$\Pr\left(\bigcup_{i=1}^{\ell} \neg A_i \right) \leq C\exp(-k), \tag{D.17}$$

$$\Pr\left(\bigcap_{i=1}^{\ell} A_i \right) = 1 - \Pr\left(\bigcup_{i=1}^{\ell} \neg A_i \right) \geq 1 - C\exp(-k), \tag{D.18}$$

where $C = \sum_{i=1}^{\ell} C_i$ and $k = \min_i k_i$.

Proof outline. Lemma D.2.1 is a consequence of the union bound, using $\exp(-k_i) \leq \exp(-k)$ and simplifying. Inequality (D.18) is just a negation of Inequality (D.17). □

Additionally, Hoeffding's inequality will be useful to bound the difference between a sum of indicator random variables and its expectation.

Theorem D.2.1. *Let $\{X_i\}_{i=1}^k$ be indicator random variables (i.e. $X_i \in \{0,1\}$), and $S_k = \sum_{i=1}^k X_i$. Then Hoeffding's inequality for indicator random variables states for any $\varepsilon > 0$ that:*

$$\Pr(|S_k - \mathbb{E}[S_k]| > \varepsilon) \leq 2\exp\left(-\frac{2\varepsilon^2}{k}\right). \tag{D.19}$$

Proof. This is a specific case of Hoeffding's inequality. See [45] for proof. □

When there is some finite L, U such that $\alpha_{\text{DNTS}}(x) \geq L > 0$ and $\beta_{\text{DNTS}}(x) \leq U < \infty$, then the DENTS process will have a strictly positive probability for all actions in Equation (D.5). In other words, under these conditions, the DENTS search policy will always have some uniform component to it.

Lemma D.2.2. *For any DENTS process with some L, U such that $\alpha_{\text{DNTS}}(x) \geq L > 0$ and $\beta_{\text{DNTS}}(x) \leq U < \infty$. Then there exists some $\pi^{\min} > 0$, for any state $s_t \in \mathcal{S}_0$, for all $a_t \in \mathcal{A}$ and any number of trials $n \in \mathbb{N}$, such that $\pi^n_{\text{DNTS}}(a_t|s_t) \geq \pi^{\min}$.*

Proof outline. Pick an arbitrary $n \in \mathbb{N}$, $s \in \mathcal{S}_0$ and $a \in \mathcal{A}$. It suffices to show that $\pi^n_{\text{DNTS}}(a|s) > 0$.

Let $R_{\min} \leq R(s,a) \leq R_{\max}$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$. Then the soft Q-values used in DENTS (Equation D.2) are bounded:

$$HR_{\min} \leq \hat{Q}^{(n)}_{\text{DNTS,sft}}(s,a) \leq HR_{\max} + U\log(|\mathcal{A}|), \tag{D.20}$$

using the assumption that $\beta_{\text{DNTS}}(x) \leq U < \infty$.

Because $\alpha_{\text{DNTS}}(x) \geq L > 0$, the arguments to all exponentials in the Boltzmann policy (Equation (D.3)) are bounded above and below, so then $\rho^n_{\text{DNTS}}(a|s) > 0$.

Because $\frac{1}{|\mathcal{A}|} > 0$, the search policy must also be positive $\pi^n_{\text{DNTS}}(a|s) > 0$, because it is the linear combination of two positive terms (Equation D.5).

Finally, because n, s, a were arbitrary, there must be some

$$\pi^{\min} = \min_{n \in \mathbb{N}, s \in \mathcal{S}_0, a \in \mathcal{A}} \pi^n_{\text{DNTS}}(a|s) > 0. \tag{D.21}$$

□

D.2.2 Inductive Steps for Exponential Convergence Theorems

For the induction, it is convenient to be able to 'translate' bounds that depend on some $N^n(s_t, a_t)$ to a corresponding bound on $N^n(s_t)$. In every application below, the event of interest concerns the value estimates in the subtree below (s_t, a_t), and so is determined by the trajectories that pass through (s_t, a_t):

Lemma D.2.3. *Consider any DENTS process with positive α_{DNTS} and bounded β_{DNTS}, and some state action pair $(s_t, a_t) \in \mathcal{S}_0 \times \mathcal{A}$. Suppose that the action a_t has some minimum probability η of being chosen from the state s_t (irrespective of the number of trials), i.e. $\Pr(a^i_t = a_t | s^i_t = s_t) \geq \eta$. Let E be an event that is determined by the trajectories passing through (s_t, a_t), so that conditioning further on $N^n(s_t)$ carries no additional information:*

$$\Pr\left(E \;\middle|\; N^n(s_t, a_t), N^n(s_t)\right) = \Pr\left(E \;\middle|\; N^n(s_t, a_t)\right). \tag{D.22}$$

And suppose for some $C', k' > 0$ that E admits a bound that holds almost surely, conditional on the visit count:

$$\Pr\left(E \;\middle|\; N^n(s_t, a_t)\right) \leq C' \exp(-k' N^n(s_t, a_t)). \tag{D.23}$$

Then, there exists $C, k > 0$ such that, almost surely:

$$\Pr\left(E \;\middle|\; N^n(s_t)\right) \leq C \exp(-k N^n(s_t)). \tag{D.24}$$

Proof. Recall from Equation (4.44) that

$$N^n(s_t, a_t) = \sum_{i \in T^n(s_t)} \mathbb{1}[a_t^i = a_t] = \sum_{i \in T^n(s_t)} \mathbb{1}[a_t^i = a_t | s_t^i = s_t]. \tag{D.25}$$

Taking the expectation over a_t and given $N^n(s_t)$ gives:

$$\mathbb{E}\left[N^n(s_t, a_t) \mid N^n(s_t)\right] = \sum_{i \in T^n(s_t)} \Pr\left(a_t^i = a_t | s_t^i = s_t\right). \tag{D.26}$$

Using the assumed lower bound $\Pr(a_t^i = a_t | s_t^i = s_t) \geq \eta$, $|T^n(s_t)| = N^n(s_t)$ and rearranging gives:

$$\mathbb{E}\left[N^n(s_t, a_t) \mid N^n(s_t)\right] - \frac{\eta N^n(s_t)}{2} \geq \frac{\eta N^n(s_t)}{2}. \tag{D.27}$$

So then the probability of $N^n(s_t, a_t)$ being below a multiplicative ratio of $N^n(s_t)$ can be bounded above as follows, where all of the probabilities are conditional on $N^n(s_t)$, so that (D.31) is a sum of $N^n(s_t)$ indicator random variables:

$$\Pr\left(N^n(s_t, a_t) < \frac{1}{2}\eta N^n(s_t) \;\middle|\; N^n(s_t)\right) \tag{D.28}$$

$$\leq \Pr\left(N^n(s_t, a_t) < \mathbb{E}\left[N^n(s_t, a_t) | N^n(s_t)\right] - \frac{1}{2}\eta N^n(s_t) \;\middle|\; N^n(s_t)\right) \tag{D.29}$$

$$= \Pr\left(\mathbb{E}\left[N^n(s_t, a_t) | N^n(s_t)\right] - N^n(s_t, a_t) > \frac{1}{2}\eta N^n(s_t) \;\middle|\; N^n(s_t)\right) \tag{D.30}$$

$$\leq \Pr\left(\left|\mathbb{E}\left[N^n(s_t, a_t) | N^n(s_t)\right] - N^n(s_t, a_t)\right| > \frac{1}{2}\eta N^n(s_t) \;\middle|\; N^n(s_t)\right) \tag{D.31}$$

$$\leq 2\exp\left(-\frac{1}{2}\eta^2 N^n(s_t)\right). \tag{D.32}$$

The first inequality follows from Equation (D.27), the second line is a rearrangement, the third line comes from the inequality in (D.30) implying the inequality in (D.31), and the final line uses Theorem D.2.1, a Hoeffding bound for the sum of indicator random variables.

Finally, the bound using $N^n(s_t, a_t)$ can be converted into one depending on $N^n(s_t)$ by conditioning on the visit counts. Let E' denote the event that $N^n(s_t, a_t) \geq \frac{1}{2}\eta N^n(s_t)$, and note that the indicator $\mathbb{1}[E']$ is determined by $N^n(s_t)$ and $N^n(s_t, a_t)$.

Splitting on E' using $\mathbb{1}[E'] + \mathbb{1}[\neg E'] = 1$ then gives almost surely:

$$\Pr\left(E \;\middle|\; N^n(s_t, a_t)\right) \tag{D.33}$$

$$= \mathbb{1}[E'] \Pr\left(E \;\middle|\; N^n(s_t, a_t)\right) + \mathbb{1}[\neg E'] \Pr\left(E \;\middle|\; N^n(s_t, a_t)\right) \tag{D.34}$$

$$\leq \mathbb{1}[E']\; C' \exp\left(-k' N^n(s_t, a_t)\right) + \mathbb{1}[\neg E'] \tag{D.35}$$

$$\leq C' \exp\left(-\frac{1}{2} k' \eta N^n(s_t)\right) + \mathbb{1}[\neg E']. \tag{D.36}$$

Inequality (D.35) follows from the assumed Inequality D.23 and the trivial bound $\Pr(E \mid N^n(s_t, a_t)) \leq 1$. The final line follows by considering when E' is true or false. When $\mathbb{1}[E'] = 0$ the inequality is trivial. When $\mathbb{1}[E'] = 1$, the event E' is true, so $N^n(s_t, a_t) \geq \frac{1}{2}\eta N^n(s_t)$, which gives $C' \exp\left(-k' N^n(s_t, a_t)\right) \leq C' \exp\left(-\frac{1}{2} k' \eta N^n(s_t)\right)$.

Taking expectations conditional on $N^n(s_t)$, the left hand side (D.33) becomes $\Pr(E \mid N^n(s_t))$ by the tower property, and $\mathbb{E}\left[\mathbb{1}[\neg E'] \mid N^n(s_t)\right] = \Pr(\neg E' \mid N^n(s_t))$, which is precisely the quantity bounded by (D.32):

$$\Pr\left(E \;\middle|\; N^n(s_t)\right) \leq C' \exp\left(-\frac{1}{2} k' \eta N^n(s_t)\right) + 2 \exp\left(-\frac{1}{2} \eta^2 N^n(s_t)\right) \tag{D.37}$$

$$\leq C \exp(-k N^n(s_t)). \tag{D.38}$$

The final inequality (D.38) follows from Lemma D.2.1, with $C = C' + 2$ and $k = \min(k'\eta/2, \eta^2/2)$. □

Similar to Lemma D.2.3, it is convenient to be able to translate bounds that depend on $N^n(s_{t+1})$, for some $s_{t+1} \in \mathrm{Succ}(s_t, a_t)$, into bounds that depend on $N^n(s_t, a_t)$:

Lemma D.2.4. *Consider any DENTS process with positive α_{DNTS} and bounded β_{DNTS}. For some state action pair $(s_t, a_t) \in \mathcal{S}_0 \times \mathcal{A}$, and for some $s'_{t+1} \in \mathrm{Succ}(s_t, a_t)$, let E be an event that is determined by the trajectories passing through s'_{t+1}, so that:*

$$\Pr\left(E \;\middle|\; N^n(s'_{t+1}), N^n(s_t, a_t)\right) = \Pr\left(E \;\middle|\; N^n(s'_{t+1})\right). \tag{D.39}$$

And suppose for some $C', k' > 0$ that E admits a bound that holds almost surely, conditional on the visit count:

$$\Pr\left(E \;\middle|\; N^n(s'_{t+1})\right) \leq C' \exp(-k' N^n(s'_{t+1})). \tag{D.40}$$

Then, there exists $C, k > 0$ such that, almost surely:

$$\Pr\left(E \;\middle|\; N^n(s_t, a_t)\right) \leq C \exp(-k N^n(s_t, a_t)). \tag{D.41}$$

Proof outline. Proof is similar to Lemma D.2.3. Instead of having a minimum probability of selecting an action η, replace it with the corresponding probability from the transition distribution $p(s_{t+1}|s_t, a_t)$. Then swapping any $N^n(s_t)$ with $N^n(s_t, a_t)$, any $N^n(s_t, a_t)$ with $N^n(s_{t+1})$, and using $N^n(s_{t+1}) = \sum_{i \in T^n(s_t, a_t)} \mathbb{1}[s^i_{t+1} = s_{t+1}]$ (Equation (4.45)) as the sum of indicator random variables will give the result. □

Next, the induction step from $\hat{V}_{\text{DNTS}}$ to $\hat{Q}_{\text{DNTS}}$ is given. It is shown that if $\hat{V}^{(N^n(s_{t+1}))}_{\text{DNTS}}(s_{t+1})$ admit exponential concentration bounds, then $\hat{Q}^{(N^n(s_t, a_t))}_{\text{DNTS}}(s_t, a_t)$ must also admit exponential concentration bounds:

Lemma D.2.5. *Consider any DENTS process with positive α_{DNTS} and bounded β_{DNTS}, and some state action pair $(s_t, a_t) \in \mathcal{S}_0 \times \mathcal{A}$. Suppose that for all $s_{t+1} \in \text{Succ}(s_t, a_t)$ there is some $C_{s_{t+1}}, k_{s_{t+1}} > 0$ such that for all $\varepsilon_0 > 0$:*

$$\Pr\left(\left|\hat{V}^{(N^n(s_{t+1}))}_{\text{DNTS}}(s_{t+1}) - V^*(s_{t+1})\right| > \varepsilon_0\right) \leq C_{s_{t+1}} \exp(-k_{s_{t+1}} \varepsilon_0^2 N^n(s_{t+1})). \tag{D.42}$$

Then there exists some $C, k > 0$, for all $\varepsilon > 0$ such that:

$$\Pr\left(\left|\hat{Q}^{(N^n(s_t, a_t))}_{\text{DNTS}}(s_t, a_t) - Q^*(s_t, a_t)\right| > \varepsilon\right) \leq C \exp(-k \varepsilon^2 N^n(s_t, a_t)). \tag{D.43}$$

Proof. By the assumed bounds, Lemma D.2.1 and Lemma D.2.4, there is some $C_1, k_1 > 0$, such that for any $\varepsilon_1 > 0$:

$$\Pr\left(\forall s_{t+1}. \left|\hat{V}^{(N^n(s_{t+1}))}_{\text{DNTS}}(s_{t+1}) - V^*(s_{t+1})\right| \leq \varepsilon_1\right) > 1 - C_1 \exp(-k_1 \varepsilon_1^2 N^n(s_t, a_t)). \tag{D.44}$$

And recall that for any $\varepsilon_2 > 0$, using Corollary 4.5.1 that:

$$\begin{aligned}&\Pr\left(\max_{s_{t+1}\in\text{Succ}(s_t,a_t)}\left|\frac{N^n(s_{t+1})}{N^n(s_t,a_t)} - p(s_{t+1}|s_t,a_t)\right| \leq \varepsilon_2\right)\\ >&1 - 2\exp\left(-\frac{1}{2}\varepsilon_2^2 N^n(s_t,a_t)\right).\end{aligned} \tag{D.45}$$

If the events in Inequalities (D.44) and (D.45) hold, then the following inequalities must also hold:

$$V^*(s_{t+1}) - \varepsilon_1 \leq \hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1}) \leq V^*(s_{t+1}) + \varepsilon_1 \tag{D.46}$$

$$p(s_{t+1}|s_t,a_t) - \varepsilon_2 \leq \frac{N^n(s_{t+1})}{N^n(s_t,a_t)} \leq p(s_{t+1}|s_t,a_t) + \varepsilon_2. \tag{D.47}$$

The upper bounds on $\hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1})$ and $N^n(s_{t+1})/N^n(s_t,a_t)$ can be used to obtain an upper bound on $\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t)$, starting from the backup from Equation (D.7):

$$\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) \tag{D.48}$$

$$= R(s_t,a_t) + \sum_{s_{t+1}\in\text{Succ}(s_t,a_t)} \frac{N^n(s_{t+1})}{N^n(s_t,a_t)} \hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1}) \tag{D.49}$$

$$\leq R(s_t,a_t) + \sum_{s_{t+1}\in\text{Succ}(s_t,a_t)} \frac{N^n(s_{t+1})}{N^n(s_t,a_t)} \left|\hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1})\right| \tag{D.50}$$

$$\leq R(s_t,a_t) + \sum_{s_{t+1}\in\text{Succ}(s_t,a_t)} (p(s_{t+1}|s_t,a_t) + \varepsilon_2)(|V^*(s_{t+1})| + \varepsilon_1) \tag{D.51}$$

$$\leq R(s_t,a_t) + \mathbb{E}_{s_{t+1}\sim p(\cdot|s_t,a_t)}[V^*(s_{t+1})] + \varepsilon_2\left|\sum_{s_{t+1}\in\text{Succ}(s_t,a_t)} V^*(s_{t+1})\right| + \varepsilon_1 + \varepsilon_1\varepsilon_2 \tag{D.52}$$

$$= Q^*(s_t,a_t) + \varepsilon_2\left|\sum_{s_{t+1}\in\text{Succ}(s_t,a_t)} V^*(s_{t+1})\right| + \varepsilon_1 + \varepsilon_1\varepsilon_2. \tag{D.53}$$

Following similar reasoning but using the lower bounds on $\hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1})$ and $N^n(s_{t+1})/N^n(s_t,a_t)$ gives:

$$\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) \geq Q^*(s_t,a_t) - \varepsilon_2\left|\sum_{s_{t+1}\in\text{Succ}(s_t,a_t)} V^*(s_{t+1})\right| - \varepsilon_1 - \varepsilon_1\varepsilon_2. \tag{D.54}$$

Now given an arbitrary $\varepsilon > 0$, recalling that $\varepsilon_1, \varepsilon_2 > 0$ were arbitrary, set $\varepsilon_1 = \varepsilon/3$ and $\varepsilon_2 = \min(\varepsilon/3, \varepsilon/3|\sum_{s_{t+1}} V^*(s_{t+1})|)$ to give:

$$Q^*(s_t,a_t) - \varepsilon \leq \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) \leq Q^*(s_t,a_t) + \varepsilon. \tag{D.55}$$

Using Lemma D.2.1 liberally, there is some $C_2, C_3, k_2, k_3 > 0$ such that:

$$\Pr\left(\left|\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) - Q^*(s_t,a_t)\right| \leq \varepsilon\right) \tag{D.56}$$

$$> \left(1 - C_1 \exp(-k_1\varepsilon_1^2 N^n(s_t,a_t))\right)\left(1 - 2\exp\left(-\frac{1}{2}\varepsilon_2^2 N^n(s_t,a_t)\right)\right) \tag{D.57}$$

$$= \left(1 - C_2 \exp(-k_2\varepsilon^2 N^n(s_t,a_t))\right) \cdot \left(1 - C_3 \exp(-k_3\varepsilon^2 N^n(s_t,a_t))\right) \tag{D.58}$$

$$> 1 - C_2 \exp(-k_2\varepsilon^2 N^n(s_t,a_t)) - C_3 \exp(-k_3\varepsilon^2 N^n(s_t,a_t)). \tag{D.59}$$

Finally, by negating, setting $C = C_2 + C_3$ and $k = \min(k_2, k_3)$ and using Lemma D.2.1 again, the result follows:

$$\Pr\left(\left|\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) - Q^*(s_t,a_t)\right| > \varepsilon\right) \leq C\exp(-k\varepsilon^2 N^n(s_t,a_t)). \tag{D.60}$$

□

Lemma D.2.5 is now used to show a larger induction step, that if exponential concentration bounds exist for $\hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1})$, then they also exist for $\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t)$:

Lemma D.2.6. *Consider a DENTS process with positive α_{DNTS} and bounded β_{DNTS}. Let $s_t \in \mathcal{S}_0$, with $0 \leq t < H$. If for all $s_{t+1} \in \bigcup_{a\in\mathcal{A}} \text{Succ}(s_t, a)$ there is some $C_{s_{t+1}}, k_{s_{t+1}} > 0$ such that for any $\varepsilon_{s_{t+1}} > 0$:*

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N^n(s_{t+1}))}(s_{t+1}) - V^*(s_{t+1})\right| > \varepsilon_{s_{t+1}}\right) \leq C_{s_{t+1}} \exp\left(-k_{s_{t+1}}\varepsilon_{s_{t+1}}^2 N^n(s_{t+1})\right), \tag{D.61}$$

then there is some $C, k > 0$, for any $\varepsilon > 0$:

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) - V^*(s_t)\right| > \varepsilon\right) \leq C\exp\left(-k\varepsilon^2 N^n(s_t)\right). \tag{D.62}$$

Proof. Given the assumptions and by Lemmas D.2.5, D.2.1 and D.2.3, for some $C, k > 0$ and for any $\varepsilon_1 > 0$:

$$\Pr\left(\forall a_t \in \mathcal{A}.\left|\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t,a_t) - Q^*(s_t,a_t)\right| \leq \varepsilon_1\right) > 1 - C\exp(-k\varepsilon_1^2 N^n(s_t)). \tag{D.63}$$

Let $\varepsilon > 0$, and set $\varepsilon_1 = \min(\varepsilon, \Delta_{\mathcal{M}}/2)$ (recalling $\Delta_{\mathcal{M}}$ from Definition D.2.1). So with probability at least $1 - C_1 \exp(-k_1 \varepsilon_1^2 N^n(s_t))$ for any a_t it must be that:

$$Q^*(s_t, a_t) - \varepsilon_1 \leq \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) \leq Q^*(s_t, a_t) + \varepsilon_1. \tag{D.64}$$

Let $a^* = \arg\max_{a \in \mathcal{A}} Q^*(s_t, a)$. Using $\varepsilon_1 \leq \Delta_{\mathcal{M}}/2$ in (D.64), then for any $a \in \mathcal{A} - \{a^*\}$:

$$\begin{aligned}
\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) \leq & Q^*(s_t, a) + \Delta_{\mathcal{M}}/2 && \text{(D.65)}\\
\leq & Q^*(s_t, a^*) - \Delta_{\mathcal{M}}/2 && \text{(D.66)}\\
\leq & \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a^*))}(s_t, a^*), && \text{(D.67)}
\end{aligned}$$

and hence $\arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) = a^*$. As a consequence:

$$\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) = \max_a \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) = \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a^*))}(s_t, a^*). \tag{D.68}$$

Then using (D.64) with $a_t = a^*$ (noting $V^*(s_t) = Q^*(s_t, a^*)$), using (D.68) and using $\varepsilon_1 \leq \varepsilon$ gives:

$$\begin{aligned}
V^*(s_t) - \varepsilon \leq & V^*(s_t) - \varepsilon_1 && \text{(D.69)}\\
\leq & \hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) && \text{(D.70)}\\
\leq & V^*(s_t) + \varepsilon_1 && \text{(D.71)}\\
\leq & V^*(s_t) + \varepsilon. && \text{(D.72)}
\end{aligned}$$

Hence:

$$\begin{aligned}
\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) - V^*(s_t)\right| > \varepsilon\right) \leq & C \exp(-k\varepsilon_1^2 N^n(s_t)) && \text{(D.73)}\\
\leq & C \exp(-k\varepsilon^2 N^n(s_t)), && \text{(D.74)}
\end{aligned}$$

which is the result. □

Now the following Theorem performs the induction using Lemma D.2.6 to show that the value estimates for all reachable states converge to the optimal values in the DENTS process:

Theorem D.2.2. *Consider a DENTS process with positive α_{DNTS} and bounded β_{DNTS}, let $s_t \in \mathcal{S}_0$ then there is some $C, k > 0$ for any $\varepsilon > 0$:*

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) - V^*(s_t)\right| > \varepsilon\right) \leq C \exp\left(-k\varepsilon^2 N^n(s_t)\right). \tag{D.75}$$

Moreover, at the root node s_0:

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N^n(s_0))}(s_0) - V^*(s_0)\right| > \varepsilon\right) \leq C \exp\left(-k\varepsilon^2 n\right). \tag{D.76}$$

Proof. The result vacuously holds for $t = H$ because $\hat{V}_{\text{DNTS}}^{(N^n(s_H))}(s_H) = V^*(s_H) = 0$. Hence the result holds for all $t = 0, ..., H$ by induction using Lemma D.2.6. Noting that $N^n(s_0) = n$ gives (D.76). □

Finally, the concentration inequalities over the value estimates of the DENTS process can then be used to show that the immediate simple regret at all states tends to zero exponentially:

Lemma D.2.7. *Consider a DENTS process with positive α_{DNTS} and bounded β_{DNTS}. Let $s_t \in \mathcal{S}_0$, with $0 \leq t \leq H$ then there is some $C', k' > 0$ such that:*

$$\mathbb{E}\left[\texttt{inst_regr}(s_t, \psi_{\text{DNTS}}^n)\right] \leq C' \exp\left(-k' N^n(s_t)\right). \tag{D.77}$$

Proof. Let a^* be the locally optimal standard action, so $a^* = \arg\max_{a \in \mathcal{A}} Q^*(s_t, a)$. By Theorem D.2.2 and Lemmas D.2.5 and D.2.3 there exists $C_1, k_1 > 0$ such that for all $\varepsilon_1 > 0$:

$$\Pr\left(\forall a_t \in \mathcal{A}. \left|\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) - Q^*(s_t, a_t)\right| \leq \varepsilon_1\right) > 1 - C_1 \exp(-k_1 \varepsilon_1^2 N^n(s_t)). \tag{D.78}$$

Setting $\varepsilon_1 = \Delta_{\mathcal{M}}/2$ then gives with probability $1 - C_1 \exp(-k_2 N^n(s_t))$ (where $k_2 = k_1 \Delta_{\mathcal{M}}^2/4$) for all actions $a \in \mathcal{A}$ that:

$$Q^*(s_t, a) - \Delta_{\mathcal{M}}/2 \leq \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) \leq Q^*(s_t, a) + \Delta_{\mathcal{M}}/2. \tag{D.79}$$

And hence, with probability $1 - C_1 \exp(-k_2 N^n(s_t))$, for all $a_t \in \mathcal{A} - \{a^*\}$ it must be that:

$$\begin{aligned}
\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) \leq & Q^*(s_t, a_t) + \Delta_{\mathcal{M}}/2 && \text{(D.80)}\\
\leq & Q^*(s_t, a^*) - \Delta_{\mathcal{M}}/2 && \text{(D.81)}\\
\leq & \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a^*))}(s_t, a^*). && \text{(D.82)}
\end{aligned}$$

Negating the probability of (D.82) then gives:

$$\Pr\left(\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) > \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a^*))}(s_t, a^*)\right) \leq C_1 \exp(-k_2 N^n(s_t)) \qquad \text{(D.83)}$$

Finally, the bound on the expected immediate regret follows:

$$\begin{aligned}
& \mathbb{E}\left[\texttt{inst_regr}(s_t, \psi_{\text{DNTS}}^n)\right] && \text{(D.84)}\\
= & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \Pr\left(\psi_{\text{DNTS}}^n(s_t) = a\right) && \text{(D.85)}\\
= & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \Pr\left(a = \arg\max_{a'} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a'))}(s_t, a')\right) && \text{(D.86)}\\
\leq & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \Pr\left(\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) > \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a^*))}(s_t, a^*)\right) && \text{(D.87)}\\
\leq & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \, C_1 \exp(-k_2 N^n(s_t)), && \text{(D.88)}
\end{aligned}$$

and setting $k' = k_2$ and $C' = C_1 \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a))$ gives the result. □

Bibliography

[1] Axel Abels, Diederik M Roijers, Tom Lenaerts, Ann Nowé, and Denis Steckelmacher. Dynamic weights in multi-objective deep reinforcement learning. In *International Conference on Machine Learning*, pages 11–20. PMLR, 2019.

[2] Bruce Abramson. Expected-outcome: A general model of static evaluation. *IEEE transactions on pattern analysis and machine intelligence*, 12(2):182–193, 1990.

[3] Guillermo Aparicio, Leocadio G Casado, Eligius MT Hendrix, Boglárka G.-Tóth, and Inmaculada Garcia. On the minimum number of simplex shapes in longest edge bisection refinement of a regular n-simplex. *Informatica*, 26(1): 17–32, 2015.

[4] P Auer. Finite-time analysis of the multiarmed bandit problem, 2002.

[5] David Auger, Adrien Couetoux, and Olivier Teytaud. Continuous upper confidence trees with polynomial exploration–consistency. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part I 13*, pages 194–209. Springer, 2013.

[6] C Bradford Barber, David P Dobkin, and Hannu Huhdanpaa. Qhull: Quickhull algorithm for computing the convex hull. *Astrophysics Source Code Library*, pages ascl–1304, 2013.

[7] Leon Barrett and Srini Narayanan. Learning all optimal policies with multiple criteria. In *Proceedings of the 25th international conference on Machine learning*, pages 41–47, 2008.

[8] Andrew G Barto, Steven J Bradtke, and Satinder P Singh. Learning to act using real-time dynamic programming. *Artificial intelligence*, 72(1-2):81–138, 1995.

[9] Richard Bellman. *Dynamic Programming*. Dover Publications, 1957. ISBN 9780486428093.

[10] Vincent Berthier, Hassen Doghmen, and Olivier Teytaud. Consistency modifications for automatically tuned Monte-Carlo tree search. In *International Conference on Learning and Intelligent Optimization*, pages 111–124. Springer, 2010.

[11] Lilian Besson and Emilie Kaufmann. What doubling tricks can and can't do for multi-armed bandits. *arXiv preprint arXiv:1803.06971*, 2018.

[12] Patrick Billingsley. *Probability and measure*. John Wiley & Sons, 2017.

[13] Julian Blank, Kalyanmoy Deb, Yashesh Dhebar, Sunith Bandaru, and Haitham Seada. Generating well-spaced points on a unit simplex for evolutionary many-objective optimization. *IEEE Transactions on Evolutionary Computation*, 25(1):48–60, 2020.

[14] Blai Bonet and Hector Geffner. Labeled RTDP: Improving the convergence of real-time dynamic programming. In *ICAPS*, volume 3, pages 12–21, 2003.

[15] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in games*, 4(1):1–43, 2012.

[16] Sébastien Bubeck, Gilles Stoltz, Csaba Szepesvári, and Rémi Munos. Online optimization in X-armed bandits. *Advances in neural information processing systems*, 21, 2008.

[17] Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure exploration in multi-armed bandits problems. In *Algorithmic Learning Theory: 20th International Conference, ALT 2009, Porto, Portugal, October 3-5, 2009. Proceedings 20*, pages 23–37. Springer, 2009.

[18] Sébastien Bubeck, Rémi Munos, Gilles Stoltz, and Csaba Szepesvári. X-armed bandits. *Journal of Machine Learning Research*, 12(5), 2011.

[19] Amber Cassimon, Reinout Eyckerman, Siegfried Mercelis, Steven Latré, and Peter Hellinckx. A review of the Deep Sea Treasure problem as a multi-objective reinforcement learning benchmark. *arXiv preprint arXiv:2110.06742*, 2021.

[20] Andrea Castelletti, Francesca Pianosi, and Marcello Restelli. Tree-based fitted q-iteration for multi-objective Markov decision problems. In *The 2012 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2012.

[21] Tristan Cazenave. Sequential halving applied to trees. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(1):102–105, 2014.

[22] Tristan Cazenave, Flavien Balbo, Suzanne Pinson, et al. Monte-Carlo bus regulation. In *12th international IEEE conference on intelligent transportation systems*, volume 340345, 2009.

[23] Weizhe Chen and Lantao Liu. Pareto Monte Carlo tree search for multi-objective informative planning. *arXiv preprint arXiv:2111.01825*, 2021.

[24] Benjamin E Childs, James H Brodeur, and Levente Kocsis. Transpositions and move groups in Monte Carlo tree search. In *2008 IEEE Symposium On Computational Intelligence and Games*, pages 389–395. IEEE, 2008.

[25] Pierre-Arnaud Coquelin and Rémi Munos. Bandit algorithms for tree search. *arXiv preprint cs/0703062*, 2007.

[26] Adrien Couëtoux, Jean-Baptiste Hoock, Nataliya Sokolovska, Olivier Teytaud, and Nicolas Bonnard. Continuous upper confidence trees. In *International conference on learning and intelligent optimization*, pages 433–445. Springer, 2011.

[27] Tuan Q Dam, Carlo D'Eramo, Jan Peters, and Joni Pajarinen. Convex regularization in Monte-Carlo tree search. In *International Conference on Machine Learning*, pages 2365–2375. PMLR, 2021.

[28] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.

[29] Madalina M Drugan and Ann Nowe. Designing multi-objective multi-armed bandits algorithms: A study. In *The 2013 international joint conference on neural networks (IJCNN)*, pages 1–8. IEEE, 2013.

[30] Mădălina M Drugan, Ann Nowé, and Bernard Manderick. Pareto upper confidence bounds algorithms: an empirical study. In *2014 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pages 1–8. IEEE, 2014.

[31] Aryeh Dvoretzky, Jack Kiefer, and Jacob Wolfowitz. Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *The Annals of Mathematical Statistics*, pages 642–669, 1956.

[32] Zohar Feldman and Carmel Domshlak. Monte-Carlo planning: Theoretically fast convergence meets practical efficiency. *arXiv preprint arXiv:1309.6828*, 2013.

[33] William Feller et al. *An introduction to probability theory and its applications*. Wiley New York, 1971.

[34] Florian Felten, Lucas N. Alegre, Ann Nowé, Ana L. C. Bazzan, El Ghazali Talbi, Grégoire Danoy, and Bruno C. da Silva. A toolkit for reliable benchmarking and research in multi-objective reinforcement learning. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023.

[35] Vojtěch Forejt, Marta Kwiatkowska, and David Parker. Pareto curves for probabilistic model checking. In *International Symposium on Automated Technology for Verification and Analysis*, pages 317–332. Springer, 2012.

[36] Sylvain Gelly and David Silver. Combining online and offline knowledge in UCT. In *Proceedings of the 24th international conference on Machine learning*, pages 273–280, 2007.

[37] Sylvain Gelly and David Silver. Monte-Carlo tree search and rapid action value estimation in computer Go. *Artificial Intelligence*, 175(11):1856–1875, 2011.

[38] Irving J Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3-4):237–264, 1953.

[39] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR, 2017.

[40] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[41] Eric A Hansen and Shlomo Zilberstein. LAO*: A heuristic search algorithm that finds solutions with loops. *Artificial Intelligence*, 129(1-2):35–62, 2001.

[42] Conor F Hayes, Mathieu Reymond, Diederik M Roijers, Enda Howley, and Patrick Mannion. Risk aware and multi-objective decision making with distributional Monte Carlo tree search. *arXiv preprint arXiv:2102.00966*, 2021.

[43] Conor F Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M Zintgraf, Richard Dazeley, Fredrik Heintz, et al. A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1):26, 2022.

[44] Conor F Hayes, Mathieu Reymond, Diederik M Roijers, Enda Howley, and Patrick Mannion. Monte Carlo tree search algorithms for risk-aware and multi-objective reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 37(2):26, 2023.

[45] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American statistical association*, 58(301):13–30, 1963.

[46] Zohar Karnin, Tomer Koren, and Oren Somekh. Almost optimal exploration in multi-armed bandits. In *International conference on machine learning*, pages 1238–1246. PMLR, 2013.

[47] Thomas Keller. *Anytime optimal MDP planning with trial-based heuristic tree search*. PhD thesis, University of Freiburg, Freiburg im Breisgau, Germany, 2015.

[48] Thomas Keller and Patrick Eyerich. PROST: Probabilistic planning based on UCT. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 22, pages 119–127, 2012.

[49] Thomas Keller and Malte Helmert. Trial-based heuristic tree search for finite horizon MDPs. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 23, pages 135–143, 2013.

[50] Robert Kleinberg, Aleksandrs Slivkins, and Eli Upfal. Multi-armed bandits in metric spaces. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 681–690, 2008.

[51] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.

[52] Levente Kocsis, Csaba Szepesvári, and Jan Willemson. Improved Monte-Carlo search. *Univ. Tartu, Estonia, Tech. Rep*, 1:1–22, 2006.

[53] Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985.

[54] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[55] Kyungjae Lee, Sungjoon Choi, and Songhwai Oh. Sparse Markov decision processes with causal sparse Tsallis entropy regularization for reinforcement learning. *IEEE Robotics and Automation Letters*, 3(3):1466–1473, 2018.

[56] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, pages 661–670, 2010.

[57] Daniel J Lizotte, Michael H Bowling, and Susan A Murphy. Efficient reinforcement learning with multiple reward functions for randomized controlled trial analysis. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pages 695–702, 2010.

[58] Daniel J Lizotte, Michael Bowling, and Susan A Murphy. Linear fitted-q iteration with multiple reward functions. *The Journal of Machine Learning Research*, 13(1):3253–3295, 2012.

[59] Tyler Lu, Dávid Pál, and Martin Pál. Contextual multi-armed bandits. In *Proceedings of the Thirteenth international conference on Artificial Intelligence and Statistics*, pages 485–492. JMLR Workshop and Conference Proceedings, 2010.

[60] Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 16(1):1–118, 2023.

[61] Ruben Martinez-Cantin. BayesOpt: A Bayesian optimization library for nonlinear optimization, experimental design and bandits. *Journal of Machine Learning Research*, 15(1):3735–3739, 2014.

[62] David A McAllester and Robert E Schapire. On the convergence rate of Good-Turing estimators. In *COLT*, pages 1–6, 2000.

[63] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.

[64] Thomas M Moerland, Joost Broekens, Aske Plaat, and Catholijn M Jonker. A0C: Alpha Zero in continuous action space. *arXiv preprint arXiv:1805.09613*, 2018.

[65] Hossam Mossalam, Yannis M Assael, Diederik M Roijers, and Shimon Whiteson. Multi-objective deep reinforcement learning. *arXiv preprint arXiv:1610.02707*, 2016.

[66] Mausam Natarajan and Andrey Kolobov. *Planning with Markov decision processes: An AI perspective*. Springer Nature, 2022.

[67] Ricardo H Nochetto, Kunibert G Siebert, and Andreas Veeser. Theory of adaptive finite element methods: an introduction. In *Multiscale, nonlinear*

and adaptive approximation: Dedicated to Wolfgang Dahmen on the occasion of his 60th birthday, pages 409–542. Springer, 2009.

[68] Michael Painter. THTS++. Software, version 0.0, 2026. Available at `https://github.com/MWPainter/thts-plus-plus`.

[69] Michael Painter, Bruno Lacerda, and Nick Hawes. Convex hull Monte-Carlo Tree-Search. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30, pages 217–225, 2020.

[70] Michael Painter, Mohamed Baioumy, Nick Hawes, and Bruno Lacerda. Monte Carlo tree search with Boltzmann exploration. *Advances in Neural Information Processing Systems*, 36:78181–78192, 2023.

[71] Tom Pepels, Tristan Cazenave, Mark HM Winands, and Marc Lanctot. Minimizing simple and cumulative regret in Monte-Carlo tree search. In *Workshop on Computer Games*, pages 1–15. Springer, 2014.

[72] Francisco Perdomo and Ángel Plaza. Properties of triangulations obtained by the longest-edge bisection. *Central European Journal of Mathematics*, 12(12): 1796–1810, 2014.

[73] Laurent Péret and Frédérick Garcia. On-line search for solving Markov decision processes via heuristic sampling. *learning*, 16:2, 2004.

[74] Diego Perez, Spyridon Samothrakis, and Simon Lucas. Online and offline learning in multi-objective Monte Carlo tree search. In *2013 IEEE conference on computational inteligence in games (CIG)*, pages 1–8. IEEE, 2013.

[75] Diego Perez, Sanaz Mostaghim, Spyridon Samothrakis, and Simon M Lucas. Multiobjective Monte Carlo tree search for real-time games. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(4):347–360, 2014.

[76] Diego Perez-Liebana, Sanaz Mostaghim, and Simon M Lucas. Multi-objective tree search approaches for general video game playing. In *2016 IEEE Congress on Evolutionary Computation (CEC)*, pages 624–631. IEEE, 2016.

[77] Patrice Perny and Paul Weng. On finding compromise solutions in multi-objective Markov decision processes. In *ECAI*, volume 215, pages 969–970, 2010.

[78] Werner C Rheinboldt. On a theory of mesh-refinement processes. *SIAM Journal on Numerical Analysis*, 17(6):766–778, 1980.

[79] M Cecilia Rivara. Algorithms for refining triangular grids suitable for adaptive and multigrid techniques. *International journal for numerical methods in Engineering*, 20(4):745–756, 1984.

[80] Maria-Cecilia Rivara. Mesh refinement processes based on the generalized bisection of simplices. *SIAM Journal on Numerical Analysis*, 21(3):604–613, 1984.

[81] Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

[82] Diederik M Roijers, Peter Vamplew, Shimon Whiteson, and Richard Dazeley. A survey of multi-objective sequential decision-making. *Journal of Artificial Intelligence Research*, 48:67–113, 2013.

[83] Diederik M Roijers, Denis Steckelmacher, and Ann Nowé. Multi-objective reinforcement learning for the expected utility of the return. In *Proceedings of the Adaptive and Learning Agents Workshop at FAIM*, 2018.

[84] Diederik Marijn Roijers, Shimon Whiteson, and Frans A Oliehoek. Computing convex coverage sets for faster multi-objective coordination. *Journal of Artificial Intelligence Research*, 52:399–443, 2015.

[85] Ivo G Rosenberg and Frank Stenger. A lower bound on the angles of triangles constructed by bisecting the longest side. *Mathematics of Computation*, 29 (130):390–395, 1975.

[86] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

[87] Abdallah Saffidine, Tristan Cazenave, and Jean Méhat. UCD: Upper Confidence bound for rooted Directed acyclic graphs. *Knowledge-Based Systems*, 34:26–33, 2012.

[88] Jose MG Salmerón, Guillermo Aparicio, Leocadio G Casado, Inmaculada García, Eligius MT Hendrix, and B G.-Tóth. Generating a smallest binary tree by proper selection of the longest edges to bisect in a unit simplex refinement. *Journal of Combinatorial Optimization*, 33(2):389–402, 2017.

[89] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[90] Claude Elwood Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

[91] David Silver and Joel Veness. Monte-Carlo planning in large POMDPs. *Advances in neural information processing systems*, 23, 2010.

[92] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[93] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *nature*, 550(7676):354–359, 2017.

[94] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[95] Aleksandrs Slivkins. Contextual bandits with similarity information. In *Proceedings of the 24th annual Conference On Learning Theory*, pages 679–702. JMLR Workshop and Conference Proceedings, 2011.

[96] Dennis JNJ Soemers, Chiara F Sironi, Torsten Schuster, and Mark HM Winands. Enhancements for real-time Monte-Carlo tree search in general video game playing. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8. IEEE, 2016.

[97] Ralph E Steuer and Eng-Ung Choo. An interactive weighted Tchebycheff procedure for multiple objective programming. *Mathematical programming*, 26(3):326–344, 1983.

[98] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[99] David Tolpin and Solomon Shimony. MCTS based on simple regret. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 26, pages 570–576, 2012.

[100] Marko Tot, Michelangelo Conserva, Diego Perez Liebana, and Sam Devlin. Turning zeroes into non-zeroes: Sample efficient exploration with Monte Carlo graph search. In *2022 IEEE Conference on Games (CoG)*, pages 300–306. IEEE, 2022.

[101] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[102] Peter Vamplew, Dean Webb, Luisa M Zintgraf, Diederik M Roijers, Richard Dazeley, Rustam Issabekov, and Evan Dekker. MORL-Glue: A benchmark suite for multi-objective reinforcement learning. In *29th Benelux Conference on Artificial Intelligence November 8–9, 2017, Groningen*, volume 389, 2017.

[103] Kristof Van Moffaert and Ann Nowé. Multi-objective reinforcement learning using sets of Pareto dominating policies. *The Journal of Machine Learning Research*, 15(1):3483–3512, 2014.

[104] Michael D Vose. A linear algorithm for generating random numbers with a given distribution. *IEEE Transactions on software engineering*, 17(9):972–975, 1991.

[105] Alastair J Walker. An efficient method for generating discrete random variables with general distributions. *ACM Transactions on Mathematical Software (TOMS)*, 3(3):253–256, 1977.

[106] Weijia Wang and Michele Sebag. Multi-objective Monte-Carlo tree search. In *Asian conference on machine learning*, pages 507–522. PMLR, 2012.

[107] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8 (3):279–292, 1992.

[108] D.J. White. Multi-objective infinite-horizon discounted Markov decision processes. *Journal of Mathematical Analysis and Applications*, 89(2):639–647, 1982.

[109] Marco A Wiering and Edwin D De Jong. Computing optimal stationary policies for multi-objective Markov decision processes. In *IEEE International Symposium on Approximate Dynamic Programming and Reinforcement Learning*, pages 158–165. IEEE, 2007.

[110] Ronald J Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

[111] David J Wu. Accelerating self-play learning in Go. *arXiv preprint arXiv:1902.10565*, 2019.

[112] David J. Wu. Monte-Carlo graph search from first principles. `https://github.com/lightvector/KataGo/blob/master/docs/GraphSearch.md`, 2023. Accessed: 2026-03-19.

[113] Chenjun Xiao, Ruitong Huang, Jincheng Mei, Dale Schuurmans, and Martin Müller. Maximum entropy Monte-Carlo planning. *Advances in Neural Information Processing Systems*, 32, 2019.

[114] Jie Xu, Yunsheng Tian, Pingchuan Ma, Daniela Rus, Shinjiro Sueda, and Wojciech Matusik. Prediction-guided multi-objective reinforcement learning for continuous robot control. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[115] Xiao Xu and Ge Li. Chebyshev metric based multi-objective Monte Carlo tree search for combat simulations. In *2017 21st International Conference on System Theory, Control and Computing (ICSTCC)*, pages 607–612. IEEE, 2017.

[116] Brian D Ziebart. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.

[117] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.

[118] Olgierd Cecil Zienkiewicz, Robert Leroy Taylor, Perumal Nithiarasu, and JZ Zhu. *The finite element method*, volume 3. Elsevier, 1977.